GUÍA DE APRENDIZAJE

Construir el backend de IT Service & Demand Manager

Del monolito Spring Boot a dos microservicios en Docker · Java 21 · Spring Boot 4.1.1 · Spring Data JPA · Hibernate · PostgreSQL 18 · Flyway · Testcontainers · RestClient · Docker Compose

Edición completa · 27 capítulos y 5 anexos

Preparada para Alberto de Rafael Fernández
Octubre de 2026

Índice

Cada entrada es un enlace. Desde cualquier capítulo o sección, el enlace **↑ Índice** junto al título te devuelve a su entrada en esta página, y el enlace **Paso n.m ↖** te lleva a su línea del mapa cronológico.

0 Cómo usar esta guía 8

Bloque I Mapa cronológico del desarrollo

Bloque I Mapa cronológico 12

1 Mapa cronológico del desarrollo 13
- 1.1 Las fases de un vistazo 13
- 1.2 Cómo leer el mapa 14
- 1.3 El mapa paso a paso 14
- 1.4 Rutas de lectura 16

Bloque II El desarrollo explicado, fase a fase

Fase 1 Concepción y preparación 17

2 Del servidor de aplicaciones a Spring Boot 4 18
- 2.1 El punto de partida 18
- 2.2 Generar el proyecto: Java 21, Spring Boot 4.1.1 y sus *starters* 18
- 2.3 Qué hace Spring Boot al arrancar 21
- 2.4 Organizar el código por módulo de negocio 22
- 2.5 Errores típicos y depuración 22
- 2.6 Ejercicios 23

3 PostgreSQL en Docker y la configuración de la aplicación 24
- 3.1 PostgreSQL 18 en un contenedor 24
- 3.2 La configuración de la conexión 26
- 3.3 `ddl-auto=validate`: Hibernate comprueba, Flyway decide 27
- 3.4 `open-in-view=false`: la sesión no llega al controlador 28
- 3.5 `show-sql=true`: ver lo que hace Hibernate 28
- 3.6 Errores típicos y depuración 29
- 3.7 Ejercicios 29

4 Migraciones de esquema con Flyway 30
- 4.1 Por qué migraciones versionadas 30
- 4.2 Cómo funciona Flyway por dentro 30
- 4.3 La migración V1 crea `applications` 31
- 4.4 La migración V2 carga las aplicaciones iniciales 32
- 4.5 Las reglas que evitan problemas 33
- 4.6 Errores típicos y depuración 33
- 4.7 Ejercicios 34

Fase 2 El módulo Application 35

5 La entidad Application y el contexto de persistencia 36
5.1 Tres capas: JPA, Hibernate y Spring Data 36
5.2 Mapear la entidad `Application` 36
5.3 El contexto de persistencia 38
5.4 *Dirty checking* por dentro 39
5.5 El coste de `GenerationType.IDENTITY` 40
5.6 Errores típicos y depuración 40
5.7 Ejercicios 41

6 Repositorios de Spring Data 42
6.1 `ApplicationRepository` 42
6.2 La jerarquía de `JpaRepository` 43
6.3 Cómo se genera la implementación 43
6.4 Consultas derivadas del nombre del método 44
6.5 La paginación por dentro 44
6.6 Otras formas de consultar 45
6.7 Errores típicos y depuración 45
6.8 Ejercicios 45

7 El servicio y las transacciones 47
7.1 El papel del servicio 47
7.2 Consulta, detalle y alta 47
7.3 Modificar sin llamar a `save` 49
7.4 `@Transactional` por dentro 50
7.5 Excepciones HTTP en el servicio 51
7.6 Convertir a DTO dentro de la transacción 51
7.7 Errores típicos y depuración 51
7.8 Ejercicios 52

8 DTO y Bean Validation 53
8.1 Por qué DTO separados de la entidad 53
8.2 *Records* de Java 53
8.3 Las reglas de validación 54
8.4 Dos DTO de entrada, no uno 56
8.5 Cadenas en lugar de enumerados 56
8.6 La misma regla en tres sitios 57
8.7 Errores típicos y depuración 57
8.8 Ejercicios 57

9 El controlador REST y el contrato HTTP 59
9.1 Exponer `/api/applications` 59
9.2 Cómo construye Spring un `Pageable` 61
9.3 La página en un DTO propio 61
9.4 El recorrido completo de una petición 62
9.5 El contrato HTTP del módulo 63
9.6 Errores típicos y depuración 63
9.7 Ejercicios 64

10 El tratamiento de errores 65
10.1 Un formato de error común 65
10.2 Cómo encuentra Spring el manejador 67
10.3 Rechazar filtros y ordenaciones desconocidos 67
10.4 Tipos incorrectos y cuerpos mal formados 68
10.5 Lo que todavía no está cubierto 69
10.6 Errores típicos y depuración 70
10.7 Ejercicios 70

11 Las pruebas del módulo Application 71
11.1 Tres niveles de prueba 71
11.2 Probar el servicio con Mockito 72
11.3 Probar el controlador con `@WebMvcTest` y MockMvc 74
11.4 Ejecutar las pruebas 76
11.5 Errores típicos y depuración 76
11.6 Ejercicios 76

Fase 3 El módulo Demand 78

12 El modelo de la demanda 79
12.1 Qué es una demanda 79
12.2 La migración V3 crea `demands` 79
12.3 La entidad `Demand` en el monolito 80
12.4 Las fechas de creación y modificación 82
12.5 El ciclo de vida en `changeStatus` 83
12.6 Errores típicos y depuración 85
12.7 Ejercicios 85

13 Consultas dinámicas con `Specification` 87
13.1 El problema de las combinaciones 87
13.2 La Criteria API en pocas palabras 87
13.3 Filtrar el listado con `Specification` 88
13.4 Validar estado, prioridad y ordenación 90
13.5 Errores típicos y depuración 91
13.6 Ejercicios 91

14 Carga perezosa y el problema N+1 93
14.1 Cómo aparece el problema 93
14.2 Por qué la configuración del proyecto lo hace visible 94
14.3 Resolverlo con `@EntityGraph` 94
14.4 Comprobar que el problema no vuelve 95
14.5 Otras formas de evitar el N+1 96
14.6 Errores típicos y depuración 97
14.7 Ejercicios 97

15 Servicio, ciclo de vida y API de demandas 98
15.1 Crear una demanda 98
15.2 Modificar y cambiar de estado 100
15.3 El controlador de demandas 101
15.4 Probar el dominio, el servicio y el controlador 102

15.5 Errores típicos y depuración 104
15.6 Ejercicios 104

Fase 4 El monolito terminado 105

16 El monolito terminado 106
16.1 El monolito completo 106
16.2 Pruebas contra un PostgreSQL real con Testcontainers 107
16.3 El acoplamiento entre Demand y Application 110
16.4 Balance del monolito 110
16.5 Errores típicos y depuración 111
16.6 Ejercicios 111

Fase 5 Evolución a dos microservicios 112

17 Decidir la extracción 113
17.1 Por qué extraer el catálogo de aplicaciones 113
17.2 Qué cambia cuando la llamada cruza la red 113
17.3 La estrategia: pasos pequeños y reversibles 114
17.4 Lo que se decidió no hacer 115
17.5 Riesgos aceptados 116
17.6 Ejercicios 116

18 Una frontera dentro del monolito: puertos y adaptadores 117
18.1 Puertos y adaptadores en pocas palabras 117
18.2 El puerto `ApplicationCatalog` 118
18.3 De la asociación a un identificador 118
18.4 El catálogo con JPA dentro del monolito 119
18.5 Una consulta por página 121
18.6 Aislar la ordenación por nombre 122
18.7 Cómo se preparó y verificó este refactor 123
18.8 Errores típicos y depuración 124
18.9 Ejercicios 124

19 Nace `application-catalog-service` 126
19.1 Un proyecto nuevo con su propia base de datos 126
19.2 El *endpoint* de consulta por lotes 127
19.3 Trasladar los datos existentes 129
19.4 Las pruebas del servicio nuevo 130
19.5 Errores típicos y depuración 131
19.6 Ejercicios 131

20 El adaptador HTTP con `RestClient` 132
20.1 `RestClient` y la configuración del adaptador 132
20.2 Las dos operaciones del puerto por HTTP 134
20.3 Traducir los fallos remotos 135
20.4 Una llamada remota dentro de una transacción 137
20.5 Probar el adaptador contra un servidor real 138
20.6 Errores típicos y depuración 139
20.7 Ejercicios 140

21 Separar las bases de datos 141
21.1 La migración V4 elimina la clave foránea 141
21.2 El estado final de la arquitectura 143
21.3 Retirar la ordenación por nombre de aplicación 143
21.4 Probar Demand sin clave foránea 144
21.5 Errores típicos y depuración 146
21.6 Ejercicios 147

Fase 6 Validación, estado final y pendientes 148

22 Validar el sistema completo 149
22.1 Arrancar los dos servicios y el frontend 149
22.2 Ejecutar las suites de pruebas 150
22.3 Monolito frente a dos servicios 152
22.4 Ejercicios 153

23 Errores y lecciones aprendidas 154
23.1 Recoger los errores 154
23.2 Lo que ocurrió 154
23.3 Lo detectado en la revisión del código 155
23.4 Trampas evitadas 157
23.5 Las lecciones, en una página 157
23.6 Ejercicios 158

24 Estado final y próximos pasos 159
24.1 Qué está hecho 159
24.2 Qué no está hecho 160
24.3 Próximos pasos, por prioridad 160
24.4 Lo que vino después 162
24.5 Ejercicios 162

Fase 7 Cierre de la separación y despliegue con Docker 164

25 Cerrar la separación 165
25.1 Por qué una fase más 165
25.2 Una red de seguridad: Git antes de tocar nada 165
25.3 La llamada remota, fuera de la transacción 166
25.4 Una traducción de errores completa 169
25.5 Degradar en lugar de fallar 171
25.6 La configuración, en variables de entorno 172
25.7 Errores típicos y depuración 173
25.8 Ejercicios 174

26 Imágenes de contenedor 175
26.1 Comprobaciones de salud con Actuator 175
26.2 Qué son una imagen y un contenedor 176
26.3 El Dockerfile en dos etapas 177
26.4 Seguridad básica de la imagen 179
26.5 Probar una imagen sola 179
26.6 Errores típicos y depuración 180

26.7 Ejercicios ... 180

27 La plataforma con Docker Compose ... 182
27.1 Un repositorio para la infraestructura ... 182
27.2 El `compose.yaml`, servicio a servicio ... 183
27.3 Reutilizar los datos: el volumen externo ... 184
27.4 La inicialización de las bases de datos ... 185
27.5 Comprobaciones de salud sin `curl` ... 186
27.6 El frontend detrás de nginx ... 187
27.7 Validar y retirar lo antiguo ... 188
27.8 Para terminar ... 189
27.9 Errores típicos y depuración ... 189
27.10 Ejercicios ... 190

Anexos Material de consulta

A Soluciones y respuestas ... 191

B Glosario ... 210

C Equivalencias Java EE ↔ Spring Boot ... 214

D Chuleta de comandos ... 218

E Mapa comentado de ficheros ... 222

ANTES DE EMPEZAR

Cómo usar esta guía

↑ Volver al índice

Esta guía cuenta, en el orden en que ocurrió, cómo se construyó el backend de **IT Service & Demand Manager**: primero un monolito Spring Boot con los módulos de aplicaciones y demandas, y después su evolución hacia dos servicios independientes. Cada paso se explica con el código real de los proyectos y con el razonamiento que lo justifica.

A quién va dirigida

Está escrita para un ingeniero con años de experiencia en Java EE sobre servidores de aplicaciones como WebSphere: alguien que conoce EJB, JPA gestionado por el contenedor, JNDI, JAX-RS y las transacciones JTA, pero que no ha trabajado con Spring Boot moderno. Por eso no se explica qué es una transacción o una clave foránea, pero sí cómo Spring decide cuándo empieza una transacción, qué hace Hibernate al terminarla o por qué una prueba arranca su propio PostgreSQL en Docker.

El tono es el de un arquitecto que acompaña a otro ingeniero: se cuentan las decisiones, las alternativas que se descartaron y también los errores.

Cómo está organizada

La guía tiene dos bloques que se complementan:

Bloque	Qué contiene	Para qué sirve
I. Mapa cronológico	Todos los pasos del desarrollo, una línea por paso, agrupados en siete fases	Ver el recorrido completo de un vistazo y saltar a cualquier punto
II. Desarrollo explicado	Un capítulo o más por fase, con el código, los conceptos y las decisiones	Entender cada paso a fondo

Las siete fases siguen el orden real del proyecto, y ese orden no se rompe en ningún capítulo:

Fase	Contenido	Capítulos
1. Concepción y preparación	Spring Boot 4, PostgreSQL en Docker, configuración y Flyway	2-4
2. El módulo Application	Entidad, repositorio, servicio, DTO, controlador, errores y pruebas	5-11
3. El módulo Demand	Relación con Application, ciclo de vida, consultas dinámicas y N+1	12-15
4. El monolito terminado	Revisión del conjunto y del acoplamiento entre módulos	16
5. Evolución a dos microservicios	Puertos y adaptadores, nuevo servicio, cliente HTTP y separación de bases de datos	17-21
6. Validación y pendientes	Puesta en marcha, comparación, lecciones aprendidas y próximos pasos	22-24

Fase	Contenido	Capítulos
7. Cierre y despliegue con Docker	Corrección de los defectos, Actuator, imágenes, Docker Compose y nginx	25-27

Mientras se explica el monolito (fases 1 a 4) no aparece ningún elemento de la arquitectura de microservicios: ni el puerto 8081, ni `RestClient`, ni las migraciones V4 y V5. Todo eso llega en la fase 5, igual que llegó en el proyecto.

Cinco anexos completan la guía: **A**, soluciones de los ejercicios y respuestas de autoevaluación; **B**, glosario; **C**, equivalencias entre Java EE y Spring Boot; **D**, chuleta de comandos; y **E**, mapa comentado de los ficheros de los tres proyectos.

Las fuentes y su grado de certeza

La guía se basa en tres fuentes, todas leídas sin modificarlas:

1. **El repositorio publicado en GitHub** (`it-service-demand-manager`): la primera versión del monolito, solo con el módulo de aplicaciones. Es la foto de la fase 2.
2. `C:\Workspace\it-service-manager`: el servicio de demandas en su estado actual (puerto 8080), con las migraciones V1 a V5 y documentos de trabajo del refactor.
3. `C:\Workspace\application-catalog-service`: el servicio del catálogo de aplicaciones (puerto 8081).

Los proyectos locales no conservan historial de Git, así que algunos estados intermedios del monolito ya no existen en ningún fichero. Cuando la guía necesita mostrarlos, el código se **reconstruye** a partir de los documentos del propio proyecto, de las pruebas que los verificaban y del código final, y se marca siempre como tal.

Las marcas de estado

Cada fragmento de código y cada afirmación relevante llevan una marca que indica su situación:

Marca	Significado
IMPLEMENTADO	Código que existe tal cual en alguno de los proyectos
AÑADIDO DESPUÉS	Existe, pero se incorporó más tarde que el paso que se está explicando
REFACTORIZADO	Código que cambió de forma respecto a una versión anterior que también se explica
ESTADO FINAL	Estado del código al cerrar la fase en la que se explica: la fase 5 en los capítulos 17 a 21 y la fase 7 en los capítulos 25 a 27. Cuando la fase 7 cambió algo marcado así, el capítulo original lo indica
RECONSTRUIDO	Estado intermedio que ya no existe; se reconstruye y no debe tomarse como copia literal
PENDIENTE	Algo que no está hecho y se propone como trabajo futuro

Cómo está construido cada capítulo

1. **Qué se hizo**, con el paso del mapa al que corresponde.
2. **Cómo funciona por dentro**: lo justo para que Spring, Hibernate o PostgreSQL dejen de ser una caja negra.
3. **El código del proyecto**, con el fichero de origen y su marca de estado sobre cada bloque.
4. **Decisión y alternativas**: por qué se hizo así y qué se descartó.
5. **Errores típicos y depuración**.
6. **Ejercicios** y **autoevaluación**, con las soluciones en el anexo A.

Los recuadros

Si vienes de Java EE...

Relaciona el concepto con su equivalente en Java EE o WebSphere y señala dónde deja de funcionar la analogía.

En el proyecto

Señala exactamente dónde y cómo aparece el concepto en los proyectos.

Decisión de diseño

Recoge una decisión tomada, el motivo y las alternativas valoradas.

Cuidado

Avisa de un error habitual o de una trampa de la herramienta.

Lección aprendida

Algo que salió mal o que costó más de lo previsto, y lo que enseña.

Código reconstruido

Explica de dónde sale un fragmento reconstruido y qué grado de fidelidad tiene.

Pendiente

Trabajo que queda por hacer.

Cómo moverte por el PDF

- **Índice enlazado.** Cada fase, capítulo y sección del índice es un enlace a su página, y junto a cada título hay un enlace ↑ **Índice** que devuelve a su entrada.
- **Mapa ↔ desarrollo.** Cada paso del mapa cronológico enlaza con la sección que lo desarrolla, y esa sección tiene junto a su título un enlace **Paso n.m** ↖ que vuelve a su línea del mapa.
- **Índice de capítulo.** Cada capítulo empieza con la lista enlazada de sus secciones.
- **Ejercicios y soluciones.** Cada ejercicio enlaza con su solución y cada solución con su ejercicio; lo mismo ocurre con la autoevaluación.
- **Marcadores.** El panel de marcadores del lector muestra la estructura completa.

Cómo sacarle el máximo partido

Lee con los dos proyectos abiertos en el IDE y con Docker arrancado. Los ejercicios proponen cambios sobre una **copia** o una rama de trabajo: nunca sobre la base de datos con datos reales ni sobre el volumen persistente de PostgreSQL.

> **Cuidado: Seguridad**
>
> Ningún fragmento de la guía contiene credenciales reales. Donde la configuración usa contraseñas, aparecen como variables de entorno (`${DB_PASSWORD}`). No copies contraseñas en ficheros versionados ni en los comandos de los ejercicios.

> **Nota: Versiones**
>
> La guía describe el proyecto en octubre de 2026: Java 21, Spring Boot 4.1.1 (Spring Framework 7, Spring Data JPA 4, Hibernate 7), PostgreSQL 18 en Docker, Flyway, JUnit 5, Mockito y Testcontainers 2. Los conceptos son estables; los nombres de algunos módulos han cambiado entre Spring Boot 3 y 4, y se señalan donde importa.

BLOQUE I

Mapa cronológico

1 Mapa cronológico del desarrollo pág. 13

↑ Volver al índice

CAPÍTULO 1

Mapa cronológico del desarrollo

↑ Volver al índice

EN ESTE CAPÍTULO

1.1 Las fases de un vistazo
1.2 Cómo leer el mapa
1.3 El mapa paso a paso
1.4 Rutas de lectura

Este capítulo es el plano de toda la guía. Reúne los pasos reales del desarrollo del backend, en su orden y agrupados en siete fases, con una línea por paso. Cada número enlaza con la sección que lo explica, y cada sección tiene un enlace de vuelta a su línea.

1.1 Las fases de un vistazo

↑ Índice

El backend no se diseñó como microservicios desde el principio. Empezó como un **monolito modular**: una sola aplicación Spring Boot, una sola base de datos PostgreSQL y dos módulos de negocio, `application` y `demand`, que vivían en paquetes separados. Solo cuando el monolito estaba terminado y probado se decidió separar el catálogo de aplicaciones en un servicio propio.



Figura 1.1. Las seis fases del desarrollo. La línea divisoria entre la fase 4 y la 5 separa el monolito de la arquitectura de dos servicios.

La figura muestra las seis fases que llevaron del monolito a los dos servicios. Después llegó una **séptima fase**, que cerró los defectos encontrados en la sexta y empaquetó todo el sistema en contenedores: dos servicios en Docker, un PostgreSQL y el frontend detrás de nginx, levantados con un solo comando.

La figura resume la idea más importante de la guía: **la frontera entre módulos se diseñó antes de convertirse en una frontera de red**. Las fases 1 a 4 construyen un monolito correcto; la fase 5 lo transforma paso a paso, de forma que cada paso deja el sistema funcionando.

1.2 Cómo leer el mapa

↑ Índice

Cada fila del mapa es un paso real del desarrollo. La primera columna es su número (fase.paso) y enlaza con la sección que lo desarrolla; la última columna indica el número de esa sección. Algunos pasos llevan una marca de estado:

- RECONSTRUIDO : el código de ese paso ya no existe, porque la fase 5 lo sustituyó. La sección lo reconstruye y explica de dónde sale.
- AÑADIDO DESPUÉS : el paso se incorporó más tarde que los de su alrededor, aunque se explica junto a ellos porque pertenece al mismo tema.
- REFACTORIZADO : el paso cambia código que ya se había explicado antes.

El número de pasos de cada fase no es arbitrario: es el número de cambios con entidad propia que hubo. La fase 2 tiene más porque construye todas las capas por primera vez; la fase 4 tiene pocos porque no añade funcionalidad, solo revisa.

1.3 El mapa paso a paso

↑ Índice

Fase 1 Concepción y preparación

1.1	Elegir Java 21 y Spring Boot 4.1.1 y generar el proyecto con sus *starters*	§ 2.2 →
1.2	Organizar el código por módulo de negocio, con un paquete por capa dentro de cada módulo	§ 2.4 →
1.3	Levantar PostgreSQL 18 en Docker con volumen persistente y comprobación de salud	§ 3.1 →
1.4	Configurar la conexión con variables de entorno, `ddl-auto=validate` y `open-in-view=false`	§ 3.2 →
1.5	Adoptar Flyway: la migración V1 crea la tabla `applications`	§ 4.3 →
1.6	La migración V2 carga las aplicaciones iniciales	§ 4.4 →

Fase 2 El módulo Application

2.1	Mapear la entidad `Application`, sin *setters* y con un método `update`	§ 5.2 →
2.2	Crear `ApplicationRepository` con consultas derivadas paginadas	§ 6.1 →
2.3	Implementar la consulta, el detalle con 404 y el alta como `ACTIVE` en `ApplicationService`	§ 7.2 →
2.4	Implementar la modificación por *dirty checking*, sin llamar a `save`	§ 7.3 →
2.5	Definir los DTO como *records* y validarlos con Bean Validation	§ 8.1 →
2.6	Exponer `/api/applications` con paginación por defecto y `201 Created`	§ 9.1 →
2.7	Devolver la página en un DTO propio, `ApplicationPageResponse`	§ 9.3 →
2.8	Unificar los errores con `GlobalExceptionHandler` y `ApiErrorResponse`	§ 10.1 →
2.9	AÑADIDO DESPUÉS Rechazar con 400 los filtros y campos de ordenación desconocidos	§ 10.3 →
2.10	AÑADIDO DESPUÉS Añadir manejadores para tipos incorrectos y cuerpos JSON mal formados	§ 10.4 →
2.11	Probar el servicio con Mockito y el controlador con MockMvc	§ 11.2 →

Fase 3 El módulo Demand

3.1	La migración V3 crea `demands` con clave foránea, índices y fechas con zona horaria	§ 12.2 →
3.2	RECONSTRUIDO Mapear `Demand` con una asociación `@ManyToOne` hacia `Application`	§ 12.3 →
3.3	Encapsular el ciclo de vida de la demanda en `changeStatus`	§ 12.5 →
3.4	Filtrar el listado con `Specification` y `JpaSpecificationExecutor`	§ 13.3 →
3.5	Validar estado, prioridad y campos de ordenación del listado	§ 13.4 →
3.6	RECONSTRUIDO Evitar el problema N+1 del listado con `@EntityGraph`	§ 14.3 →
3.7	Crear y modificar demandas; traducir una transición inválida a 409	§ 15.1 →
3.8	Probar el dominio, el servicio y el controlador de demandas	§ 15.4 →

Fase 4 El monolito terminado

4.1	Revisar el monolito completo: capas, *endpoints* y contrato con el frontend	§ 16.1 →
4.2	Ejecutar las pruebas contra un PostgreSQL real con Testcontainers	§ 16.2 →
4.3	Identificar el acoplamiento entre Demand y Application	§ 16.3 →

Fase 5 Evolución a dos microservicios

5.1	Decidir extraer Application y planificar la extracción en pasos reversibles	§ 17.1 →
5.2	Definir el puerto `ApplicationCatalog` y el *record* `ApplicationSummary`	§ 18.2 →
5.3	REFACTORIZADO Sustituir la asociación por un identificador escalar `applicationId`	§ 18.3 →
5.4	RECONSTRUIDO Implementar el catálogo con JPA dentro del monolito	§ 18.4 →
5.5	Consultar el catálogo una sola vez por página en el listado	§ 18.5 →
5.6	RECONSTRUIDO Aislar la ordenación por nombre de aplicación en un adaptador de consultas	§ 18.6 →
5.7	Crear `application-catalog-service` con su propia base de datos	§ 19.1 →
5.8	Añadir `GET /api/applications/summaries` para consultas por lote	§ 19.2 →
5.9	Trasladar los datos de aplicaciones a la nueva base de datos	§ 19.3 →
5.10	Implementar `RestApplicationCatalog` con `RestClient` y tiempos máximos	§ 20.1 →
5.11	Traducir los fallos remotos: 404 a «no existe», error 5xx o de red a 503	§ 20.3 →
5.12	Probar el adaptador contra un servidor HTTP real del JDK	§ 20.5 →
5.13	La migración V4 elimina la clave foránea y la V5 la tabla heredada	§ 21.1 →
5.14	Retirar la ordenación por nombre de aplicación	§ 21.3 →
5.15	Probar Demand sin clave foránea y con el catálogo simulado	§ 21.4 →

Fase 6 Validación, estado final y pendientes

6.1	Arrancar los dos servicios y comprobarlos junto con el frontend	§ 22.1 →
6.2	Ejecutar las suites de pruebas de ambos servicios	§ 22.2 →
6.3	Comparar el monolito con la arquitectura de dos servicios	§ 22.3 →

6.4	Recoger los errores cometidos y las lecciones aprendidas	§ 23.1 →
6.5	Documentar el estado final y los próximos pasos	§ 24.1 →

Fase 7 **Cierre de la separación y despliegue con Docker**

7.1	Poner los dos servicios bajo control de versiones antes de tocar nada	§ 25.2 →
7.2	Sacar las llamadas al catálogo de las transacciones con `TransactionOperations`	§ 25.3 →
7.3	Traducir cualquier respuesta inesperada del catálogo a 502 y añadir un manejador de último recurso	§ 25.4 →
7.4	Degradar el listado y el detalle cuando falta una aplicación, en lugar de fallar	§ 25.5 →
7.5	Leer toda la configuración de variables de entorno y retirar el perfil `docker`	§ 25.6 →
7.6	Exponer el estado de salud con Spring Boot Actuator	§ 26.1 →
7.7	Construir cada servicio con un Dockerfile de dos etapas y el JAR por capas	§ 26.3 →
7.8	Ejecutar con un usuario sin privilegios y excluir secretos con `.dockerignore`	§ 26.4 →
7.9	Crear el repositorio `it-service-platform` con el Compose del sistema completo	§ 27.1 →
7.10	Inicializar las bases de datos y sus usuarios con un *script* versionado	§ 27.4 →
7.11	Comprobar la salud de cada contenedor sin añadir herramientas a la imagen	§ 27.5 →
7.12	Servir el frontend con nginx como proxy inverso hacia los dos servicios	§ 27.6 →
7.13	Validar el sistema en contenedores y retirar el Compose antiguo	§ 27.7 →

1.4 Rutas de lectura

↑ Índice

La lectura recomendada es secuencial: cada fase se apoya en la anterior. Si buscas algo concreto, estas rutas te llevan directamente:

Si quieres...	Lee
Entender JPA e Hibernate en Spring	Capítulos 5, 6, 7 y 14
Seguir una petición HTTP de principio a fin	Capítulo 9 (figura del flujo completo)
Entender cómo se trocea un monolito sin romperlo	Capítulos 17 y 18
Ver cómo se llama a otro servicio de forma robusta	Capítulo 20
Saber qué queda por hacer	Capítulo 24
Meter un sistema Spring en contenedores	Capítulos 26 y 27

FASE 1

Concepción y preparación

2	Del servidor de aplicaciones a Spring Boot 4	pág. 18
3	PostgreSQL en Docker y la configuración de la aplicación	pág. 24
4	Migraciones de esquema con Flyway	pág. 30

↑ Volver al índice

CAPÍTULO 2

Del servidor de aplicaciones a Spring Boot 4

↑ Volver al índice

EN ESTE CAPÍTULO

2.1 El punto de partida
2.2 Generar el proyecto: Java 21, Spring Boot 4.1.1 y sus *starters*
2.3 Qué hace Spring Boot al arrancar
2.4 Organizar el código por módulo de negocio
2.5 Errores típicos y depuración
2.6 Ejercicios

El primer paso no fue escribir código de negocio, sino elegir la plataforma y darle forma al proyecto. Este capítulo explica qué se generó, qué hace Spring Boot cuando arranca y por qué el código se organizó por módulos de negocio desde el primer día.

2.1 El punto de partida

↑ Índice

El objetivo era construir una API REST para gestionar el catálogo de aplicaciones de una organización y, después, las demandas de trabajo asociadas a cada aplicación. La API la consumiría un frontend independiente en React, y los datos vivirían en PostgreSQL.

En un proyecto Java EE clásico, el primer día habría consistido en preparar un servidor: instalar WebSphere, definir un origen de datos JNDI, configurar el gestor de transacciones y empaquetar un EAR con un módulo web y un módulo EJB. En Spring Boot ese trabajo desaparece casi por completo: **la aplicación lleva el servidor dentro**.

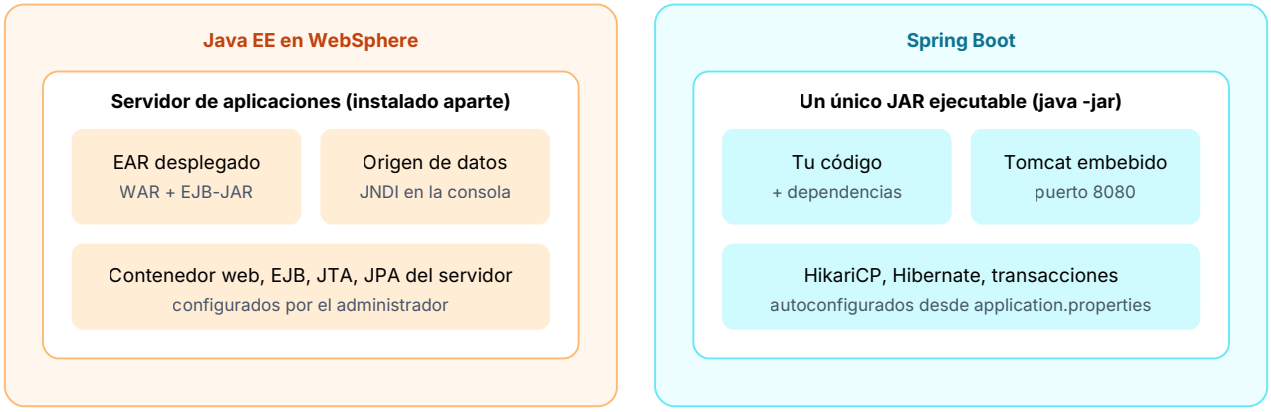


Figura 2.1. En Java EE la aplicación se despliega dentro de un servidor ya configurado; en Spring Boot la aplicación contiene su servidor y su configuración.

2.2 Generar el proyecto: Java 21, Spring Boot 4.1.1 y sus *starters*

↑ Índice

Paso 1.1 ↖

El proyecto se generó con Spring Initializr (la herramienta web, o el asistente equivalente del IDE), eligiendo Maven, Java 21 y Spring Boot 4.1.1. El resultado es un `pom.xml` que hereda de `spring-boot-starter-parent` y declara una lista corta de dependencias.

it-service-demand-manager/backend/pom.xml (extracto) IMPLEMENTADO

```xml
<parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>4.1.1</version>
    <relativePath/>
</parent>

<properties>
    <java.version>21</java.version>
</properties>

<dependencies>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-webmvc</artifactId>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-data-jpa</artifactId>
    </dependency>
    <dependency>
        <groupId>org.postgresql</groupId>
        <artifactId>postgresql</artifactId>
        <scope>runtime</scope>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-flyway</artifactId>
    </dependency>
    <dependency>
        <groupId>org.flywaydb</groupId>
        <artifactId>flyway-database-postgresql</artifactId>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-validation</artifactId>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-webmvc-test</artifactId>
        <scope>test</scope>
    </dependency>
</dependencies>
```

2.2.1 Qué es un *starter*

Un *starter* es una dependencia que no contiene código propio: agrupa las librerías necesarias para una capacidad y activa su **autoconfiguración**. Declarar `spring-boot-starter-data-jpa` trae Hibernate, Spring Data JPA, el *pool* de conexiones HikariCP y la integración de transacciones, todo en versiones probadas entre sí.

Las versiones no aparecen en el `pom.xml` porque las fija el *parent*. `spring-boot-starter-parent` importa una lista de materiales (BOM) con la versión de cada librería del ecosistema. Esto evita el problema clásico de combinar, por ejemplo, una versión de Hibernate con una de Spring Data que no la soporta.

Starter	Qué aporta	Dónde se estudia
`spring-boot-starter-webmvc`	Spring MVC, Tomcat embebido y Jackson para JSON	Capítulo 9
`spring-boot-starter-data-jpa`	Hibernate, Spring Data JPA, HikariCP y transacciones	Capítulos 5-7
`postgresql` (*runtime*)	El driver JDBC; solo se necesita al ejecutar, no al compilar	Capítulo 3
`spring-boot-starter-flyway` y `flyway-database-postgresql`	Migraciones de esquema al arrancar, con soporte específico de PostgreSQL	Capítulo 4
`spring-boot-starter-validation`	Bean Validation (Hibernate Validator)	Capítulo 8
`spring-boot-starter-webmvc-test`	JUnit 5, Mockito, AssertJ y el soporte de `@WebMvcTest` y MockMvc	Capítulo 11

Cuidado: Spring Boot 4 es modular

En Spring Boot 4 la autoconfiguración se repartió en módulos más pequeños, y algunos *starters* cambiaron de nombre o pasaron a ser obligatorios. El starter web se llama ahora `spring-boot-starter-webmvc`, y Flyway necesita `spring-boot-starter-flyway`: si solo se añade `flyway-core`, como se hacía en Spring Boot 3, la librería está en el *classpath* pero **nadie la ejecuta al arrancar**. Los ejemplos que encuentres en Internet escritos para Spring Boot 3 pueden no funcionar tal cual.

2.2.2 La clase principal

`ItServiceManagerApplication.java` IMPLEMENTADO

```
@SpringBootApplication
public class ItServiceManagerApplication {

    public static void main(String[] args) {
        SpringApplication.run(ItServiceManagerApplication.class, args);
    }
}
```

`@SpringBootApplication` combina tres anotaciones:

- `@SpringBootConfiguration`: la clase es una fuente de configuración, como una clase `@Configuration`.
- `@EnableAutoConfiguration`: activa la autoconfiguración de lo que haya en el *classpath*.
- `@ComponentScan`: busca componentes (`@Service`, `@RestController`, `@Repository`...) **en el paquete de esta clase y en todos sus subpaquetes**. Por eso la clase principal está en `com.albertoderafael.itmanager`, la raíz de todos los demás paquetes.

Si vienes de Java EE...

`@ComponentScan` hace el papel del descubrimiento de EJB y recursos JAX-RS que el servidor de aplicaciones realiza al desplegar un módulo. La diferencia es que aquí lo hace la propia aplicación al ejecutar `main`, sin descriptores `ejb-jar.xml` ni `web.xml`.

2.3 Qué hace Spring Boot al arrancar

↑ Índice

Entender el arranque evita muchas sorpresas. Cuando se ejecuta `SpringApplication.run`, ocurre lo siguiente, en este orden:

1. **Se prepara el entorno.** Spring lee `application.properties`, las variables de entorno y los argumentos de línea de comandos, y los combina en un único `Environment`. Si una propiedad aparece en varios sitios, gana la de mayor prioridad: una variable de entorno sobrescribe al fichero.
2. **Se crea el contexto de aplicación**, el contenedor de *beans*.
3. **Se escanean los componentes** del paquete raíz y se registran como *beans*.
4. **Se evalúan las autoconfiguraciones.** Cada una lleva condiciones como «si existe la clase `DataSource` en el *classpath* y el usuario no ha definido un *bean* `DataSource` propio, crea uno con HikariCP». Así, lo que defines tú siempre tiene preferencia sobre lo que Spring crearía.
5. **Se inicializan los *beans*** en orden de dependencias. En este proyecto el orden importa: Flyway migra la base de datos *antes* de que Hibernate valide el esquema (capítulo 4).
6. **Se arranca Tomcat** en el puerto 8080 y la aplicación queda lista para recibir peticiones.

> **En el proyecto**
>
> Si quieres ver qué autoconfiguraciones se aplicaron y cuáles no, arranca con la propiedad `debug=true`. Spring imprime el *condition evaluation report*: una lista de cada autoconfiguración con el motivo por el que se activó o se descartó. Es la primera herramienta cuando algo «no se configura solo».

2.3.1 Inyección por constructor

Todos los componentes del proyecto reciben sus dependencias por constructor, sin `@Autowired`:

`ApplicationService.java (extracto)` **IMPLEMENTADO**

```java
@Service
public class ApplicationService {

    private final ApplicationRepository applicationRepository;

    public ApplicationService(ApplicationRepository applicationRepository) {
        this.applicationRepository = applicationRepository;
    }
}
```

Cuando una clase tiene un único constructor, Spring lo usa para inyectar sin necesidad de anotación. La dependencia queda `final`, el objeto no puede existir sin ella y la clase se puede instanciar en una prueba unitaria con `new`, pasándole un *mock*. El capítulo 11 aprovecha exactamente eso.

> **Decisión de diseño: Inyección por constructor en lugar de por atributo**
>
> **Motivo:** dependencias obligatorias e inmutables, clases fáciles de probar sin Spring y ciclos de dependencias detectados al arrancar. **Alternativa descartada:** `@Autowired` sobre atributos privados, habitual en código antiguo: oculta las dependencias y obliga a usar reflexión o el contenedor en las pruebas.

2.4 Organizar el código por módulo de negocio

Paso 1.2 ↖ ↑ Índice

El código se organizó primero por **módulo de negocio** y, dentro de cada módulo, por capa técnica:

Estructura del monolito en la fase 2 IMPLEMENTADO

```
com.albertoderafael.itmanager
├── ItServiceManagerApplication.java
├── application/          ← módulo de negocio
│   ├── controller/       ApplicationController
│   ├── dto/              ApplicationCreateRequest, …Response, …PageResponse
│   ├── entity/           Application
│   ├── repository/       ApplicationRepository
│   └── service/          ApplicationService
└── exception/            ← transversal
    ├── ApiErrorResponse
    └── GlobalExceptionHandler
```

La alternativa clásica es la contraria: un paquete `controllers` con todos los controladores, otro `services` con todos los servicios, y así sucesivamente. Funciona en proyectos pequeños, pero a medida que crecen dos problemas se acentúan: para entender una funcionalidad hay que saltar entre cinco paquetes, y nada impide que cualquier servicio use cualquier repositorio.

Con la organización por módulo, todo lo que pertenece a las aplicaciones vive junto. Y, sobre todo, **la frontera entre módulos se ve en los nombres de los paquetes**. Cuando más adelante aparezca el módulo `demand`, cualquier `import` de `itmanager.application` dentro de `itmanager.demand` será una dependencia visible entre módulos. Esa visibilidad es la que permitió, en la fase 5, encontrar exactamente qué había que cortar.

Decisión de diseño: Paquetes por módulo de negocio

Motivo: cohesión (lo que cambia junto vive junto) y fronteras visibles entre módulos, que facilitan extraer uno más tarde. **Alternativa descartada:** paquetes por capa técnica, que mezclan todos los dominios en los mismos paquetes. **Coste aceptado:** el paquete `exception` es transversal y lo comparten todos los módulos.

Si vienes de Java EE...

Equivale a separar cada dominio en su propio módulo EJB dentro del EAR, pero sin la ceremonia: aquí son simples paquetes Java. Java no impide que un paquete importe otro, así que la frontera es una convención del equipo. Herramientas como ArchUnit o Spring Modulith permiten convertirla en una regla comprobada por las pruebas; el proyecto no las usa todavía.

2.5 Errores típicos y depuración

↑ Índice

Síntoma	Causa probable	Cómo resolverlo
Un `@Service` o `@RestController` no se registra (404 en todos los *endpoints*, o `NoSuchBeanDefinitionException`)	La clase está fuera del paquete de la clase principal	Moverla bajo `com.albertoderafael.itmanager`

Síntoma	Causa probable	Cómo resolverlo
`Web server failed to start. Port 8080 was already in use`	Otra instancia del backend sigue arrancada	Pararla, o usar `server.port` para elegir otro puerto
Flyway no se ejecuta aunque está en el `pom.xml`	Solo se añadió `flyway-core`, sin `spring-boot-starter-flyway`	Usar el *starter* de Spring Boot 4
`UnsupportedClassVersionError` al ejecutar	Se compila con Java 21 pero se ejecuta con una JVM anterior	Comprobar `java -version` y la variable `JAVA_HOME`
Una dependencia aparece con versión distinta a la esperada	Se fijó una versión a mano que contradice el BOM	Quitar la versión y dejar que la gestione el *parent*

2.6 Ejercicios

↑ Índice

Ejercicio 2.1 · Leer el informe de autoconfiguración Ver solución →

Arranca el backend con `--debug` (o con `debug=true` en la configuración) y localiza en el informe la auto-configuración de `DataSource` y la de Flyway. Anota qué condición hizo que cada una se activara.

Ejercicio 2.2 · Romper el escaneo de componentes Ver solución →

En una rama de trabajo, mueve temporalmente `ApplicationController` a un paquete `com.albertodera-fael.otro`. Arranca y llama a `GET /api/applications`. Explica el resultado y devuelve la clase a su sitio.

Autoevaluación del capítulo 2 Ver respuestas →

1. ¿Por qué no hace falta indicar la versión de Hibernate en el `pom.xml`?
2. ¿Qué tres anotaciones combina `@SpringBootApplication` y qué hace cada una?
3. ¿Qué ocurre en Spring Boot 4 si se declara `flyway-core` pero no `spring-boot-starter-flyway`?
4. Da dos ventajas de la inyección por constructor frente a `@Autowired` en atributos.
5. ¿Qué ventaja concreta tuvo organizar el código por módulo de negocio cuando llegó la fase 5?

CAPÍTULO 3

PostgreSQL en Docker y la configuración de la aplicación

↑ Volver al índice

EN ESTE CAPÍTULO

3.1 PostgreSQL 18 en un contenedor
3.2 La configuración de la conexión
3.3 `ddl-auto=validate`: Hibernate comprueba, Flyway decide
3.4 `open-in-view=false`: la sesión no llega al controlador
3.5 `show-sql=true`: ver lo que hace Hibernate
3.6 Errores típicos y depuración
3.7 Ejercicios

Antes de la primera entidad hacía falta una base de datos reproducible y una configuración que no guardara secretos en el código. Este capítulo explica el fichero de Docker Compose, cada propiedad de `application.properties` y las dos decisiones de JPA que más condicionan el resto del proyecto.

3.1 PostgreSQL 18 en un contenedor

Paso 1.3 ↖ ↑ Índice

En lugar de instalar PostgreSQL en Windows, la base de datos se ejecuta en un contenedor Docker descrito en `compose.yaml`:

`it-service-manager/compose.yaml` IMPLEMENTADO

```yaml
services:

  postgres:
    image: postgres:18.6
    environment:
      POSTGRES_DB: it_service_manager
      POSTGRES_USER: itmanager
      POSTGRES_PASSWORD: ${DB_PASSWORD}

    ports:
      - "5433:5432"

    volumes:
      - postgres_data:/var/lib/postgresql

    healthcheck:
      test:
        [
          "CMD-SHELL",
          "pg_isready -U $${POSTGRES_USER} -d $${POSTGRES_DB}"
        ]
      interval: 5s
      timeout: 5s
      retries: 10
      start_period: 10s
```

```
volumes:
  postgres_data:
```

Cada bloque responde a una necesidad concreta:

`image: postgres:18.6`. La versión está fijada hasta el parche. Con `postgres:latest`, un `docker compose pull` cualquiera podría traer una versión mayor nueva, y PostgreSQL no puede abrir directamente los ficheros de datos de otra versión mayor: haría falta una migración con `pg_upgrade` o un volcado y restauración.

`environment`. La imagen oficial lee estas variables **solo la primera vez**, cuando el directorio de datos está vacío: crea la base de datos `it_service_manager` y el usuario `itmanager` con la contraseña indicada. La contraseña no está escrita en el fichero: `${DB_PASSWORD}` la toma Docker Compose del entorno o de un fichero `.env` situado junto a `compose.yaml`, que no se versiona.

`ports: "5433:5432"`. Dentro del contenedor PostgreSQL escucha en su puerto estándar, el 5432. Fuera, en Windows, se publica en el **5433**. Así no choca con una instalación local de PostgreSQL que ya estuviera usando el 5432.

`volumes`. Los datos se guardan en un **volumen con nombre**, `postgres_data`, que Docker gestiona fuera del contenedor. Se puede parar, borrar y recrear el contenedor sin perder los datos. En PostgreSQL 18 la imagen oficial cambió la ubicación de los datos a un subdirectorio por versión mayor, y recomienda montar el volumen en `/var/lib/postgresql`, que es exactamente lo que hace el proyecto.

`healthcheck`. Docker ejecuta `pg_isready` cada cinco segundos y marca el contenedor como *healthy* cuando PostgreSQL acepta conexiones. Un contenedor «arrancado» no significa una base de datos lista: PostgreSQL tarda unos segundos en inicializarse, sobre todo la primera vez. El doble `$$` es un escape de Compose: deja pasar `${POSTGRES_USER}` literalmente para que lo resuelva el *shell* del contenedor, y no Compose.

Cuidado: Los datos viven en el volumen

`docker compose down` para y elimina el contenedor, pero conserva el volumen y por tanto los datos. **No añadas `-v`** a ese comando ni ejecutes `docker volume rm` sobre `postgres_data`: eso borra la base de datos de forma irreversible. Para los experimentos de esta guía usa siempre una base de datos aparte o las bases temporales de Testcontainers (capítulo 16).

Si vienes de Java EE...

En WebSphere la base de datos era un recurso externo que administraba otro equipo, y la aplicación la conocía por un nombre JNDI. Aquí la base de datos de desarrollo forma parte del proyecto: cualquiera que clone el repositorio la levanta con `docker compose up -d`, con la misma versión y la misma configuración. Lo que se pierde es la separación de responsabilidades: en producción seguirá siendo un recurso gestionado aparte.

Nota: Dónde vive hoy este fichero

Este `compose.yaml` fue la infraestructura del proyecto hasta la fase 7. Desde entonces, PostgreSQL y los servicios se levantan con el Compose del repositorio `it-service-platform`, que reutiliza este mismo volumen como volumen externo (capítulo 27), y este fichero se retiró del servicio de demandas.

3.2 La configuración de la conexión

Paso 1.4 ↖ ↑ Índice

Spring Boot crea el `DataSource` a partir de tres propiedades. El proyecto pasó por dos variantes de configuración, y las dos merecen verse.

La copia publicada en GitHub no contiene ningún dato de conexión: lo recibe todo del entorno.

it-service-demand-manager/backend/src/main/resources/application.properties IMPLEMENTADO

```
spring.application.name=it-service-manager

# Configure PostgreSQL through environment variables. No credentials in Git.
spring.datasource.url=${DB_URL}
spring.datasource.username=${DB_USERNAME}
spring.datasource.password=${DB_PASSWORD}

spring.jpa.hibernate.ddl-auto=validate
spring.jpa.open-in-view=false
spring.jpa.show-sql=true
```

El proyecto local fija la URL y el usuario, y solo deja la contraseña fuera:

it-service-manager/src/main/resources/application.properties (fase 1) RECONSTRUIDO

```
spring.application.name=it-service-manager

# PostgreSQL
spring.datasource.url=jdbc:postgresql://localhost:5432/it_service_manager
spring.datasource.username=itmanager
spring.datasource.password=${DB_PASSWORD}

# JPA / Hibernate
spring.jpa.hibernate.ddl-auto=validate
spring.jpa.open-in-view=false
spring.jpa.show-sql=true
```

Código reconstruido

El fichero actual del proyecto local tiene exactamente estas líneas y, además, la URL del catálogo de aplicaciones, que se añadió en la fase 5. Aquí se muestra sin esa línea para respetar la cronología.

3.2.1 Cómo se resuelve `${DB_PASSWORD}`

La sintaxis `${...}` es un **marcador de posición**. Al arrancar, Spring busca una propiedad llamada `DB_PASSWORD` en todas sus fuentes: argumentos de línea de comandos, propiedades del sistema de la JVM, variables de entorno del sistema operativo y ficheros de propiedades. Si no la encuentra, el arranque falla con `Could not resolve placeholder 'DB_PASSWORD'`, lo cual es preferible a arrancar con una contraseña vacía.

Cuidado: Spring Boot no lee ficheros `.env`

El fichero `.env` lo lee **Docker Compose**, no Spring Boot. Para que la aplicación reciba la contraseña hay que definir la variable de entorno en la sesión de PowerShell, en la configuración de ejecución del IDE o en el sistema. La guía de instalación del repositorio publicado lo resuelve pidiéndola con `Get-Credential`, para que no quede escrita ni en el historial de comandos.

3.2.2 El perfil `docker`

El proyecto local tiene un segundo fichero de configuración:

`it-service-manager/src/main/resources/application-docker.properties` IMPLEMENTADO

```
spring.datasource.url=jdbc:postgresql://localhost:5433/it_service_manager
```

Un fichero `application-<perfil>.properties` solo se carga cuando ese perfil está activo, y sus propiedades **sobrescriben** las del fichero general. Arrancando con `spring.profiles.active=docker` (por ejemplo, con la variable de entorno `SPRING_PROFILES_ACTIVE=docker`), la aplicación se conecta al 5433, el puerto publicado por el contenedor. Sin el perfil, intenta el 5432.

> **Lección aprendida: Dos puertos, un descuido fácil**
>
> La configuración por defecto apunta al 5432, pero el contenedor del proyecto se publica en el 5433. Quien arranca sin el perfil `docker` y no tiene otro PostgreSQL local obtiene un `Connection refused` que no parece tener relación con los perfiles. Conviene que la configuración por defecto coincida con la forma habitual de trabajar, o documentarlo junto al `compose.yaml`. Se recoge entre los pendientes del capítulo 24 y se resolvió en la fase 7: el valor por defecto pasó a ser el 5433 (capítulo 25).

3.2.3 El *pool* de conexiones

Spring Boot no abre una conexión por petición: crea un *pool* con **HikariCP** (por defecto, hasta diez conexiones) y las reutiliza. Cuando un método transaccional empieza, toma una conexión del *pool*; cuando termina, la devuelve. Este detalle tendrá importancia en la fase 5, cuando una transacción incluya una llamada HTTP a otro servicio y retenga su conexión mientras espera.

3.3 `ddl-auto=validate`: Hibernate comprueba, Flyway decide

↑ Índice

La propiedad `spring.jpa.hibernate.ddl-auto` indica a Hibernate qué hacer con el esquema al arrancar:

Valor	Qué hace Hibernate	Uso razonable
`none`	Nada	Producción, si otra herramienta gestiona el esquema
`validate`	Compara las entidades con las tablas y **falla al arrancar** si no coinciden	Desarrollo y producción con Flyway
`update`	Intenta añadir tablas y columnas que falten; nunca borra	Prototipos rápidos
`create`	Borra y crea el esquema al arrancar	Pruebas desechables
`create-drop`	Como `create`, y además borra al parar	Pruebas en memoria

El proyecto usa `validate` porque el esquema lo crea Flyway (capítulo 4) y Hibernate solo debe comprobarlo. Si alguien añade un atributo a una entidad y olvida la migración, la aplicación no arranca y el mensaje dice exactamente qué columna falta. Es un error temprano y barato.

Decisión de diseño: `validate` en lugar de `update`

Motivo: el esquema tiene una única fuente de verdad, las migraciones versionadas, y cualquier desajuste se detecta al arrancar. **Alternativa descartada:** `update`, cómodo al principio pero peligroso: no elimina columnas ni cambia tipos, no deja rastro de qué cambió ni cuándo, y produce esquemas distintos en cada máquina.

3.4 `open-in-view=false`: la sesión no llega al controlador

↑ Índice

Por defecto, Spring Boot activa un patrón llamado *Open Session in View* (OSIV): abre el contexto de persistencia de Hibernate al empezar cada petición HTTP y lo mantiene abierto hasta que la respuesta se ha escrito. Mientras está abierto, cualquier asociación perezosa se puede cargar en cualquier punto, incluso al serializar el JSON. Spring Boot muestra un aviso al arrancar cuando la propiedad no se configura explícitamente.

El proyecto lo desactiva. Con `open-in-view=false`, el contexto de persistencia vive **solo dentro de los métodos transaccionales** del servicio. Fuera de ellos, acceder a una asociación no cargada lanza `LazyInitializationException`.

Puede parecer una incomodidad, pero obliga a una disciplina sana: el servicio decide qué datos necesita la respuesta y los carga dentro de la transacción. Con OSIV activado, las consultas se disparan desde lugares inesperados (un *getter* llamado por Jackson), la conexión a la base de datos se retiene durante toda la petición y los problemas de rendimiento quedan ocultos. El capítulo 14 muestra cómo esta decisión hace visible el problema N+1 en el módulo de demandas.

Si vienes de Java EE...

En un EJB con transacciones gestionadas por el contenedor, el `EntityManager` transaccional se cerraba al terminar el método de negocio, y las entidades devueltas a la capa web quedaban *detached*. `open-in-view=false` reproduce ese comportamiento, que probablemente ya conoces. OSIV es lo excepcional.

3.5 `show-sql=true`: ver lo que hace Hibernate

↑ Índice

`spring.jpa.show-sql=true` imprime por la salida estándar cada sentencia SQL que Hibernate envía. Durante el aprendizaje es imprescindible: permite ver cuándo se ejecuta un `UPDATE`, cuántas consultas genera un listado o si aparece una consulta inesperada.

Tiene dos limitaciones: escribe en la consola sin pasar por el sistema de *logs* y no muestra los valores de los parámetros. Una alternativa más completa es activar los *loggers* de Hibernate:

Alternativa (no está en el proyecto) PENDIENTE

```
logging.level.org.hibernate.SQL=DEBUG
logging.level.org.hibernate.orm.jdbc.bind=TRACE
```

En producción no debe usarse ninguna de las dos: genera muchísimo volumen y puede escribir datos personales en los *logs*.

3.6 Errores típicos y depuración

↑ Índice

Síntoma	Causa probable	Cómo resolverlo
`Connection to localhost:5432 refused`	El contenedor se publica en el 5433 y no está activo el perfil `docker`	Activar el perfil o corregir la URL
`password authentication failed for user "itmanager"`	La variable `DB_PASSWORD` no coincide con la que se usó al crear el volumen	Recuerda que la imagen solo lee la contraseña la primera vez; usa la original
`Could not resolve placeholder 'DB_PASSWORD'`	La variable no está definida en el entorno de la JVM	Definirla en la configuración de ejecución del IDE o en la sesión
La aplicación arranca antes de que PostgreSQL esté listo y falla	Se lanzó justo después de `docker compose up`	Esperar al estado *healthy* (`docker compose ps`)
`Schema-validation: missing column [...]`	Una entidad tiene un atributo sin columna	Crear la migración correspondiente (capítulo 4)
Aviso `spring.jpa.open-in-view is enabled by default`	Falta la propiedad	Declararla explícitamente a `false`

3.7 Ejercicios

↑ Índice

Ejercicio 3.1 · Inspeccionar el contenedor Ver solución →

Con el contenedor arrancado, ejecuta `docker compose ps` y localiza la columna de estado de salud. Después conéctate con `docker compose exec postgres psql -U itmanager -d it_service_manager` y lista las tablas con `\dt` . Sal con `\q` .

Ejercicio 3.2 · Provocar un fallo de validación del esquema Ver solución →

En una rama de trabajo, añade a `Application` un atributo `private String owner;` sin crear ninguna migración. Arranca la aplicación, lee el mensaje de error completo y explica qué componente lo produce. Deshaz el cambio.

Autoevaluación del capítulo 3 Ver respuestas →

1. ¿Por qué el puerto publicado del contenedor es el 5433 y no el 5432?
2. ¿Qué ocurre con los datos al ejecutar `docker compose down` ? ¿Y qué opción no debes añadir nunca a ese comando?
3. ¿Quién lee el fichero `.env` : Docker Compose o Spring Boot?
4. ¿Qué diferencia hay entre `ddl-auto=validate` y `ddl-auto=update` , y por qué el proyecto elige la primera?
5. ¿Qué excepción aparece al acceder a una asociación perezosa fuera de la transacción con `open-in-view=false` , y por qué es una ventaja?

CAPÍTULO 4

Migraciones de esquema con Flyway

↑ Volver al índice

EN ESTE CAPÍTULO

4.1 Por qué migraciones versionadas
4.2 Cómo funciona Flyway por dentro
4.3 La migración V1 crea `applications`
4.4 La migración V2 carga las aplicaciones iniciales
4.5 Las reglas que evitan problemas
4.6 Errores típicos y depuración
4.7 Ejercicios

El esquema de la base de datos se trata como código: cada cambio es un fichero SQL numerado, versionado con el resto del proyecto y aplicado automáticamente al arrancar. Este capítulo explica cómo lo hace Flyway, las dos primeras migraciones y las reglas que evitan los problemas más serios.

4.1 Por qué migraciones versionadas

↑ Índice

Sin una herramienta de migraciones, el esquema de cada entorno evoluciona a mano: alguien ejecuta un `ALTER TABLE` en desarrollo, otro lo olvida en pruebas y nadie sabe con certeza qué versión tiene producción. Flyway resuelve tres problemas a la vez:

1. **Reproducibilidad.** Una base de datos vacía llega siempre al mismo esquema aplicando las mismas migraciones en el mismo orden.
2. **Trazabilidad.** Cada cambio tiene un fichero, un número y un autor en el control de versiones.
3. **Automatización.** La aplicación aplica las migraciones pendientes al arrancar; no hay pasos manuales que olvidar.

> **Si vienes de Java EE...**
>
> En muchos proyectos Java EE los cambios de esquema se entregaban como *scripts* al equipo de base de datos, que los aplicaba en cada entorno. Flyway formaliza ese mismo flujo y lo automatiza, y añade lo que solía faltar: un registro, dentro de la propia base de datos, de qué *scripts* se han aplicado ya.

4.2 Cómo funciona Flyway por dentro

↑ Índice

Flyway busca las migraciones en `src/main/resources/db/migration`. El nombre de cada fichero sigue un patrón estricto:

```
V2__insert_initial_applications.sql
| |  └─ descripción (los guiones bajos se leen como espacios)
| └──── dos guiones bajos como separador
└────── V de «versionada» + número de versión
```

Al arrancar, la autoconfiguración de Spring Boot ejecuta Flyway **antes** de crear el `EntityManagerFactory` de JPA. Flyway hace lo siguiente:

1. Si no existe, crea la tabla `flyway_schema_history`.
2. Lee de esa tabla qué versiones están aplicadas y con qué *checksum* (una huella calculada a partir del contenido de cada fichero).
3. Compara con los ficheros disponibles. Si el *checksum* de una migración ya aplicada no coincide con el de su fichero, **se detiene con un error de validación**.
4. Aplica en orden las migraciones pendientes, cada una en su propia transacción, y registra cada una en la tabla de historial.

Después, Hibernate valida el esquema resultante (`ddl-auto=validate`). El orden garantiza que Hibernate siempre valida un esquema ya migrado.

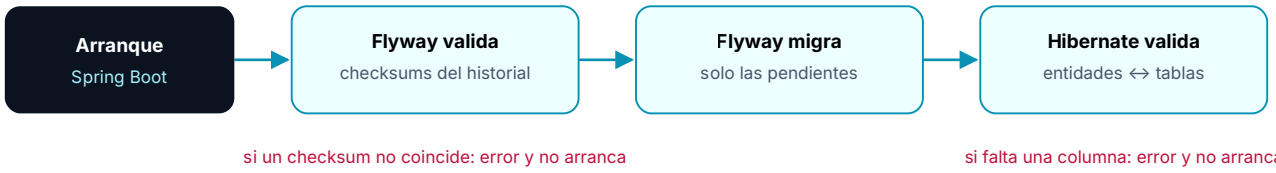


Figura 4.1. Orden de arranque: el esquema se valida dos veces, primero contra el historial de Flyway y después contra las entidades de JPA.

4.3 La migración V1 crea `applications`

Paso 1.5 ↖ ↑ Índice

`V1__create_applications.sql` IMPLEMENTADO

```
CREATE TABLE applications (
    id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    description VARCHAR(500),
    criticality VARCHAR(20) NOT NULL,
    status VARCHAR(20) NOT NULL
);
```

Tres detalles de esta migración se reflejan después en el código Java:

La clave primaria es una columna de identidad. `GENERATED BY DEFAULT AS IDENTITY` es la forma estándar de SQL para un autoincremento; sustituye al antiguo tipo `BIGSERIAL` de PostgreSQL. PostgreSQL asigna el valor al insertar la fila. En la entidad le corresponde `@GeneratedValue(strategy = GenerationType.IDENTITY)` (capítulo 5).

La variante `BY DEFAULT`, frente a `ALWAYS`, permite insertar una fila indicando el `id` explícitamente. Es útil para importar datos que ya tienen identificadores, pero tiene una trampa: la secuencia interna no avanza con esas inserciones manuales, y la siguiente inserción normal puede intentar reutilizar un `id` existente. Tras una importación hay que ajustar la secuencia, normalmente con `setval`.

Las longitudes coinciden con las validaciones. `VARCHAR(100)` para el nombre corresponde a `@Column(length = 100)` en la entidad y a `@Size(max = 100)` en el DTO de entrada (capítulo 8). Si los tres no

coinciden, el error aparece tarde y en el sitio equivocado: una cadena de 120 caracteres pasaría la validación y fallaría en el `INSERT` con un error de base de datos.

Estado y criticidad son texto. No hay tipo enumerado de PostgreSQL ni restricción `CHECK`: los valores válidos se controlan en Java. Es una decisión consciente, con un coste que se discute en el capítulo 8.

4.4 La migración V2 carga las aplicaciones iniciales

Paso 1.6 ↖ ↑ Índice

`it-service-manager/…/V2__insert_initial_applications.sql` IMPLEMENTADO

```sql
INSERT INTO applications (
    name,
    description,
    criticality,
    status
)
VALUES
(
    'SIGMA',
    'Portal de gestión de expedientes',
    'HIGH',
    'ACTIVE'
),
(
    'ORION',
    'Aplicación backend',
    'MEDIUM',
    'ACTIVE'
);
```

Una migración no tiene por qué cambiar la estructura: también puede cargar datos de referencia. Aquí se insertan las dos primeras aplicaciones del catálogo, de modo que una base de datos recién creada ya tiene algo que mostrar en el frontend. Los `id` no se indican; los asigna la columna de identidad.

4.4.1 La copia pública y su V2 distinta

Al preparar la versión publicada en GitHub se sustituyeron esos datos por registros ficticios:

`it-service-demand-manager/…/V2__insert_initial_applications.sql` IMPLEMENTADO

```sql
-- Fictional sample records for a new demonstration database.
INSERT INTO applications (name, description, criticality, status)
VALUES
    ('Portal de ejemplo', 'Portal ficticio de servicios internos', 'HIGH', 'ACTIVE'),
    ('API de ejemplo', 'Servicio backend ficticio', 'MEDIUM', 'ACTIVE');
```

El nombre del fichero es el mismo, pero el contenido no, así que su *checksum* tampoco. Si alguien arrancara la copia pública contra la base de datos original, Flyway encontraría la V2 registrada con un *checksum* y el fichero con otro, y se negaría a arrancar. La guía de instalación del repositorio lo advierte explícitamente y pide usar una base de datos nueva.

Lección aprendida: Una migración aplicada es inmutable

Este episodio ilustra la regla más importante de Flyway: **una migración que ya se aplicó en alguna base de datos no se modifica nunca**. Si hay que corregir algo, se crea una migración nueva. La copia pública es un caso legítimo, porque está pensada para bases de datos nuevas, pero muestra exactamente qué ocurre cuando un fichero aplicado cambia.

4.5 Las reglas que evitan problemas

↑ Índice

Regla	Por qué
No modificar migraciones aplicadas	Rompe la validación de *checksums* en todas las bases de datos que ya la tienen
Solo hacia delante	Flyway Community no deshace migraciones; para revertir un cambio se escribe otra migración que lo revierta
Una migración, un propósito	Facilita entender el historial y localizar el origen de un problema
Probar en una base de datos vacía	Garantiza que la secuencia completa V1…Vn funciona desde cero
No usar `repair` para «arreglar» un *checksum*	`repair` reescribe el historial para que coincida con los ficheros: oculta la discrepancia, no la resuelve

Cuidado: `clean` borra todo

Flyway tiene un comando `clean` que elimina **todos** los objetos del esquema. Spring Boot lo mantiene desactivado por defecto (`spring.flyway.clean-disabled=true`). No lo actives en una base de datos con datos que quieras conservar.

Decisión de diseño: SQL escrito a mano en lugar de generado

Motivo: el SQL de las migraciones es exactamente el que se ejecutará en todos los entornos; se revisa como cualquier otro código y aprovecha la sintaxis de PostgreSQL. **Alternativas descartadas:** dejar que Hibernate genere el esquema (`ddl-auto=update`), sin historial ni control; o Liquibase, válido y con más funciones (*changelogs* en XML o YAML, *rollbacks*), pero más complejo de lo que el proyecto necesitaba.

4.6 Errores típicos y depuración

↑ Índice

Síntoma	Causa probable	Cómo resolverlo
`Validate failed: Migration checksum mismatch for migration version 2`	Se modificó un fichero ya aplicado	Restaurar el contenido original y crear una migración nueva con el cambio
`Detected resolved migration not applied to database`	Hay una migración con número menor que la última aplicada	Renumerarla por encima de la última, o revisar el orden

Síntoma	Causa probable	Cómo resolverlo
Una migración nueva no se ejecuta	El nombre no sigue el patrón (un solo guion bajo, falta la `V`)	Corregir el nombre: `V3__descripcion.sql`
`relation "applications" does not exist` al arrancar	Flyway no se ejecutó (falta el *starter* en Spring Boot 4)	Ver el capítulo 2
`duplicate key value violates unique constraint "applications_pkey"`	Se insertaron filas con `id` explícito sin ajustar la secuencia	Ajustar la secuencia con `setval`

4.7 Ejercicios

↑ Índice

Ejercicio 4.1 · Leer el historial Ver solución →

Conéctate a la base de datos y ejecuta `SELECT installed_rank, version, description, checksum, success FROM flyway_schema_history ORDER BY installed_rank;`. Identifica la fila de cada migración e interpreta cada columna.

Ejercicio 4.2 · Diseñar una migración correctiva Ver solución →

Supón que quieres impedir en la base de datos que `criticality` tome valores distintos de LOW, MEDIUM, HIGH y CRITICAL. Escribe (sin aplicarla en la base de datos real) la migración que lo haría, con su nombre de fichero correcto, y explica por qué no se puede añadir esa restricción editando la V1.

Autoevaluación del capítulo 4 Ver respuestas →

1. ¿Qué guarda la tabla `flyway_schema_history` y para qué usa Flyway el *checksum*?
2. ¿En qué orden se ejecutan Flyway y la validación de Hibernate al arrancar, y por qué importa?
3. ¿Qué diferencia hay entre `GENERATED BY DEFAULT AS IDENTITY` y `GENERATED ALWAYS AS IDENTITY`?
4. ¿Por qué la copia pública del repositorio no debe conectarse a la base de datos original?
5. ¿Por qué no es buena idea usar `flyway repair` para resolver un error de *checksum*?

FASE 2

El módulo Application

5	La entidad Application y el contexto de persistencia	pág. 36
6	Repositorios de Spring Data	pág. 42
7	El servicio y las transacciones	pág. 47
8	DTO y Bean Validation	pág. 53
9	El controlador REST y el contrato HTTP	pág. 59
10	El tratamiento de errores	pág. 65
11	Las pruebas del módulo Application	pág. 71

↑ Volver al índice

CAPÍTULO 5

La entidad Application y el contexto de persistencia

↑ Volver al índice

EN ESTE CAPÍTULO

5.1 Tres capas: JPA, Hibernate y Spring Data
5.2 Mapear la entidad `Application`
5.3 El contexto de persistencia
5.4 *Dirty checking* por dentro
5.5 El coste de `GenerationType.IDENTITY`
5.6 Errores típicos y depuración
5.7 Ejercicios

Con la tabla creada, el siguiente paso fue representarla en Java. Este capítulo explica la entidad `Application` línea a línea y, sobre todo, el mecanismo que está detrás de ella: el contexto de persistencia de Hibernate y su comprobación automática de cambios, el *dirty checking*.

5.1 Tres capas: JPA, Hibernate y Spring Data

↑ Índice

Antes de mirar el código conviene separar tres piezas que a menudo se mezclan:

Pieza	Qué es	En el proyecto
Jakarta Persistence (JPA)	Una especificación: anotaciones (`@Entity`, `@Id`…) e interfaces (`EntityManager`)	Las anotaciones de la entidad
Hibernate	La implementación de JPA que hace el trabajo real: genera SQL, gestiona el contexto de persistencia	Configurado por Spring Boot; no se usa directamente
Spring Data JPA	Una capa encima de JPA que genera repositorios a partir de interfaces	`ApplicationRepository` (capítulo 6)

Cuando el código llama a `applicationRepository.findById(1L)`, Spring Data traduce la llamada a `entityManager.find(Application.class, 1L)`, y Hibernate ejecuta el `SELECT` y construye el objeto.

Si vienes de Java EE…

Es la misma JPA que usabas en Java EE; en WebSphere el proveedor solía ser OpenJPA o EclipseLink, aquí es Hibernate. El paquete cambió de `javax.persistence` a `jakarta.persistence` con Jakarta EE 9, así que el código antiguo necesita cambiar los `import`. Lo que no existía en Java EE es Spring Data, que elimina la mayor parte de los DAO escritos a mano.

5.2 Mapear la entidad `Application`

Paso 2.1 ↖ ↑ Índice

`application/entity/Application.java` IMPLEMENTADO

```
@Entity
@Table(name = "applications")
```

```java
public class Application {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, length = 100)
    private String name;

    @Column(length = 500)
    private String description;

    @Column(nullable = false, length = 20)
    private String criticality;

    @Column(nullable = false, length = 20)
    private String status;

    protected Application() {
        // Constructor requerido por JPA
    }

    public Application(
            String name,
            String description,
            String criticality,
            String status) {

        this.name = name;
        this.description = description;
        this.criticality = criticality;
        this.status = status;
    }

    public void update(
            String name,
            String description,
            String criticality,
            String status) {

        this.name = name;
        this.description = description;
        this.criticality = criticality;
        this.status = status;
    }

    // getters de los cinco atributos (sin setters)
}
```

Cada elemento tiene un motivo:

`@Entity` y `@Table(name = "applications")`. Marcan la clase como persistente y la asocian a la tabla. Sin `@Table`, Hibernate usaría el nombre de la clase (`application`, en singular) y la validación del esquema fallaría.

`@Id` y `@GeneratedValue(strategy = IDENTITY)`. El identificador lo genera la columna de identidad de PostgreSQL. Por eso es `Long` y no `long`: una entidad nueva tiene `id == null`, y Spring Data usa precisamente ese `null` para saber si la entidad es nueva (capítulo 6).

`@Column(nullable = false, length = 100)`. Describe la columna. Con `ddl-auto=validate`, Hibernate comprueba que la columna exista y que el tipo sea compatible. Atributos como `length` o `nullable` documen-

tan el modelo y se usarían si Hibernate generara el esquema, pero no se aplican en tiempo de ejecución: no sustituyen a la validación de entrada del capítulo 8.

El constructor `protected` sin argumentos. JPA exige un constructor sin argumentos para crear la instancia por reflexión al leer una fila. Al hacerlo `protected`, Hibernate puede usarlo, pero el resto del código no puede crear una aplicación vacía.

El constructor público con los datos. Es la única forma de crear una aplicación nueva desde el código, y obliga a dar todos los valores.

`update(...)` en lugar de *setters*. La entidad no expone un *setter* por atributo. Para modificarla hay un método que cambia los datos editables a la vez. El nombre expresa una operación de negocio, y no hay forma de dejar la entidad a medio modificar llamando solo a algunos *setters*.

> **Decisión de diseño: Entidades sin *setters* públicos**
>
> **Motivo:** las modificaciones pasan por métodos con significado de negocio, y la entidad controla su propio estado. En `Demand` (capítulo 12) esto permite que las reglas del ciclo de vida vivan en la entidad. **Alternativa descartada:** *getters* y *setters* para todo, el estilo JavaBean habitual, que convierte la entidad en una estructura de datos que cualquiera puede dejar en un estado inválido. **Coste:** algo más de código en cada entidad.

5.3 El contexto de persistencia

↑ Índice

El concepto central de JPA es el **contexto de persistencia**: el conjunto de entidades que un `EntityManager` está gestionando en un momento dado. Puedes imaginarlo como un mapa en memoria cuya clave es el tipo y el `id` de la entidad, y cuyo valor es la instancia.

Tiene tres propiedades que conviene conocer bien:

1. **Garantía de identidad.** Dentro del mismo contexto, buscar dos veces la misma fila devuelve **la misma instancia** de Java. El segundo `findById(1L)` ni siquiera va a la base de datos: la encuentra en el mapa. Por eso a veces se le llama caché de primer nivel.
2. **Gestión del ciclo de vida.** Cada entidad está en uno de cuatro estados, y el estado determina qué hará Hibernate con ella.
3. **Escritura diferida.** Los cambios no se envían a la base de datos en el momento en que se hacen, sino cuando el contexto se **sincroniza** (*flush*).

En una aplicación Spring con `open-in-view=false`, **el contexto de persistencia dura lo mismo que la transacción**: se crea cuando empieza un método `@Transactional` y se cierra cuando termina.

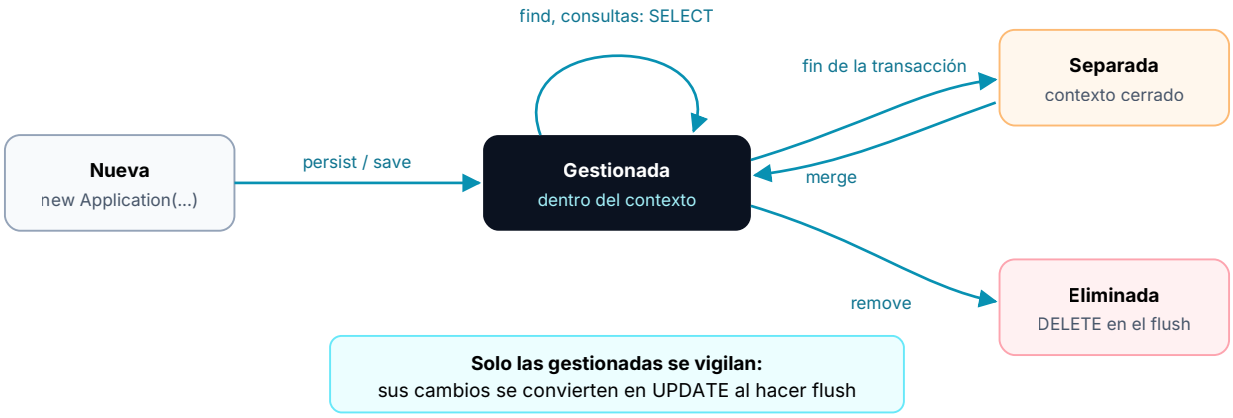


Figura 5.1. Los cuatro estados de una entidad. Una entidad cargada con `find` o con una consulta entra directamente en el estado gestionado.

Estado	Cómo se llega	Qué hace Hibernate
Nueva (*transient*)	`new Application(...)`	Nada: no la conoce
Gestionada (*managed*)	`persist`, `find`, una consulta, o `merge`	Vigila sus cambios y los escribe en el *flush*
Separada (*detached*)	Se cierra el contexto o se llama a `clear`	Nada: los cambios se pierden salvo que se haga `merge`
Eliminada (*removed*)	`remove` sobre una gestionada	Emite un `DELETE` en el *flush*

5.4 *Dirty checking* por dentro

↑ Índice

Cuando Hibernate carga una entidad, guarda en el contexto de persistencia una **instantánea** (*snapshot*) con los valores de cada atributo tal como venían de la base de datos. Al hacer *flush*, recorre todas las entidades gestionadas y compara cada atributo con su instantánea. Si alguno cambió, la entidad está «sucia» (*dirty*) y Hibernate genera un `UPDATE`.

El *flush* ocurre en tres momentos:

1. **Al confirmar la transacción**, justo antes del `COMMIT`. Es el caso normal.
2. **Antes de una consulta** que pueda verse afectada por cambios pendientes (modo `AUTO`), para que la consulta vea un estado coherente.
3. **Cuando se pide explícitamente** con `flush()` o `saveAndFlush()`, algo que las pruebas de integración usan para forzar el SQL dentro de la prueba.

La consecuencia práctica es la que hace el capítulo 7: para modificar una aplicación basta con cargarla y cambiar sus atributos dentro de una transacción. **No hace falta llamar a `save`**. Al terminar el método, Hibernate detecta el cambio y lo escribe.

Salida de show-sql al modificar una aplicación (ejemplo)

```
Hibernate: select a1_0.id,a1_0.criticality,a1_0.description,a1_0.name,a1_0.status
           from applications a1_0 where a1_0.id=?
```

```
Hibernate: update applications set criticality=?,description=?,name=?,status=?
           where id=?
```

Dos detalles de esa salida: el `UPDATE` incluye **todas** las columnas, aunque solo haya cambiado una (Hibernate prepara una sentencia fija por entidad y la reutiliza; la anotación `@DynamicUpdate` cambiaría eso), y si no cambia nada, **no hay `UPDATE`**.

> **Cuidado: El *dirty checking* solo funciona con entidades gestionadas**
>
> Si una entidad está separada, porque se cargó en otra transacción o se construyó a mano con un `id`, cambiar sus atributos no tiene ningún efecto en la base de datos. Es el error más frecuente al empezar con JPA: el código «modifica» el objeto, no hay excepción y el cambio nunca se guarda.

5.4.1 Transacciones de solo lectura

Los métodos de consulta del servicio llevan `@Transactional(readOnly = true)`. Con esa marca, Spring indica a Hibernate que no habrá escrituras, y Hibernate desactiva el *flush* automático y puede prescindir de las instantáneas. En listados grandes ahorra memoria y tiempo de CPU. Además, el *driver* JDBC marca la conexión como de solo lectura.

5.5 El coste de `GenerationType.IDENTITY`

↑ Índice

Con `IDENTITY`, el `id` lo asigna PostgreSQL al insertar. Hibernate necesita ese `id` para registrar la entidad en el contexto de persistencia, así que tiene que ejecutar el `INSERT` **inmediatamente** al llamar a `persist`, sin esperar al *flush*. Eso tiene una consecuencia: Hibernate no puede agrupar varias inserciones en un solo lote JDBC.

La alternativa es `GenerationType.SEQUENCE`: Hibernate pide a una secuencia de PostgreSQL un bloque de identificadores por adelantado, asigna el `id` en memoria y retrasa los `INSERT` hasta el *flush*, donde puede agruparlos.

> **Decisión de diseño: `IDENTITY` en lugar de `SEQUENCE`**
>
> **Motivo:** encaja con la columna de identidad estándar de la migración, es la opción más sencilla de entender y el proyecto inserta registros de uno en uno desde la API. **Alternativa:** `SEQUENCE` con `allocationSize`, preferible si algún día hay cargas masivas. Cambiar de estrategia implica una migración, así que se deja anotado como posible mejora.

5.6 Errores típicos y depuración

↑ Índice

Síntoma	Causa probable	Cómo resolverlo
`No default constructor for entity`	Falta el constructor sin argumentos	Añadirlo, `protected`
`Schema-validation: missing table [application]`	Falta `@Table(name = "applications")`	Indicar el nombre de la tabla
Un cambio no se guarda y no hay error	La entidad estaba separada o el método no era transaccional	Cargar y modificar dentro del mismo método `@Transactional`

Síntoma	Causa probable	Cómo resolverlo
Aparece un `UPDATE` inesperado al final de un método	Se modificó una entidad gestionada sin querer, por ejemplo para «formatear» un valor	No modificar entidades para presentar datos; construir el DTO aparte
`LazyInitializationException`	Se accede a una asociación perezosa fuera de la transacción	Ver el capítulo 14

5.7 Ejercicios

↑ Índice

Ejercicio 5.1 · Observar el *dirty checking* Ver solución →

Con `show-sql` activado, modifica una aplicación desde el frontend o con una petición `PUT`. Localiza en la consola el `SELECT` y el `UPDATE`. Después envía la misma petición otra vez, sin cambiar nada, y comprueba si aparece un segundo `UPDATE`. Explica el resultado.

Ejercicio 5.2 · La garantía de identidad Ver solución →

Escribe en una prueba de integración (o en un método temporal de un servicio, dentro de una transacción) dos llamadas seguidas a `findById(1L)` y compara las dos instancias con `==`. ¿Cuántos `SELECT` aparecen? ¿Qué cambia si llamas a `entityManager.clear()` entre las dos?

Autoevaluación del capítulo 5 Ver respuestas →

1. ¿Qué diferencia hay entre JPA, Hibernate y Spring Data JPA?
2. ¿Por qué el `id` de la entidad es `Long` y no `long`?
3. ¿Por qué el constructor sin argumentos es `protected` y no `public`?
4. ¿Qué compara Hibernate al hacer *flush* para decidir si genera un `UPDATE`?
5. ¿Por qué `GenerationType.IDENTITY` impide agrupar inserciones en lotes?

CAPÍTULO 6

Repositorios de Spring Data

↑ Volver al índice

EN ESTE CAPÍTULO

6.1 `ApplicationRepository`
6.2 La jerarquía de `JpaRepository`
6.3 Cómo se genera la implementación
6.4 Consultas derivadas del nombre del método
6.5 La paginación por dentro
6.6 Otras formas de consultar
6.7 Errores típicos y depuración
6.8 Ejercicios

El acceso a datos del módulo de aplicaciones es una interfaz de doce líneas sin ninguna implementación escrita a mano. Este capítulo explica cómo Spring Data la convierte en código ejecutable, qué métodos hereda y cómo funciona la paginación por dentro.

6.1 `ApplicationRepository`

Paso 2.2 ↖ ↑ Índice

`application/repository/ApplicationRepository.java` IMPLEMENTADO

```
public interface ApplicationRepository extends JpaRepository<Application, Long> {

    Page<Application> findByStatus(
            String status,
            Pageable pageable);

    Page<Application> findByCriticality(
            String criticality,
            Pageable pageable);

    Page<Application> findByStatusAndCriticality(
            String status,
            String criticality,
            Pageable pageable);
}
```

La interfaz declara dos cosas: que gestiona entidades `Application` cuyo identificador es `Long` (los parámetros genéricos de `JpaRepository`) y tres consultas adicionales para los filtros del listado. Con eso, el servicio puede buscar por `id`, listar con paginación, guardar, contar, filtrar por estado, por criticidad o por ambos.

Si vienes de Java EE...

En Java EE esto habría sido un DAO: una interfaz, una clase `@Stateless` con un `EntityManager` inyectado con `@PersistenceContext` y un método por consulta, cada uno con su JPQL y su `TypedQuery`. Spring Data elimina esa clase: la implementación se genera al arrancar.

6.2 La jerarquía de `JpaRepository`

↑ Índice

`JpaRepository` no es una interfaz aislada; reúne varias capas, cada una con un grupo de operaciones.

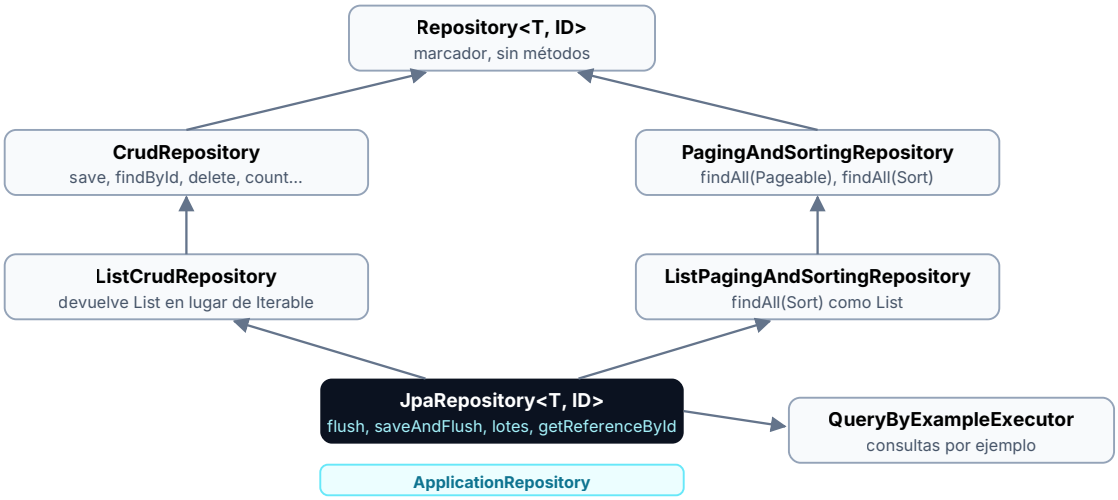


Figura 6.1. Jerarquía de interfaces de Spring Data. Las flechas apuntan a la interfaz que se extiende. `ApplicationRepository` hereda todos los métodos.

Interfaz	Métodos que usa el proyecto
`CrudRepository` / `ListCrudRepository`	`findById`, `save`, `findAllById`
`PagingAndSortingRepository`	`findAll(Pageable)`
`JpaRepository`	`saveAndFlush`, `flush` (en las pruebas)

`PagingAndSortingRepository` ya no extiende `CrudRepository`, como ocurría en versiones antiguas; por eso `JpaRepository` combina las dos ramas. En el día a día da igual: basta con extender `JpaRepository`.

6.3 Cómo se genera la implementación

↑ Índice

Al arrancar, Spring Data busca las interfaces que extienden `Repository` y, para cada una, crea un **proxy dinámico**. El proxy delega los métodos heredados en una clase real, `SimpleJpaRepository`, que contiene el código JPA de siempre. Los métodos que el desarrollador añade, como `findByStatus`, se analizan y se convierten en consultas (siguiente sección).

Conviene conocer tres detalles de `SimpleJpaRepository`:

1. **Es transaccional por sí misma.** La clase lleva `@Transactional(readOnly = true)` y los métodos de escritura, como `save`, sobrescriben esa anotación con una transacción de escritura. Si el servicio ya tiene una transacción abierta, el repositorio **se une a ella** (propagación `REQUIRED`).
2. **`save` decide entre `persist` y `merge`.** Si la entidad es nueva (su `id` es `null`), llama a `entityManager.persist`; si no, a `entityManager.merge`. `merge` copia el estado de un objeto sobre la entidad gestionada y devuelve **otra instancia**: la gestionada. Por eso se recomienda usar siempre el valor devuelto por `save`.
3. **`findById` es `entityManager.find`.** Consulta primero el contexto de persistencia y solo va a la base de datos si no encuentra la entidad (capítulo 5).

> **Cuidado: `save` sobre una entidad gestionada no hace nada útil**
>
> Llamar a `save` con una entidad que ya está gestionada no provoca ningún SQL adicional: el `UPDATE` llegará igualmente en el *flush*. No es un error, pero transmite una idea equivocada de cómo funciona JPA. El capítulo 7 explica por qué el servicio no lo llama al modificar.

6.4 Consultas derivadas del nombre del método

↑ Índice

Spring Data analiza el nombre de cada método añadido y construye la consulta a partir de él:

```
findByStatusAndCriticality(String status, String criticality, Pageable pageable)
|    | └──────┬──────┘└──────┬──────┘
|    |   propiedad   propiedad      → where a.status = ?1 and a.criticality = ?2
|    └── By: empieza el criterio
└─────── find: consulta de lectura
```

El análisis ocurre **al arrancar**. Si el nombre hace referencia a una propiedad que no existe en la entidad (`findByStatuss`), la aplicación no arranca, con un mensaje que nombra la propiedad desconocida. Es otra comprobación temprana, como la validación del esquema.

Las palabras clave admitidas son muchas (`Containing`, `Between`, `IgnoreCase`, `OrderBy`…). La regla práctica es que un nombre derivado está bien mientras se lea con facilidad. Cuando se necesitan tres o más criterios opcionales, el número de métodos crece de forma combinatoria; el capítulo 13 muestra cómo lo resolvió el módulo de demandas.

6.5 La paginación por dentro

↑ Índice

Los tres métodos reciben un `Pageable` y devuelven un `Page`. `Pageable` describe qué página se quiere: número de página (empezando en cero), tamaño y ordenación. `Page` contiene los elementos de esa página **y** los metadatos: número total de elementos y de páginas.

Para construir ese resultado, Spring Data ejecuta **dos consultas**:

```sql
-- 1. Los datos de la página
select a1_0.id, a1_0.criticality, a1_0.description, a1_0.name, a1_0.status
from applications a1_0
where a1_0.status = ?
order by a1_0.id
offset ? rows fetch first ? rows only

-- 2. El recuento, para calcular totalElements y totalPages
select count(a1_0.id) from applications a1_0 where a1_0.status = ?
```

Spring Data omite la segunda consulta cuando puede deducir el total sin ella: por ejemplo, si la primera página devuelve menos elementos que el tamaño pedido, el total es exactamente ese número.

> **Nota: `Page` frente a `Slice`**
>
> Si la interfaz no necesita saber cuántas páginas hay (un *scroll* infinito, por ejemplo), un método puede devolver `Slice` en lugar de `Page`. `Slice` pide un elemento más de los necesarios para saber si hay página siguiente y se ahorra el `count`, que en tablas muy grandes puede ser la consulta más cara. El frontend del proyecto muestra el total de registros, así que usa `Page`.

6.6 Otras formas de consultar

↑ Índice

Técnica	Cuándo usarla	En el proyecto
Consulta derivada	Pocos criterios, nombre legible	Aplicaciones
`@Query("select a from Application a where …")`	Consultas fijas que no caben en un nombre	No se usa
`Specification` (Criteria API)	Criterios opcionales que se combinan en tiempo de ejecución	Demandas (capítulo 13)
`EntityManager` directo	Casos especiales que Spring Data no cubre	No se usa

6.7 Errores típicos y depuración

↑ Índice

Síntoma	Causa probable	Cómo resolverlo
`No property 'statuss' found for type 'Application'` al arrancar	Error de escritura en el nombre de un método derivado	Corregir el nombre
`No qualifying bean of type 'ApplicationRepository'`	La interfaz está fuera del paquete escaneado o no extiende `Repository`	Revisar paquete y herencia
La página siempre tiene 20 elementos aunque se pida otro tamaño	El parámetro de la URL no se llama `size`	Usar `page`, `size` y `sort` (capítulo 9)
Un `merge` devuelve un objeto distinto y los cambios posteriores no se guardan	Se siguió usando el objeto pasado a `save`, no el devuelto	Usar siempre la instancia devuelta

6.8 Ejercicios

↑ Índice

Ejercicio 6.1 · Contar las consultas de una página Ver solución →

Con `show-sql` activado, pide `GET /api/applications?page=0&size=1` y después `GET /api/applications?page=0&size=50`. ¿Cuántas consultas aparecen en cada caso? Explica la diferencia.

Ejercicio 6.2 · Un método derivado nuevo Ver solución →

Diseña (sin necesidad de mantenerlo en el proyecto) un método derivado que busque aplicaciones cuyo nombre contenga un texto, sin distinguir mayúsculas y minúsculas, con paginación. Escribe la consulta SQL que esperas que genere.

Autoevaluación del capítulo 6

Ver respuestas →

1. ¿Qué clase contiene la implementación real de los métodos heredados de `JpaRepository`?
2. ¿Cómo decide `save` entre `persist` y `merge`?
3. ¿Cuándo detecta Spring Data un error en el nombre de un método derivado?
4. ¿Por qué un listado paginado puede ejecutar dos consultas, y cuándo se ahorra la segunda?
5. ¿Qué ocurre si un repositorio se llama desde un método que ya tiene una transacción abierta?

CAPÍTULO 7

El servicio y las transacciones

↑ Volver al índice

EN ESTE CAPÍTULO

7.1 El papel del servicio
7.2 Consulta, detalle y alta
7.3 Modificar sin llamar a `save`
7.4 `@Transactional` por dentro
7.5 Excepciones HTTP en el servicio
7.6 Convertir a DTO dentro de la transacción
7.7 Errores típicos y depuración
7.8 Ejercicios

El servicio es la capa donde viven los casos de uso: consultar, ver un detalle, dar de alta y modificar. También es donde empiezan y terminan las transacciones. Este capítulo recorre `ApplicationService` tal como se escribió en la primera versión y explica qué hace Spring cuando ve `@Transactional`.

7.1 El papel del servicio

↑ Índice

En la arquitectura del proyecto cada capa tiene una responsabilidad clara:

Capa	Responsabilidad	No debe…
Controlador	Traducir HTTP a llamadas Java y viceversa	Contener reglas de negocio ni acceder a repositorios
Servicio	Casos de uso, reglas, transacciones y conversión entidad → DTO	Conocer detalles de HTTP más allá de lo imprescindible
Repositorio	Acceso a datos	Contener reglas de negocio
Entidad	Estado y reglas propias del objeto	Saber cómo se persiste o cómo se expone

El servicio recibe DTO de entrada, trabaja con entidades y devuelve DTO de salida. **Las entidades nunca salen del servicio**: el controlador no ve un solo objeto `Application`.

7.2 Consulta, detalle y alta

Paso 2.3 ↖ ↑ Índice

7.2.1 La consulta del listado

`application/service/ApplicationService.java (versión de GitHub)` IMPLEMENTADO

```
@Transactional(readOnly = true)
public ApplicationPageResponse getApplications(
        String status,
        String criticality,
        Pageable pageable) {

    Page<Application> applications;
```

```
    if (status != null && criticality != null) {
        applications = applicationRepository
                .findByStatusAndCriticality(status, criticality, pageable);
    } else if (status != null) {
        applications = applicationRepository
                .findByStatus(status, pageable);
    } else if (criticality != null) {
        applications = applicationRepository
                .findByCriticality(criticality, pageable);
    } else {
        applications = applicationRepository.findAll(pageable);
    }

    Page<ApplicationResponse> responsePage =
            applications.map(this::toResponse);

    return new ApplicationPageResponse(
            responsePage.getContent(),
            responsePage.getNumber(),
            responsePage.getSize(),
            responsePage.getTotalElements(),
            responsePage.getTotalPages()
    );
}
```

Con dos filtros opcionales hay cuatro combinaciones, y cada una tiene su método de repositorio. Es explícito y fácil de seguir. También deja ver el límite del enfoque: con tres filtros serían ocho ramas. El módulo de demandas, con tres filtros, tomó otro camino (capítulo 13).

`Page.map` transforma cada entidad en su DTO conservando los metadatos de paginación. El resultado se copia a `ApplicationPageResponse`, un DTO propio cuyo motivo explica el capítulo 9.

En esta primera versión los filtros no se validan: un estado inventado, como `status=PEPITO`, simplemente devuelve una página vacía. Más adelante se añadió la validación (capítulo 10).

7.2.2 El detalle con 404

application/service/ApplicationService.java IMPLEMENTADO

```
@Transactional(readOnly = true)
public ApplicationResponse getApplicationById(Long id) {

    var application = applicationRepository.findById(id)
            .orElseThrow(() -> new ResponseStatusException(
                    HttpStatus.NOT_FOUND,
                    "Application not found: " + id
            ));

    return toResponse(application);
}
```

`findById` devuelve un `Optional<Application>`. Si está vacío, `orElseThrow` lanza una `ResponseStatusException` con estado 404 y un mensaje que incluye el identificador. Esa excepción la captura el manejador global (capítulo 10) y la convierte en una respuesta JSON.

7.2.3 El alta

application/service/ApplicationService.java IMPLEMENTADO

```
@Transactional
public ApplicationResponse createApplication(
        ApplicationCreateRequest request) {

    Application application = new Application(
            request.name(),
            request.description(),
            request.criticality(),
            "ACTIVE"
    );

    Application saved = applicationRepository.save(application);

    return toResponse(saved);
}
```

Dos decisiones se ven en este método. La primera: **el estado inicial lo decide el servidor**. El DTO de alta no tiene campo `status`; toda aplicación nueva nace `ACTIVE`. La segunda: se usa el valor devuelto por `save`, que es la entidad gestionada y ya tiene `id`, porque el `INSERT` se ejecutó inmediatamente (capítulo 5).

7.3 Modificar sin llamar a save

Paso 2.4 ↖ ↑ Índice

application/service/ApplicationService.java IMPLEMENTADO

```
@Transactional
public ApplicationResponse updateApplication(
        Long id,
        ApplicationUpdateRequest request) {

    Application application = applicationRepository.findById(id)
            .orElseThrow(() -> new ResponseStatusException(
                    HttpStatus.NOT_FOUND,
                    "Application not found: " + id
            ));

    application.update(
            request.name(),
            request.description(),
            request.criticality(),
            request.status()
    );

    return toResponse(application);
}
```

No hay ninguna llamada a `save`. La secuencia es esta:

1. Spring abre una transacción y, con ella, un contexto de persistencia.
2. `findById` carga la entidad, que queda **gestionada**, y Hibernate guarda su instantánea.
3. `update` cambia sus atributos en memoria.
4. `toResponse` construye el DTO con los valores nuevos.

5. Al salir del método, Spring confirma la transacción. Antes del `COMMIT`, Hibernate hace *flush*, compara con la instantánea y emite el `UPDATE`.

Si en el paso 3 o 4 se lanzara una excepción no comprobada, la transacción se desharía y no habría `UPDATE`.

> **Decisión de diseño: Confiar en el *dirty checking* en lugar de llamar a `save`**
>
> **Motivo:** el código refleja cómo funciona JPA de verdad: una entidad gestionada se sincroniza sola. Llamar a `save` no aporta nada y sugiere que sin él no se guardaría. **Prueba:** el test `shouldUpdateExistingApplicationWithoutCallingSave` (capítulo 11) verifica explícitamente que `save` no se llama. **Requisito:** el método debe ser transaccional; sin `@Transactional`, la entidad quedaría separada al volver de `findById` y el cambio se perdería.

7.4 `@Transactional` por dentro

↑ Índice

`@Transactional` no es magia del compilador: es un **proxy**. Al arrancar, Spring detecta que `ApplicationService` tiene métodos transaccionales y, en lugar de inyectar la clase tal cual en el controlador, inyecta un objeto intermediario que la envuelve.

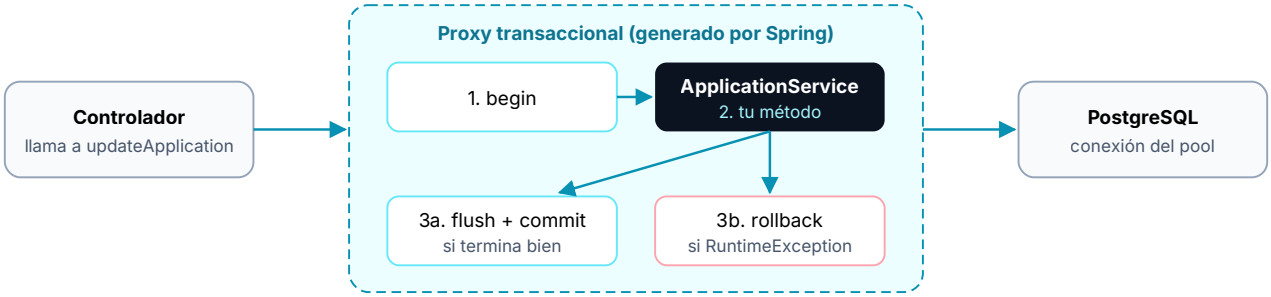


Figura 7.1. El controlador habla con un proxy. El proxy abre la transacción, delega en el método real y, según cómo termine, confirma o deshace.

De este mecanismo se derivan las reglas que hay que conocer:

Propagación. Por defecto es `REQUIRED`: si ya hay una transacción en curso, el método se une a ella; si no, abre una. Por eso los métodos del repositorio, que también son transaccionales, participan en la transacción del servicio en lugar de abrir otra.

Reglas de *rollback*. Por defecto, la transacción se deshace ante una excepción **no comprobada** (`RuntimeException` y sus subclases) o un `Error`, y **se confirma** ante una excepción comprobada. `ResponseStatusException` es no comprobada, así que un 404 lanzado a mitad de un método deshace todo lo hecho hasta ese momento.

Autoinvocación. Si un método de `ApplicationService` llama a otro método de la misma clase, la llamada no pasa por el proxy y su `@Transactional` se ignora. Las anotaciones transaccionales solo tienen efecto en llamadas que llegan desde fuera del *bean*.

Visibilidad. Las anotaciones se colocan en métodos públicos. El proyecto anota los métodos y no la clase, de forma que cada método declara si lee o escribe.

Si vienes de Java EE...

Es el equivalente a las transacciones gestionadas por el contenedor de un EJB `@Stateless`, con `REQUIRED` como valor por defecto en ambos casos. Hay una diferencia importante: en EJB, una excepción de aplicación comprobada tampoco provocaba *rollback* salvo que se marcara con `@ApplicationException(rollback = true)`; en Spring, el equivalente es `@Transactional(rollbackFor = ...)`. La trampa de la autoinvocación también existía en EJB cuando se llamaba a un método con `this.` en lugar de a través del contexto del contenedor.

7.5 Excepciones HTTP en el servicio

↑ Índice

El servicio lanza `ResponseStatusException`, una clase de Spring Web que lleva dentro un código HTTP. Es una concesión consciente: el servicio queda ligeramente acoplado a HTTP.

Decisión de diseño: `ResponseStatusException` en el servicio

Motivo: simplicidad. Una sola clase cubre 404, 400 y 409, el manejador global la traduce de forma uniforme, y no hace falta una jerarquía de excepciones propia para un proyecto de este tamaño. **Alternativa:** excepciones de dominio (`ApplicationNotFoundException`, `InvalidStatusTransitionException`) que el manejador global traduce a códigos HTTP. Es más limpio, porque el servicio no sabe nada de HTTP, y se recoge como mejora en el capítulo 24.

7.6 Convertir a DTO dentro de la transacción

↑ Índice

`application/service/ApplicationService.java` IMPLEMENTADO

```
private ApplicationResponse toResponse(Application application) {

    return new ApplicationResponse(
            application.getId(),
            application.getName(),
            application.getDescription(),
            application.getCriticality(),
            application.getStatus()
    );
}
```

La conversión ocurre **antes de que termine la transacción**, mientras la entidad está gestionada. Con `open-in-view=false` esto es imprescindible en cuanto la entidad tenga asociaciones perezosas: fuera de la transacción no se podrían cargar. En `Application` no hay asociaciones, pero la costumbre se estableció desde el principio, y en el módulo de demandas resultó decisiva.

7.7 Errores típicos y depuración

↑ Índice

Síntoma	Causa probable	Cómo resolverlo
Una modificación no se guarda	Falta `@Transactional` en el método del servicio	Añadirlo

Síntoma	Causa probable	Cómo resolverlo
Se guarda a medias tras un error	La excepción era comprobada y no provocó *rollback*	Usar excepciones no comprobadas o `rollbackFor`
Un método interno no abre transacción	Se llama desde la misma clase (autoinvocación)	Mover la lógica a otro *bean* o anotar el método llamado desde fuera
`TransactionRequiredException`	Una operación de escritura se ejecutó sin transacción	Revisar que el método de entrada sea `@Transactional`
Se escribe en un método `readOnly` y no se guarda	`readOnly` desactiva el *flush* automático	Quitar `readOnly` en métodos que modifican

7.8 Ejercicios

↑ Índice

Ejercicio 7.1 · Ver el *rollback* en acción Ver solución →

En una rama de trabajo, añade al final de `updateApplication`, después de `application.update(...)`, una línea que lance `new IllegalStateException("prueba")`. Envía una modificación y comprueba en la base de datos que el cambio no se guardó. Repite con una excepción comprobada declarada en el método (por ejemplo, `java.io.IOException`) y compara.

Ejercicio 7.2 · Detectar la autoinvocación Ver solución →

Explica qué ocurriría si `getApplications`, anotado con `readOnly = true`, llamara internamente a un método público de la misma clase anotado con `@Transactional` (sin `readOnly`) que modifica una entidad. ¿Qué transacción estaría activa y qué pasaría con el cambio?

Autoevaluación del capítulo 7 Ver respuestas →

1. ¿Por qué el servicio no llama a `save` al modificar una aplicación, y qué condición es imprescindible para que funcione?
2. ¿Qué propagación tiene `@Transactional` por defecto y qué significa?
3. ¿Ante qué tipo de excepciones hace Spring *rollback* por defecto?
4. ¿Qué es la autoinvocación y por qué hace que se ignore `@Transactional`?
5. ¿Por qué la conversión a DTO se hace dentro del método transaccional?

CAPÍTULO 8

DTO y Bean Validation

↑ Volver al índice

EN ESTE CAPÍTULO

8.1 Por qué DTO separados de la entidad
8.2 *Records* de Java
8.3 Las reglas de validación
8.4 Dos DTO de entrada, no uno
8.5 Cadenas en lugar de enumerados
8.6 La misma regla en tres sitios
8.7 Errores típicos y depuración
8.8 Ejercicios

La API no expone entidades: recibe y devuelve objetos de transferencia de datos, los DTO. Este capítulo explica por qué, cómo se escriben con *records* de Java y cómo Bean Validation rechaza una petición incorrecta antes de que llegue al servicio.

8.1 Por qué DTO separados de la entidad

Paso 2.5 ↖ ↑ Índice

Sería posible que el controlador recibiera y devolviera directamente objetos `Application`. Muchos tutoriales lo hacen. El proyecto no, por cuatro motivos:

1. **El contrato de la API es independiente del modelo de datos.** Si mañana la tabla cambia, los clientes no tienen por qué enterarse.
2. **Cada operación acepta solo lo que debe.** El alta no tiene `id` ni `status`; si recibiera la entidad, un cliente podría enviar un `id` o un estado arbitrario.
3. **No se filtran detalles de persistencia.** Serializar una entidad con asociaciones perezosas puede disparar consultas, o fallar con `LazyInitializationException`.
4. **Las reglas de validación de entrada no contaminan la entidad.** Las anotaciones de validación viven en el DTO de cada operación.

El módulo de aplicaciones tiene cuatro DTO:

DTO	Dirección	Uso
`ApplicationCreateRequest`	Entrada	`POST /api/applications`
`ApplicationUpdateRequest`	Entrada	`PUT /api/applications/{id}`
`ApplicationResponse`	Salida	Detalle y elementos del listado
`ApplicationPageResponse`	Salida	Listado paginado (capítulo 9)

8.2 *Records* de Java

↑ Índice

Todos los DTO son *records*, disponibles desde Java 16:

application/dto/ApplicationResponse.java IMPLEMENTADO

```
public record ApplicationResponse(
        Long id,
        String name,
        String description,
        String criticality,
        String status) {
}
```

Un *record* es una clase inmutable que el compilador completa: atributos `private final`, un constructor con todos ellos, métodos de acceso con el mismo nombre que el atributo (`name()`, no `getName()`), y `equals`, `hashCode` y `toString` basados en todos los valores. Jackson, la librería JSON de Spring, sabe leerlos y escribirlos sin configuración.

Si vienes de Java EE...

Un *record* sustituye a la clase JavaBean con *getters*, *setters*, constructor vacío, `equals` y `hashCode` generados por el IDE que se usaba como *transfer object* en Java EE. La diferencia esencial es la inmutabilidad: un DTO no debería cambiar después de construirse.

Cuidado: Las entidades no pueden ser *records*

JPA necesita crear entidades con un constructor sin argumentos y modificar sus atributos (para el *dirty checking*, entre otras cosas). Un *record* no permite ninguna de las dos cosas. Los *records* son perfectos para DTO y para resultados de consultas, pero no para entidades.

8.3 Las reglas de validación

↑ Índice

application/dto/ApplicationCreateRequest.java IMPLEMENTADO

```
public record ApplicationCreateRequest(

        @NotBlank
        @Size(max = 100)
        String name,

        @Size(max = 500)
        String description,

        @NotBlank
        @Pattern(regexp = "LOW|MEDIUM|HIGH|CRITICAL")
        String criticality

) {
}
```

application/dto/ApplicationUpdateRequest.java IMPLEMENTADO

```
public record ApplicationUpdateRequest(

        @NotBlank
        @Size(max = 100)
        String name,

        @Size(max = 500)
```

```
        String description,

        @NotBlank
        @Pattern(regexp = "LOW|MEDIUM|HIGH|CRITICAL")
        String criticality,

        @NotBlank
        @Pattern(regexp = "ACTIVE|INACTIVE")
        String status

) {
}
```

Las anotaciones son de Jakarta Bean Validation, y la implementación que las ejecuta es Hibernate Validator, incluida en `spring-boot-starter-validation`. Las que usa el proyecto:

Anotación	Rechaza	Acepta `null`
`@NotNull`	`null`	No
`@NotEmpty`	`null` y cadena vacía `""`	No
`@NotBlank`	`null`, `""` y cadenas solo con espacios `" "`	No
`@Size(max = n)`	Cadenas de más de *n* caracteres	**Sí**
`@Pattern(regexp = …)`	Cadenas que no encajan **completas** con la expresión	**Sí**

La última columna explica por qué `criticality` lleva dos anotaciones. `@Pattern`, como casi todas las restricciones de Bean Validation, **considera válido el `null`**: la filosofía es que cada anotación comprueba una sola cosa, y la obligatoriedad es asunto de `@NotNull` o `@NotBlank`. Sin `@NotBlank`, una petición sin criticidad pasaría la validación y fallaría después en el `INSERT`, por la restricción `NOT NULL` de la columna.

Por la misma razón, `description` solo lleva `@Size`: es opcional, puede ser `null` y, si viene, no puede pasar de 500 caracteres.

8.3.1 Cuándo se ejecuta la validación

La validación no ocurre sola: la activa la anotación `@Valid` en el parámetro del controlador.

`application/controller/ApplicationController.java (extracto)` IMPLEMENTADO

```
@PostMapping
public ResponseEntity<ApplicationResponse> createApplication(
        @Valid @RequestBody ApplicationCreateRequest request) {
```

El orden es este:

1. Spring MVC lee el cuerpo de la petición y Jackson lo convierte en un `ApplicationCreateRequest`. Si el JSON está mal formado, falla aquí, con otra excepción (capítulo 10).
2. Como el parámetro tiene `@Valid`, Spring pasa el objeto al validador, que evalúa **todas** las restricciones y reúne todos los errores, no solo el primero.
3. Si hay algún error, Spring lanza `MethodArgumentNotValidException` **sin llamar al método del controlador**. El servicio nunca se ejecuta.
4. El manejador global convierte la excepción en un 400 con la lista de campos inválidos.

En el proyecto

La prueba `shouldReturnBadRequestWhenCreatingInvalidApplication` (capítulo 11) envía un nombre con solo espacios y una criticidad `URGENT`, comprueba que la respuesta contiene un error para cada uno de esos dos campos y verifica con Mockito que `createApplication` del servicio **no se llamó nunca**.

8.4 Dos DTO de entrada, no uno

↑ Índice

El alta y la modificación tienen DTO distintos aunque se parezcan mucho. La diferencia está en lo que cada operación permite: al crear, el estado no se elige; al modificar, sí. Con un único DTO compartido, el campo `status` sería opcional en uno y obligatorio en otro, y la validación tendría que distinguir el caso con grupos de validación o con comprobaciones manuales.

Decisión de diseño: Un DTO por operación

Motivo: cada DTO describe exactamente el contrato de su operación y sus reglas se leen sin condiciones. **Alternativa descartada:** un DTO compartido con grupos de validación (`@NotBlank(groups = OnUpdate.class)`), que ahorra una clase a cambio de reglas más difíciles de leer. **Coste:** algo de duplicación entre los dos *records*.

¿Qué ocurre si el cliente envía un `status` al dar de alta? Nada: Spring Boot configura Jackson para **ignorar las propiedades desconocidas**, así que el campo se descarta en silencio. Es cómodo para la compatibilidad entre versiones, aunque puede ocultar errores del cliente.

8.5 Cadenas en lugar de enumerados

↑ Índice

Estado y criticidad son `String` en la base de datos, en la entidad y en los DTO, con los valores válidos expresados como expresiones regulares. La alternativa natural en Java habría sido un `enum`:

Alternativa con enumerados (no está en el proyecto) PENDIENTE

```
public enum Criticality { LOW, MEDIUM, HIGH, CRITICAL }

// en la entidad
@Enumerated(EnumType.STRING)
@Column(nullable = false, length = 20)
private Criticality criticality;
```

Decisión de diseño: Valores como texto validado

Motivo: máxima sencillez al empezar, y el JSON y la base de datos guardan exactamente el mismo texto. **Coste:** la lista de valores válidos se repite en varios sitios (la expresión regular del DTO y, más adelante, las constantes del servicio y el frontend), el compilador no ayuda y nada impide escribir `"ACTVE"` en el código. **Mejora propuesta:** `enum` con `@Enumerated(EnumType.STRING)`, que no cambia ni el JSON ni la columna. Se recoge en el capítulo 24. Si se adopta, nunca debe usarse `EnumType.ORDINAL`, que guarda la posición del valor y se rompe al reordenar el `enum`.

8.6 La misma regla en tres sitios

↑ Índice

Una consecuencia de las decisiones anteriores es que la longitud y la obligatoriedad de cada campo aparecen en tres capas, y deben coincidir:

Campo	Migración V1	Entidad	DTO de entrada
`name`	`VARCHAR(100) NOT NULL`	`@Column(nullable = false, length = 100)`	`@NotBlank @Size(max = 100)`
`description`	`VARCHAR(500)`	`@Column(length = 500)`	`@Size(max = 500)`
`criticality`	`VARCHAR(20) NOT NULL`	`@Column(nullable = false, length = 20)`	`@NotBlank @Pattern(...)`
`status`	`VARCHAR(20) NOT NULL`	`@Column(nullable = false, length = 20)`	`@NotBlank @Pattern(...)` (solo al modificar)

Cada capa protege algo distinto: el DTO da un error claro al cliente, la entidad documenta el modelo y la base de datos es la última barrera. El frontend añade una cuarta copia, con Zod, para dar respuesta inmediata en el formulario.

8.7 Errores típicos y depuración

↑ Índice

Síntoma	Causa probable	Cómo resolverlo
Una petición inválida llega al servicio	Falta `@Valid` en el parámetro	Añadirlo
Un campo obligatorio vacío pasa la validación	Solo tiene `@Pattern` o `@Size`, que aceptan `null`	Añadir `@NotBlank` o `@NotNull`
`value too long for type character varying(100)` desde PostgreSQL	El DTO permite más caracteres que la columna	Alinear `@Size` con la migración
`@Pattern` rechaza `"high"`	La expresión distingue mayúsculas y debe encajar completa	Enviar `HIGH`, o ajustar la expresión si se quiere aceptar
Un campo enviado por el cliente no llega	Nombre distinto entre el JSON y el componente del *record*	Revisar mayúsculas y nombres

8.8 Ejercicios

↑ Índice

Ejercicio 8.1 · Todos los errores a la vez Ver solución →

Envía un `POST /api/applications` con este cuerpo: `{"name": "", "description": null, "criticality": "urgent"}`. Analiza la respuesta: ¿cuántos errores de campo aparecen y por qué `description` no figura entre ellos?

Ejercicio 8.2 · Probar el validador sin Spring Ver solución →

Escribe una prueba unitaria que cree un validador con `Validation.buildDefaultValidatorFactory().getValidator()`, valide un `ApplicationCreateRequest` con un nombre de 101 caracteres y compruebe que hay exactamente una violación, sobre el campo `name`.

Autoevaluación del capítulo 8

Ver respuestas →

1. Da tres motivos para no exponer la entidad `Application` directamente en la API.
2. ¿Por qué un *record* no puede ser una entidad JPA?
3. ¿Por qué `criticality` necesita `@NotBlank` además de `@Pattern`?
4. ¿Qué excepción lanza Spring cuando falla la validación de un `@RequestBody` y qué ocurre con el método del controlador?
5. ¿Qué inconveniente tiene representar el estado como `String`, y cómo se mejoraría sin cambiar el JSON ni la base de datos?

CAPÍTULO 9

El controlador REST y el contrato HTTP

↑ Volver al índice

EN ESTE CAPÍTULO

9.1 Exponer `/api/applications`
9.2 Cómo construye Spring un `Pageable`
9.3 La página en un DTO propio
9.4 El recorrido completo de una petición
9.5 El contrato HTTP del módulo
9.6 Errores típicos y depuración
9.7 Ejercicios

El controlador es la puerta de entrada: convierte peticiones HTTP en llamadas al servicio y resultados en respuestas HTTP. Este capítulo explica `ApplicationController`, cómo Spring construye un `Pageable` a partir de la URL, por qué la página se devuelve en un DTO propio y, al final, el recorrido completo de una petición por todas las capas.

9.1 Exponer `/api/applications`

Paso 2.6 ↖ ↑ Índice

`application/controller/ApplicationController.java` IMPLEMENTADO

```
@RestController
@RequestMapping("/api/applications")
public class ApplicationController {

    private final ApplicationService applicationService;

    public ApplicationController(ApplicationService applicationService) {
        this.applicationService = applicationService;
    }

    @GetMapping
    public ApplicationPageResponse getApplications(
            @RequestParam(required = false) String status,
            @RequestParam(required = false) String criticality,
            @PageableDefault(
                    size = 10,
                    sort = "id",
                    direction = Sort.Direction.ASC
            ) Pageable pageable) {

        return applicationService.getApplications(
                status,
                criticality,
                pageable);
    }

    @GetMapping("/{id}")
    public ApplicationResponse getApplicationById(@PathVariable Long id) {
        return applicationService.getApplicationById(id);
    }

    @PostMapping
    public ResponseEntity<ApplicationResponse> createApplication(
            @Valid @RequestBody ApplicationCreateRequest request) {
```

```
        ApplicationResponse created =
                applicationService.createApplication(request);

        URI location = URI.create(
                "/api/applications/" + created.id()
        );

        return ResponseEntity
                .created(location)
                .body(created);
    }

    @PutMapping("/{id}")
    public ResponseEntity<ApplicationResponse> updateApplication(
            @PathVariable Long id,
            @Valid @RequestBody ApplicationUpdateRequest request) {

        ApplicationResponse response =
                applicationService.updateApplication(id, request);

        return ResponseEntity.ok(response);
    }
}
```

`@RestController` combina `@Controller` y `@ResponseBody`: el valor que devuelve cada método se serializa directamente como cuerpo de la respuesta (JSON, con Jackson), en lugar de interpretarse como el nombre de una vista.

`@RequestMapping("/api/applications")` fija el prefijo común. Cada método añade su parte con `@GetMapping`, `@PostMapping` o `@PutMapping`.

`@RequestParam(required = false)` lee un parámetro de la cadena de consulta (`?status=ACTIVE`). Si no viene, el valor es `null`, y el servicio interpreta `null` como «sin filtro».

`@PathVariable Long id` extrae el segmento `{id}` de la ruta y lo convierte a `Long`. Si no es un número (`/api/applications/abc`), la conversión falla antes de llegar al método; el capítulo 10 explica cómo se responde a eso.

El alta devuelve `201 Created` con una cabecera `Location` que apunta al recurso nuevo, y el propio recurso en el cuerpo. Es la respuesta que HTTP prevé para una creación, y permite al cliente conocer el `id` y la URL sin otra petición. La modificación devuelve `200 OK` con el recurso actualizado.

Si vienes de Java EE...

Es el equivalente directo de un recurso JAX-RS: `@Path` se convierte en `@RequestMapping`, `@GET` en `@GetMapping`, `@QueryParam` en `@RequestParam`, `@PathParam` en `@PathVariable`, y `Response.created(uri).entity(x).build()` en `ResponseEntity.created(uri).body(x)`. El servlet frontal de Spring, `DispatcherServlet`, hace el papel del servlet de la implementación JAX-RS del servidor.

Nota: La URL de `Location`

La cabecera se construye como URI relativa (`/api/applications/5`). HTTP lo permite y el cliente la resuelve contra la URL de la petición. Si se quisiera absoluta, `ServletUriComponentsBuilder.fromCurrentRequest()` la construiría a partir de la petición en curso.

9.2 Cómo construye Spring un `Pageable`

↑ Índice

El parámetro `Pageable pageable` no lleva `@RequestParam`. Lo rellena un componente de Spring Data Web, `PageableHandlerMethodArgumentResolver`, que lee tres parámetros de la URL:

Parámetro	Significado	Ejemplo
`page`	Número de página, **empezando en 0**	`page=2`
`size`	Elementos por página	`size=25`
`sort`	Propiedad y, opcionalmente, dirección; se puede repetir	`sort=name,desc&sort=id`

`@PageableDefault(size = 10, sort = "id", direction = ASC)` define qué se usa cuando el cliente no envía esos parámetros: primera página, diez elementos, ordenados por `id` ascendente. Sin la anotación, el tamaño por defecto sería 20 y no habría orden, lo que da resultados no deterministas entre páginas.

Spring Data limita además el tamaño máximo de página (2000 por defecto), de modo que una petición con `size=1000000` no carga la tabla entera.

> **Cuidado: El campo de ordenación lo elige el cliente**
>
> `sort` acepta cualquier texto. Spring Data lo traduce a un `ORDER BY` sobre la propiedad de la entidad con ese nombre. Si la propiedad no existe, la consulta falla con una excepción que, sin tratamiento, llega al cliente como un error 500. En la primera versión no había ninguna comprobación; más adelante se añadió una lista de campos permitidos (capítulo 10).

9.3 La página en un DTO propio

Paso 2.7 ↖ ↑ Índice

El servicio devuelve un `ApplicationPageResponse`, no el `Page<ApplicationResponse>` de Spring Data:

application/dto/ApplicationPageResponse.java IMPLEMENTADO

```
public record ApplicationPageResponse(
        List<ApplicationResponse> content,
        int page,
        int size,
        long totalElements,
        int totalPages
) {
}
```

Respuesta de GET /api/applications?size=2

```
{
  "content": [
    { "id": 1, "name": "SIGMA", "description": "Portal de gestión de expedientes",
      "criticality": "HIGH", "status": "ACTIVE" },
    { "id": 2, "name": "ORION", "description": "Aplicación backend",
      "criticality": "MEDIUM", "status": "ACTIVE" }
  ],
  "page": 0,
  "size": 2,
  "totalElements": 2,
```

```
  "totalPages": 1
}
```

Serializar directamente un `Page` produce un JSON con muchos campos internos (`pageable`, `sort`, `first`, `last`, `numberOfElements`…) cuya forma depende de la implementación `PageImpl` y puede cambiar entre versiones de Spring Data. De hecho, Spring Data avisa en el *log* cuando detecta que se serializa un `PageImpl` tal cual y recomienda no hacerlo.

> **Decisión de diseño: DTO de página propio**
>
> **Motivo:** el contrato con el frontend queda fijado por el proyecto, con cinco campos estables y con nombre propio, y no depende de una clase interna de Spring Data. **Alternativa:** activar `@EnableSpringDataWebSupport(pageSerializationMode = VIA_DTO)`, que hace que Spring Data serialice un `PagedModel` estable. Habría funcionado, pero con un formato anidado (`content` y `page`) que el proyecto no controla.

9.4 El recorrido completo de una petición

↑ Índice

Con todas las capas construidas, es el momento de seguir una petición de alta de principio a fin.

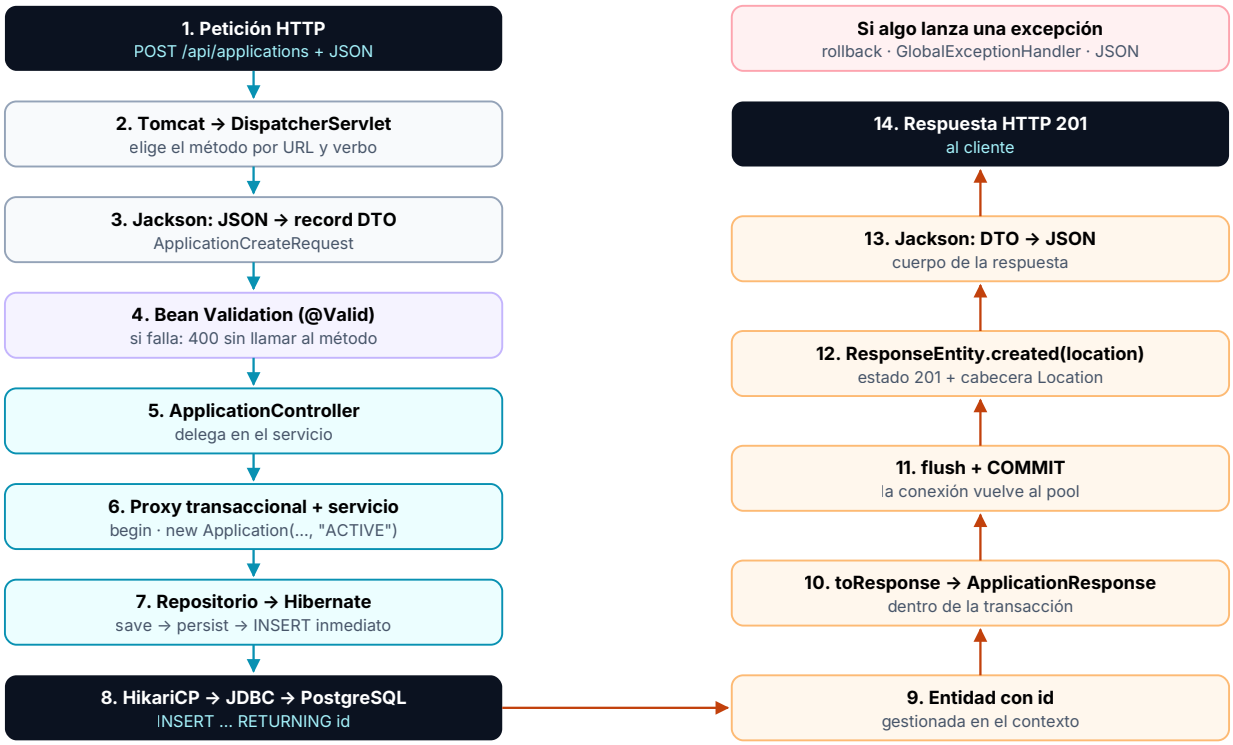


Figura 9.1. Flujo completo de un alta: HTTP → controlador → DTO → Bean Validation → servicio → repositorio → Hibernate → PostgreSQL, y la vuelta hasta la respuesta 201.

Algunos puntos de la figura merecen un comentario:

- **Paso 4.** Si la validación falla, el flujo salta directamente al manejador de errores. No se abre ninguna transacción ni se toca la base de datos.
- **Paso 7.** Con `IDENTITY`, el `INSERT` se ejecuta en este momento y no en el *commit*, porque Hibernate necesita el `id`. PostgreSQL lo devuelve en la misma sentencia.

- **Paso 10.** El DTO se construye antes del *commit*. Si el *commit* fallara (por ejemplo, por una restricción de la base de datos comprobada al final), el cliente recibiría un error y no un 201.
- **Paso 11.** La conexión se devuelve al *pool* al terminar la transacción, antes de serializar la respuesta. Con `open-in-view=false` la conexión no se retiene mientras Jackson escribe el JSON.

9.5 El contrato HTTP del módulo

↑ Índice

Método y ruta	Éxito	Errores posibles
`GET /api/applications?status&criticality&page&size&sort`	200 + página	400 (a partir del paso 2.9)
`GET /api/applications/{id}`	200 + aplicación	404, 400 si el `id` no es numérico
`POST /api/applications`	201 + `Location` + aplicación	400 por validación o JSON mal formado
`PUT /api/applications/{id}`	200 + aplicación	400, 404

No hay `DELETE`: las aplicaciones no se borran, se marcan como `INACTIVE`. Es una decisión de negocio que conserva el histórico y que, más adelante, evitó un problema de integridad cuando las demandas empezaron a referenciar aplicaciones.

> **En el proyecto**
>
> Este contrato es exactamente el que consume el frontend del proyecto a través del proxy de desarrollo de Vite, que reenvía `/api` al puerto 8080. La guía del frontend describe el otro lado: el cliente HTTP, los tipos de TypeScript alineados con estos DTO y el manejo del formato de error del capítulo 10.

9.6 Errores típicos y depuración

↑ Índice

Síntoma	Causa probable	Cómo resolverlo
`415 Unsupported Media Type` en un `POST`	Falta la cabecera `Content-Type: application/json`	Enviarla
La primera página parece la segunda	El cliente numera las páginas desde 1	`page` empieza en 0
El orden cambia entre peticiones idénticas	No hay criterio de orden estable	Mantener `sort=id` como orden por defecto o secundario
500 al ordenar por un campo	La propiedad no existe en la entidad	Ver la validación de campos de ordenación (capítulo 10)
La respuesta tiene campos `pageable`, `first`, `last`…	Se devolvió un `Page` directamente	Copiarlo a un DTO de página

9.7 Ejercicios

↑ Índice

Ejercicio 9.1 · Explorar el `Pageable` Ver solución →

Pide `GET /api/applications?page=0&size=1&sort=name,desc` y después `GET /api/applications?page=1&size=1&sort=name,desc`. Comprueba que entre las dos respuestas aparecen las dos aplicaciones iniciales en el orden esperado y que `totalPages` vale 2.

Ejercicio 9.2 · Seguir el flujo con el depurador Ver solución →

Pon puntos de ruptura en `ApplicationController.createApplication`, en `ApplicationService.createApplication` y en `toResponse`. Envía un alta y, en cada parada, observa en la pila de llamadas las clases de Spring que hay entre el controlador y el servicio. Identifica el proxy transaccional.

Autoevaluación del capítulo 9 Ver respuestas →

1. ¿Qué diferencia hay entre `@Controller` y `@RestController`?
2. ¿Qué tres parámetros de la URL lee el resolvedor de `Pageable` y desde qué número empiezan las páginas?
3. ¿Qué pasaría con el orden de los resultados si se quitara `@PageableDefault`?
4. ¿Por qué el proyecto no devuelve directamente el `Page` de Spring Data?
5. En el flujo de un alta, ¿en qué momento se ejecuta el `INSERT` y en cuál se devuelve la conexión al *pool*?

CAPÍTULO 10

El tratamiento de errores

↑ Volver al índice

EN ESTE CAPÍTULO

10.1 Un formato de error común
10.2 Cómo encuentra Spring el manejador
10.3 Rechazar filtros y ordenaciones desconocidos
10.4 Tipos incorrectos y cuerpos mal formados
10.5 Lo que todavía no está cubierto
10.6 Errores típicos y depuración
10.7 Ejercicios

Un cliente necesita distinguir entre «has enviado algo mal», «eso no existe» y «algo ha fallado en el servidor», y necesita recibirlo siempre con la misma forma. Este capítulo explica el formato de error del proyecto, cómo Spring elige el manejador de cada excepción y cómo el tratamiento se fue ampliando.

10.1 Un formato de error común

Paso 2.8 ↖ ↑ Índice

Todas las respuestas de error del proyecto tienen la misma estructura:

exception/ApiErrorResponse.java IMPLEMENTADO

```
public record ApiErrorResponse(
        int status,
        String error,
        String message,
        Map<String, String> fieldErrors
) {
}
```

Campo	Contenido
`status`	El código HTTP, repetido en el cuerpo para comodidad del cliente
`error`	Una categoría estable: `VALIDATION_ERROR` o `RESOURCE_ERROR`
`message`	Un texto legible
`fieldErrors`	Mapa campo → mensaje; vacío si el error no es de campos

La primera versión del manejador global trataba dos casos:

exception/GlobalExceptionHandler.java (versión de GitHub) IMPLEMENTADO

```
@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<ApiErrorResponse> handleValidation(
            MethodArgumentNotValidException ex) {

        Map<String, String> fieldErrors = new LinkedHashMap<>();
```

```
        ex.getBindingResult().getFieldErrors().forEach(error ->
            fieldErrors.putIfAbsent(
                error.getField(),
                error.getDefaultMessage()
            )
        );

        ApiErrorResponse body = new ApiErrorResponse(
            400,
            "VALIDATION_ERROR",
            "La petición contiene campos inválidos",
            fieldErrors
        );

        return ResponseEntity.badRequest().body(body);
    }

    @ExceptionHandler(ResponseStatusException.class)
    public ResponseEntity<ApiErrorResponse> handleResponseStatus(
            ResponseStatusException ex) {

        ApiErrorResponse body = new ApiErrorResponse(
            ex.getStatusCode().value(),
            "RESOURCE_ERROR",
            ex.getReason(),
            Map.of()
        );

        return ResponseEntity
                .status(ex.getStatusCode())
                .body(body);
    }
}
```

El primer manejador recoge los errores de Bean Validation. Recorre los errores de campo y guarda **uno por campo** (`putIfAbsent` conserva el primero), en un `LinkedHashMap` para mantener el orden. Si un campo viola dos restricciones, como un nombre vacío que incumple `@NotBlank`, el cliente recibe un único mensaje para ese campo.

El segundo traduce cualquier `ResponseStatusException` lanzada por el servicio: toma su código HTTP y su motivo (`getReason()`), que es el texto pasado al constructor.

400: petición inválida

```
{
  "status": 400,
  "error": "VALIDATION_ERROR",
  "message": "La petición contiene campos inválidos",
  "fieldErrors": {
    "name": "no debe estar vacío",
    "criticality": "debe coincidir con \"LOW|MEDIUM|HIGH|CRITICAL\""
  }
}
```

404: recurso inexistente

```
{
  "status": 404,
  "error": "RESOURCE_ERROR",
  "message": "Application not found: 999",
```

```
  "fieldErrors": {}
}
```

> **Nota: El idioma de los mensajes de campo**
>
> Los mensajes de `fieldErrors` son los mensajes por defecto de Hibernate Validator, que se eligen según el idioma de la petición o de la JVM: en un equipo configurado en español aparecen en español. El resto de los mensajes del proyecto están escritos en inglés en el código, salvo el texto general de validación, que está en español. El frontend no muestra estos textos directamente, sino que usa `status`, `error` y los nombres de los campos, así que la mezcla no afecta al usuario, pero es una incoherencia a corregir.

10.2 Cómo encuentra Spring el manejador

↑ Índice

`@RestControllerAdvice` marca una clase cuyos métodos `@ExceptionHandler` se aplican a **todos** los controladores. Cuando un controlador (o algo que se ejecuta por él, como la validación o el servicio) lanza una excepción, Spring MVC la intercepta y busca un manejador:

1. Primero, métodos `@ExceptionHandler` en el propio controlador.
2. Después, en las clases `@ControllerAdvice` y `@RestControllerAdvice`.
3. Entre los candidatos, elige el que declara **el tipo más específico** de la jerarquía de la excepción.
4. Si no hay ninguno, la excepción sigue su camino: Spring aplica sus resolutores por defecto y, en último término, Spring Boot responde con su propio formato de error.

> **Si vienes de Java EE...**
>
> El equivalente en JAX-RS es un `ExceptionMapper<T>` registrado como `@Provider`. La idea es idéntica: un único punto que traduce excepciones a respuestas HTTP. La diferencia es que en Spring un mismo *advice* agrupa varios manejadores como métodos.

10.3 Rechazar filtros y ordenaciones desconocidos

Paso 2.9 ↖ ↑ Índice

En la primera versión, un filtro inventado devolvía una página vacía y un campo de ordenación inexistente provocaba un error 500. Más adelante el servicio empezó a validar sus parámetros de consulta:

`application/service/ApplicationService.java (extracto)` **AÑADIDO DESPUÉS**

```
private static final Set<String> VALID_STATUSES = Set.of(
        "ACTIVE",
        "INACTIVE"
);

private static final Set<String> VALID_CRITICALITIES = Set.of(
        "LOW",
        "MEDIUM",
        "HIGH",
        "CRITICAL"
);

private static final Set<String> VALID_SORT_FIELDS = Set.of(
        "id",
        "name",
        "criticality",
```

```
        "status"
);

@Transactional(readOnly = true)
public ApplicationPageResponse getApplications(
        String status,
        String criticality,
        Pageable pageable) {

    Page<Application> applications;

    validateStatus(status);
    validateCriticality(criticality);
    validateSort(pageable.getSort());

    // … mismas cuatro ramas que en la primera versión
}

private void validateSort(Sort sort) {
    for (Sort.Order order : sort) {
        if (!VALID_SORT_FIELDS.contains(order.getProperty())) {
            throw new ResponseStatusException(
                    HttpStatus.BAD_REQUEST,
                    "Invalid application sort field: " + order.getProperty()
            );
        }
    }
}
```

`validateStatus` y `validateCriticality` siguen el mismo patrón, con los mensajes `Invalid application status: X` e `Invalid application criticality: X`. Como lanzan `ResponseStatusException`, el manejador existente ya los convierte en respuestas 400 sin tocarlo.

Código reconstruido

Este fragmento procede del código final del módulo de aplicaciones, donde se conserva sin cambios (capítulo 19), porque la versión del monolito en la que se introdujo la validación no se conserva. Lo seguro es que la copia publicada en GitHub no la tiene y que el servicio de demandas del monolito sí validaba sus filtros con este mismo patrón (capítulo 13). Por eso se marca como añadido después, sin fecha exacta.

Decisión de diseño: Lista blanca de campos de ordenación

Motivo: el cliente solo puede ordenar por campos que la API ofrece de forma deliberada. Un campo desconocido es un error del cliente (400), no del servidor (500), y el mensaje dice cuál es. Además, la lista evita que la API dependa de los nombres internos de la entidad. **Coste:** la lista debe mantenerse cuando cambian los campos ordenables, y el frontend debe conocerla.

10.4 Tipos incorrectos y cuerpos mal formados

Paso 2.10 ↖ ↑ Índice

Dos errores de cliente no los cubrían los manejadores iniciales: un parámetro que no se puede convertir al tipo esperado (`/api/applications/abc`, cuando `id` es `Long`) y un cuerpo JSON que no se puede leer (una coma de más, un texto donde se esperaba un objeto). Se añadieron dos manejadores:

exception/GlobalExceptionHandler.java (añadidos) AÑADIDO DESPUÉS

```
@ExceptionHandler(MethodArgumentTypeMismatchException.class)
public ResponseEntity<ApiErrorResponse> handleTypeMismatch(
        MethodArgumentTypeMismatchException ex) {

    ApiErrorResponse body = new ApiErrorResponse(
            400,
            "VALIDATION_ERROR",
            "Invalid value for parameter: " + ex.getName(),
            Map.of()
    );

    return ResponseEntity.badRequest().body(body);
}

@ExceptionHandler(HttpMessageNotReadableException.class)
public ResponseEntity<ApiErrorResponse> handleUnreadableMessage(
        HttpMessageNotReadableException ex) {

    ApiErrorResponse body = new ApiErrorResponse(
            400,
            "VALIDATION_ERROR",
            "Malformed request body",
            Map.of()
    );

    return ResponseEntity.badRequest().body(body);
}
```

Sin ellos, Spring ya respondía con un 400, pero con el formato de error por defecto de Spring Boot, distinto del del proyecto. El frontend habría tenido que entender dos formatos. Ahora ambos casos llegan como `VALIDATION_ERROR`.

Observa que el mensaje del cuerpo mal formado es genérico: no incluye `ex.getMessage()`, que contendría detalles internos del analizador JSON y nombres de clases Java. No conviene exponer esa información a un cliente.

10.5 Lo que todavía no está cubierto

↑ Índice

El manejador cubre los casos habituales, pero no todos. Estas excepciones siguen llegando al cliente con el formato por defecto de Spring Boot (`timestamp`, `status`, `error`, `path`) o, peor, como un 500:

Situación	Excepción	Respuesta actual
Falta un parámetro obligatorio	`MissingServletRequestParameterException`	400 con formato de Spring Boot
Verbo no permitido (`DELETE`)	`HttpRequestMethodNotSupportedException`	405 con formato de Spring Boot
Violación de una restricción de la base de datos	`DataIntegrityViolationException`	500
Cualquier error inesperado	`Exception`	500 con formato de Spring Boot

Pendiente: Cerrar el tratamiento de errores

Añadir un manejador de último recurso para `Exception` que registre el error completo en el *log* y devuelva un 500 con el formato del proyecto y un mensaje genérico. Valorar también el estándar **Problem Details** (RFC 9457), que Spring soporta con la clase `ProblemDetail` y la propiedad `spring.mvc.problemdetails.enabled`. Adoptarlo cambiaría el contrato con el frontend, así que debería hacerse de forma coordinada.

Resuelto en la fase 7 (capítulo 25): manejador de último recurso y manejador de parámetros ausentes. Problem Details sigue siendo una opción futura.

10.6 Errores típicos y depuración

↑ Índice

Síntoma	Causa probable	Cómo resolverlo
Un error llega con `timestamp` y `path` en lugar de `fieldErrors`	Ningún manejador del proyecto cubre esa excepción	Añadir el manejador o revisar el tipo exacto
`message` es `null` en un 404	La `ResponseStatusException` se creó sin motivo	Pasar siempre el texto como segundo argumento
El manejador no se ejecuta	La clase no está en un paquete escaneado	Comprobar que cuelga del paquete raíz
Se exponen trazas o nombres de clases	Se copió `ex.getMessage()` al cuerpo	Usar mensajes propios y registrar el detalle en el *log*

10.7 Ejercicios

↑ Índice

Ejercicio 10.1 · Recorrer todos los formatos Ver solución →

Provoca, con peticiones manuales, cada uno de estos casos y anota la respuesta: un alta con nombre vacío, un detalle con `id` inexistente, un detalle con `id` no numérico, un alta con JSON mal formado y un listado con `sort=pepito`. Clasifica cada respuesta por su campo `error`.

Ejercicio 10.2 · Diseñar el manejador de último recurso Ver solución →

Escribe un método `@ExceptionHandler(Exception.class)` para `GlobalExceptionHandler` que devuelva un 500 con el formato del proyecto, el mensaje `Unexpected error` y que registre la excepción completa con un *logger*. Explica por qué no interfiere con los manejadores más específicos.

Autoevaluación del capítulo 10 Ver respuestas →

1. ¿Qué significan las categorías `VALIDATION_ERROR` y `RESOURCE_ERROR`?
2. ¿Por qué el manejador de validación usa `putIfAbsent` y un `LinkedHashMap`?
3. Si dos métodos `@ExceptionHandler` pueden tratar una excepción, ¿cuál elige Spring?
4. ¿Qué devolvía la primera versión ante `sort=pepito`, y qué devuelve después del paso 2.9?
5. ¿Por qué el manejador de cuerpos mal formados no incluye el mensaje de la excepción?

CAPÍTULO 11

Las pruebas del módulo Application

↑ Volver al índice

EN ESTE CAPÍTULO

11.1 Tres niveles de prueba
11.2 Probar el servicio con Mockito
11.3 Probar el controlador con `@WebMvcTest` y MockMvc
11.4 Ejecutar las pruebas
11.5 Errores típicos y depuración
11.6 Ejercicios

El módulo de aplicaciones cierra su fase con pruebas automáticas de dos tipos: pruebas unitarias del servicio, que no arrancan Spring, y pruebas de la capa web, que arrancan solo los controladores. Este capítulo explica las dos, qué garantiza cada una y qué queda fuera de su alcance.

11.1 Tres niveles de prueba

↑ Índice

Nivel	Qué arranca	Velocidad	Qué demuestra	En el proyecto
Unitaria	Nada: `new` y *mocks*	Milisegundos	La lógica de una clase aislada	`ApplicationServiceTest`, `DemandTest`
De capa (*slice*)	Una parte de Spring: la capa web	Pocos segundos	El contrato HTTP: rutas, JSON, validación, códigos	`ApplicationControllerTest`
De integración	Toda la aplicación y un PostgreSQL real	Decenas de segundos	Que las capas funcionan juntas: SQL, Flyway, transacciones	Capítulo 16

Cuanto más abajo en la tabla, más realista y más lenta es la prueba. Lo razonable es tener muchas pruebas rápidas para la lógica y unas pocas de integración para lo que solo se puede comprobar con todo montado.

11.1.1 El punto de partida

La versión publicada en GitHub solo tiene la prueba que genera Spring Initializr:

`ItServiceManagerApplicationTests.java (versión de GitHub)` IMPLEMENTADO

```
@SpringBootTest
class ItServiceManagerApplicationTests {

    @Test
    void contextLoads() {
    }
}
```

Parece trivial, pero `@SpringBootTest` arranca el contexto completo: conecta con la base de datos, ejecuta Flyway y valida el esquema. Por eso necesita un PostgreSQL configurado, y la guía de instalación del repo-

sitorio advierte que solo debe ejecutarse contra una base de datos de pruebas. Es una prueba de integración disfrazada, y depende del entorno de quien la ejecuta. El capítulo 16 explica cómo se resolvió.

Código reconstruido

Las pruebas que siguen se muestran tal como están en el código final del módulo de aplicaciones, donde se conservaron al extraerlo (capítulo 19). El documento de verificación del refactor confirma que el monolito ya tenía una batería de pruebas antes de la fase 5, pero no se conserva su versión exacta. Lo único que cambia respecto al monolito es el paquete.

11.2 Probar el servicio con Mockito

Paso 2.11 ↖ ↑ Índice

`application/service/ApplicationServiceTest.java (extracto)` ESTADO FINAL

```
@ExtendWith(MockitoExtension.class)
class ApplicationServiceTest {

    @Mock
    private ApplicationRepository applicationRepository;

    private ApplicationService applicationService;

    @BeforeEach
    void setUp() {
        applicationService = new ApplicationService(applicationRepository);
    }

    @Test
    void shouldThrowNotFoundWhenApplicationDoesNotExist() {

        when(applicationRepository.findById(999L))
                .thenReturn(Optional.empty());

        ResponseStatusException exception =
                assertThrows(
                        ResponseStatusException.class,
                        () -> applicationService.getApplicationById(999L)
                );

        assertEquals(HttpStatus.NOT_FOUND, exception.getStatusCode());
        assertEquals(
                "Application not found: 999",
                exception.getReason()
        );
    }
}
```

La prueba no arranca Spring. `@ExtendWith(MockitoExtension.class)` activa Mockito en JUnit 5, `@Mock` crea un doble del repositorio y el servicio se construye con `new`, gracias a la inyección por constructor (capítulo 2). El *mock* devuelve lo que se le indica con `when(...).thenReturn(...)`, y la prueba comprueba el comportamiento del servicio: el código 404 y el mensaje exacto.

11.2.1 Probar el alta sin base de datos

application/service/ApplicationServiceTest.java (extracto) ESTADO FINAL

```
@Test
void shouldCreateApplicationWithActiveStatus() {

    ApplicationCreateRequest request =
            new ApplicationCreateRequest(
                    "TEST_APP",
                    "Aplicación creada desde test",
                    "HIGH"
            );

    when(applicationRepository.save(
            org.mockito.ArgumentMatchers.any(Application.class)))
            .thenAnswer(invocation -> invocation.getArgument(0));

    ApplicationResponse response =
            applicationService.createApplication(request);

    assertEquals("TEST_APP", response.name());
    assertEquals("HIGH", response.criticality());
    assertEquals("ACTIVE", response.status());

    verify(applicationRepository).save(
            org.mockito.ArgumentMatchers.any(Application.class));
}
```

`thenAnswer` permite devolver algo calculado a partir de la llamada: aquí, la misma entidad que recibió `save`. Así la prueba puede comprobar que el servicio construyó la entidad con el estado `ACTIVE`, aunque el DTO de entrada no lo traía. `verify` comprueba que `save` se llamó exactamente una vez.

11.2.2 Probar que *no* se llama a save

application/service/ApplicationServiceTest.java (extracto) ESTADO FINAL

```
@Test
void shouldUpdateExistingApplicationWithoutCallingSave() {

    Application application = new Application(
            "TEST_APP",
            "Descripción original",
            "LOW",
            "ACTIVE"
    );

    when(applicationRepository.findById(1L))
            .thenReturn(Optional.of(application));

    ApplicationUpdateRequest request =
            new ApplicationUpdateRequest(
                    "TEST_APP_UPDATED",
                    "Descripción modificada",
                    "CRITICAL",
                    "INACTIVE"
            );

    ApplicationResponse response =
            applicationService.updateApplication(1L, request);

    assertEquals("TEST_APP_UPDATED", response.name());
```

```
    assertEquals("INACTIVE", response.status());

    verify(applicationRepository, never())
            .save(any(Application.class));
}
```

Esta prueba fija por escrito la decisión del capítulo 7: la modificación se apoya en el *dirty checking*. Si alguien añade un `save` innecesario, la prueba falla y obliga a reconsiderarlo.

> **Cuidado: Lo que esta prueba no puede demostrar**
>
> Sin Spring ni base de datos no hay transacción ni contexto de persistencia: la prueba demuestra que la entidad cambia en memoria, **no que el cambio llegue a PostgreSQL**. Eso solo lo puede comprobar una prueba de integración, que el proyecto incorpora más adelante con `flush` y una consulta posterior.

El resto de pruebas del servicio cubre la búsqueda correcta, la modificación de un `id` inexistente y, una vez añadida la validación del paso 2.9, los filtros inválidos y un campo de ordenación inventado.

11.3 Probar el controlador con `@WebMvcTest` y MockMvc

↑ Índice

`application/controller/ApplicationControllerTest.java (extracto)` ESTADO FINAL

```
@WebMvcTest(ApplicationController.class)
class ApplicationControllerTest {

    @Autowired
    private MockMvc mockMvc;

    @MockitoBean
    private ApplicationService applicationService;

    @Test
    void shouldCreateApplication() throws Exception {

        ApplicationResponse response =
                new ApplicationResponse(
                        5L,
                        "TEST_APP",
                        "Aplicación creada desde test MVC",
                        "HIGH",
                        "ACTIVE"
                );

        when(applicationService.createApplication(
                any(ApplicationCreateRequest.class)))
                .thenReturn(response);

        String requestBody = """
                {
                  "name": "TEST_APP",
                  "description": "Aplicación creada desde test MVC",
                  "criticality": "HIGH"
                }
                """;

        mockMvc.perform(post("/api/applications")
                        .contentType(MediaType.APPLICATION_JSON)
                        .content(requestBody))
```

```
                .andExpect(status().isCreated())
                .andExpect(header().string(
                        "Location",
                        "/api/applications/5"))
                .andExpect(jsonPath("$.id").value(5))
                .andExpect(jsonPath("$.status").value("ACTIVE"));
    }
}
```

`@WebMvcTest(ApplicationController.class)` arranca un contexto de Spring reducido: el controlador indicado, los `@RestControllerAdvice`, Jackson, la validación y la configuración de Spring MVC. **No** carga servicios, repositorios ni la base de datos. En Spring Boot 4, esta anotación la aporta `spring-boot-starter-webmvc-test`.

`@MockitoBean` registra un *mock* de Mockito como *bean* del contexto, en lugar del servicio real que no se ha cargado. Sustituye a `@MockBean`, que Spring Boot marcó como obsoleto en la versión 3.4 y que ya no existe en Spring Boot 4.

`MockMvc` ejecuta peticiones contra el `DispatcherServlet` **sin abrir un puerto de red**. La petición recorre los mismos pasos que una real (resolución de la ruta, Jackson, validación, manejadores de errores), pero en memoria.

`jsonPath("$.id")` consulta el cuerpo JSON de la respuesta con expresiones JsonPath.

11.3.1 La validación, comprobada desde fuera

`application/controller/ApplicationControllerTest.java (extracto)` ESTADO FINAL

```
@Test
void shouldReturnBadRequestWhenCreatingInvalidApplication() throws Exception {

    String requestBody = """
            {
              "name": " ",
              "description": "Aplicación inválida",
              "criticality": "URGENT"
            }
            """;

    mockMvc.perform(post("/api/applications")
                    .contentType(MediaType.APPLICATION_JSON)
                    .content(requestBody))
            .andExpect(status().isBadRequest())
            .andExpect(jsonPath("$.error").value("VALIDATION_ERROR"))
            .andExpect(jsonPath("$.fieldErrors.name").exists())
            .andExpect(jsonPath("$.fieldErrors.criticality").exists());

    verify(applicationService, never())
            .createApplication(any(ApplicationCreateRequest.class));
}
```

Esta prueba ejerce tres capas reales a la vez: Bean Validation, el `GlobalExceptionHandler` y el formato `ApiErrorResponse`. Y comprueba lo más importante: que una petición inválida **nunca llega al servicio**.

Decisión de diseño: Pruebas de capa web con *mocks* del servicio

Motivo: el contrato HTTP (rutas, códigos, cabeceras, formato de error) se prueba en segundos y sin base de datos, y cada prueba falla por una sola razón. **Alternativa:** probarlo todo con `@SpringBootTest` y un servidor real, más realista pero mucho más lento y frágil. **Complemento:** las pruebas de integración con PostgreSQL real del capítulo 16.

11.4 Ejecutar las pruebas

↑ Índice

```
.\mvnw.cmd test                                      # todas
.\mvnw.cmd test -Dtest=ApplicationServiceTest        # una clase
.\mvnw.cmd test -Dtest="ApplicationServiceTest#shouldCreate*"   # métodos por patrón
```

Maven ejecuta las pruebas con el *plugin* Surefire, que por defecto busca clases cuyo nombre termina en `Test` o `Tests`. Una clase llamada `PruebasServicio` no se ejecutaría sin configuración adicional.

11.5 Errores típicos y depuración

↑ Índice

Síntoma	Causa probable	Cómo resolverlo
`NoSuchBeanDefinitionException: ApplicationService` en un `@WebMvcTest`	Falta el `@MockitoBean` del servicio	Declararlo en la prueba
`UnnecessaryStubbingException`	Se preparó un `when(...)` que la prueba nunca usa	Eliminarlo; Mockito estricto avisa de preparación sobrante
`contextLoads` falla en otra máquina	Necesita una base de datos configurada	Usar Testcontainers (capítulo 16)
`jsonPath` no encuentra el campo	El JSON no tiene la forma esperada	Añadir `.andDo(print())` para ver la respuesta completa
Una clase de pruebas no se ejecuta	Su nombre no termina en `Test` o `Tests`	Renombrarla

11.6 Ejercicios

↑ Índice

Ejercicio 11.1 · Una prueba nueva de servicio Ver solución →

Escribe una prueba unitaria que compruebe que `updateApplication` lanza un 404 con el mensaje `Application not found: 42` cuando el repositorio no encuentra el `id` 42, y que en ese caso tampoco se llama a `save`.

Ejercicio 11.2 · Probar la cabecera de un error Ver solución →

Escribe una prueba `@WebMvcTest` que haga `GET /api/applications/abc` y compruebe que la respuesta es 400, con `error` igual a `VALIDATION_ERROR` y un mensaje que nombra el parámetro `id`. ¿Hace falta preparar algún comportamiento del *mock* del servicio? ¿Por qué?

Autoevaluación del capítulo 11 Ver respuestas →

1. ¿Por qué `contextLoads` no es realmente una prueba unitaria?
2. ¿Qué hace `thenAnswer(invocation -> invocation.getArgument(0))` y por qué se usa en la prueba del alta?
3. ¿Qué carga y qué no carga `@WebMvcTest`?
4. ¿Qué sustituye `@MockitoBean` y por qué es necesario en las pruebas de controlador?
5. ¿Qué no puede demostrar la prueba `shouldUpdateExistingApplicationWithoutCallingSave`?

FASE 3

El módulo Demand

12 El modelo de la demanda pág. 79

13 Consultas dinámicas con `Specification` pág. 87

14 Carga perezosa y el problema N+1 pág. 93

15 Servicio, ciclo de vida y API de demandas pág. 98

↑ Volver al índice

CAPÍTULO 12

El modelo de la demanda

↑ Volver al índice

EN ESTE CAPÍTULO

12.1 Qué es una demanda
12.2 La migración V3 crea `demands`
12.3 La entidad `Demand` en el monolito
12.4 Las fechas de creación y modificación
12.5 El ciclo de vida en `changeStatus`
12.6 Errores típicos y depuración
12.7 Ejercicios

El segundo módulo introduce dos cosas que el primero no tenía: una relación con otra entidad y reglas de negocio que dependen del estado. Este capítulo explica la tabla `demands`, la entidad tal como era en el monolito, sus fechas y la máquina de estados que gobierna su ciclo de vida.

12.1 Qué es una demanda

↑ Índice

Una demanda es una petición de trabajo sobre una aplicación del catálogo: una mejora, una adaptación normativa, un cambio. Tiene título, descripción opcional, prioridad, estado y, siempre, una aplicación a la que pertenece. A diferencia de una aplicación, una demanda **evoluciona**: se abre, se trabaja en ella, se completa o se cancela, y no todos los cambios de estado tienen sentido.

Atributo	Tipo	Reglas
`title`	Texto, máx. 150	Obligatorio
`description`	Texto, máx. 1000	Opcional
`priority`	`LOW`, `MEDIUM`, `HIGH`, `CRITICAL`	Obligatoria
`status`	`OPEN`, `IN_PROGRESS`, `COMPLETED`, `CANCELLED`	Lo fija el servidor al crear; cambia según reglas
aplicación	Referencia a `applications`	Obligatoria y debe existir
`createdAt`, `updatedAt`	Instantes en UTC	Los fija el servidor

12.2 La migración V3 crea `demands`

Paso 3.1 ↖ ↑ Índice

`V3__create_demands.sql` IMPLEMENTADO

```
CREATE TABLE demands (
    id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,

    title VARCHAR(150) NOT NULL,
    description VARCHAR(1000),

    priority VARCHAR(20) NOT NULL,
    status VARCHAR(20) NOT NULL,
```

```
    application_id BIGINT NOT NULL,

    created_at TIMESTAMP WITH TIME ZONE NOT NULL,
    updated_at TIMESTAMP WITH TIME ZONE NOT NULL,

    CONSTRAINT fk_demands_application
        FOREIGN KEY (application_id)
        REFERENCES applications(id)
);

CREATE INDEX idx_demands_application_id
    ON demands(application_id);

CREATE INDEX idx_demands_status
    ON demands(status);

CREATE INDEX idx_demands_priority
    ON demands(priority);
```

La clave foránea. `fk_demands_application` garantiza en la base de datos que toda demanda apunta a una aplicación existente. Su nombre explícito no es un capricho: para eliminarla algún día con `ALTER TABLE … DROP CONSTRAINT` hay que conocerlo, y si PostgreSQL lo hubiera generado, dependería de sus reglas internas. No tiene `ON DELETE`, así que PostgreSQL impide borrar una aplicación con demandas, lo cual encaja con la decisión de no borrar aplicaciones (capítulo 9).

Los índices. PostgreSQL crea automáticamente un índice para la clave primaria, pero **no** para las claves foráneas. El índice sobre `application_id` acelera el filtro «demandas de esta aplicación», que el frontend usa en la ficha de cada aplicación, y las comprobaciones de integridad. Los índices sobre `status` y `priority` corresponden a los otros dos filtros del listado.

`TIMESTAMP WITH TIME ZONE`. A pesar del nombre, PostgreSQL no guarda la zona horaria: convierte el valor a UTC al escribir y lo devuelve en la zona de la sesión al leer. Es decir, guarda **un instante** en el tiempo, sin ambigüedad. Hibernate 6 y posteriores asocian por defecto el tipo `java.time.Instant` a este tipo de columna, de modo que `ddl-auto=validate` los acepta como compatibles.

> **Decisión de diseño: Fechas como instantes en UTC**
>
> **Motivo:** un instante no depende de dónde esté el servidor ni el usuario; el frontend lo muestra en la hora local del navegador. **Alternativa descartada:** `TIMESTAMP` sin zona con `LocalDateTime`, que guarda una fecha y hora «de pared» sin referencia: dos servidores en zonas distintas, o un cambio de horario de verano, producen valores ambiguos.

12.3 La entidad `Demand` en el monolito

Paso 3.2 ↖ ↑ Índice

En el monolito, la demanda se relacionaba con la aplicación a través de una **asociación JPA**: la entidad tenía un atributo de tipo `Application`, no un número.

`demand/entity/Demand.java (versión del monolito)` RECONSTRUIDO

```
@Entity
@Table(name = "demands")
public class Demand {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
```

```
    private Long id;

    @Column(nullable = false, length = 150)
    private String title;

    @Column(length = 1000)
    private String description;

    @Column(nullable = false, length = 20)
    private String priority;

    @Column(nullable = false, length = 20)
    private String status;

    @ManyToOne(fetch = FetchType.LAZY, optional = false)
    @JoinColumn(name = "application_id", nullable = false)
    private Application application;

    @Column(name = "created_at", nullable = false)
    private Instant createdAt;

    @Column(name = "updated_at", nullable = false)
    private Instant updatedAt;

    protected Demand() {
        // Constructor requerido por JPA
    }

    public Demand(String title, String description, String priority,
                  String status, Application application) {
        this.title = title;
        this.description = description;
        this.priority = priority;
        this.status = status;
        this.application = application;
    }

    public void update(String title, String description, String priority,
                       Application application) {
        this.title = title;
        this.description = description;
        this.priority = priority;
        this.application = application;
        this.updatedAt = Instant.now();
    }

    // prePersist, changeStatus y getters: igual que en la versión final
}
```

Código reconstruido

Esta versión ya no existe: la fase 5 sustituyó la asociación por un identificador (capítulo 18). Se reconstruye a partir de tres fuentes del propio proyecto: el documento `frontera-catalogo.md`, que dice que tras el refactor «la entidad ya no contiene una asociación JPA hacia Application» y que «las pruebas de dominio usan identificadores»; la mención a una prueba que verificaba un `EntityGraph` sobre esa asociación; y la entidad final, que conserva el resto de atributos y métodos. Los detalles de anotación (`LAZY`, `optional = false`) son los que exige el diseño descrito en el capítulo 14, pero no pueden verificarse literalmente.

12.3.1 Qué significa cada anotación de la asociación

`@ManyToOne`. Muchas demandas pertenecen a una aplicación. En la tabla es la columna `application_id`; en Java, una referencia al objeto.

`@JoinColumn(name = "application_id")`. Indica qué columna contiene la clave foránea.

`fetch = FetchType.LAZY`. En JPA, las asociaciones `@ManyToOne` son **`EAGER` por defecto**: al cargar una demanda, Hibernate carga también su aplicación, siempre, se necesite o no. Con `LAZY`, Hibernate coloca en el atributo un **proxy**: un objeto de una subclase generada de `Application` que solo conoce el `id`. La primera vez que se llama a un método como `getName()`, el proxy ejecuta el `SELECT` de la aplicación. Esto exige que el contexto de persistencia siga abierto; con `open-in-view=false`, fuera de la transacción ese acceso lanza `LazyInitializationException`.

`optional = false`. Informa a Hibernate de que la asociación nunca es nula, lo que le permite usar uniones internas en lugar de externas al consultarla.

> **Cuidado: `EAGER` por defecto es una trampa**
>
> Dejar `@ManyToOne` con su valor por defecto parece cómodo, pero hace que cualquier consulta de demandas arrastre consultas adicionales de aplicaciones, también cuando no se van a usar. La práctica recomendada es declarar todas las asociaciones `LAZY` y decidir en cada consulta qué se carga, que es lo que el proyecto hizo con `@EntityGraph` (capítulo 14).

12.4 Las fechas de creación y modificación

↑ Índice

La versión final de la entidad, que en esta parte coincide con la del monolito, gestiona las fechas así:

`demand/entity/Demand.java (extracto)` **IMPLEMENTADO**

```
@PrePersist
private void prePersist() {
    Instant now = Instant.now();
    this.createdAt = now;
    this.updatedAt = now;
}
```

`@PrePersist` es una **retrollamada del ciclo de vida** de JPA: Hibernate ejecuta el método justo antes de insertar la entidad. Puede ser privado. Las dos fechas reciben el mismo instante, de modo que una demanda recién creada tiene `createdAt == updatedAt`.

La fecha de modificación, en cambio, no se gestiona con `@PreUpdate`, sino a mano: `update(...)` y `changeStatus(...)` asignan `updatedAt = Instant.now()`.

> **Decisión de diseño: `updatedAt` explícito en los métodos de negocio**
>
> **Motivo:** la fecha cambia exactamente cuando cambia la demanda a través de una operación de negocio, y la respuesta del servicio ya lleva la fecha nueva, porque se asigna antes de construir el DTO. Con `@PreUpdate`, la fecha se asignaría durante el *flush*, que ocurre **después** de construir la respuesta: el cliente recibiría la fecha antigua. **Alternativas:** `@PreUpdate`; las anotaciones de Hibernate `@CreationTimestamp` y `@UpdateTimestamp`; o la auditoría de Spring Data (`@CreatedDate`, `@LastModifiedDate`). **Coste:** cada nuevo método que modifique la demanda debe acordarse de actualizar la fecha.

Una consecuencia sutil: `update(...)` asigna la fecha aunque los valores recibidos sean idénticos a los actuales. La entidad queda «sucia» por el cambio de `updatedAt` y Hibernate emite un `UPDATE` en cada petición `PUT`, cambie algo o no.

12.5 El ciclo de vida en `changeStatus`

Paso 3.3 ↖ ↑ Índice



Figura 12.1. Máquina de estados de una demanda. Cualquier transición no dibujada se rechaza; repetir el estado actual no es un cambio.

`demand/entity/Demand.java` IMPLEMENTADO

```
public void changeStatus(String newStatus) {

    if (this.status.equals(newStatus)) {
        return;
    }

    boolean validTransition =
            ("OPEN".equals(this.status)
                    && ("IN_PROGRESS".equals(newStatus)
                        || "CANCELLED".equals(newStatus)))
            ||
            ("IN_PROGRESS".equals(this.status)
                    && ("COMPLETED".equals(newStatus)
                        || "CANCELLED".equals(newStatus)));

    if (!validTransition) {
        throw new IllegalStateException(
                "Invalid demand status transition: "
                        + this.status + " -> " + newStatus
        );
    }

    this.status = newStatus;
    this.updatedAt = Instant.now();
}
```

Las reglas son cuatro transiciones válidas, y todo lo demás es inválido:

Desde	Hacia	Resultado
Cualquiera	El mismo estado	Se acepta sin hacer nada
`OPEN`	`IN_PROGRESS` o `CANCELLED`	Se aplica

Desde	Hacia	Resultado
`IN_PROGRESS`	`COMPLETED` o `CANCELLED`	Se aplica
`COMPLETED` o `CANCELLED`	Cualquier otro	`IllegalStateException`
`IN_PROGRESS`	`OPEN`	`IllegalStateException`

Que repetir el estado actual no sea un error tiene una razón práctica: el formulario de modificación envía siempre el estado. Si el usuario solo cambia el título, el estado llega igual que estaba, y eso no debe rechazarse.

Decisión de diseño: La regla vive en la entidad

Motivo: la entidad es el único sitio por el que pasa cualquier cambio de estado, así que la regla no se puede saltar desde ningún servicio. Además se prueba con pruebas unitarias puras, sin *mocks* ni Spring. **Alternativa descartada:** comprobar las transiciones en el servicio, donde cualquier otro código con acceso a la entidad podría cambiarle el estado directamente. **Detalle:** la entidad lanza `IllegalStateException`, una excepción de Java sin relación con HTTP; es el servicio quien la traduce a un 409 (capítulo 15).

12.5.1 Las pruebas del dominio

`demand/entity/DemandTest.java (extracto)` ESTADO FINAL

```
class DemandTest {

    private final Long applicationId = 1L;

    @Test
    void shouldRejectInProgressToOpen() {

        Demand demand = createDemand("IN_PROGRESS");

        assertThrows(
                IllegalStateException.class,
                () -> demand.changeStatus("OPEN")
        );

        assertEquals("IN_PROGRESS", demand.getStatus());
    }

    @Test
    void shouldAllowSameStatus() {

        Demand demand = createDemand("OPEN");

        assertDoesNotThrow(() ->
                demand.changeStatus("OPEN"));

        assertEquals("OPEN", demand.getStatus());
    }

    private Demand createDemand(String status) {
        return new Demand(
                "Demanda test",
                "Descripción de prueba",
                "HIGH",
                status,
                applicationId
        );
```

```
    }
}
```

La clase tiene cinco pruebas: dos transiciones válidas, dos inválidas y la repetición del estado. Cada prueba inválida comprueba también que **el estado no cambió** tras la excepción. En la versión final, la demanda se crea con un identificador de aplicación; en el monolito, con un objeto `Application`, como recoge el documento del refactor.

> **Pendiente: Huecos en las pruebas del ciclo de vida**
>
> Ninguna prueba cubre las transiciones hacia `CANCELLED` ni el rechazo de salidas desde `CANCELLED`. Son reglas tan importantes como las demás. El ejercicio 12.1 propone completarlas.

12.6 Errores típicos y depuración

↑ Índice

Síntoma	Causa probable	Cómo resolverlo
Las fechas aparecen desplazadas una o dos horas	Se mezclaron fechas sin zona (`LocalDateTime`) con instantes	Usar `Instant` en todo el recorrido y convertir a hora local solo al mostrar
Cada demanda cargada dispara un `SELECT` de su aplicación	La asociación se dejó `EAGER` por defecto	Declararla `LAZY` (y ver el capítulo 14)
`LazyInitializationException: could not initialize proxy`	Se accede a la aplicación fuera de la transacción	Construir el DTO dentro del servicio o cargarla con la consulta
`createdAt` es `null` al insertar	Se olvidó `@PrePersist` o el método no se ejecutó	Comprobar la anotación y que se inserta con JPA
Un `PUT` sin cambios genera un `UPDATE`	`update(...)` siempre asigna `updatedAt`	Comportamiento esperado con este diseño

12.7 Ejercicios

↑ Índice

> **Ejercicio 12.1 · Completar las pruebas del ciclo de vida** Ver solución →
>
> Añade a `DemandTest` las pruebas que faltan: `OPEN → CANCELLED` y `IN_PROGRESS → CANCELLED` se aceptan; `CANCELLED → OPEN` y `COMPLETED → CANCELLED` se rechazan sin cambiar el estado. Usa nombres coherentes con los existentes.

> **Ejercicio 12.2 · La fecha en la respuesta** Ver solución →
>
> Explica, paso a paso, qué fecha `updatedAt` recibiría el cliente en la respuesta de una modificación si la entidad usara `@PreUpdate` en lugar de asignarla en `update(...)`. Ayúdate de la figura 9.1.

> **Autoevaluación del capítulo 12** Ver respuestas →
>
> 1. ¿Por qué hace falta crear a mano el índice `idx_demands_application_id`?
> 2. ¿Qué guarda realmente PostgreSQL en una columna `TIMESTAMP WITH TIME ZONE`?

3. ¿Cuál es el modo de carga por defecto de `@ManyToOne` y por qué el monolito lo cambió?
4. ¿Por qué repetir el estado actual no se considera una transición inválida?
5. ¿Por qué la entidad lanza `IllegalStateException` y no una excepción con código HTTP?

CAPÍTULO 13

Consultas dinámicas con `Specification`

↑ Volver al índice

EN ESTE CAPÍTULO

13.1 El problema de las combinaciones
13.2 La Criteria API en pocas palabras
13.3 Filtrar el listado con `Specification`
13.4 Validar estado, prioridad y ordenación
13.5 Errores típicos y depuración
13.6 Ejercicios

El listado de demandas admite tres filtros opcionales: aplicación, estado y prioridad. Con el enfoque del módulo de aplicaciones harían falta ocho métodos de repositorio. Este capítulo explica cómo se resolvió con la Criteria API de JPA envuelta en las `Specification` de Spring Data, y cómo se validan los parámetros del listado.

13.1 El problema de las combinaciones

↑ Índice

En el capítulo 7, dos filtros opcionales producían cuatro ramas `if`, una por combinación. Con tres filtros son ocho, y con cuatro serían dieciséis. Cada combinación exige un método derivado con un nombre cada vez más largo, como `findByApplicationIdAndStatusAndPriority`. El enfoque no escala.

Lo que se necesita es **construir la consulta en tiempo de ejecución**, añadiendo una condición por cada filtro que venga informado. JPA ofrece para eso la Criteria API.

13.2 La Criteria API en pocas palabras

↑ Índice

La Criteria API construye consultas con objetos Java en lugar de con texto JPQL. Tiene cuatro piezas:

Pieza	Qué representa	Equivalente en SQL
`CriteriaBuilder`	La fábrica de expresiones: `equal`, `and`, `like`, `conjunction`…	Los operadores
`CriteriaQuery<T>`	La consulta en construcción	La sentencia `SELECT`
`Root<T>`	La entidad raíz de la consulta	La tabla del `FROM`
`Predicate`	Una condición	Una expresión del `WHERE`

Escrita directamente, una consulta con Criteria es bastante verbosa: hay que obtener el `CriteriaBuilder` del `EntityManager`, crear la consulta, el `Root`, los predicados, ejecutarla y, para paginar, construir aparte la consulta de recuento. Spring Data reduce todo eso a una sola idea: la **`Specification`**.

Spring Data JPA (simplificado)

```
public interface Specification<T> {
    Predicate toPredicate(Root<T> root,
                          CriteriaQuery<?> query,
```

```
                        CriteriaBuilder criteriaBuilder);
}
```

Una `Specification` es **una condición reutilizable**: dada la raíz de la consulta y el constructor de expresiones, devuelve un predicado. Como es una interfaz funcional, se escribe con una *lambda*. Y se puede combinar con `and` y `or` para formar condiciones mayores. Spring Data se encarga de crear la consulta, aplicar la condición, la paginación y la ordenación, y de generar el recuento con la misma condición.

> **Si vienes de Java EE...**
>
> Si usaste la Criteria API en Java EE, todo esto te resultará familiar; la diferencia es que no escribes la consulta completa, solo la condición. Es parecido a componer `Predicate<T>` de `java.util.function`, con la diferencia de que una `Specification` no se evalúa en memoria: se traduce a SQL.

13.3 Filtrar el listado con `Specification`

Paso 3.4 ↖ ↑ Índice

El repositorio declara que acepta especificaciones extendiendo una segunda interfaz:

demand/repository/DemandRepository.java IMPLEMENTADO

```
public interface DemandRepository extends JpaRepository<Demand, Long>,
        JpaSpecificationExecutor<Demand> {

    List<Demand> findByApplicationId(Long applicationId);
}
```

`JpaSpecificationExecutor` añade métodos como `findAll(Specification, Pageable)` y `count(Specification)`. El método derivado `findByApplicationId` lo usan las pruebas de integración.

El servicio construye la condición filtro a filtro. En el monolito, el filtro por aplicación navegaba por la asociación:

demand/service/DemandService.java · getDemands (monolito) RECONSTRUIDO

```
Specification<Demand> specification =
        (root, query, criteriaBuilder) ->
                criteriaBuilder.conjunction();

if (applicationId != null) {
    specification = specification.and(
            (root, query, criteriaBuilder) ->
                    criteriaBuilder.equal(
                            root.get("application").get("id"),
                            applicationId
                    )
    );
}

if (status != null) {
    specification = specification.and(
            (root, query, criteriaBuilder) ->
                    criteriaBuilder.equal(root.get("status"), status)
    );
}

if (priority != null) {
    specification = specification.and(
```

```
            (root, query, criteriaBuilder) ->
                    criteriaBuilder.equal(root.get("priority"), priority)
    );
}

Page<Demand> demands =
        demandRepository.findAll(specification, pageable);
```

Código reconstruido

La estructura (especificación inicial con `conjunction()`, un `and` por filtro, `findAll` paginado) es la del código final. Solo cambia la expresión del filtro por aplicación: en el código final es `root.get("applicationId")`, porque la entidad ya no tiene asociación (capítulo 18). En el monolito, con el atributo `application` de tipo `Application`, la expresión natural es la que se muestra.

Lectura paso a paso:

1. **La especificación inicial** devuelve `criteriaBuilder.conjunction()`, un predicado que siempre es verdadero (una conjunción vacía). Sirve como elemento neutro: si no llega ningún filtro, la consulta devuelve todo; si llegan, cada `and` añade su condición a algo que ya existe. Las versiones recientes de Spring Data ofrecen también `Specification.unrestricted()` para lo mismo.
2. **Cada filtro informado añade un `and`**. Las *lambdas* no se ejecutan en ese momento; se guardan y Spring Data las evaluará al construir la consulta.
3. **`root.get("status")`** obtiene la ruta al atributo `status` de la entidad. Se escribe el nombre del **atributo Java**, no el de la columna.
4. **`root.get("application").get("id")`** navega por la asociación hasta el identificador. Hibernate sabe que el `id` de la aplicación es la propia columna `application_id` de `demands`, así que no necesita unir las tablas.
5. **`findAll(specification, pageable)`** ejecuta la consulta de la página y, cuando hace falta, la de recuento, ambas con la misma condición.

Con estado `OPEN` y prioridad `HIGH`, el SQL resultante es, en esencia:

```
select … from demands d1_0
where 1=1 and d1_0.status = ? and d1_0.priority = ?
order by d1_0.id
offset ? rows fetch first ? rows only
```

Cuidado: Los nombres de atributo son cadenas

`root.get("status")` no se comprueba al compilar: un error de escritura falla al ejecutar la consulta, con `IllegalArgumentException` o una excepción de ruta inválida. Hibernate puede generar un **metamodelo estático** (clases como `Demand_` con atributos `Demand_.status`) que convierte esos nombres en constantes comprobadas por el compilador. El proyecto no lo usa; con tres filtros, las pruebas del listado cubren el riesgo.

Decisión de diseño: `Specification` para los filtros de demandas

Motivo: un único método de repositorio cubre cualquier combinación de filtros, añadir un cuarto filtro es añadir un bloque `if`, y la paginación y el recuento siguen funcionando igual. **Alternativas:** métodos derivados por combinación (no escala); una consulta `@Query` con condiciones del tipo `(:status is null or d.status = :status)`, que es compacta pero da planes de ejecución peores y, en PostgreSQL, problemas al inferir el tipo de un parámetro nulo; o Querydsl, potente pero con generación de código y una dependencia más. **Coste:** algo más de código que un método derivado y nombres de atributo como texto.

13.4 Validar estado, prioridad y ordenación

Paso 3.5 ↖ ↑ Índice

Antes de construir la consulta, el servicio valida sus parámetros, con el mismo patrón de listas blancas que el capítulo 10 describe para las aplicaciones:

`demand/service/DemandService.java (extracto)` IMPLEMENTADO

```
private static final Set<String> VALID_STATUSES = Set.of(
        "OPEN",
        "IN_PROGRESS",
        "COMPLETED",
        "CANCELLED"
);

private static final Set<String> VALID_PRIORITIES = Set.of(
        "LOW",
        "MEDIUM",
        "HIGH",
        "CRITICAL"
);

private void validateStatus(String status) {
    if (status != null && !VALID_STATUSES.contains(status)) {
        throw new ResponseStatusException(
                HttpStatus.BAD_REQUEST,
                "Invalid demand status: " + status
        );
    }
}

private void validateSort(Sort sort) {
    for (Sort.Order order : sort) {
        if (!VALID_SORT_FIELDS.contains(order.getProperty())) {
            throw new ResponseStatusException(
                    HttpStatus.BAD_REQUEST,
                    "Invalid demand sort field: " + order.getProperty()
            );
        }
    }
}
```

El método `getDemands` empieza llamando a `validateStatus`, `validatePriority` y `validateSort`. Un parámetro inválido produce un 400 antes de tocar la base de datos.

13.4.1 La lista de campos de ordenación en el monolito

En el monolito, la lista de campos ordenables incluía uno que no pertenece a la demanda: el **nombre de la aplicación**.

`demand/service/DemandService.java · VALID_SORT_FIELDS (monolito)` RECONSTRUIDO

```
private static final Set<String> VALID_SORT_FIELDS = Set.of(
        "id",
        "title",
        "priority",
        "status",
        "updatedAt",
        "application.name"
);
```

Código reconstruido

El código final tiene los cinco primeros campos. El sexto se deduce del documento `frontera-catalogo.md`, que afirma: «La API existente permite `sort=application.name`», y de una prueba final que comprueba precisamente que ese campo ahora se rechaza (capítulo 21). No es posible verificar si la lista del monolito tenía algún otro campo.

Con `sort=application.name,asc`, Spring Data ve una propiedad anidada y añade a la consulta una **unión** con `applications` para poder ordenar por su columna `name`. La base de datos ordena todo el conjunto filtrado y después pagina. Funciona porque en el monolito las dos tablas están en la misma base de datos. Ese detalle, aparentemente menor, se convertirá en uno de los problemas centrales de la fase 5.

13.5 Errores típicos y depuración

↑ Índice

Síntoma	Causa probable	Cómo resolverlo
`Unable to locate Attribute with the given name [statuss]`	Nombre de atributo mal escrito en `root.get`	Corregirlo; valorar el metamodelo estático
`findAll(spec, pageable)` no existe en el repositorio	Falta extender `JpaSpecificationExecutor`	Añadir la interfaz
El filtro por aplicación no devuelve nada	Se compara con el nombre de la columna (`application_id`) en lugar del atributo	Usar nombres de atributos Java
La ordenación por un campo anidado duplica o pierde filas	La unión no es uno a uno	En `@ManyToOne` obligatoria no ocurre; revisar si la relación lo es
400 `Invalid demand sort field` desde el frontend	El cliente pide un campo fuera de la lista	Alinear la lista del frontend con la del servidor

13.6 Ejercicios

↑ Índice

Ejercicio 13.1 · Un cuarto filtro Ver solución →

Diseña cómo añadirías un filtro opcional `text` que busque, sin distinguir mayúsculas, en el título de la demanda. Escribe la *lambda* de la especificación usando `criteriaBuilder.like` y `criteriaBuilder.lower`, y el SQL que esperas.

Ejercicio 13.2 · Ver el recuento Ver solución →

Con `show-sql` activado, pide `GET /api/demands?status=OPEN&size=1` cuando haya al menos dos demandas abiertas. Identifica las dos consultas y comprueba que ambas llevan la misma condición `WHERE`.

Autoevaluación del capítulo 13 Ver respuestas →

1. ¿Por qué el número de métodos derivados crece de forma combinatoria con los filtros opcionales?
2. ¿Qué devuelve `criteriaBuilder.conjunction()` y para qué se usa como especificación inicial?
3. ¿En qué momento se ejecutan las *lambdas* de una `Specification`?
4. ¿Qué interfaz debe extender un repositorio para aceptar especificaciones?
5. ¿Qué necesita la base de datos para ordenar por `application.name`, y por qué funcionaba en el monolito?

CAPÍTULO 14

Carga perezosa y el problema N+1

↑ Volver al índice

EN ESTE CAPÍTULO

14.1 Cómo aparece el problema
14.2 Por qué la configuración del proyecto lo hace visible
14.3 Resolverlo con `@EntityGraph`
14.4 Comprobar que el problema no vuelve
14.5 Otras formas de evitar el N+1
14.6 Errores típicos y depuración
14.7 Ejercicios

Cada demanda del listado muestra el nombre de su aplicación. Con una asociación perezosa, ese detalle inocente puede multiplicar el número de consultas. Este capítulo explica el problema N+1, por qué la configuración del proyecto lo hace visible y cómo lo resolvió el monolito con `@EntityGraph`.

14.1 Cómo aparece el problema

↑ Índice

La respuesta de cada demanda incluye `applicationName`. En el monolito, el servicio lo obtenía navegando por la asociación:

`demand/service/DemandService.java · toResponse (monolito)` RECONSTRUIDO

```
private DemandResponse toResponse(Demand demand) {
    return new DemandResponse(
            demand.getId(),
            demand.getTitle(),
            demand.getDescription(),
            demand.getPriority(),
            demand.getStatus(),
            demand.getApplication().getId(),
            demand.getApplication().getName(),
            demand.getCreatedAt(),
            demand.getUpdatedAt()
    );
}
```

Código reconstruido

El DTO `DemandResponse` y el orden de sus campos son los del código final. En el monolito, el `id` y el nombre de la aplicación salían de la asociación; en la versión final salen del catálogo (capítulo 18).

Supón una página de diez demandas que pertenecen a diez aplicaciones distintas. Con la asociación perezosa y sin nada más, ocurre esto:

1. `findAll(specification, pageable)` ejecuta **una** consulta y devuelve diez demandas. En el atributo `application` de cada una hay un proxy que solo conoce el `id`.
2. `Page.map(this::toResponse)` recorre las diez demandas.

3. En la primera, `getApplication().getName()` despierta el proxy, que ejecuta un `SELECT` de esa aplicación.
4. Lo mismo en la segunda, la tercera... hasta la décima.

Total: **1 + 10 consultas**. Para una página de *N* elementos, *N + 1*. De ahí el nombre.

Lo que muestra show-sql (esquema)

```
select … from demands d1_0 where … order by d1_0.id offset ? rows fetch first ? rows only
select … from applications a1_0 where a1_0.id=?   -- demanda 1
select … from applications a1_0 where a1_0.id=?   -- demanda 2
…                                                  -- hasta 10 veces
```

El contexto de persistencia suaviza el problema, pero no lo elimina: si varias demandas pertenecen a la misma aplicación, el segundo acceso encuentra la aplicación ya cargada y no vuelve a la base de datos (capítulo 5). Así que el número real de consultas extra es el **número de aplicaciones distintas** de la página. Con dos aplicaciones en el catálogo, como en los datos iniciales, apenas se nota; con cientos, sí.

14.2 Por qué la configuración del proyecto lo hace visible

↑ Índice

En una aplicación con `open-in-view=true` (el valor por defecto de Spring Boot), el problema puede pasar completamente desapercibido: si la entidad llegara hasta Jackson, la serialización despertaría los proxies sin que nadie lo viera en el código.

El proyecto lo hace visible por dos lados:

- Con **`open-in-view=false`**, cualquier acceso perezoso fuera de la transacción falla con `LazyInitializationException`. Eso obliga a construir el DTO dentro del servicio, donde está claro qué datos se tocan.
- Con **`show-sql=true`**, las consultas repetidas aparecen en la consola en cuanto se pide un listado.

Dentro de la transacción de solo lectura de `getDemands`, el acceso perezoso funciona, así que no hay error: hay consultas de más. El N+1 es un problema de rendimiento, no de corrección, y por eso hay que buscarlo activamente.

14.3 Resolverlo con `@EntityGraph`

Paso 3.6 ↖ ↑ Índice

Un **grafo de entidad** indica a JPA qué asociaciones cargar junto con la entidad principal **en la misma consulta**. Spring Data permite declararlo sobre un método del repositorio con `@EntityGraph`. El monolito lo aplicó a la consulta del listado:

demand/repository/DemandRepository.java (monolito) RECONSTRUIDO

```java
public interface DemandRepository extends JpaRepository<Demand, Long>,
        JpaSpecificationExecutor<Demand> {

    @Override
    @EntityGraph(attributePaths = "application")
    Page<Demand> findAll(Specification<Demand> specification,
                         Pageable pageable);
}
```

Código reconstruido

El documento `frontera-catalogo.md` menciona «la prueba que verificaba el antiguo EntityGraph», lo que confirma que el monolito usaba `@EntityGraph` sobre la asociación con `Application`. La forma concreta (redeclarar `findAll` de `JpaSpecificationExecutor` con `attributePaths`) es la habitual en Spring Data para este caso, pero no se conserva el fichero original.

Al redeclarar el método heredado con la anotación, Spring Data aplica el grafo a la consulta de datos. Hibernate añade una unión con `applications` y rellena la asociación con objetos reales en lugar de proxies:

Con el grafo de entidad (esquema)

```
select d1_0.id, …, a1_0.id, a1_0.name, …
from demands d1_0
join applications a1_0 on a1_0.id = d1_0.application_id
where … order by d1_0.id
offset ? rows fetch first ? rows only
```

Una consulta para los datos de la página (más el recuento, cuando hace falta), sea cual sea el número de demandas.

14.3.1 Por qué aquí se puede paginar con una unión

Unir y paginar a la vez es seguro en este caso porque la asociación es **a uno**: cada demanda tiene exactamente una aplicación, así que la unión no multiplica las filas y `OFFSET … FETCH` sigue contando demandas.

Con una asociación **a muchos** sería distinto. Si se cargara, por ejemplo, una colección de comentarios de cada demanda, cada demanda aparecería tantas veces como comentarios, y Hibernate no podría paginar en SQL: traería todas las filas y paginaría en memoria, con un aviso en el *log*. Para colecciones, la técnica adecuada es otra (carga por lotes, o dos consultas).

Nota: El recuento no lleva el grafo

Spring Data no aplica el grafo de entidad a la consulta de recuento: para contar demandas no hace falta cargar aplicaciones. El `count` sigue siendo una consulta simple sobre `demands`.

14.4 Comprobar que el problema no vuelve

↑ Índice

El problema N+1 es fácil de reintroducir sin darse cuenta: basta con que alguien quite la anotación o añada otro acceso perezoso. Por eso el monolito tenía una prueba que lo verificaba:

DemandIntegrationTest.java · prueba del grafo (monolito) RECONSTRUIDO

```
@Test
void shouldLoadApplicationWithTheDemandPage() {

    // … guardar una demanda asociada a una aplicación,
    //     flush() y clear() para vaciar el contexto

    Page<Demand> result = demandRepository.findAll(
            (root, query, cb) -> cb.conjunction(),
            PageRequest.of(0, 10, Sort.by(Sort.Direction.DESC, "id")));

    Demand loaded = result.getContent().getFirst();
```

```
    PersistenceUnitUtil util = entityManager.getEntityManagerFactory()
            .getPersistenceUnitUtil();

    assertTrue(util.isLoaded(loaded, "application"));
}
```

Código reconstruido

Se conserva la versión adaptada de esta prueba, `shouldLoadApplicationIdWithoutAnEntityAssociation` (capítulo 21): guarda una demanda, vacía el contexto con `clear()`, la recupera con `findAll` y una especificación neutra ordenada por `id` descendente, y comprueba el identificador de aplicación. La comprobación de carga con `PersistenceUnitUtil.isLoaded` es una forma estándar de verificar un grafo de entidad; no puede asegurarse que la prueba original usara exactamente esta técnica.

La clave está en `clear()`: si la aplicación siguiera en el contexto de persistencia de la prueba, la asociación estaría cargada aunque el grafo no funcionara. Vaciar el contexto obliga a que la carga dependa solo de la consulta.

14.5 Otras formas de evitar el N+1

↑ Índice

Técnica	Cómo funciona	Cuándo encaja
`@EntityGraph`	Une y carga las asociaciones indicadas en la misma consulta	Asociaciones a uno en consultas concretas (la elegida)
`JOIN FETCH` en `@Query`	Lo mismo, escrito en JPQL	Consultas fijas; con paginación exige escribir también el `countQuery`
Carga por lotes (`hibernate.default_batch_fetch_size` o `@BatchSize`)	Al despertar un proxy, Hibernate carga de golpe los de todas las entidades pendientes con `IN (…)`	Colecciones y casos generales; convierte N+1 en 1 + N/lote
Proyección a DTO	La consulta devuelve directamente los campos de la respuesta, sin entidades	Listados de solo lectura muy grandes
Consulta por lote explícita	Reunir los identificadores de la página y pedirlos todos de una vez	Cuando los datos asociados están en otro sitio (la técnica de la fase 5)

La última fila es la que el proyecto acabaría usando en la fase 5, cuando las aplicaciones dejaron de estar en la misma base de datos: no se puede hacer una unión con una tabla que está en otro servicio, pero sí se puede pedir de una vez todo lo que la página necesita.

Decisión de diseño: `@EntityGraph` en la consulta del listado

Motivo: resuelve el problema exactamente donde aparece, sin cambiar el modelo ni escribir JPQL, y sigue funcionando con `Specification` y paginación. **Alternativas:** carga por lotes global, que habría reducido el problema en toda la aplicación pero sin eliminarlo; o proyecciones a DTO, que habrían duplicado la lógica de los filtros.

14.6 Errores típicos y depuración

↑ Índice

Síntoma	Causa probable	Cómo resolverlo
Un listado de *N* elementos ejecuta *N + 1* consultas	Acceso perezoso a una asociación en un bucle	Grafo de entidad, `JOIN FETCH` o carga por lotes
`@EntityGraph` no tiene efecto	Se anotó un método distinto del que llama el servicio	Redeclarar exactamente la firma usada
Aviso `HHH90003004: firstResult/maxResults specified with collection fetch; applying in memory`	Se unió una colección y se paginó	No cargar colecciones con unión en consultas paginadas
La prueba del grafo pasa aunque se quite la anotación	El contexto de persistencia ya tenía la entidad	Llamar a `flush()` y `clear()` antes de la consulta

14.7 Ejercicios

↑ Índice

Ejercicio 14.1 · Contar consultas con estadísticas de Hibernate Ver solución →

Activa temporalmente `spring.jpa.properties.hibernate.generate_statistics=true` y observa en el *log* el número de sentencias JDBC por petición al pedir un listado de demandas. Explica cómo usarías esta información en una prueba automática que detecte el N+1.

Ejercicio 14.2 · Calcular el coste real Ver solución →

Una página de 20 demandas pertenece a 4 aplicaciones distintas. Calcula cuántas consultas se ejecutarían sin grafo de entidad, con grafo de entidad, y con carga por lotes de tamaño 10. Ten en cuenta el recuento y la caché del contexto de persistencia.

Autoevaluación del capítulo 14 Ver respuestas →

1. ¿Por qué el problema se llama N+1, y de qué depende el número real de consultas extra?
2. ¿Cómo contribuyen `open-in-view=false` y `show-sql=true` a detectar el problema?
3. ¿Qué hace `@EntityGraph(attributePaths = "application")`?
4. ¿Por qué es seguro paginar con una unión a una asociación `@ManyToOne` y no a una colección?
5. ¿Por qué una prueba que verifique el grafo debe llamar a `clear()` antes de consultar?

CAPÍTULO 15

Servicio, ciclo de vida y API de demandas

↑ Volver al índice

EN ESTE CAPÍTULO

15.1 Crear una demanda
15.2 Modificar y cambiar de estado
15.3 El controlador de demandas
15.4 Probar el dominio, el servicio y el controlador
15.5 Errores típicos y depuración
15.6 Ejercicios

Con el modelo y las consultas listos, faltaban los casos de uso: crear una demanda sobre una aplicación existente, consultarla y modificarla respetando su ciclo de vida. Este capítulo explica cómo el servicio del monolito resolvió la referencia a la aplicación, por qué una transición inválida es un 409 y cómo se probó todo.

15.1 Crear una demanda

Paso 3.7 ↖ ↑ Índice

Crear una demanda exige que la aplicación indicada exista. En el monolito, el servicio de demandas lo comprobaba usando directamente el repositorio del otro módulo:

demand/service/DemandService.java · createDemand (monolito) RECONSTRUIDO

```
@Service
public class DemandService {

    private final DemandRepository demandRepository;
    private final ApplicationRepository applicationRepository;  // otro módulo

    @Transactional
    public DemandResponse createDemand(DemandCreateRequest request) {

        Application application = applicationRepository
                .findById(request.applicationId())
                .orElseThrow(() -> new ResponseStatusException(
                        HttpStatus.NOT_FOUND,
                        "Application not found: " + request.applicationId()
                ));

        Demand demand = new Demand(
                request.title(),
                request.description(),
                request.priority(),
                "OPEN",
                application
        );

        Demand saved = demandRepository.save(demand);

        return toResponse(saved);
    }
}
```

Código reconstruido

El código final conserva la misma secuencia (comprobar la aplicación, crear con estado `OPEN`, guardar, responder) y el mismo mensaje de error, pero obtiene la aplicación a través de un puerto (capítulo 18). El documento del refactor afirma que, tras él, «Demand y su servicio no conocen la entidad ni el repositorio Application», lo que confirma que antes sí los conocían. El uso de `ApplicationRepository` es la forma directa de hacerlo en el monolito.

El método repite el patrón del alta de aplicaciones, con dos diferencias:

- **El estado inicial es `OPEN`**, siempre. El DTO de alta no tiene campo de estado.
- **Una aplicación inexistente es un 404**, con el mensaje `Application not found: <id>`. Se podría discutir si debería ser un 400 o un 422, porque el recurso que se intenta crear es la demanda, no la aplicación. El proyecto eligió 404 por coherencia con el resto de búsquedas por `id`, y el frontend lo interpreta como «la aplicación seleccionada ya no existe».

La comprobación verifica **existencia**, no estado: una demanda puede crearse sobre una aplicación `INACTIVE`. Es una regla de negocio deliberada, recogida en el documento `frontera-catalogo.md`.

Observa la línea marcada con el comentario: el servicio de demandas depende de un repositorio del módulo de aplicaciones. En el monolito funciona y es lo más sencillo, pero es exactamente el tipo de dependencia que el capítulo 16 identificará como problema.

15.1.1 Los DTO de demandas

`demand/dto/DemandCreateRequest.java` IMPLEMENTADO

```
public record DemandCreateRequest(

        @NotBlank
        @Size(max = 150)
        String title,

        @Size(max = 1000)
        String description,

        @NotBlank
        @Pattern(regexp = "LOW|MEDIUM|HIGH|CRITICAL")
        String priority,

        @NotNull
        Long applicationId
) {
}
```

`DemandUpdateRequest` añade `status`, con `@NotBlank` y `@Pattern(regexp = "OPEN|IN_PROGRESS|COMPLETED|CANCELLED")`. El identificador de aplicación usa `@NotNull`, porque es un número y `@NotBlank` solo se aplica a texto.

La respuesta incluye datos de la propia demanda y dos de su aplicación, el `id` y el nombre, para que el frontend pueda mostrar la tabla sin hacer otra petición por fila:

`demand/dto/DemandResponse.java` IMPLEMENTADO

```
public record DemandResponse(
        Long id,
        String title,
```

```
        String description,
        String priority,
        String status,
        Long applicationId,
        String applicationName,
        Instant createdAt,
        Instant updatedAt
) {
}
```

15.2 Modificar y cambiar de estado

↑ Índice

demand/service/DemandService.java · updateDemand (monolito) RECONSTRUIDO

```
@Transactional
public DemandResponse updateDemand(Long id, DemandUpdateRequest request) {

    Demand demand = demandRepository.findById(id)
            .orElseThrow(() -> new ResponseStatusException(
                    HttpStatus.NOT_FOUND,
                    "Demand not found: " + id
            ));

    Application application = applicationRepository
            .findById(request.applicationId())
            .orElseThrow(() -> new ResponseStatusException(
                    HttpStatus.NOT_FOUND,
                    "Application not found: " + request.applicationId()
            ));

    demand.update(
            request.title(),
            request.description(),
            request.priority(),
            application
    );

    try {
        demand.changeStatus(request.status());
    } catch (IllegalStateException ex) {
        throw new ResponseStatusException(
                HttpStatus.CONFLICT,
                ex.getMessage()
        );
    }

    return toResponse(demand);
}
```

Código reconstruido

El bloque `try` / `catch` y el orden de las operaciones son idénticos en el código final; solo cambia cómo se obtiene la aplicación.

El orden de las comprobaciones define qué error recibe el cliente cuando hay varios problemas a la vez: primero, que la demanda exista (404 `Demand not found`); después, que exista la aplicación (404 `Application not found`); por último, que la transición sea válida (409).

15.2.1 Qué pasa con los cambios cuando la transición falla

Fíjate en el orden: `update(...)` **ya ha modificado** título, descripción, prioridad y aplicación cuando `changeStatus` lanza la excepción. ¿Se guardan esos cambios?

No. El servicio captura la `IllegalStateException` y lanza una `ResponseStatusException`, que es no comprobada. Al salir del método con esa excepción, el proxy transaccional **deshace la transacción**. Hibernate nunca hace *flush*, así que la entidad modificada en memoria se descarta junto con el contexto de persistencia. La operación es todo o nada, sin que el código tenga que deshacer nada a mano.

> **Si vienes de Java EE...**
>
> Es exactamente el comportamiento de una transacción gestionada por el contenedor ante una `RuntimeException` en un EJB, con la diferencia de que allí el contenedor envolvía la excepción en una `EJBException`. Aquí la excepción llega intacta al manejador global.

15.2.2 Por qué 409 Conflict

Una transición inválida no es un error de formato: el JSON es correcto y el estado `OPEN` es un valor válido. El problema es que **choca con el estado actual del recurso**. Para eso está el código 409 Conflict: la petición no puede completarse porque entra en conflicto con el estado del recurso en el servidor.

Código	Cuándo se usa en el proyecto
400 Bad Request	La petición está mal construida: campos inválidos, filtros desconocidos
404 Not Found	El recurso, o uno al que se hace referencia, no existe
409 Conflict	La petición es correcta, pero choca con el estado actual

El frontend aprovecha esta distinción: ante un 409 muestra el mensaje de transición inválida y ofrece recargar la demanda, porque lo más probable es que otra persona la haya cambiado mientras tanto.

> **Decisión de diseño: Cambios de estado a través de `PUT`**
>
> **Motivo:** el formulario de modificación edita todos los campos, incluido el estado, y los envía juntos; un único *endpoint* mantiene la API pequeña. La regla de transición se aplica igual. **Alternativa:** un *endpoint* específico, como `PATCH /api/demands/{id}/status` o `POST /api/demands/{id}/start`, que expresa mejor la intención y facilita permisos distintos por acción. Podría añadirse si las transiciones se complican.

15.3 El controlador de demandas

↑ Índice

`demand/controller/DemandController.java (extracto)` IMPLEMENTADO

```java
@RestController
@RequestMapping("/api/demands")
public class DemandController {

    @PostMapping
    public ResponseEntity<DemandResponse> createDemand(
            @Valid @RequestBody DemandCreateRequest request) {

        DemandResponse response = demandService.createDemand(request);
```

```
        return ResponseEntity
                .created(URI.create("/api/demands/" + response.id()))
                .body(response);
    }

    @GetMapping
    public DemandPageResponse getDemands(
            @RequestParam(required = false) Long applicationId,
            @RequestParam(required = false) String status,
            @RequestParam(required = false) String priority,
            @PageableDefault(size = 10, sort = "id",
                    direction = Sort.Direction.ASC) Pageable pageable) {

        return demandService.getDemands(
                applicationId, status, priority, pageable);
    }

    // GET /{id} y PUT /{id}: igual que en aplicaciones
}
```

El controlador es casi idéntico al de aplicaciones, con un filtro más. `applicationId` es `Long`: si llega un valor no numérico, el manejador de tipos incorrectos responde con un 400 (capítulo 10).

Método y ruta	Éxito	Errores posibles
`GET /api/demands?applicationId&status&priority&page&size&sort`	200 + página	400
`GET /api/demands/{id}`	200 + demanda	404
`POST /api/demands`	201 + `Location` + demanda	400, 404 (aplicación)
`PUT /api/demands/{id}`	200 + demanda	400, 404 (demanda o aplicación), 409

15.4 Probar el dominio, el servicio y el controlador

Paso 3.8 ↖ ↑ Índice

Las pruebas del módulo siguen la misma estrategia que las de aplicaciones: el dominio con pruebas unitarias puras (`DemandTest`, capítulo 12), el servicio con Mockito y el controlador con `@WebMvcTest`.

15.4.1 El 409, comprobado desde HTTP

demand/controller/DemandControllerTest.java (extracto) ESTADO FINAL

```
@Test
void shouldReturnConflictForInvalidStatusTransition()
        throws Exception {

    when(demandService.updateDemand(
            eq(1L),
            any(DemandUpdateRequest.class)))
            .thenThrow(new ResponseStatusException(
                    HttpStatus.CONFLICT,
                    "Invalid demand status transition: IN_PROGRESS -> OPEN"
            ));

    String requestBody = """
```

```
            {
              "title": "Adaptacion a la nueva normativa",
              "description": "Intento invalido",
              "priority": "CRITICAL",
              "status": "OPEN",
              "applicationId": 1
            }
            """;

    mockMvc.perform(put("/api/demands/1")
                    .contentType(MediaType.APPLICATION_JSON)
                    .content(requestBody))
            .andExpect(status().isConflict())
            .andExpect(jsonPath("$.status").value(409))
            .andExpect(jsonPath("$.error").value("RESOURCE_ERROR"))
            .andExpect(jsonPath("$.message").value(
                    "Invalid demand status transition: IN_PROGRESS -> OPEN"));
}
```

La prueba no comprueba la regla de transición (eso lo hace `DemandTest`), sino el contrato: que una `ResponseStatusException` con 409 llega al cliente con el formato de error del proyecto. Cada nivel prueba lo suyo.

La clase tiene ocho pruebas: alta correcta con `Location`, alta inválida (con `verify(..., never())`), detalle, detalle inexistente, modificación, conflicto y dos filtros inválidos.

15.4.2 El servicio con `@InjectMocks`

`demand/service/DemandServiceTest.java (extracto, monolito)` **RECONSTRUIDO**

```
@ExtendWith(MockitoExtension.class)
public class DemandServiceTest {

    @Mock
    private DemandRepository demandRepository;

    @Mock
    private ApplicationRepository applicationRepository;

    @InjectMocks
    private DemandService demandService;

    @Test
    void shouldRejectInvalidSortField() {

        ResponseStatusException exception = assertThrows(
                ResponseStatusException.class,
                () -> demandService.getDemands(
                        null, null, null,
                        PageRequest.of(0, 10, Sort.by("pepito"))));

        assertEquals(HttpStatus.BAD_REQUEST, exception.getStatusCode());
        assertEquals("Invalid demand sort field: pepito",
                exception.getReason());
    }
}
```

A diferencia de `ApplicationServiceTest`, que construía el servicio con `new`, esta clase usa `@InjectMocks`: Mockito crea el servicio e inyecta en su constructor los *mocks* declarados. Es más breve, pero menos explícito, y si el constructor cambia, Mockito puede inyectar `null` en silencio en lugar de fallar al compilar.

Nota: El segundo *mock*

La única diferencia con la versión final es el segundo *mock*: en el monolito el servicio dependía de `ApplicationRepository`; en la fase 5 esa dependencia cambia (capítulo 18) y la prueba cambia con ella. Las tres pruebas de esta clase validan parámetros antes de consultar nada, así que su cuerpo es idéntico en ambas versiones.

15.5 Errores típicos y depuración

↑ Índice

Síntoma	Causa probable	Cómo resolverlo
Una transición inválida devuelve 500	La `IllegalStateException` no se captura en el servicio	Traducirla a `ResponseStatusException` 409
Una transición inválida guarda los demás cambios	La excepción lanzada es comprobada, o se captura sin relanzar	Relanzar una excepción no comprobada para provocar el *rollback*
`applicationId` nulo pasa la validación	Se usó `@NotBlank` en un `Long`	Usar `@NotNull`
`@InjectMocks` deja un atributo a `null`	El constructor cambió y falta un `@Mock`	Declarar todos los *mocks* o construir con `new`

15.6 Ejercicios

↑ Índice

Ejercicio 15.1 · Comprobar el todo o nada Ver solución →

Con el backend arrancado, toma una demanda en estado `IN_PROGRESS` y envía un `PUT` que cambie el título **y** pida volver a `OPEN`. Verifica el 409 y, después, consulta la demanda para comprobar que el título no cambió. Explica qué componente ha evitado que se guardara.

Ejercicio 15.2 · Una prueba de servicio con transición inválida Ver solución →

Escribe una prueba unitaria de `updateDemand` en la que el repositorio devuelva una demanda `COMPLETED` y la petición pida `IN_PROGRESS`. Comprueba que se lanza una `ResponseStatusException` con estado 409 y el mensaje exacto. Indica qué *mocks* necesitas preparar.

Autoevaluación del capítulo 15 Ver respuestas →

1. ¿Qué devuelve el servicio si se intenta crear una demanda sobre una aplicación inexistente? ¿Y sobre una `INACTIVE`?
2. ¿En qué orden se comprueban los errores en `updateDemand`?
3. ¿Por qué no se guardan los cambios de título cuando `changeStatus` lanza la excepción?
4. ¿Qué diferencia hay entre un 400 y un 409 en este proyecto?
5. ¿Qué riesgo tiene `@InjectMocks` frente a construir el servicio con `new`?

FASE 4

El monolito terminado

16 El monolito terminado pág. 106

↑ Volver al índice

CAPÍTULO 16

El monolito terminado

↑ Volver al índice

EN ESTE CAPÍTULO

16.1 El monolito completo
16.2 Pruebas contra un PostgreSQL real con Testcontainers
16.3 El acoplamiento entre Demand y Application
16.4 Balance del monolito
16.5 Errores típicos y depuración
16.6 Ejercicios

Al terminar el módulo de demandas, el backend estaba completo para lo que el frontend necesitaba. Antes de seguir, conviene detenerse: revisar el conjunto, asegurar las pruebas contra una base de datos real y, sobre todo, mirar con atención el único punto en el que los dos módulos se tocan.

16.1 El monolito completo

Paso 4.1 ↖ ↑ Índice

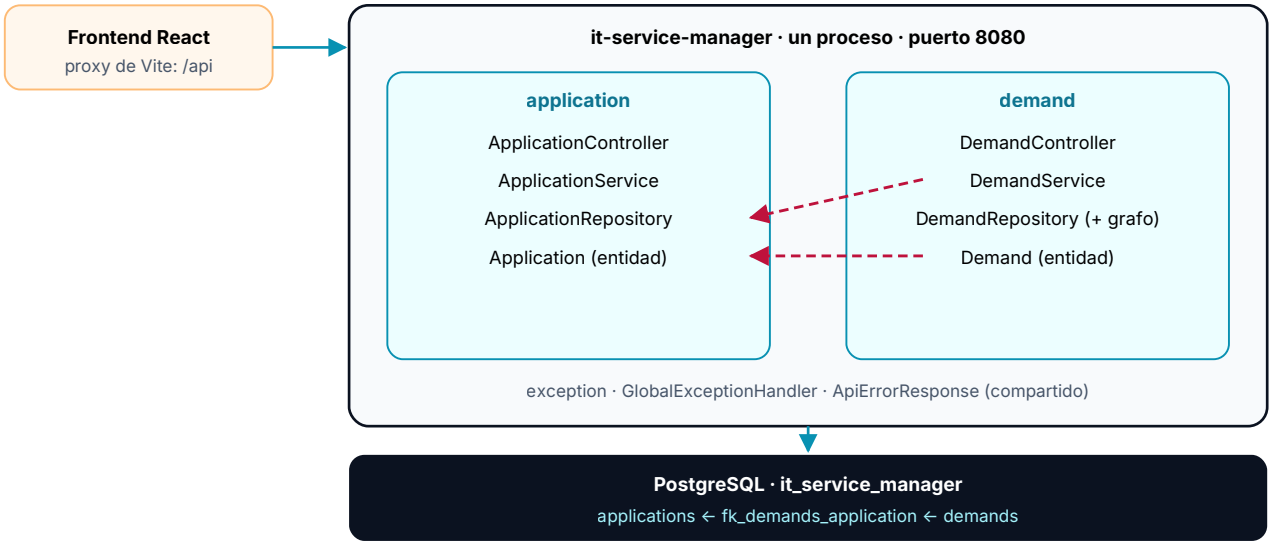


Figura 16.1. El monolito terminado. Las flechas discontinuas son las dependencias del módulo de demandas hacia el de aplicaciones: el servicio usa su repositorio y la entidad tiene una asociación con la suya.

Aspecto	Estado al final de la fase 4
Proceso	Una aplicación Spring Boot, puerto 8080
Base de datos	Una, `it_service_manager`, migraciones V1 a V3
Módulos	`application` y `demand`, más `exception` compartido
Endpoints	4 de aplicaciones y 4 de demandas
Integridad	Clave foránea entre `demands` y `applications`
Transacciones	Locales; crear una demanda lee la aplicación en la misma transacción

Aspecto	Estado al final de la fase 4
Pruebas	Unitarias, de capa web y de integración con PostgreSQL real

El frontend consume las dos APIs a través del proxy de desarrollo de Vite, que reenvía todo lo que empieza por `/api` al puerto 8080. Para el navegador, frontend y backend comparten origen, así que no hace falta configurar CORS.

> **En el proyecto**
>
> Un detalle del contrato quedó desalineado entre los dos lados: el frontend ofrece ordenar las demandas por más campos de los que el backend acepta en su lista blanca, y el backend responde a esos casos con un 400. La guía del frontend lo documenta con una prueba que lo demuestra. En la fase 5 la discrepancia crece, porque uno de esos campos, el nombre de la aplicación, deja de estar disponible (capítulo 21).

16.2 Pruebas contra un PostgreSQL real con Testcontainers

Paso 4.2 ↖ ↑ Índice

La prueba `contextLoads` original dependía de que cada desarrollador tuviera un PostgreSQL configurado (capítulo 11). La solución fue que la propia prueba arranque su base de datos en Docker con **Testcontainers**:

`ItServiceManagerApplicationTests.java` IMPLEMENTADO

```
@Testcontainers
@SpringBootTest
class ItServiceManagerApplicationTests {

    @Container
    @ServiceConnection
    static PostgreSQLContainer<?> postgres =
            new PostgreSQLContainer<>("postgres:18");

    @Test
    void contextLoads() {
    }
}
```

`pom.xml (dependencias de prueba añadidas)` IMPLEMENTADO

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-testcontainers</artifactId>
    <scope>test</scope>
</dependency>
<dependency>
    <groupId>org.testcontainers</groupId>
    <artifactId>testcontainers-junit-jupiter</artifactId>
    <scope>test</scope>
</dependency>
<dependency>
    <groupId>org.testcontainers</groupId>
    <artifactId>testcontainers-postgresql</artifactId>
    <scope>test</scope>
</dependency>
```

16.2.1 Qué ocurre al ejecutar la prueba

1. `@Testcontainers` activa la extensión de JUnit 5 que gestiona los contenedores declarados en la clase.
2. `@Container` sobre un atributo `static` indica que el contenedor se arranca **una vez para toda la clase**, antes de la primera prueba, y se para al terminar. Sin `static`, se arrancaría uno nuevo para cada método.
3. Testcontainers pide a Docker un contenedor `postgres:18`, con la base de datos y el usuario que configura por defecto, y lo publica en **un puerto aleatorio libre** del equipo. Espera a que PostgreSQL acepte conexiones.
4. `@ServiceConnection` hace que Spring Boot lea del contenedor la URL, el usuario y la contraseña reales, y cree con ellos los detalles de conexión del `DataSource`, **por encima** de lo que diga `application.properties`. No hay que escribir ninguna URL.
5. `@SpringBootTest` arranca la aplicación completa contra esa base de datos vacía: Flyway aplica V1 a V3, Hibernate valida el esquema y el contexto queda listo.
6. Al terminar, el contenedor se elimina con todos sus datos.

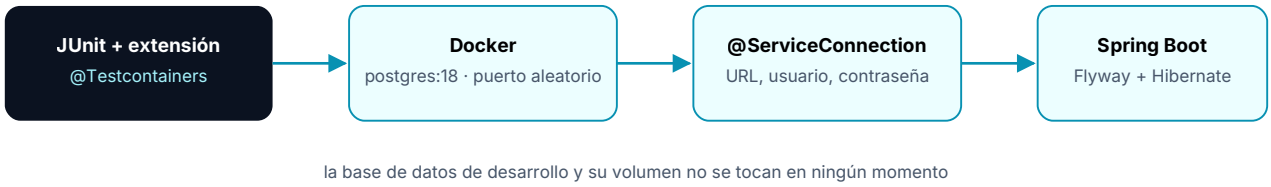


Figura 16.2. Una prueba con Testcontainers crea su propio PostgreSQL desechable y conecta la aplicación a él.

Decisión de diseño: PostgreSQL real en lugar de una base de datos en memoria

Motivo: las pruebas se ejecutan contra el mismo motor y la misma versión mayor que el entorno real, con las mismas migraciones de Flyway, los mismos tipos (`TIMESTAMP WITH TIME ZONE`, columnas de identidad) y las mismas restricciones. **Alternativa descartada:** H2 en memoria, más rápida y sin Docker, pero con otro dialecto SQL: una migración válida en H2 puede fallar en PostgreSQL y al revés. **Coste:** las pruebas de integración necesitan Docker en marcha y tardan más.

Si vienes de Java EE...

En Java EE las pruebas de integración solían necesitar Arquillian y un servidor de aplicaciones gestionado, o una base de datos compartida del equipo, con el riesgo de que dos ejecuciones se pisaran. Testcontainers da a cada ejecución su propia base de datos limpia.

16.2.2 Una prueba de integración del monolito

Con la infraestructura lista, las pruebas de integración pueden comprobar lo que las unitarias no alcanzan: que el SQL funciona, que la transacción se confirma y que Flyway deja la base de datos como Hibernate espera.

`DemandIntegrationTest.java (monolito)` RECONSTRUIDO

```
@Testcontainers
@SpringBootTest
@Transactional
```

```
class DemandIntegrationTest {

    @Container
    @ServiceConnection
    static PostgreSQLContainer postgres =
            new PostgreSQLContainer("postgres:18")
                    .withDatabaseName("it_service_manager")
                    .withUsername("itmanager")
                    .withPassword("test");

    @Autowired
    private DemandService demandService;

    @Test
    void shouldCreateDemandLinkedToApplication() {

        // La aplicación 1 la crea la migración V2
        var response = demandService.createDemand(
                new DemandCreateRequest(
                        "Adaptación a la nueva normativa",
                        "Incorporar nuevos tipos de expediente",
                        "HIGH",
                        1L));

        assertNotNull(response.id());
        assertEquals("OPEN", response.status());
        assertEquals("SIGMA", response.applicationName());
        assertNotNull(response.createdAt());
    }
}
```

Código reconstruido

La declaración del contenedor y el cuerpo de la prueba proceden de `shouldCreateDemandLinkedToExternalApplication`, en el código final. En el monolito no hacía falta simular el catálogo: la aplicación existía de verdad en la base de datos de la prueba, creada por la migración V2.

`@Transactional` **en la clase de prueba** tiene un efecto distinto que en el código de producción: cada método de prueba se ejecuta en una transacción que **se deshace al terminar**. Así, lo que crea una prueba no lo ve la siguiente, y todas parten del estado que dejan las migraciones.

Cuidado: Una transacción que nunca se confirma

Como la transacción de la prueba se deshace, Hibernate puede no llegar a enviar nunca el SQL a la base de datos: el `UPDATE` de una modificación se queda pendiente hasta un *flush* que no llega. Por eso las pruebas de integración del proyecto llaman a `flush()` o `saveAndFlush()` cuando quieren comprobar que el SQL realmente funciona, y a `clear()` cuando quieren releer de la base de datos en lugar de la memoria.

Lección aprendida: Dos versiones de la misma clase

`ItServiceManagerApplicationTests` usa `org.testcontainers.containers.PostgreSQLContainer<?>`, la clase clásica, genérica; las pruebas de integración más recientes usan `org.testcontainers.postgresql.PostgreSQLContainer`, sin genérico, la ubicación que introdujo Testcontainers 2 y que sustituye a la anterior, marcada como obsoleta. Las dos funcionan, pero conviene unificar en la nueva. Se recoge en el capítulo 23.

16.3 El acoplamiento entre Demand y Application

Paso 4.3 ↖ ↑ Índice

El monolito funciona y está probado. Pero la figura 16.1 tiene dos flechas discontinuas que conviene examinar, porque determinan cuánto costaría separar los módulos. Revisando el código, el módulo de demandas dependía del de aplicaciones en siete puntos:

#	Punto de contacto	Dónde	Tipo
1	La entidad `Demand` tiene un atributo `Application`	Modelo	Código
2	`DemandService` usa `ApplicationRepository` para comprobar que existe	Servicio	Código
3	`toResponse` navega por la asociación para obtener el nombre	Servicio	Código
4	`@EntityGraph` une las dos tablas en el listado	Repositorio	SQL
5	`sort=application.name` ordena con una unión	Repositorio	SQL
6	`fk_demands_application`	Base de datos	Esquema
7	Crear una demanda lee la aplicación en la misma transacción	Servicio	Transacción

Lo que el módulo de demandas necesita de una aplicación es muy poco: **saber si existe** y **conocer su nombre**. Sin embargo, para obtener esas dos cosas, depende de la entidad completa, del repositorio, de la tabla y de que ambas estén en la misma base de datos.

Mientras todo sea un monolito, esa dependencia es cómoda. Pero si el catálogo de aplicaciones tuviera que vivir aparte (porque lo usan otros sistemas, porque lo mantiene otro equipo o porque escala de forma distinta), cada uno de esos siete puntos sería un obstáculo. Y los obstáculos no son iguales: los puntos 1 a 3 se resuelven cambiando código Java; los 4 y 5 obligan a renunciar a consultas SQL que hoy funcionan; el 6 exige una migración; y el 7 cambia las garantías de consistencia.

> **Decisión de diseño: Detenerse antes de crecer**
>
> **Motivo:** antes de añadir más módulos (incidencias, equipos), interesaba saber si la estructura aguantaría una separación. Identificar los puntos de contacto con el código delante permitió estimar el coste real de extraer el catálogo. **Consecuencia:** esta tabla se convirtió en la lista de trabajo de la fase 5.

16.4 Balance del monolito

↑ Índice

Fortalezas	Debilidades
Una sola unidad que desplegar y depurar	Los módulos solo están separados por convención
Transacciones ACID entre módulos sin esfuerzo	El módulo de demandas conoce la estructura interna del de aplicaciones
Integridad referencial garantizada por la base de datos	No se puede escalar ni desplegar un módulo por separado
Consultas que unen datos de ambos módulos	Un cambio en `Application` puede romper `Demand`
Pruebas de integración completas con un solo contenedor	La base de datos es un punto de acoplamiento compartido

Un monolito así no es un error. Para un equipo pequeño y un dominio todavía en evolución, suele ser la mejor opción. La fase 5 no se emprendió porque el monolito fallara, sino para aprender a separarlo bien y para preparar una arquitectura en la que el catálogo pudiera servir a otros módulos.

16.5 Errores típicos y depuración

↑ Índice

Síntoma	Causa probable	Cómo resolverlo
`Could not find a valid Docker environment`	Docker Desktop no está arrancado	Arrancarlo antes de las pruebas de integración
Las pruebas de integración tardan mucho la primera vez	Docker descarga la imagen `postgres:18`	Solo ocurre la primera vez
Una prueba ve datos de otra	Falta `@Transactional` en la clase o se confirma explícitamente	Mantener el *rollback* automático
Una prueba de modificación pasa aunque el SQL sea incorrecto	El *flush* nunca ocurrió	Llamar a `flush()` antes de comprobar
`@ServiceConnection` no tiene efecto	Falta la dependencia `spring-boot-testcontainers`	Añadirla con *scope* `test`

16.6 Ejercicios

↑ Índice

Ejercicio 16.1 · Observar el contenedor efímero Ver solución →

Ejecuta una prueba de integración y, mientras corre, lista los contenedores con `docker ps`. Identifica el contenedor de PostgreSQL de la prueba y su puerto publicado. Comprueba que desaparece al terminar y que tu contenedor de desarrollo sigue intacto.

Ejercicio 16.2 · Clasificar el acoplamiento Ver solución →

Para cada uno de los siete puntos de contacto de la tabla, indica qué tendría que cambiar si las aplicaciones pasaran a estar en otra base de datos, y ordénalos de más fácil a más difícil de resolver. Compara tu orden con el que siguió la fase 5.

Autoevaluación del capítulo 16 Ver respuestas →

1. ¿Qué hace `@ServiceConnection` y qué propiedades de `application.properties` deja sin efecto?
2. ¿Por qué el contenedor se declara como atributo `static`?
3. ¿Qué efecto tiene `@Transactional` en una clase de pruebas, y por qué obliga a usar `flush()` en algunas comprobaciones?
4. ¿Qué dos cosas necesita realmente el módulo de demandas de una aplicación?
5. ¿Por qué los puntos de contacto de tipo SQL son más difíciles de eliminar que los de código?

?>

FASE 5

Evolución a dos microservicios

17 Decidir la extracción pág. 113

18 Una frontera dentro del monolito: puertos y adaptadores pág. 117

19 Nace `application-catalog-service` pág. 126

20 El adaptador HTTP con `RestClient` pág. 132

21 Separar las bases de datos pág. 141

↑ Volver al índice

CAPÍTULO 17

Decidir la extracción

↑ Volver al índice

EN ESTE CAPÍTULO

17.1 Por qué extraer el catálogo de aplicaciones
17.2 Qué cambia cuando la llamada cruza la red
17.3 La estrategia: pasos pequeños y reversibles
17.4 Lo que se decidió no hacer
17.5 Riesgos aceptados
17.6 Ejercicios

Con el monolito terminado, se tomó la decisión de separar el catálogo de aplicaciones en un servicio propio. Este capítulo explica por qué, qué cambia cuando una llamada cruza la red y, sobre todo, cómo se planificó la extracción para que el sistema siguiera funcionando después de cada paso.

17.1 Por qué extraer el catálogo de aplicaciones

Paso 5.1 ↖ ↑ Índice

De los dos módulos, el candidato natural era el catálogo, por su posición en el dominio:

- **Es un dato de referencia.** Las demandas apuntan a aplicaciones; las incidencias y los equipos previstos también lo harán. El catálogo es lo que comparten todos los módulos futuros, y conviene que tenga un dueño y una API claros.
- **Cambia a otro ritmo.** El catálogo es relativamente estable; las demandas se crean y modifican a diario.
- **La dependencia va en un solo sentido.** Demand necesita a Application, pero Application no sabe nada de Demand. Extraer el lado del que otros dependen, sin dependencias de vuelta, es el corte más limpio posible.
- **Es una oportunidad de aprendizaje controlada.** El proyecto tiene también un objetivo formativo, y el catálogo es lo bastante pequeño para hacer la separación completa sin que el riesgo sea grande.

> **Decisión de diseño: Extraer Application, no Demand**
>
> **Motivo:** el catálogo no depende de nadie y es el dato que compartirán los módulos futuros. **Alternativa descartada:** extraer las demandas y dejar el catálogo en el monolito, que habría obligado al servicio nuevo a llamar de vuelta al monolito para todo y no habría preparado la arquitectura para los módulos siguientes.

Hay que decir también lo contrario: con un solo equipo y dos módulos, un monolito modular bien organizado es perfectamente defendible, y la distribución tiene costes reales. La decisión se tomó sabiendo que se pagaban esos costes a cambio de una arquitectura preparada para crecer y del aprendizaje.

17.2 Qué cambia cuando la llamada cruza la red

↑ Índice

En el monolito, comprobar que una aplicación existe es una llamada a un método Java que lee de la misma base de datos, en la misma transacción. Al separar el catálogo, esa misma pregunta se convierte en una petición HTTP a otro proceso. Parece un cambio de transporte, pero cambia la naturaleza de la operación:

Aspecto	Llamada local (monolito)	Llamada remota (dos servicios)
Latencia	Nanosegundos	Milisegundos, variable
Fallos posibles	Prácticamente ninguno	El otro servicio está caído, lento, o la red falla
Resultado	Siempre hay respuesta	Puede no haber respuesta: hay que decidir cuánto esperar
Transacción	Compartida: todo o nada	Independiente en cada servicio
Integridad	Clave foránea	Solo comprobación en la aplicación
Consultas que unen datos	Una unión SQL	Imposibles: los datos están en bases distintas
Versionado	Se despliega todo junto	Cada servicio puede cambiar su API por su cuenta

Cada fila se convirtió en una decisión concreta de los capítulos siguientes: los tiempos máximos y la traducción de errores del capítulo 20, la consulta por lotes del capítulo 18, la retirada de la ordenación por nombre del capítulo 21 y la eliminación de la clave foránea.

Si vienes de Java EE...

Si trabajaste con EJB remotos sobre RMI-IIOP en WebSphere, conoces la tentación: la llamada remota se escribe igual que la local y es fácil olvidar que no lo es. Las «falacias de la computación distribuida» (la red es fiable, la latencia es cero, el ancho de banda es infinito...) siguen siendo igual de ciertas con HTTP y JSON.

17.3 La estrategia: pasos pequeños y reversibles

↑ Índice

La extracción **no** se hizo en un solo cambio. Se descompuso en pasos tales que, después de cada uno, el sistema compilaba, las pruebas pasaban y la aplicación funcionaba igual para el usuario. Es la idea de la técnica conocida como *branch by abstraction*: primero se introduce una abstracción entre quien usa algo y su implementación; después se cambia la implementación detrás de la abstracción; y por último se retira lo que ya no se usa.

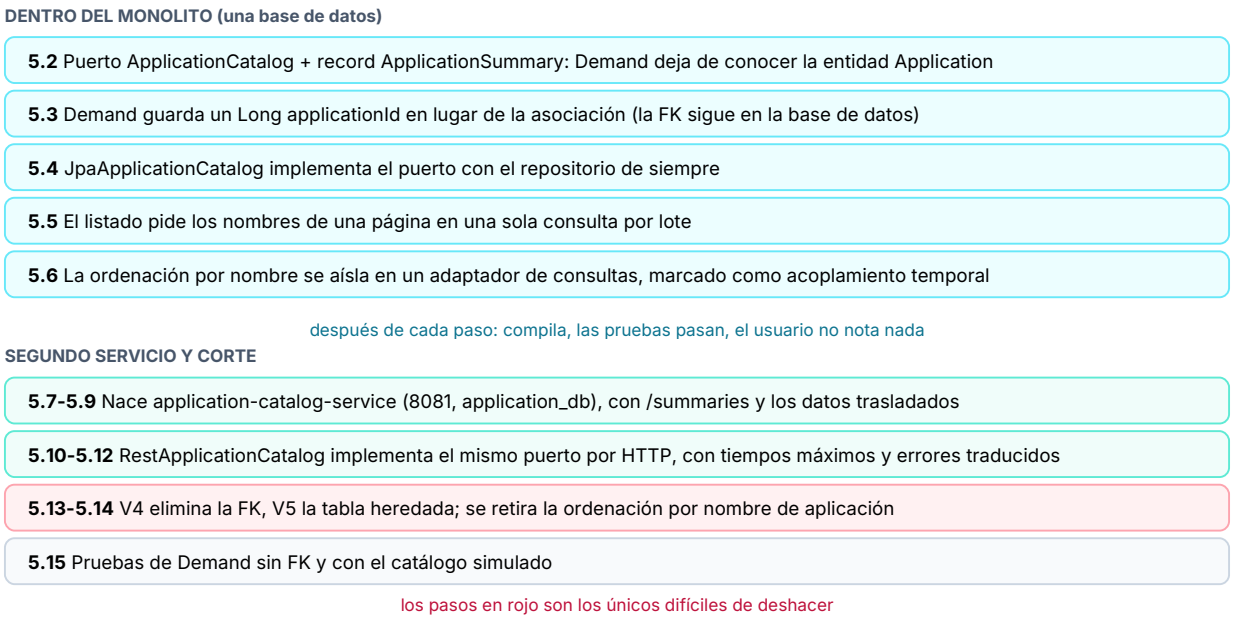


Figura 17.1. Plan de la extracción. La primera mitad cambia la estructura del monolito sin cambiar su comportamiento; la segunda introduce el servicio nuevo detrás de la abstracción ya preparada.

El orden importa por tres razones:

1. **Lo arriesgado va al final.** Los pasos 5.2 a 5.6 son refactorizaciones dentro del monolito: si algo sale mal, se revierten con el control de versiones. La separación de bases de datos de los pasos 5.13 y 5.14 es lo único difícil de deshacer, y se hace cuando todo lo demás ya funciona.
2. **Cada problema se resuelve donde es barato.** El N+1 del listado y la ordenación por nombre se resuelven todavía en el monolito, con la base de datos compartida a mano para comparar resultados. Cuando llega el HTTP, la lógica ya está preparada para trabajar por lotes.
3. **El servicio nuevo encaja en un hueco que ya existe.** Cuando aparece `RestApplicationCatalog`, implementa una interfaz que el servicio de demandas ya usa. El cambio de implementación es local.

17.4 Lo que se decidió no hacer

↑ Índice

Una extracción también se define por lo que deja fuera. Estas piezas habituales en arquitecturas de microservicios se descartaron deliberadamente, por ahora:

Pieza	Por qué no
Base de datos compartida entre servicios	Anularía la separación: los servicios seguirían acoplados por el esquema
Transacciones distribuidas (XA, dos fases)	Complejidad y acoplamiento desproporcionados para comprobar que algo existe
Mensajería y eventos	El catálogo solo se lee desde Demand; no hay que propagar cambios
Copia local del catálogo en Demand	Habría que sincronizarla; con dos aplicaciones de referencia no compensa todavía
Gateway, descubrimiento de servicios	Dos servicios con URL fija no lo necesitan; el proxy de Vite hace de entrada en desarrollo

Pieza	Por qué no
Despliegue conjunto en contenedores	Queda pendiente (capítulo 24)

Decisión de diseño: Un contrato de solo lectura

El servicio de demandas solo **lee** del catálogo: si existe una aplicación y cómo se llama. Nunca la crea ni la modifica. Esa restricción mantiene la frontera estrecha y explica por qué basta con dos operaciones (una por `id` y otra por lote). Las altas y modificaciones de aplicaciones las hace el frontend directamente contra el servicio del catálogo.

17.5 Riesgos aceptados

↑ Índice

- **Integridad eventual.** Sin clave foránea, nada impide en la base de datos que una demanda apunte a un `id` que no existe en el catálogo. Se mitiga porque el servicio de demandas lo comprueba al crear y modificar, y porque las aplicaciones no se borran.
- **Disponibilidad acoplada.** Si el catálogo cae, el servicio de demandas no puede crear ni listar demandas, porque necesita los nombres. Se acepta, y se responde con un error claro (503) en lugar de un fallo opaco.
- **Funcionalidad perdida.** Ordenar las demandas por el nombre de su aplicación deja de ser posible de forma correcta y eficiente. Se retira de la API.

17.6 Ejercicios

↑ Índice

Ejercicio 17.1 · Elegir qué extraer Ver solución →

Imagina que el siguiente módulo es Incidencias, que también referencia aplicaciones y además tiene un responsable de un futuro módulo de Equipos. Razona qué módulos serían candidatos a servicio propio y en qué orden, aplicando los criterios de la primera sección.

Ejercicio 17.2 · Descomponer en pasos Ver solución →

Supón que quisieras extraer también el manejo de errores compartido (`exception`) a una librería común. Propón una secuencia de pasos en la que el sistema funcione después de cada uno, siguiendo la idea de la figura 17.1.

Autoevaluación del capítulo 17 Ver respuestas →

1. ¿Por qué el catálogo de aplicaciones era mejor candidato a extraer que las demandas?
2. Cita tres diferencias entre una llamada local y una remota que obligaron a tomar decisiones de diseño.
3. ¿En qué consiste la técnica de *branch by abstraction*?
4. ¿Por qué los pasos que separan las bases de datos se dejaron para el final?
5. ¿Qué riesgo introduce eliminar la clave foránea y cómo se mitiga?

CAPÍTULO 18

Una frontera dentro del monolito: puertos y adaptadores

↑ Volver al índice

EN ESTE CAPÍTULO

18.1 Puertos y adaptadores en pocas palabras
18.2 El puerto `ApplicationCatalog`
18.3 De la asociación a un identificador
18.4 El catálogo con JPA dentro del monolito
18.5 Una consulta por página
18.6 Aislar la ordenación por nombre
18.7 Cómo se preparó y verificó este refactor
18.8 Errores típicos y depuración
18.9 Ejercicios

El primer tramo de la extracción no creó ningún servicio. Cambió la forma en que el módulo de demandas obtiene lo que necesita de las aplicaciones, de modo que deje de importar cómo y dónde se guardan. Este capítulo explica el puerto `ApplicationCatalog`, el paso de la asociación a un identificador, la consulta por lotes y el aislamiento de la ordenación por nombre.

18.1 Puertos y adaptadores en pocas palabras

↑ Índice

La arquitectura de **puertos y adaptadores** (también llamada hexagonal) separa el núcleo de una aplicación de sus detalles técnicos:

- Un **puerto** es una interfaz que el núcleo define con sus propios términos: «necesito saber si existe una aplicación y cómo se llama».
- Un **adaptador** es una implementación de esa interfaz con una tecnología concreta: con JPA, con HTTP, con un fichero.

El punto clave es **quién es dueño de la interfaz**. El puerto pertenece a quien lo usa (el módulo de demandas), no a quien lo implementa. Así, la dependencia apunta hacia el núcleo, y cambiar de adaptador no obliga a tocar el núcleo. Es el principio de inversión de dependencias aplicado a la frontera entre módulos.



Figura 18.1. Inversión de la dependencia. Después del cambio, el servicio de demandas solo conoce una interfaz propia; el adaptador implementa esa interfaz.

Si vienes de Java EE...

Si alguna vez definiste una interfaz de negocio local en un EJB y dejaste que el contenedor inyectara la implementación por JNDI, ya conoces la mitad de la idea. La otra mitad es la propiedad: aquí la interfaz la escribe el consumidor, con exactamente lo que necesita, y no el proveedor con todo lo que ofrece.

18.2 El puerto `ApplicationCatalog`

Paso 5.2 ↖ ↑ Índice

`demand/port/ApplicationCatalog.java` IMPLEMENTADO

```java
/** Read-only module boundary. No persistence entities cross this contract. */
public interface ApplicationCatalog {
    Optional<ApplicationSummary> findById(Long id);

    /** Returns existing applications only; missing IDs are absent from the map. */
    Map<Long, ApplicationSummary> findByIds(Set<Long> ids);
}
```

`demand/port/ApplicationSummary.java` IMPLEMENTADO

```java
public record ApplicationSummary(Long id, String name) {
}
```

Cada detalle del contrato es deliberado:

- **Vive en el paquete `demand.port`.** Lo define el consumidor.
- **Es de solo lectura.** El comentario lo dice expresamente: el módulo de demandas no crea ni modifica aplicaciones.
- **No cruza ninguna entidad.** Devuelve `ApplicationSummary`, un *record* inmutable con los dos únicos datos que Demand necesita. Nada de proxies de Hibernate, nada de asociaciones perezosas, nada que dependa de un contexto de persistencia.
- **`findById` devuelve `Optional`.** «No existe» es un resultado normal, no una excepción. El servicio decide qué hacer con él.
- **`findByIds` devuelve un mapa con los que existen.** Los identificadores que no existen simplemente no aparecen. Así, una sola llamada sirve para una página entera, y el servicio puede detectar los ausentes.

Decisión de diseño: Un *record* de dos campos en lugar de la entidad

Motivo: el contrato expresa exactamente la necesidad (existencia y nombre), no expone el modelo interno del otro módulo y será idéntico tanto si los datos vienen de JPA como de HTTP. **Alternativa descartada:** devolver la entidad `Application`, que habría mantenido el acoplamiento con su estructura y, en la fase siguiente, habría sido imposible de reproducir desde una respuesta HTTP.

18.3 De la asociación a un identificador

Paso 5.3 ↖ ↑ Índice

El segundo cambio afecta a la entidad. El atributo `Application application` se sustituye por un número:

demand/entity/Demand.java (extracto) REFACTORIZADO

```
@Column(name = "application_id", nullable = false)
private Long applicationId;

public Demand(
        String title,
        String description,
        String priority,
        String status,
        Long applicationId) {

    this.title = title;
    this.description = description;
    this.priority = priority;
    this.status = status;
    this.applicationId = applicationId;
}

public void update(
        String title,
        String description,
        String priority,
        Long applicationId) {

    this.title = title;
    this.description = description;
    this.priority = priority;
    this.applicationId = applicationId;
    this.updatedAt = Instant.now();
}
```

La columna es la misma, `application_id`. **El esquema no cambia**: no hay migración nueva y la clave foránea sigue en la base de datos, protegiendo la integridad durante toda la transición. Lo único que cambia es cómo lo ve Java: antes, un objeto; ahora, un valor.

Los efectos se propagan a tres sitios, todos pequeños:

Sitio	Antes	Después
Filtro del listado	`root.get("application").get("id")`	`root.get("applicationId")`
Repositorio	Grafo de entidad sobre `application`	Sin grafo: no hay asociación que cargar
Pruebas de dominio	Creaban la demanda con un objeto `Application`	La crean con un identificador

La entidad `Demand` ya no importa nada del módulo de aplicaciones. Es el primer punto de la tabla del capítulo 16 que desaparece.

18.4 El catálogo con JPA dentro del monolito

Paso 5.4 ↖ ↑ Índice

Mientras ambos módulos convivan en el mismo proceso, el puerto se implementa con el repositorio de siempre:

application/integration/JpaApplicationCatalog.java RECONSTRUIDO

```
@Component
class JpaApplicationCatalog implements ApplicationCatalog {
```

```
    private final ApplicationRepository applicationRepository;

    JpaApplicationCatalog(ApplicationRepository applicationRepository) {
        this.applicationRepository = applicationRepository;
    }

    @Override
    public Optional<ApplicationSummary> findById(Long id) {
        return applicationRepository.findById(id)
                .map(application -> new ApplicationSummary(
                        application.getId(), application.getName()));
    }

    @Override
    public Map<Long, ApplicationSummary> findByIds(Set<Long> ids) {
        return applicationRepository.findAllById(ids).stream()
                .collect(Collectors.toUnmodifiableMap(
                        Application::getId,
                        application -> new ApplicationSummary(
                                application.getId(), application.getName())));
    }
}
```

Código reconstruido

El documento `frontera-catalogo.md` describe esta clase: « `JpaApplicationCatalog`, dentro del módulo Application, implementa el contrato usando el repositorio existente». El código no se conserva, porque se retiró al extraer el catálogo. La implementación mostrada es la mínima que cumple el contrato del puerto y reproduce lo que hace el adaptador HTTP final con los datos que recibe. Se desconoce el paquete exacto.

Observa la dirección de las dependencias: el módulo de **aplicaciones** depende del puerto definido en el módulo de **demandas**. Es intencionado y temporal: es el módulo que provee los datos quien se adapta al contrato del que los consume.

El servicio de demandas, por su parte, ya solo conoce el puerto:

`demand/service/DemandService.java (extracto)` ESTADO FINAL

```
@Transactional
public DemandResponse createDemand(
        DemandCreateRequest request) {

    ApplicationSummary application = requireApplication(request.applicationId());

    Demand demand = new Demand(
            request.title(),
            request.description(),
            request.priority(),
            "OPEN",
            application.id()
    );

    Demand saved = demandRepository.save(demand);

    return toResponse(saved, application);
}

private ApplicationSummary requireApplication(Long id) {
    return applicationCatalog.findById(id).orElseThrow(() -> applicationNotFound(id));
}
```

```
private ResponseStatusException applicationNotFound(Long id) {
    return new ResponseStatusException(HttpStatus.NOT_FOUND, "Application not found: " + id);
}
```

`toResponse` recibe ahora dos argumentos: la demanda y el resumen de su aplicación. El nombre ya no sale de navegar por una asociación, sino del catálogo. El mismo resumen que se usó para comprobar la existencia sirve para construir la respuesta, sin una segunda consulta.

18.5 Una consulta por página

Paso 5.5 ↖ ↑ Índice

Sin asociación ni grafo de entidad, el listado necesita otra forma de obtener los nombres sin caer en el N+1: pedirle al catálogo, **de una sola vez**, todas las aplicaciones de la página.

`demand/service/DemandService.java · getDemands (extracto)` ESTADO FINAL

```
Page<Demand> demands =
        demandRepository.findAll(specification, pageable);
Set<Long> ids = demands.getContent().stream()
        .map(Demand::getApplicationId).collect(Collectors.toSet());
Map<Long, ApplicationSummary> applications = ids.isEmpty()
        ? Map.of() : applicationCatalog.findByIds(ids);
Page<DemandResponse> responsePage = demands.map(demand -> {
    ApplicationSummary application = applications.get(demand.getApplicationId());
    if (application == null) {
        throw applicationNotFound(demand.getApplicationId());
    }
    return toResponse(demand, application);
});
```

1. Se obtiene la página de demandas, con sus filtros, orden y paginación, como siempre.
2. Se reúnen los identificadores de aplicación **distintos** en un `Set`: diez demandas de dos aplicaciones producen un conjunto de dos elementos.
3. Si la página está vacía, no se llama al catálogo.
4. Se hace **una única llamada** `findByIds` y se obtiene un mapa.
5. Cada demanda busca su nombre en el mapa, en memoria.

El número de consultas al catálogo es siempre **una por página** (o cero), sea cual sea el tamaño de la página. Con JPA, `findAllById` se traduce en un `SELECT … WHERE id IN (…)`. Con HTTP será una única petición. La lógica del servicio no cambiará.

Decisión de diseño: Si falta una aplicación, falla la página entera

Comportamiento: si el catálogo no devuelve alguna de las aplicaciones de la página, el servicio lanza un 404 `Application not found` y el listado completo falla. **Motivo:** una demanda sin aplicación es un dato incoherente, y ocultarlo sería peor. **Alternativa:** degradar con elegancia, mostrando esa fila con un nombre como «Aplicación no disponible», para que una referencia rota no impida ver el resto. En el monolito, con clave foránea, el caso no puede darse; tras la separación de bases de datos sí podría, y por eso se recoge como mejora en el capítulo 24. **Cambiado en la fase 7** (capítulo 25): ahora la fila se devuelve con `applicationName: null`.

18.5.1 Probar la consulta por lotes

demand/service/DemandCatalogTest.java (extracto) ESTADO FINAL

```
@Test
void listUsesOneBatchForDistinctIdsAndPreservesPageMetadata() {
    var pageable = PageRequest.of(1, 3);
    when(repository.findAll(
            org.mockito.ArgumentMatchers
                    .<Specification<Demand>>any(),
            eq(pageable)))
            .thenReturn(new PageImpl<>(
                    List.of(demand(7L), demand(2L), demand(7L)),
                    pageable,
                    12));
    when(catalog.findByIds(Set.of(7L, 2L))).thenReturn(Map.of(
            7L, new ApplicationSummary(7L, "Seven"), 2L, new ApplicationSummary(2L, "Two")));
    var response = service.getDemands(null, null, null, pageable);
    assertEquals(List.of("Seven", "Two", "Seven"),
            response.content().stream().map(item -> item.applicationName()).toList());
    assertEquals(1, response.page());
    assertEquals(3, response.size());
    assertEquals(12, response.totalElements());
    assertEquals(4, response.totalPages());
    verify(catalog).findByIds(Set.of(7L, 2L));
    verifyNoMoreInteractions(catalog);
}
```

La prueba prepara una página de tres demandas con dos aplicaciones distintas (la 7 repetida) y comprueba cuatro cosas: que se llama al catálogo **una vez** con el conjunto `{7, 2}`, que cada demanda recibe su nombre en el orden correcto, que los metadatos de paginación se conservan (página 1 de 4, doce elementos en total) y, con `verifyNoMoreInteractions`, que no hubo ninguna otra llamada al catálogo. La sintaxis `ArgumentMatchers.<Specification<Demand>>any()` indica a Java el tipo genérico del comodín, que no puede deducir por sí solo.

La misma clase, `DemandCatalogTest`, tiene siete pruebas: alta con datos del catálogo, alta y modificación con aplicación inexistente, modificación que cambia la referencia sin llamar a `save`, el lote, la página vacía (con `verifyNoInteractions(catalog)`) y el detalle con el nombre actual.

18.6 Aislar la ordenación por nombre

Paso 5.6 ↖ ↑ Índice

Quedaba un punto de contacto que no se podía resolver con el puerto: ordenar las demandas por el **nombre** de su aplicación (`sort=application.name`). Con la asociación eliminada, Spring Data ya no puede añadir la unión automáticamente.

La tentación es ordenar en memoria: obtener la página, pedir los nombres y ordenarla en Java. Pero eso es **incorrecto**. La paginación ya se ha aplicado: ordenar los diez elementos de la página dos no los convierte en los elementos once a veinte del orden global por nombre. Para ordenar bien, la base de datos tiene que ordenar **todo el conjunto filtrado antes de paginar**, y para eso necesita los nombres.

La solución fue aislar esa consulta concreta en un adaptador propio del monolito:

demand/port/DemandQueries.java y integration/JpaDemandQueries.java RECONSTRUIDO

```
public interface DemandQueries {
    Page<Demand> find(Specification<Demand> filters, Pageable pageable);
}
```

```
@Component
class JpaDemandQueries implements DemandQueries {

    // Ordenaciones locales: el repositorio de siempre.
    // Ordenación por application.name: consulta Criteria que une
    // Demand y Application por applicationId = id, aplica los mismos
    // filtros, la dirección pedida y las ordenaciones secundarias,
    // pagina en la base de datos y calcula el total con otra consulta.
    @Override
    public Page<Demand> find(Specification<Demand> filters, Pageable pageable) {
        if (!sortsByApplicationName(pageable.getSort())) {
            return demandRepository.findAll(filters, pageable);
        }
        return findSortedByApplicationName(filters, pageable);
    }
}
```

Código reconstruido

El documento `frontera-catalogo.md` describe este adaptador con precisión: «`DemandQueries` aísla esa lectura. Su implementación `integration.JpaDemandQueries` usa el repositorio habitual para las ordenaciones locales y una consulta Criteria entre Demand y Application exclusivamente cuando se ordena por nombre». También dice que «la consulta respeta filtros, ordenaciones secundarias, dirección y total de elementos» y que «la FK garantiza que la unión no pierda ni multiplique demandas». El esqueleto mostrado refleja esa descripción; el cuerpo de la consulta Criteria no se conserva y no se reproduce para no inventarlo.

Sin asociación, la unión ya no se puede expresar navegando por un atributo. La Criteria API permite declarar **dos raíces** en la misma consulta (`query.from(Demand.class)` y `query.from(Application.class)`) y relacionarlas con una condición de igualdad entre `applicationId` e `id`. Es una unión explícita entre entidades que no tienen asociación mapeada.

El documento del proyecto es muy claro sobre la naturaleza de esta pieza:

> *Este adaptador es deliberadamente propio del monolito: todavía conoce ambas entidades. [...] Antes de extraer Application a otro proceso habrá que reemplazar también este adaptador, además de la implementación del catálogo. No bastará con cambiar el catálogo por HTTP para conservar la ordenación global por nombre.*

Decisión de diseño: Un acoplamiento temporal, explícito y aislado

Motivo: mantener la funcionalidad existente mientras dure el monolito, sin cargar todo el catálogo ni todas las demandas en memoria, y dejar el acoplamiento en **una sola clase** con nombre propio en lugar de repartido por el servicio. **Consecuencia conocida:** cuando las bases de datos se separen, esta consulta no podrá existir. La decisión sobre qué hacer entonces (retirar la funcionalidad, como se hizo en el capítulo 21, o copiar los nombres en el servicio de demandas) se pospuso conscientemente.

18.7 Cómo se preparó y verificó este refactor

↑ Índice

El refactor de los pasos 5.2 a 5.6 se preparó en una copia de trabajo, y su documento de entrega deja constancia de algo importante:

> *El refactor está preparado en esta copia de trabajo. No se ha aplicado a `C:\Workspace\it-service-manager` [...]. Se conservan los 35 casos existentes, adaptando los fixtures y la comprobación de la antigua asociación JPA. Se añaden 12 casos: 7 unitarios y 5 de integración. [...] La suite NO ha podido ejecutarse.*

Es decir, el cambio se entregó con pruebas nuevas, pero **sin haber visto pasar la suite**: la compilación con Maven falló en aquel entorno por problemas de acceso a los JAR. El propio documento lo dice sin rodeos y deja como pendiente ejecutar `mvnw.cmd test` en un entorno normal antes de incorporar el cambio.

Lección aprendida: Un cambio no está terminado hasta que las pruebas pasan donde importa

El documento hizo lo correcto: no afirmó que las pruebas pasaran y dejó la verificación como pendiente explícito. La lección es tratar ese pendiente como bloqueante: ninguna fase posterior debería construirse sobre un refactor cuya suite no se ha ejecutado. El capítulo 22 retoma la ejecución de las suites, y el 23 recoge el episodio.

18.8 Errores típicos y depuración

↑ Índice

Síntoma	Causa probable	Cómo resolverlo
El listado vuelve a hacer una consulta por demanda	Se llama a `findById` del catálogo dentro del `map`	Reunir los identificadores y usar `findByIds`
Se llama al catálogo con una página vacía	Falta la comprobación del conjunto vacío	Devolver `Map.of()` sin llamar
La ordenación por nombre «funciona» pero las páginas no cuadran	Se ordenó cada página en memoria	Ordenar en la base de datos antes de paginar
El módulo de demandas sigue importando `Application`	Queda algún uso de la entidad (en pruebas, en DTO)	Buscar los `import` del paquete `application` en `demand`
`NoUniqueBeanDefinitionException: ApplicationCatalog`	Hay dos implementaciones del puerto activas a la vez	Activar solo una (ver el capítulo 20)

18.9 Ejercicios

↑ Índice

Ejercicio 18.1 · Comprobar la frontera Ver solución →

Con una búsqueda en el IDE o con `grep`, comprueba que ningún fichero del paquete `demand` (en `src/main`) importa nada de `itmanager.application`. Explica qué tendría que pasar para que una herramienta como ArchUnit convirtiera esta comprobación en una prueba automática.

Ejercicio 18.2 · Diseñar la degradación Ver solución →

Modifica, sobre el papel, el bloque `demands.map(...)` para que una aplicación ausente no haga fallar la página, sino que la fila muestre `applicationName` como `null` y el resto funcione. ¿Qué cambiaría en el contrato con el frontend? ¿Qué prueba de `DemandCatalogTest` habría que adaptar?

Autoevaluación del capítulo 18 Ver respuestas →

1. ¿Quién es el dueño de la interfaz en la arquitectura de puertos y adaptadores, y por qué importa?
2. ¿Por qué el puerto devuelve `ApplicationSummary` y no la entidad `Application`?
3. ¿Cambió el esquema de la base de datos al sustituir la asociación por `applicationId`?
4. ¿Cuántas llamadas al catálogo hace el listado para una página de diez demandas de tres aplicaciones? ¿Y para una página vacía?
5. ¿Por qué ordenar cada página en memoria no es una forma correcta de ordenar por nombre de aplicación?

CAPÍTULO 19

Nace `application-catalog-service`

↑ Volver al índice

EN ESTE CAPÍTULO

19.1 Un proyecto nuevo con su propia base de datos
19.2 El *endpoint* de consulta por lotes
19.3 Trasladar los datos existentes
19.4 Las pruebas del servicio nuevo
19.5 Errores típicos y depuración
19.6 Ejercicios

Con la frontera preparada dentro del monolito, llegó el momento de crear el segundo servicio. Este capítulo explica cómo se construyó `application-catalog-service`, su base de datos propia, el *endpoint* de consulta por lotes que necesitaba el servicio de demandas y cómo se trasladaron los datos existentes.

19.1 Un proyecto nuevo con su propia base de datos

Paso 5.7 ↖ ↑ Índice

El catálogo se convirtió en un proyecto Spring Boot independiente, en `C:\Workspace\application-catalog-service`, con su propio `pom.xml`, su propio paquete raíz (`com.albertoderafael.applicationcatalog`) y su propio proceso. El código del módulo `application` del monolito se trasladó casi sin cambios: entidad, repositorio, servicio con la validación de filtros, DTO, controlador y el manejador de errores con sus cuatro casos.

`application-catalog-service/src/main/resources/application.properties` ESTADO FINAL

```
spring.application.name=application-catalog-service

server.port=8081

# PostgreSQL
spring.datasource.url=jdbc:postgresql://localhost:5433/application_db
spring.datasource.username=application_user
spring.datasource.password=${APPLICATION_DB_PASSWORD}

# Hibernate
spring.jpa.hibernate.ddl-auto=validate
spring.jpa.open-in-view=false

# Flyway
spring.flyway.enabled=true
```

Tres decisiones se leen en este fichero:

Puerto 8081. El servicio de demandas sigue en el 8080. En desarrollo, ambos procesos conviven en la misma máquina.

Una base de datos propia, `application_db`, con un usuario propio. El servicio no comparte tablas con nadie. El patrón se conoce como *base de datos por servicio*: cada servicio es el único dueño de sus datos, y los demás solo acceden a ellos a través de su API. Que la base de datos esté en la misma instancia de PostgreSQL (el contenedor del puerto 5433) es una comodidad de desarrollo; lo importante es la separa-

ción lógica. Con un usuario distinto, ni siquiera por error podría el servicio de demandas leer esas tablas con sus credenciales.

Otra variable para la contraseña, `APPLICATION_DB_PASSWORD`, distinta de la del servicio de demandas. Cada servicio tiene sus propios secretos.

> **Pendiente: La creación de `application_db` no está versionada**
>
> El `compose.yaml` del proyecto de demandas solo crea la base de datos `it_service_manager` y su usuario. La base de datos `application_db` y el usuario `application_user` se crearon a mano en la misma instancia, y ningún fichero del repositorio lo refleja. Quien parta de cero no podrá arrancar el catálogo sin saberlo. La solución habitual es un *script* de inicialización montado en `/docker-entrypoint-initdb.d`, que la imagen oficial de PostgreSQL ejecuta solo al crear el directorio de datos, o un servicio de base de datos propio para el catálogo. Se recoge en el capítulo 24 y **se resolvió en la fase 7** con un *script* de inicialización (capítulo 27).

19.1.1 Las dependencias del servicio nuevo

El `pom.xml` del catálogo usa los mismos *starters* que el monolito y añade, para las pruebas, los módulos de prueba de Spring Boot 4 que corresponden a cada uno (`spring-boot-starter-data-jpa-test`, `spring-boot-starter-flyway-test`, `spring-boot-starter-validation-test`, `spring-boot-starter-webmvc-test`) y las tres dependencias de Testcontainers. No necesita cliente HTTP: el catálogo no llama a nadie.

19.1.2 La migración V1, sin datos

`application-catalog-service/…/V1__create_applications.sql` ESTADO FINAL

```
CREATE TABLE applications (
    id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    description VARCHAR(500),
    criticality VARCHAR(20) NOT NULL,
    status VARCHAR(20) NOT NULL
);
```

La tabla es idéntica a la del monolito, pero el servicio nuevo **no tiene migración V2** con datos iniciales. Los datos reales venían del monolito, con sus identificadores, y había que conservarlos tal cual (siguiente sección). Una V2 con datos de ejemplo habría ocupado esos identificadores o duplicado los registros.

> **Si vienes de Java EE…**
>
> Crear un servicio así equivale, en el mundo WebSphere, a separar un módulo EJB en su propio EAR, desplegado en su propio servidor y con su propio origen de datos. Lo que en Java EE requería coordinar con administración de sistemas aquí es un proyecto Maven, un puerto y una base de datos.

19.2 El *endpoint* de consulta por lotes

Paso 5.8 ↖ ↑ Índice

El puerto `ApplicationCatalog` tiene dos operaciones. `findById` se resuelve con el *endpoint* de detalle que ya existía. `findByIds` necesitaba uno nuevo:

application-catalog-service · ApplicationController.java (extracto) ESTADO FINAL

```java
@GetMapping("/summaries")
public List<ApplicationSummaryResponse> getApplicationSummaries(
        @RequestParam Set<Long> ids) {

    return applicationService.getApplicationSummaries(ids);
}
```

application-catalog-service · ApplicationService.java (extracto) ESTADO FINAL

```java
@Transactional(readOnly = true)
public List<ApplicationSummaryResponse> getApplicationSummaries(Set<Long> ids) {

    if (ids == null || ids.isEmpty()) {
        return List.of();
    }

    return applicationRepository.findAllById(ids)
            .stream()
            .map(application -> new ApplicationSummaryResponse(
                    application.getId(),
                    application.getName()
            ))
            .toList();
}
```

application-catalog-service · ApplicationSummaryResponse.java ESTADO FINAL

```java
public record ApplicationSummaryResponse(
        Long id,
        String name
) {
}
```

GET /api/applications/summaries?ids=1,2

```
[
  { "id": 1, "name": "SIGMA" },
  { "id": 2, "name": "ORION" }
]
```

Varios detalles merecen explicación:

`@RequestParam Set<Long> ids`. Spring convierte automáticamente `ids=1,2` (valores separados por comas) o `ids=1&ids=2` (parámetro repetido) en un conjunto de `Long`. Un valor no numérico produce un 400 a través del manejador de tipos incorrectos.

`/summaries` frente a `/{id}`. Las dos rutas podrían coincidir con `/api/applications/summaries`. Spring MVC resuelve el conflicto eligiendo la ruta **más específica**: un segmento literal gana a una variable. Por eso `/summaries` no se interpreta como un `id` igual a `summaries`.

Solo los que existen. `findAllById` genera un `SELECT … WHERE id IN (…)` y devuelve solo las filas encontradas. Si se piden los `id` 1 y 99 y el 99 no existe, la respuesta tiene un elemento. Es exactamente la semántica que el puerto documenta: «los identificadores ausentes no aparecen».

Un DTO mínimo. La respuesta lleva solo `id` y `name`, lo que el consumidor necesita. Pedir resúmenes de cien aplicaciones no transfiere sus descripciones.

Decisión de diseño: `GET` con los identificadores en la URL

Motivo: es una lectura idempotente que se puede almacenar en caché y depurar desde el navegador, y una página del listado tiene como mucho unas decenas de identificadores distintos. **Límite:** las URL tienen una longitud máxima práctica (del orden de miles de caracteres según servidor y *proxies*), así que no serviría para miles de identificadores. **Alternativa para ese caso:** un `POST /api/applications/summaries` con la lista en el cuerpo, que sacrifica la semántica de `GET` a cambio de no tener límite.

Cuidado: El parámetro es obligatorio

`@RequestParam` sin `required = false` exige el parámetro. Una petición sin `ids` produce una `MissingServletRequestParameterException`, que el manejador del proyecto no trata: llega al cliente como un 400 con el formato por defecto de Spring Boot (capítulo 10). El servicio de demandas nunca hace esa petición, porque con una página vacía no llama al catálogo, pero conviene saberlo.

19.3 Trasladar los datos existentes

Paso 5.9 ↖ ↑ Índice

Las demandas existentes apuntaban a aplicaciones por su `id`. Al mover el catálogo a otra base de datos, **los identificadores tenían que conservarse exactamente**: si la aplicación SIGMA tenía el `id` 1 en el monolito, debía tener el `id` 1 en el catálogo nuevo; si no, todas las demandas apuntarían a la aplicación equivocada.

La secuencia que encaja con los ficheros del proyecto es esta:

1. Arrancar el catálogo contra la base de datos `application_db` vacía. Flyway aplica su V1 y crea la tabla sin filas.
2. Volcar **solo los datos** de la tabla `applications` del monolito con `pg_dump`.
3. Restaurar ese volcado en `application_db`. Como la columna es `GENERATED BY DEFAULT AS IDENTITY`, PostgreSQL acepta los `id` explícitos del volcado.
4. Ajustar la secuencia de identidad con `setval`, para que la siguiente aplicación creada desde la API no reutilice un `id` existente (capítulo 4).
5. Comprobar en ambos lados que los `id` y los nombres coinciden.

Código reconstruido

El procedimiento exacto no está documentado. Se deduce de una evidencia indirecta: en la raíz del proyecto de demandas existe un fichero `applications_backup.sql` generado por `pg_dump` 18.6 que no contiene sentencias `CREATE TABLE` (es un volcado de datos) e incluye una llamada a `setval`. La guía no reproduce su contenido. El hecho de que el catálogo arranque sin una V2 de datos y que sus datos sean los de la V2 del monolito es coherente con esta secuencia.

Forma general de los comandos (ejemplo, sin credenciales) RECONSTRUIDO

```
# Volcar solo los datos de una tabla
pg_dump --data-only --table=applications -h localhost -p 5433 -U itmanager \
        -d it_service_manager -f applications_backup.sql

# Restaurar en la base de datos del catálogo
psql -h localhost -p 5433 -U application_user -d application_db -f applications_backup.sql
```

Lección aprendida: Las migraciones de datos también merecen un fichero

La estructura de las bases de datos está totalmente versionada con Flyway, pero el traslado de datos entre ellas fue una operación manual. Funcionó, pero quien quiera reproducirlo, o repetirlo en otro entorno, no tiene un procedimiento escrito. Un *script* o un documento de unas líneas habría bastado. Además, el volcado quedó en la carpeta del proyecto: un fichero con datos reales que conviene excluir del control de versiones o retirar una vez cumplida su función.

19.4 Las pruebas del servicio nuevo

↑ Índice

El catálogo nació con pruebas en los tres niveles. Las unitarias y las de capa web son las que el capítulo 11 ya explicó, más una para `/summaries`. Las de integración usan Testcontainers con la clase de Testcontainers 2:

`application-catalog-service · ApplicationIntegrationTest.java (extracto)` ESTADO FINAL

```
@Testcontainers
@SpringBootTest
@Transactional
class ApplicationIntegrationTest {

    @Container
    @ServiceConnection
    static PostgreSQLContainer postgres =
            new PostgreSQLContainer("postgres:18")
                    .withDatabaseName("application_db")
                    .withUsername("application_user")
                    .withPassword("test");

    @Autowired
    private ApplicationRepository applicationRepository;

    @Test
    void shouldStartWithEmptyCatalogAfterFlywayMigration() {

        assertEquals(0, applicationRepository.count());
    }
}
```

La prueba `shouldStartWithEmptyCatalogAfterFlywayMigration` fija por escrito una decisión: las migraciones del catálogo crean la estructura, no datos. Si alguien añade una V2 con datos de ejemplo, la prueba falla y obliga a pensar en los identificadores.

La otra prueba de integración, `shouldCreateAndUpdateApplicationInPostgreSQL`, crea una aplicación, la modifica, llama a `flush()` para forzar el `UPDATE` contra PostgreSQL y vuelve a leerla con `findById` para comprobar los valores.

Cuidado: Releer no siempre es releer

Después del `flush()`, la prueba no llama a `clear()`. La entidad sigue en el contexto de persistencia, así que `findById` la devuelve **desde la memoria**, sin ir a la base de datos. La prueba demuestra que el `UPDATE` es SQL válido (si fallara, `flush()` lanzaría una excepción), pero no que los valores leídos vengan de PostgreSQL. Para eso haría falta `entityManager.clear()` entre el `flush()` y la lectura, como hacen las pruebas de integración del servicio de demandas.

19.5 Errores típicos y depuración

↑ Índice

Síntoma	Causa probable	Cómo resolverlo
`database "application_db" does not exist`	La base de datos del catálogo no se creó en la instancia	Crearla, o añadir un *script* de inicialización
`Port 8081 was already in use`	Otra instancia del catálogo sigue arrancada	Pararla
`/summaries` responde 400 «Invalid value for parameter: id»	La petición llegó a `/{id}` por un error en la ruta	Revisar la URL exacta
Las demandas muestran nombres de otras aplicaciones	Los `id` no se conservaron al trasladar los datos	Restaurar con los `id` originales
`duplicate key … applications_pkey` al crear una aplicación	No se ajustó la secuencia tras la importación	Ejecutar `setval` con el máximo `id`

19.6 Ejercicios

↑ Índice

Ejercicio 19.1 · Probar el *endpoint* de lotes Ver solución →

Con el catálogo arrancado, llama a `GET /api/applications/summaries?ids=1,2,999`, después a `…?ids=1&ids=2` y por último a `…?ids=abc`. Explica cada respuesta.

Ejercicio 19.2 · Escribir el *script* de inicialización Ver solución →

Escribe el contenido de un fichero `init-application-db.sql` que cree el usuario `application_user` (con una contraseña que se lea de una variable, no escrita en el fichero; investiga cómo hacerlo con un *script* de *shell* en `/docker-entrypoint-initdb.d`) y la base de datos `application_db` de la que sea dueño. Explica por qué ese *script* no se ejecutaría sobre el volumen ya existente.

Autoevaluación del capítulo 19 Ver respuestas →

1. ¿Qué significa el patrón *base de datos por servicio* y por qué no lo invalida usar la misma instancia de PostgreSQL?
2. ¿Por qué el servicio del catálogo no tiene una migración V2 con datos?
3. ¿Cómo resuelve Spring MVC la ambigüedad entre `/summaries` y `/{id}`?
4. ¿Por qué era imprescindible conservar los identificadores al trasladar los datos, y qué hay que hacer después con la secuencia?
5. ¿Qué demuestra y qué no demuestra la relectura de `shouldCreateAndUpdateApplicationInPostgreSQL`?

CAPÍTULO 20

El adaptador HTTP con `RestClient`

↑ Volver al índice

EN ESTE CAPÍTULO

20.1 `RestClient` y la configuración del adaptador
20.2 Las dos operaciones del puerto por HTTP
20.3 Traducir los fallos remotos
20.4 Una llamada remota dentro de una transacción
20.5 Probar el adaptador contra un servidor real
20.6 Errores típicos y depuración
20.7 Ejercicios

Con el catálogo funcionando en su propio proceso, el servicio de demandas necesitaba una nueva implementación del puerto `ApplicationCatalog`: una que hablara HTTP. Este capítulo explica `RestApplicationCatalog`, por qué los tiempos máximos son imprescindibles, cómo se traducen los fallos remotos a códigos que el frontend entiende y cómo se probó el adaptador contra un servidor real.

20.1 `RestClient` y la configuración del adaptador

Paso 5.10 ↖ ↑ Índice

`RestClient` es el cliente HTTP síncrono de Spring, introducido en Spring Framework 6.1. Tiene una API fluida, parecida a la de `WebClient`, pero bloqueante, y es el sustituto natural de `RestTemplate` para código nuevo. En Spring Boot 4 su autoconfiguración llega con un *starter* propio, que el servicio de demandas añadió en este paso:

it-service-manager/pom.xml (añadido) IMPLEMENTADO

```xml
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-restclient</artifactId>
</dependency>
```

it-service-manager/…/application.properties (añadido) IMPLEMENTADO

```
application-catalog.base-url=http://localhost:8081
```

demand/adapter/RestApplicationCatalog.java (extracto) ESTADO FINAL

```java
@Service
public class RestApplicationCatalog implements ApplicationCatalog {

    private final RestClient restClient;

    public RestApplicationCatalog(
            RestClient.Builder builder,
            @Value("${application-catalog.base-url}") String baseUrl) {

        this.restClient = builder
                .baseUrl(baseUrl)
                .requestFactory(createRequestFactory())
                .build();
```

```
    }

    private static JdkClientHttpRequestFactory createRequestFactory() {

        HttpClient httpClient = HttpClient.newBuilder()
                .connectTimeout(Duration.ofSeconds(2))
                .build();

        JdkClientHttpRequestFactory factory =
                new JdkClientHttpRequestFactory(httpClient);

        factory.setReadTimeout(Duration.ofSeconds(3));

        return factory;
    }
}
```

`RestClient.Builder` inyectado. Spring Boot proporciona un *builder* ya configurado con los convertidores de mensajes de la aplicación (el mismo Jackson que usan los controladores). Se inyecta uno nuevo en cada punto de inyección, así que configurarlo aquí no afecta a otros clientes.

La URL base viene de la configuración. `@Value("${application-catalog.base-url}")` la lee de `application.properties`, y se puede sobrescribir con una variable de entorno (`APPLICATION_CATALOG_BASE_URL`) en otro entorno, sin recompilar.

Recibir la URL por constructor permite que la prueba del adaptador cree la clase con `new` y la apunte a un servidor de pruebas (última sección).

El adaptador es un `@Service` que implementa el puerto. Como el módulo de aplicaciones ya no existe en este proceso, `JpaApplicationCatalog` desaparece con él, y `RestApplicationCatalog` es la única implementación: Spring la inyecta en `DemandService` sin que este cambie una línea.

> **Nota: La transición entre los dos adaptadores**
>
> Durante el tiempo en que conviven el monolito con su módulo de aplicaciones y el servicio nuevo, es habitual mantener las dos implementaciones del puerto y elegir una al arrancar con `@Profile` o `@ConditionalOnProperty`, para poder volver atrás cambiando solo la configuración. El código final no conserva ningún mecanismo de ese tipo, así que la guía no puede afirmar cómo se hizo esa transición en el proyecto.

20.1.1 Por qué los tiempos máximos son imprescindibles

El adaptador configura dos límites:

Límite	Valor	Qué protege
Conexión (`connectTimeout`)	2 s	Que el servicio de demandas espere indefinidamente a que se establezca la conexión TCP
Lectura (`setReadTimeout`)	3 s	Que espere indefinidamente a que llegue la respuesta una vez conectado

Sin ellos, el `HttpClient` del JDK no impone un límite al tiempo de espera de una respuesta. Si el catálogo se queda colgado (un bloqueo en la base de datos, una pausa larga del recolector de basura), cada petición al servicio de demandas que necesite el catálogo se quedaría esperando. Cada una ocupa un hilo de Tomcat, que por defecto tiene un número limitado. En poco tiempo, todos los hilos estarían bloqueados esperando al catálogo y el servicio de demandas dejaría de responder **a todo**, incluidas las peticiones que no necesitan el catálogo. Es un **fallo en cascada**: el problema de un servicio se propaga al que lo llama.

Con los límites, una petición que no obtiene respuesta en, como mucho, unos cinco segundos falla de forma controlada, libera su hilo y devuelve un error claro.

> **Si vienes de Java EE...**
>
> Es el equivalente a configurar los tiempos máximos de un cliente remoto en WebSphere (por ejemplo, el tiempo de espera de las peticiones RMI-IIOP o de un cliente JAX-WS). En Java EE solía estar en la configuración del servidor; aquí está en el código del adaptador, junto a la llamada que protege.

20.2 Las dos operaciones del puerto por HTTP

↑ Índice

demand/adapter/RestApplicationCatalog.java · findById ESTADO FINAL

```
@Override
public Optional<ApplicationSummary> findById(Long id) {

    try {
        return executeRequest(() -> {

            ApplicationSummaryResponse response = restClient.get()
                    .uri("/api/applications/{id}", id)
                    .retrieve()
                    .body(ApplicationSummaryResponse.class);

            return Optional.ofNullable(response)
                    .map(item -> new ApplicationSummary(
                            item.id(),
                            item.name()
                    ));
        });

    } catch (RestClientResponseException ex) {

        if (ex.getStatusCode().value() == 404) {
            return Optional.empty();
        }

        throw ex;
    }
}
```

demand/adapter/RestApplicationCatalog.java · findByIds ESTADO FINAL

```
@Override
public Map<Long, ApplicationSummary> findByIds(Set<Long> ids) {

    if (ids.isEmpty()) {
        return Map.of();
    }

    String joinedIds = ids.stream()
            .map(String::valueOf)
            .collect(Collectors.joining(","));

    return executeRequest(() -> {

        ApplicationSummaryResponse[] response = restClient.get()
                .uri("/api/applications/summaries?ids={ids}", joinedIds)
                .retrieve()
```

```
                .body(ApplicationSummaryResponse[].class);

        if (response == null) {
            return Map.of();
        }

        return Arrays.stream(response)
                .collect(Collectors.toUnmodifiableMap(
                        ApplicationSummaryResponse::id,
                        item -> new ApplicationSummary(
                                item.id(),
                                item.name()
                        )
                ));
    });
}

private record ApplicationSummaryResponse(
        Long id,
        String name
) {
}
```

La cadena `get().uri(...).retrieve().body(...)` construye la petición, la ejecuta y convierte el cuerpo JSON en el tipo indicado. Las variables de la plantilla (`{id}`, `{ids}`) se codifican correctamente en la URL.

`retrieve()` lanza excepciones ante errores HTTP. Una respuesta 4xx produce una `HttpClientErrorException` y una 5xx una `HttpServerErrorException`; ambas son subclases de `RestClientResponseException`, que lleva el código de estado.

Un *record* privado para leer la respuesta. El adaptador no depende del DTO del catálogo (está en otro proyecto) ni lo copia entero. Declara un *record* con los dos campos que le interesan. La respuesta del detalle trae además descripción, criticidad y estado, pero la configuración de Jackson de Spring Boot **ignora las propiedades desconocidas**, así que esos campos se descartan. Es el principio del *lector tolerante*: leer solo lo que se necesita para que los cambios del proveedor que no afectan a esos campos no rompan al consumidor.

De la respuesta al dominio. El adaptador convierte el *record* de transporte en `ApplicationSummary`, el *record* del puerto. Los detalles de HTTP no salen de esta clase.

20.3 Traducir los fallos remotos

Paso 5.11 ↖ ↑ Índice

demand/adapter/RestApplicationCatalog.java · executeRequest ESTADO FINAL

```
private <T> T executeRequest(Supplier<T> request) {

    try {
        return request.get();

    } catch (RestClientResponseException ex) {

        if (ex.getStatusCode().is5xxServerError()) {
            throw new ResponseStatusException(
                    HttpStatus.SERVICE_UNAVAILABLE,
                    "Application Catalog is unavailable",
                    ex
            );
        }
```

```
            throw ex;

        } catch (ResourceAccessException ex) {

            throw new ResponseStatusException(
                    HttpStatus.SERVICE_UNAVAILABLE,
                    "Application Catalog is unavailable",
                    ex
            );
        }
    }
```

Combinando `executeRequest` con el `catch` de `findById`, cada situación remota tiene una traducción precisa:

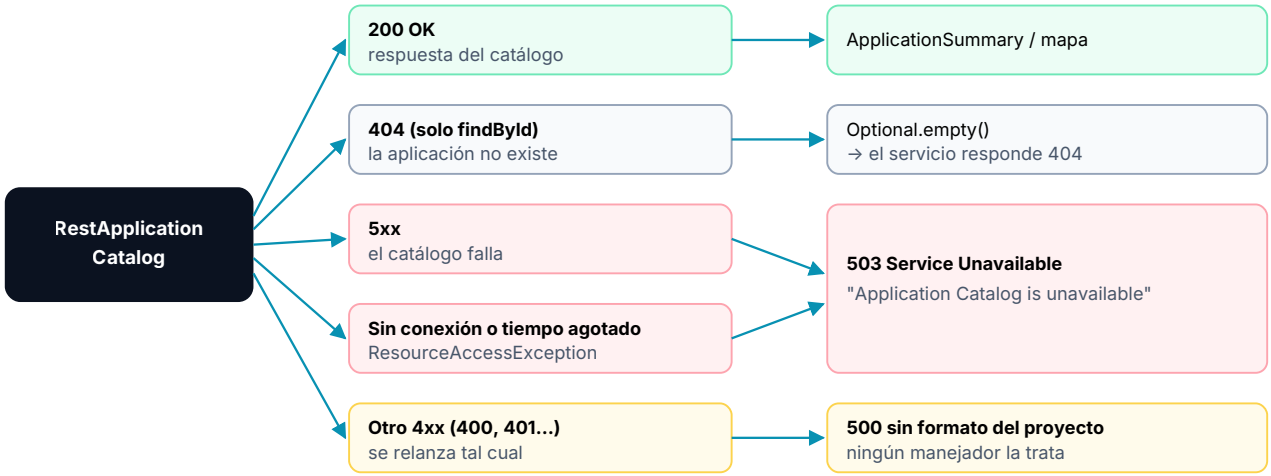


Figura 20.1. Traducción de las respuestas del catálogo. El caso inferior es un hueco: no está traducido.

20.3.1 404 frente a 503: dos significados que no deben mezclarse

La distinción más importante del adaptador es entre «la aplicación no existe» y «no puedo saber si existe».

- **404 del catálogo** significa que el catálogo respondió y la aplicación no está. Es una respuesta de negocio: el adaptador devuelve `Optional.empty()` y el servicio de demandas responde al cliente con un 404 `Application not found`, igual que en el monolito.
- **5xx, conexión rechazada o tiempo agotado** significan que no hubo respuesta fiable. Responder 404 aquí sería **mentir**: el frontend diría al usuario que la aplicación no existe cuando en realidad existe y el catálogo está caído. Por eso se responde 503.

Código	Significado en HTTP	Uso en el proyecto
404 Not Found	El recurso no existe	El catálogo confirma que la aplicación no existe
500 Internal Server Error	Error inesperado del propio servidor	Fallos no previstos
502 Bad Gateway	Un servidor intermedio recibió una respuesta inválida de otro	No se usa

Código	Significado en HTTP	Uso en el proyecto
503 Service Unavailable	El servicio no puede atender temporalmente	El catálogo no responde o falla
504 Gateway Timeout	Un servidor intermedio no recibió respuesta a tiempo	No se usa; el tiempo agotado también es 503

Decisión de diseño: Un único 503 para toda indisponibilidad del catálogo

Motivo: para el cliente del servicio de demandas lo relevante es que la operación no se puede completar **ahora** y que conviene reintentar más tarde. 503 lo expresa, y el frontend lo trata como un error temporal. **Alternativa:** distinguir 502 (respuesta inválida) y 504 (tiempo agotado), más preciso para quien opera el sistema, pero sin diferencia práctica para el usuario. **Detalle:** la excepción original se pasa como causa (tercer argumento), así que queda en el *log* aunque el cliente solo vea el mensaje genérico.

20.3.2 El hueco de los otros 4xx

Si el catálogo respondiera con un 400 o un 401 (por ejemplo, si algún día exigiera autenticación), `executeRequest` relanzaría la `RestClientResponseException` original. Ningún manejador del proyecto la trata, así que llegaría al frontend como un 500 con el formato por defecto de Spring Boot. Hoy no ocurre, porque el adaptador construye siempre peticiones válidas y el catálogo no tiene seguridad, pero es un caso sin traducir.

Pendiente: Completar la traducción de errores

Traducir cualquier 4xx inesperado del catálogo (salvo el 404 de `findById`) a un 502, porque significa que el servicio de demandas hizo una petición que el catálogo no acepta: un fallo de integración, no del usuario. Y añadir un manejador global para `RestClientException` como red de seguridad.

Resuelto en la fase 7 (capítulo 25).

20.4 Una llamada remota dentro de una transacción

↑ Índice

Hay un efecto del nuevo adaptador que no se ve en su código, sino en el del servicio:

`demand/service/DemandService.java (extracto)` ESTADO FINAL

```
@Transactional
public DemandResponse createDemand(DemandCreateRequest request) {

    ApplicationSummary application = requireApplication(request.applicationId());
    // …
```

`createDemand` es `@Transactional`, y lo primero que hace es llamar al catálogo. La transacción ya está abierta cuando empieza la llamada HTTP, así que **la conexión a la base de datos queda retenida mientras se espera al catálogo**: hasta cinco segundos en el peor caso, sin hacer nada con ella. Con el *pool* por defecto de diez conexiones, diez peticiones simultáneas esperando a un catálogo lento bastarían para que ninguna otra operación del servicio pudiera acceder a la base de datos. Ocurre lo mismo en `updateDemand` y en los métodos de consulta.

En el monolito esto no importaba: la comprobación era una consulta más en la misma conexión. Al convertirse en una llamada remota, el coste cambió sin que el código del servicio cambiara.

Pendiente: Sacar la llamada remota de la transacción

Hacer la llamada al catálogo **antes** de abrir la transacción y abrirla solo para la parte que toca la base de datos. Por ejemplo, separando la orquestación en un método sin `@Transactional` que llama al catálogo y después a otro *bean* transaccional, o usando `TransactionTemplate` para delimitar solo la escritura. La regla general: no mantener transacciones abiertas mientras se espera a la red.

Resuelto en la fase 7 con `TransactionOperations` (capítulo 25).

20.5 Probar el adaptador contra un servidor real

Paso 5.12 ↖ ↑ Índice

`demand/adapter/RestApplicationCatalogTest.java (extracto)` ESTADO FINAL

```
class RestApplicationCatalogTest {

    private HttpServer server;
    private RestApplicationCatalog catalog;
    private final AtomicInteger requests = new AtomicInteger();

    @BeforeEach
    void setUp() throws IOException {
        server = HttpServer.create(
                new InetSocketAddress("127.0.0.1", 0), 0);

        server.createContext("/api/applications", exchange -> {
            requests.incrementAndGet();

            String path = exchange.getRequestURI().getPath();

            if (path.endsWith("/summaries")) {
                respond(exchange, 200,
                        """
                        [
                          {"id":1,"name":"SIGMA"},
                          {"id":2,"name":"ORION"}
                        ]
                        """);
            } else if (path.endsWith("/404")) {
                respond(exchange, 404, "{}");
            } else if (path.endsWith("/500")) {
                respond(exchange, 500, "{}");
            } else if (path.endsWith("/1")) {
                respond(exchange, 200,
                        "{\"id\":1,\"name\":\"SIGMA\"}");
            } else {
                respond(exchange, 404, "{}");
            }
        });

        server.start();

        String baseUrl = "http://127.0.0.1:"
                + server.getAddress().getPort();

        catalog = new RestApplicationCatalog(
                RestClient.builder(), baseUrl);
```

```
    }

    @Test
    void fetchesApplicationsInSingleBatch() {
        Map<Long, ?> result = catalog.findByIds(Set.of(1L, 2L));

        assertEquals(2, result.size());
        assertEquals(1, requests.get());
    }

    @Test
    void convertsConnectionFailureTo503() {
        server.stop(0);

        ResponseStatusException ex = assertThrows(
                ResponseStatusException.class,
                () -> catalog.findById(1L));

        assertEquals(
                HttpStatus.SERVICE_UNAVAILABLE,
                ex.getStatusCode());
    }
}
```

La prueba arranca un **servidor HTTP real**, el que trae el propio JDK (`com.sun.net.httpserver.HttpServer`), en `127.0.0.1` y en el puerto 0, que significa «cualquier puerto libre». El servidor responde según la ruta: lotes, una aplicación existente, un 404 y un 500. El adaptador se crea con `new`, apuntando a ese servidor, sin arrancar Spring.

Las cinco pruebas cubren: encontrar una aplicación, un 404 que se convierte en `Optional.empty()`, el lote en **una sola petición** (el contador `requests`), un 500 que se convierte en 503 y, parando el servidor, un fallo de conexión que también se convierte en 503.

> **Decisión de diseño: Un servidor HTTP real en lugar de un simulador de Spring**
>
> **Motivo:** el adaptador se prueba de extremo a extremo, con conexiones TCP reales, el `JdkClientHttpRequestFactory` configurado y la deserialización de Jackson. Spring ofrece `MockRestServiceServer`, que intercepta las peticiones dentro del propio `RestClient` y **no** abre conexiones: no habría podido probar un fallo de conexión. **Alternativa:** WireMock, más completo (retrasos, verificación de peticiones), a cambio de una dependencia más. El servidor del JDK no necesita ninguna.

> **Pendiente: Dos casos sin probar**
>
> No hay prueba del tiempo máximo de lectura (un servidor que tarde más de tres segundos en responder) ni de un 4xx distinto del 404. El primero es el escenario más peligroso en producción; se puede probar con un manejador que haga `Thread.sleep` antes de responder, idealmente configurando el adaptador con un límite más corto para que la prueba no tarde.

20.6 Errores típicos y depuración

↑ Índice

Síntoma	Causa probable	Cómo resolverlo
`No qualifying bean of type 'RestClient.Builder'`	Falta `spring-boot-starter-restclient` (Spring Boot 4)	Añadir el *starter*

Síntoma	Causa probable	Cómo resolverlo
`Could not resolve placeholder 'application-catalog.base-url'`	Falta la propiedad	Añadirla o definir la variable de entorno
Todas las operaciones de demandas responden 503	El catálogo no está arrancado o escucha en otro puerto	Arrancarlo en el 8081 o corregir la URL
El servicio de demandas deja de responder cuando el catálogo va lento	Faltan los tiempos máximos o son demasiado largos	Configurarlos en la fábrica de peticiones
Error de deserialización al leer el detalle	El *record* de transporte tiene un tipo incompatible con el JSON	Revisar los tipos de `id` y `name`

20.7 Ejercicios

↑ Índice

Ejercicio 20.1 · Provocar un 503 Ver solución →

Con los dos servicios arrancados, para el catálogo y pide desde el frontend o con una petición manual el listado de demandas. Observa la respuesta del servicio de demandas, el mensaje y el tiempo que tarda. Vuelve a arrancar el catálogo y comprueba que todo se recupera sin reiniciar el servicio de demandas.

Ejercicio 20.2 · Probar el tiempo máximo de lectura Ver solución →

Añade a `RestApplicationCatalogTest` un contexto `/api/applications/slow` cuyo manejador espere cuatro segundos antes de responder, y una prueba que llame a `findById` sobre ese recurso y compruebe que se lanza una `ResponseStatusException` con 503. Piensa cómo podrías evitar que la prueba tarde cuatro segundos.

Autoevaluación del capítulo 20 Ver respuestas →

1. ¿Qué es un fallo en cascada y cómo lo evitan los tiempos máximos?
2. ¿Por qué el adaptador declara su propio *record* `ApplicationSummaryResponse` en lugar de reutilizar el DTO del catálogo?
3. ¿Por qué un 404 del catálogo se traduce en `Optional.empty()` y un fallo de conexión en un 503, y no en un 404?
4. ¿Qué problema aparece al llamar al catálogo dentro de un método `@Transactional`?
5. ¿Por qué `MockRestServiceServer` no habría servido para probar el fallo de conexión?

CAPÍTULO 21

Separar las bases de datos

↑ Volver al índice

EN ESTE CAPÍTULO

21.1 La migración V4 elimina la clave foránea
21.2 El estado final de la arquitectura
21.3 Retirar la ordenación por nombre de aplicación
21.4 Probar Demand sin clave foránea
21.5 Errores típicos y depuración
21.6 Ejercicios

Con el catálogo sirviendo los datos por HTTP, la tabla `applications` de la base de datos de demandas había quedado como una copia obsoleta, unida a las demandas por una clave foránea. Este capítulo explica las dos migraciones que cortaron ese último vínculo, la funcionalidad que hubo que retirar y las pruebas que verifican el resultado.

21.1 La migración V4 elimina la clave foránea

Paso 5.13 ↖ ↑ Índice

`V4__detach_demands_from_applications.sql` IMPLEMENTADO

```
ALTER TABLE demands
    DROP CONSTRAINT fk_demands_application;
```

`V5__drop_legacy_applications.sql` IMPLEMENTADO

```
DROP TABLE applications;
```

Dos migraciones de una línea, pero con consecuencias importantes. Conviene entender por qué son dos, por qué en este orden y qué garantías se pierden.

Por qué hace falta quitar la clave foránea. Desde el paso 5.10, las aplicaciones nuevas se crean en el catálogo, en otra base de datos. Si la clave foránea siguiera existiendo, una demanda sobre una aplicación creada después de la separación fallaría al insertarse: su `application_id` no existiría en la tabla local `applications`, que ya nadie actualiza. La restricción, que antes protegía la integridad, ahora impediría trabajar.

Por qué en este orden. PostgreSQL no permite borrar una tabla a la que hace referencia una clave foránea (salvo con `DROP TABLE … CASCADE`, que borraría la restricción sin dejar constancia explícita en el historial). Primero se elimina la restricción y después la tabla.

Por qué dos migraciones. Cada una tiene un propósito distinto: V4 desacopla las demandas, V5 retira datos heredados. En el historial de Flyway quedan como dos pasos con nombre propio, y cada una se aplica en su propia transacción.

Qué se conserva. La columna `application_id` y su índice `idx_demands_application_id` siguen en `demands`: el filtro por aplicación los sigue usando. Lo que cambia es su significado: ya no es una clave foránea, sino una **referencia a un recurso de otro servicio**.

Cuidado: La V5 no tiene vuelta atrás

`DROP TABLE` borra los datos. Flyway no deshace migraciones, y aunque se escribiera una V6 que recreara la tabla, estaría vacía. Antes de aplicar la V5 en cualquier base de datos con datos reales hay que asegurarse de que esos datos ya están en el catálogo (paso 5.9) y de que existe una copia de seguridad. En el proyecto, los datos se habían trasladado y verificado antes.

21.1.1 ¿Era imprescindible la V5?

Técnicamente no. Con `ddl-auto=validate`, Hibernate comprueba que existan las tablas y columnas de las entidades, pero **ignora las tablas que no corresponden a ninguna entidad**. El servicio de demandas habría arrancado igual con la tabla `applications` todavía ahí.

Se eliminó porque dejarla habría creado **dos fuentes de verdad**: unos datos de aplicaciones en el catálogo, que se actualizan, y otros en la base de datos de demandas, que se quedan congelados. Tarde o temprano, alguien consultaría la copia equivocada.

Decisión de diseño: Eliminar la copia heredada

Motivo: cada dato tiene un único dueño; una tabla huérfana con datos que parecen válidos es una fuente segura de errores. **Alternativa:** renombrarla (por ejemplo, `applications_legacy`) durante un tiempo de transición antes de borrarla, una práctica prudente cuando hay dudas sobre si algún proceso la sigue usando.

21.1.2 El historial se reproduce entero

Una consecuencia de que las migraciones sean inmutables y solo hacia delante es que **una base de datos nueva del servicio de demandas reproduce toda la historia**: V1 crea `applications`, V2 la llena, V3 crea `demands` con su clave foránea, V4 la quita y V5 borra `applications`. Es lo que ocurre cada vez que una prueba de integración arranca su contenedor de PostgreSQL.

Funciona y es la garantía de que todas las bases de datos tienen el mismo historial. Su coste es que el servicio de demandas arrastra para siempre la creación de una tabla que ya no le pertenece. En proyectos con un historial largo se suele resolver con una **línea base** (*baseline*): un *script* que crea directamente el estado actual para las instalaciones nuevas, mientras las existentes siguen su historial.

21.2 El estado final de la arquitectura

↑ Índice

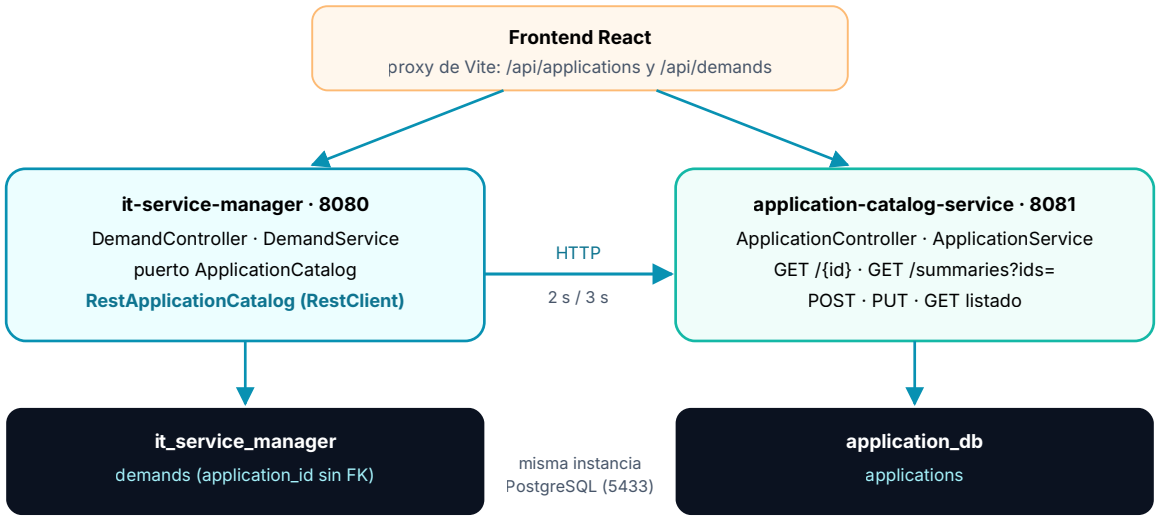


Figura 21.1. Arquitectura al final de la fase 5. El frontend habla con los dos servicios; el de demandas consulta el catálogo por HTTP; cada servicio es dueño de su base de datos.

21.3 Retirar la ordenación por nombre de aplicación

Paso 5.14 ↖ ↑ Índice

El adaptador `JpaDemandQueries` del capítulo 18 ordenaba por nombre uniendo `demands` y `applications` en la misma base de datos. Tras la V5, esa unión es imposible: las aplicaciones están en otra base de datos, detrás de otro servicio. El adaptador se retiró, y con él la posibilidad de ordenar por nombre:

demand/service/DemandService.java (extracto) ESTADO FINAL

```
private static final Set<String> VALID_SORT_FIELDS = Set.of(
        "id",
        "title",
        "priority",
        "status",
        "updatedAt"
);
```

Una petición con `sort=application.name` recibe ahora un 400 `Invalid demand sort field: application.name`. Una prueba de integración lo fija:

DemandIntegrationTest.java (extracto) ESTADO FINAL

```
@Test
void shouldRejectSortingByApplicationName() {

    ResponseStatusException exception = assertThrows(
            ResponseStatusException.class,
            () -> demandService.getDemands(
                    null,
                    null,
                    null,
                    PageRequest.of(
                            0, 10,
                            Sort.by("application.name"))));

    assertEquals(
```

```
            HttpStatus.BAD_REQUEST,
            exception.getStatusCode());

    assertEquals(
            "Invalid demand sort field: application.name",
            exception.getReason());
}
```

Decisión de diseño: Retirar una funcionalidad antes que implementarla mal

Motivo: las alternativas eran peores. Ordenar cada página en memoria da resultados incorrectos (capítulo 18). Pedir todas las demandas y todos los nombres para ordenar en memoria no escala. **Alternativas para recuperarla en el futuro:** guardar una copia del nombre de la aplicación en cada demanda (desnormalización), que exige mantenerla actualizada cuando el nombre cambie en el catálogo, normalmente con eventos; o una vista de lectura que combine datos de ambos servicios. Ambas son proyectos en sí mismos.

En el proyecto

El frontend seguía ofreciendo la ordenación por aplicación en la tabla de demandas, así que esa columna produce un 400 si se pulsa. La guía del frontend documenta el desajuste en sus notas de integración. Es el ejemplo más claro de que separar servicios puede cambiar el contrato con los clientes, y de que esos cambios deben comunicarse y coordinarse.

21.4 Probar Demand sin clave foránea

Paso 5.15 ↖ ↑ Índice

Las pruebas de integración del servicio de demandas arrancan un PostgreSQL real con Testcontainers, pero **no** arrancan el catálogo. En su lugar, simulan el puerto:

DemandIntegrationTest.java (extracto) ESTADO FINAL

```
@Testcontainers
@SpringBootTest
@Transactional
class DemandIntegrationTest {

    @Container
    @ServiceConnection
    static PostgreSQLContainer postgres =
            new PostgreSQLContainer("postgres:18")
                    .withDatabaseName("it_service_manager")
                    .withUsername("itmanager")
                    .withPassword("test");

    @Autowired
    private DemandService demandService;

    @Autowired
    private DemandRepository demandRepository;

    @Autowired
    private EntityManager entityManager;

    @MockitoBean
    private ApplicationCatalog catalog;
}
```

`@MockitoBean` sobre el puerto sustituye a `RestApplicationCatalog` en el contexto de la prueba. Todo lo demás es real: Spring, Flyway con las cinco migraciones, Hibernate, las transacciones y PostgreSQL. Es la ventaja de haber definido un puerto: la frontera con el otro servicio es exactamente el punto donde se corta la prueba.

21.4.1 Una referencia que no existe en ninguna parte

DemandIntegrationTest.java (extracto) ESTADO FINAL

```
@Test
void shouldPersistApplicationIdWithoutForeignKey() {

    Long externalApplicationId = 999999L;

    Demand demand = new Demand(
            "External application reference",
            "Verify persistence without a database FK",
            "HIGH",
            "OPEN",
            externalApplicationId);

    Demand saved = demandRepository.saveAndFlush(demand);

    entityManager.clear();

    Demand reloaded = demandRepository.findById(saved.getId())
            .orElseThrow();

    assertEquals(
            externalApplicationId,
            reloaded.getApplicationId());
}
```

La prueba guarda una demanda con un identificador de aplicación que no existe en ninguna base de datos, fuerza el `INSERT` con `saveAndFlush`, vacía el contexto con `clear()` y la relee **desde PostgreSQL**. Si la clave foránea siguiera existiendo, el `flush` fallaría. Demuestra que la V4 se aplicó.

Código reconstruido

Durante la fase del monolito con puerto (pasos 5.2 a 5.6), el documento del refactor menciona pruebas de «conservación de la FK», es decir, la comprobación opuesta: que la restricción seguía activa mientras las dos tablas convivían. Esta prueba es su contrapartida tras la V4. La versión anterior no se conserva.

21.4.2 Una página, una llamada

DemandIntegrationTest.java (extracto) ESTADO FINAL

```
@Test
void shouldFetchCatalogOnceForAWholePage() {

    for (int i = 0; i < 12; i++) {
        demandRepository.save(
                new Demand("Batch " + i, null, "HIGH", "OPEN", 7L));
    }

    entityManager.flush();
    entityManager.clear();

    when(catalog.findByIds(Set.of(7L)))
```

```
            .thenReturn(Map.of(
                    7L,
                    new ApplicationSummary(7L, "External app")));

    var page = demandService.getDemands(
            7L, null, null,
            PageRequest.of(0, 10, Sort.by("id")));

    assertEquals(10, page.content().size());
    assertEquals(12, page.totalElements());

    verify(catalog, times(1)).findByIds(Set.of(7L));
    verify(catalog, never()).findById(anyLong());
}
```

Es la versión de integración de la prueba unitaria del capítulo 18, ahora con PostgreSQL real: doce demandas, una página de diez, el total correcto (doce, calculado por la consulta de recuento) y exactamente **una** llamada al catálogo, nunca una por demanda.

21.4.3 El nombre siempre es el actual

La prueba `shouldComposeListWithCurrentNames` crea una demanda cuando la aplicación se llama «Original», cambia la respuesta del catálogo simulado a «Renamed» y comprueba que tanto el detalle como el listado muestran el nombre nuevo. Es una consecuencia directa del diseño: el servicio de demandas no guarda el nombre, lo pide cada vez. Lo que se pierde en rendimiento y en disponibilidad se gana en consistencia: el nombre nunca está desactualizado.

Prueba de `DemandIntegrationTest`	Qué garantiza
`shouldCreateDemandLinkedToExternalApplication`	El alta completa funciona con el catálogo y PostgreSQL
`shouldComposeListWithCurrentNames`	Detalle y listado muestran el nombre actual del catálogo
`shouldFetchCatalogOnceForAWholePage`	Una sola llamada por página, con paginación correcta
`shouldPersistApplicationIdWithoutForeignKey`	La clave foránea ya no existe
`shouldLoadApplicationIdWithoutAnEntityAssociation`	La referencia escalar se carga bien con `Specification` (heredera de la prueba del grafo de entidad)
`shouldRejectSortingByApplicationName`	La ordenación por nombre se rechaza con 400

21.5 Errores típicos y depuración

↑ Índice

Síntoma	Causa probable	Cómo resolverlo
`cannot drop table applications because other objects depend on it`	Se intentó la V5 sin la V4	Eliminar primero la restricción
`insert or update on table "demands" violates foreign key constraint`	La V4 no se aplicó en esa base de datos	Revisar `flyway_schema_history`
400 al ordenar la tabla de demandas por aplicación	Funcionalidad retirada en el paso 5.14	Quitar la opción del frontend

Síntoma	Causa probable	Cómo resolverlo
Las pruebas de integración intentan llamar al puerto 8081	Falta el `@MockitoBean` del puerto	Simular `ApplicationCatalog` en la prueba

21.6 Ejercicios

↑ Índice

Ejercicio 21.1 · Leer el historial completo Ver solución →

Conéctate a la base de datos de demandas y consulta `flyway_schema_history`. Comprueba que aparecen las cinco migraciones y explica qué tablas y restricciones existían después de cada una.

Ejercicio 21.2 · Diseñar la desnormalización Ver solución →

Propón cómo recuperar la ordenación por nombre de aplicación añadiendo una columna `application_name` a `demands`. Escribe la migración, indica en qué momentos habría que rellenarla o actualizarla y qué pasaría si una aplicación se renombra en el catálogo. ¿Qué mecanismo usarías para enterarte de ese cambio?

Autoevaluación del capítulo 21 Ver respuestas →

1. ¿Por qué la clave foránea tuvo que eliminarse antes de que las aplicaciones nuevas se crearan solo en el catálogo?
2. ¿Por qué no se podía ejecutar la V5 antes que la V4?
3. Si Hibernate ignora las tablas sin entidad, ¿por qué se eliminó la tabla `applications`?
4. ¿Qué sustituye `@MockitoBean` en `DemandIntegrationTest`, y qué partes del sistema siguen siendo reales?
5. ¿Qué demuestra `shouldPersistApplicationIdWithoutForeignKey` y por qué necesita `saveAndFlush` y `clear`?

FASE 6

Validación, estado final y pendientes

22 Validar el sistema completo pág. 149

23 Errores y lecciones aprendidas pág. 154

24 Estado final y próximos pasos pág. 159

↑ Volver al índice

CAPÍTULO 22

Validar el sistema completo

↑ Volver al índice

EN ESTE CAPÍTULO

22.1 Arrancar los dos servicios y el frontend
22.2 Ejecutar las suites de pruebas
22.3 Monolito frente a dos servicios
22.4 Ejercicios

La fase 6 no añade funcionalidad: comprueba que todo lo construido funciona junto. Este capítulo explica cómo arrancar los dos servicios con el frontend, qué comprobaciones manuales confirman cada decisión importante, cómo ejecutar las pruebas de ambos proyectos y en qué se diferencian, al final, el monolito y la arquitectura de dos servicios.

22.1 Arrancar los dos servicios y el frontend

Paso 6.1 ↖ ↑ Índice

El orden de arranque importa, porque cada pieza depende de la anterior:

Orden	Pieza	Puerto	Depende de
1	PostgreSQL en Docker	5433	—
2	`application-catalog-service`	8081	PostgreSQL (`application_db`)
3	`it-service-manager` (demandas)	8080	PostgreSQL (`it_service_manager`) y, en tiempo de ejecución, el catálogo
4	Frontend (Vite)	5173	Los dos servicios, a través de su proxy

Arranque en desarrollo (PowerShell, una terminal por servicio)

```
# 1. Base de datos (desde la carpeta it-service-manager)
docker compose up -d
docker compose ps            # esperar al estado "healthy"

# 2. Catálogo (desde application-catalog-service)
$cred = Get-Credential -UserName application_user -Message 'Base de datos del catálogo'
$env:APPLICATION_DB_PASSWORD = $cred.GetNetworkCredential().Password
.\mvnw.cmd spring-boot:run

# 3. Demandas (desde it-service-manager)
$cred = Get-Credential -UserName itmanager -Message 'Base de datos de demandas'
$env:DB_PASSWORD = $cred.GetNetworkCredential().Password
$env:SPRING_PROFILES_ACTIVE = 'docker'      # para conectar al puerto 5433
.\mvnw.cmd spring-boot:run

# 4. Frontend (desde it-service-manager-web)
npm run dev
```

Las contraseñas se piden con un diálogo y se guardan solo en variables de la sesión de PowerShell, sin quedar escritas en ningún fichero ni en el historial. El perfil `docker` del servicio de demandas es necesario porque su configuración por defecto apunta al puerto 5432 (capítulo 3).

El servicio de demandas arranca aunque el catálogo no esté en marcha: no lo llama al arrancar, solo cuando una petición lo necesita. Es un comportamiento deseable, porque permite arrancar los servicios en cualquier orden, pero significa que «arrancó bien» no garantiza que pueda trabajar.

22.1.1 Comprobaciones de humo

Una vez arrancado todo, una batería corta de peticiones confirma que cada decisión importante de la guía se comporta como se espera. Pueden hacerse desde el navegador, con `curl` o con `Invoke-RestMethod`:

#	Petición	Resultado esperado	Confirma
1	`GET :8081/api/applications`	200, página con SIGMA y ORION	Catálogo, datos trasladados
2	`GET :8081/api/applications/summaries?ids=1,2`	200, dos resúmenes	*Endpoint* de lotes (capítulo 19)
3	`GET :8080/api/demands`	200, página con `applicationName`	Composición con el catálogo (capítulo 18)
4	`POST :8080/api/demands` con `applicationId: 1`	201 y cabecera `Location`	Alta con comprobación remota
5	`POST :8080/api/demands` con `applicationId: 999999`	404 `Application not found: 999999`	404 del catálogo → 404 de negocio
6	`PUT :8080/api/demands/{id}` de `IN_PROGRESS` a `OPEN`	409 con el mensaje de transición	Máquina de estados (capítulo 12)
7	`GET :8080/api/demands?sort=application.name`	400 `Invalid demand sort field`	Funcionalidad retirada (capítulo 21)
8	Parar el catálogo y repetir la 3	503 `Application Catalog is unavailable`	Traducción de fallos (capítulo 20)
9	Arrancar el catálogo y repetir la 3	200 sin reiniciar demandas	Recuperación automática

Las comprobaciones 8 y 9 son las más reveladoras: muestran que la caída del catálogo no tumba al servicio de demandas, que el error que recibe el cliente es honesto (503, no 404 ni 500) y que el sistema se recupera solo.

> **En el proyecto**
>
> En desarrollo, el frontend no llama a los puertos 8080 y 8081 directamente: el proxy de Vite reenvía `/api/applications` al catálogo y `/api/demands` al servicio de demandas. Para el navegador todo tiene el mismo origen, así que no hace falta CORS. La guía del frontend explica esa configuración y qué cambiaría en producción.

22.2 Ejecutar las suites de pruebas

Paso 6.2 ↖ ↑ Índice

Cada proyecto tiene su suite, y las dos necesitan Docker arrancado para sus pruebas de integración:

```
# En application-catalog-service
.\mvnw.cmd test

# En it-service-manager
.\mvnw.cmd test
```

Recuento de las pruebas que existen en cada proyecto al cierre de la fase 5:

Proyecto	Clase	Tipo	Pruebas
Catálogo	`ApplicationServiceTest`	Unitaria	8
	`ApplicationControllerTest`	Capa web	7
	`ApplicationIntegrationTest`	Integración	2
	`ApplicationCatalogServiceApplicationTests`	Integración (contexto)	1
	Total		**18**
Demandas	`DemandTest`	Unitaria (dominio)	5
	`DemandServiceTest`	Unitaria	3
	`DemandCatalogTest`	Unitaria	7 (10 tras el cierre)
	`DemandControllerTest`	Capa web	8
	`RestApplicationCatalogTest`	Adaptador con servidor HTTP real	5 (7 tras el cierre)
	`DemandIntegrationTest`	Integración	6
	`ItServiceManagerApplicationTests`	Integración (contexto)	1
	Total		**35 (40 tras el cierre)**

En el proyecto: Resultado de la ejecución (26 de octubre de 2026)

La primera edición de esta guía se escribió leyendo el código, sin ejecutarlo, y por eso no afirmaba que las pruebas pasaran. Las suites se ejecutaron después en Windows con Java 21, Maven y Docker: **18 pruebas del catálogo y 40 del servicio de demandas, todas en verde** (`BUILD SUCCESS`). Las 40 incluyen 5 pruebas nuevas añadidas en el cierre de la fase 7 (capítulo 25): tres en `DemandCatalogTest` (demanda cuya aplicación ya no existe en el listado y en el detalle, y demanda inexistente sin llegar a llamar al catálogo) y dos en `RestApplicationCatalogTest` (4xx inesperado y respuesta ilegible, ambos traducidos a 502). El frontend pasa sus 39 pruebas.

22.2.1 Qué cubre la suite y qué no



Figura 22.1. Cobertura de la suite. Cada servicio se prueba por separado; nada comprueba automáticamente que los dos encajan.

El hueco más relevante es la **compatibilidad entre los dos servicios**. Las pruebas del servicio de demandas simulan el catálogo (con `@MockitoBean` o con el servidor del JDK), y las respuestas simuladas están escritas a mano. Si mañana el catálogo cambiara el nombre del campo `name` en `/summaries`, todas las pruebas de ambos proyectos seguirían pasando y el sistema fallaría en ejecución. Este riesgo se aborda con **pruebas de contrato** (por ejemplo, Spring Cloud Contract o Pact), en las que el consumidor declara lo que espera y el proveedor verifica que lo cumple.

22.3 Monolito frente a dos servicios

Paso 6.3 ↖ ↑ Índice

Aspecto	Monolito (fase 4)	Dos servicios (fase 5)
Procesos	1 (8080)	2 (8080 y 8081)
Bases de datos	1, compartida por los módulos	2, una por servicio (misma instancia en desarrollo)
Relación Demand → Application	Asociación `@ManyToOne`	`Long applicationId` + puerto `ApplicationCatalog`
Comprobar que la aplicación existe	Consulta local en la misma transacción	Petición HTTP con tiempos máximos
Nombre de la aplicación en el listado	Unión SQL con `@EntityGraph`	Una petición por lote por página
Integridad referencial	Clave foránea	Comprobación en el servicio al crear y modificar
Ordenar demandas por aplicación	Sí, en SQL	No; retirado
Consistencia del nombre	Siempre actual (misma base de datos)	Siempre actual (se pide cada vez)
Si falla la parte de aplicaciones	Falla todo el proceso	Demandas responde 503 en lo que necesita el catálogo
Transacciones	ACID entre módulos	Locales a cada servicio

Aspecto	Monolito (fase 4)	Dos servicios (fase 5)
Latencia de un listado	Una consulta (más el recuento)	Consulta + recuento + una petición HTTP
Despliegue	Uno	Dos, independientes
Pruebas de integración	Un contenedor, todo real	Un contenedor por servicio; el otro servicio simulado
Complejidad operativa	Baja	Mayor: dos procesos, dos configuraciones, dos bases de datos
Preparado para nuevos módulos que usen el catálogo	No: todos dependerían de la entidad	Sí: cualquier módulo puede usar la API del catálogo

La tabla deja claro que la separación **no es gratis**: se perdió una funcionalidad, aumentó la latencia del listado y apareció un modo de fallo nuevo. A cambio, el catálogo tiene un dueño, una API explícita y una base de datos propia, y los módulos futuros pueden apoyarse en él sin acoplarse a su modelo interno.

Decisión de diseño: ¿Mereció la pena?

Para el tamaño actual del sistema, un monolito modular con el puerto `ApplicationCatalog` y su adaptador JPA (el estado al final del paso 5.6) habría sido igual de mantenible y más sencillo de operar. La extracción completa se justifica por el crecimiento previsto (incidencias, equipos) y por el objetivo de aprendizaje del proyecto. Lo más valioso del proceso no fue el resultado, sino el método: la frontera se diseñó dentro del monolito y la red llegó después.

22.4 Ejercicios

↑ Índice

Ejercicio 22.1 · Ejecutar la batería de humo Ver solución →

Arranca el sistema completo y realiza las nueve comprobaciones de la tabla. Anota el código de estado y el tiempo de respuesta de la comprobación 3 en condiciones normales y de la 8 con el catálogo parado. ¿Por qué la 8 responde casi al instante y no tarda los cinco segundos de los tiempos máximos?

Ejercicio 22.2 · Ejecutar y registrar las suites Ver solución →

Ejecuta `mvnw test` en los dos proyectos y comprueba que el número de pruebas ejecutadas coincide con la tabla (18 y 35). Si alguna falla, identifica a qué nivel pertenece y qué capítulo explica lo que prueba.

Autoevaluación del capítulo 22 Ver respuestas →

1. ¿Por qué el servicio de demandas puede arrancar aunque el catálogo esté parado, y qué implica eso?
2. ¿Qué comprobación de humo demuestra que el sistema distingue «no existe» de «no está disponible»?
3. ¿Por qué ninguna de las 53 pruebas detectaría un cambio de nombre de campo en `/summaries`?
4. Cita tres cosas que se perdieron y tres que se ganaron al pasar a dos servicios.
5. ¿Qué estado intermedio de la fase 5 habría sido suficiente para el tamaño actual del sistema?

CAPÍTULO 23

Errores y lecciones aprendidas

↑ Volver al índice

EN ESTE CAPÍTULO

23.1 Recoger los errores
23.2 Lo que ocurrió
23.3 Lo detectado en la revisión del código
23.4 Trampas evitadas
23.5 Las lecciones, en una página
23.6 Ejercicios

Un proyecto real no avanza en línea recta. Este capítulo reúne lo que salió mal, lo que costó más de lo previsto y los defectos que la revisión del código para esta guía ha encontrado. Para cada caso se indica de dónde sale la evidencia, para separar lo ocurrido de lo detectado después.

23.1 Recoger los errores

Paso 6.4 ↖ ↑ Índice

Los casos se agrupan en tres categorías:

- **Ocurrido:** hay constancia en los ficheros del proyecto de que pasó.
- **Detectado en la revisión:** está en el código actual y se ha encontrado al preparar la guía; puede que nadie lo haya sufrido todavía.
- **Trampa evitada:** un error habitual que el proyecto esquivó, y conviene saber por qué.

23.2 Lo que ocurrió

↑ Índice

23.2.1 Un refactor entregado sin ver pasar las pruebas

El refactor de la frontera (pasos 5.2 a 5.6) se preparó en una copia de trabajo, con doce pruebas nuevas, pero la suite no pudo ejecutarse por problemas de acceso a los JAR de Maven en aquel entorno. El documento `verificacion-refactor.md` lo dejó escrito con honestidad: «No se ha obtenido un BUILD SUCCESS y no se afirma que las pruebas pasen».

Lección aprendida

Dejar constancia fue lo correcto. Lo que hay que añadir es la disciplina de tratarlo como **bloqueante**: las fases posteriores no deberían construirse sobre un cambio cuya suite no se ha visto pasar. Y un entorno en el que no se pueden ejecutar las pruebas es un problema que resolver antes de seguir, no una circunstancia que aceptar.

23.2.2 La copia pública con una migración distinta

La copia publicada en GitHub cambió el contenido de `V2__insert_initial_applications.sql` para usar datos ficticios. Es legítimo para una base de datos nueva, pero conectada a la base de datos original haría fallar la validación de Flyway. La guía de instalación lo advierte (capítulo 4).

Lección aprendida

Una migración aplicada es inmutable. Si se necesita una versión pública con datos distintos, lo más limpio es que los datos de ejemplo no formen parte de las migraciones versionadas (por ejemplo, un *script* aparte o una migración condicionada a un perfil de demostración).

23.2.3 Operaciones manuales sin registro

Dos operaciones de la fase 5 se hicieron a mano y no dejaron un procedimiento escrito: la creación de la base de datos `application_db` con su usuario, y el traslado de los datos de aplicaciones con un volcado de `pg_dump` (capítulo 19). Además, ese volcado, con datos reales, quedó en la carpeta raíz del proyecto de demandas.

Lección aprendida

Todo lo que cambia el estado de un entorno debería poder repetirse: un *script* de inicialización para la base de datos y unas líneas de documentación para el traslado. Los volcados de datos son artefactos temporales: deben excluirse del control de versiones y retirarse cuando cumplen su función.

23.2.4 Un cambio de contrato que el frontend no recibió

La separación eliminó la ordenación por nombre de aplicación (capítulo 21). El frontend seguía ofreciéndola, igual que la ordenación por fecha de creación, que el backend tampoco acepta. La guía del frontend lo detectó con una prueba que falla.

Lección aprendida

Separar servicios cambia contratos. Cualquier cambio que haga que una petición antes válida pase a ser rechazada debe tratarse como un cambio incompatible: anunciarse, coordinarse con los clientes y, si es posible, detectarse automáticamente con pruebas de contrato.

23.3 Lo detectado en la revisión del código

↑ Índice

#	Defecto	Dónde	Riesgo	Capítulo
1	La llamada HTTP al catálogo ocurre dentro de una transacción abierta	`DemandService`	Agotar el *pool* de conexiones si el catálogo va lento	20
2	Un 4xx del catálogo distinto de 404 se relanza sin traducir	`RestApplicationCatalog`	El cliente recibe un 500 con otro formato	20
3	Una aplicación ausente hace fallar la página entera	`DemandService.getDemands`	Una referencia rota impide ver todo el listado	18
4	La configuración por defecto apunta al 5432, pero el contenedor se publica en el 5433	`application.properties`	`Connection refused` sin el perfil `docker`	3

#	Defecto	Dónde	Riesgo	Capítulo
5	Dos clases distintas de Testcontainers para el mismo contenedor	Pruebas de demandas	Mantenimiento; una de ellas está obsoleta	16
6	Una prueba «relee» de la base de datos sin vaciar el contexto	`ApplicationIntegrationTest`	Comprueba valores en memoria, no en PostgreSQL	19
7	No hay pruebas de las transiciones hacia `CANCELLED`	`DemandTest`	Una regla de negocio sin red de seguridad	12
8	No hay prueba del tiempo máximo de lectura	`RestApplicationCatalogTest`	El escenario más peligroso no está cubierto	20
9	Excepciones sin manejador (parámetro ausente, error de integridad, genéricas)	`GlobalExceptionHandler`	Respuestas con otro formato o 500 opacos	10
10	Mensajes de error en dos idiomas	Servicios y manejador	Incoherencia en la API	10
11	Los valores válidos de estados y prioridades se repiten como texto en varios sitios	DTO, servicios, frontend	Un valor nuevo hay que añadirlo en varios lugares	8
12	`import java.time.Duration` duplicado, un `import` sin usar y sangría mezclada	`RestApplicationCatalog`	Ninguno funcional; síntoma de código copiado sin repasar	20

En el proyecto: Estado tras el cierre del 26 de octubre de 2026

Los defectos **1, 2, 3, 4 y 12 están resueltos**: la llamada al catálogo se hace fuera de la transacción, cualquier respuesta inesperada del catálogo se traduce a 502, una aplicación ausente ya no tumba el listado ni el detalle, la configuración por defecto apunta al puerto del contenedor y el adaptador está limpio. El **9** está resuelto con un manejador de último recurso y otro para parámetros ausentes. El **10** queda mitigado: el frontend ya no muestra los textos técnicos de los errores 5xx, sino mensajes propios en español. Siguen pendientes el 5, el 6, el 7, el 8 y el 11. Los detalles están en el capítulo 25.

El defecto 12 merece un comentario aunque no tenga consecuencias: Java permite repetir una importación idéntica y un `import` sin usar solo provoca un aviso. Pero son señales de que un fichero se montó a partir de fragmentos y no se revisó al final. Un formateador automático y el análisis estático del IDE (o herramientas como Checkstyle o SpotBugs en el *build*) lo habrían detectado.

Lección aprendida: La regla que más defectos habría evitado

Los defectos 1, 2 y 3 tienen la misma raíz: el código del servicio de demandas se escribió cuando el catálogo era una llamada local, y siguió igual cuando pasó a ser remota. El adaptador cambió, pero las **suposiciones** del código que lo usa (que la llamada es rápida, que siempre responde, que nunca falta un dato) no se revisaron. Al cambiar la implementación de un puerto por una remota, hay que revisar también a sus consumidores.

23.4 Trampas evitadas

↑ Índice

Trampa	Cómo se evitó	Capítulo
Esquemas distintos en cada máquina con `ddl-auto=update`	Flyway + `validate`	3, 4
Consultas ocultas al serializar con *Open Session in View*	`open-in-view=false` y DTO construidos en el servicio	3, 7
Exponer entidades en la API	DTO por operación con *records*	8
Serializar `PageImpl`	DTO de página propio	9
Asociaciones `EAGER` por defecto	`LAZY` + grafo de entidad en el listado	12, 14
Un N+1 al pasar a HTTP	Consulta por lotes preparada antes de la red	18
Ordenar páginas en memoria	Retirar la funcionalidad antes que dar resultados incorrectos	18, 21
Responder 404 cuando el catálogo está caído	Distinguir 404 de 503	20
Llamadas remotas sin límite de tiempo	Tiempos máximos de conexión y lectura	20
Pruebas contra H2 que no detectan fallos de PostgreSQL	Testcontainers con PostgreSQL 18	16
Base de datos compartida entre servicios	Una base de datos y un usuario por servicio	19
Una gran migración «*big bang*»	Pasos pequeños y reversibles	17

23.5 Las lecciones, en una página

↑ Índice

1. **La frontera primero, la red después.** Separar dentro del monolito (puerto, referencia escalar, lotes) hizo que la llegada de HTTP fuera un cambio local.
2. **Pasos pequeños, cada uno funcionando.** Lo irreversible (borrar la clave foránea y la tabla) va al final, cuando todo lo demás está probado.
3. **Cada dato, un solo dueño.** Una copia sin dueño es una fuente de errores, aunque hoy coincida.
4. **Errores honestos.** «No existe» y «no puedo saberlo» son respuestas distintas y deben llegar distintas al cliente.
5. **Las pruebas fijan decisiones.** `verify(…, never()).save(…)` o «el catálogo empieza vacío» convierten una decisión en algo que no se puede deshacer sin darse cuenta.
6. **Ver pasar la suite es parte del trabajo.** Un cambio con pruebas sin ejecutar no está terminado.
7. **Lo manual también se versiona.** Bases de datos, usuarios y traslados de datos necesitan un *script* o, como mínimo, un documento.
8. **Al cambiar una implementación por otra remota, revisa a los que la usan.** Tiempos, transacciones y datos ausentes cambian de naturaleza.

23.6 Ejercicios

↑ Índice

Ejercicio 23.1 · Priorizar los defectos Ver solución →

Ordena los doce defectos de la tabla por la combinación de probabilidad e impacto en un entorno con usuarios reales. Justifica los tres primeros y propón, para cada uno, la prueba automática que lo detectaría.

Ejercicio 23.2 · Arreglar la relectura Ver solución →

Reescribe la parte final de `shouldCreateAndUpdateApplicationInPostgreSQL` para que la relectura venga realmente de PostgreSQL. Indica qué tienes que inyectar en la prueba y en qué línea exacta va el cambio.

Autoevaluación del capítulo 23 Ver respuestas →

1. ¿Qué tienen en común los defectos 1, 2 y 3 de la tabla?
2. ¿Por qué el traslado manual de datos se considera un error si funcionó?
3. ¿Qué tipo de prueba habría detectado el cambio de contrato con el frontend?
4. ¿Qué herramientas habrían señalado los problemas del defecto 12?
5. De las ocho lecciones finales, ¿cuál habría evitado el problema del refactor sin ejecutar?

CAPÍTULO 24

Estado final y próximos pasos

↑ Volver al índice

EN ESTE CAPÍTULO

24.1 Qué está hecho
24.2 Qué no está hecho
24.3 Próximos pasos, por prioridad
24.4 Lo que vino después
24.5 Ejercicios

Este último capítulo hace inventario: qué está construido y funcionando, qué no lo está, y en qué orden conviene abordar lo que queda. Es también el punto de partida para la siguiente etapa del proyecto.

24.1 Qué está hecho

Paso 6.5 ↖ ↑ Índice

24.1.1 `application-catalog-service` (puerto 8081) ESTADO FINAL

Área	Estado
API	Listado con filtros, paginación y ordenación validada; detalle; alta; modificación; resúmenes por lote
Persistencia	Base de datos propia `application_db`, migración V1, `ddl-auto=validate`
Validación	Bean Validation en los DTO; listas blancas de filtros y campos de ordenación
Errores	Formato `ApiErrorResponse` con cuatro manejadores
Pruebas	18: unitarias, de capa web y de integración con Testcontainers

24.1.2 `it-service-manager`, servicio de demandas (puerto 8080) ESTADO FINAL

Área	Estado
API	Listado con tres filtros y ordenación validada; detalle; alta; modificación con ciclo de vida
Dominio	Máquina de estados en la entidad; fechas en UTC
Persistencia	Base de datos `it_service_manager`, migraciones V1 a V5, sin clave foránea hacia aplicaciones
Integración	Puerto `ApplicationCatalog` con adaptador HTTP: tiempos máximos, consulta por lotes, 404 y 503 diferenciados
Errores	Formato `ApiErrorResponse` con cuatro manejadores
Pruebas	35: dominio, servicio, capa web, adaptador con servidor HTTP real e integración con Testcontainers

24.1.3 Entorno de desarrollo ESTADO FINAL

- PostgreSQL 18.6 en Docker, con volumen persistente y comprobación de salud, definido en el `compose.yaml` del proyecto de demandas.

- Las dos bases de datos en esa misma instancia, publicada en el puerto 5433.
- Los dos servicios se arrancan como procesos locales con Maven; el frontend los consume a través del proxy de Vite.

24.2 Qué no está hecho

↑ Índice

Para evitar malentendidos, conviene decirlo explícitamente:

- **No hay despliegue conjunto en contenedores.** El `compose.yaml` solo levanta PostgreSQL. Los dos servicios no tienen imagen Docker ni se arrancan con Compose. (Resuelto en la fase 7, capítulos 26 y 27).
- **No hay seguridad.** Ninguno de los dos servicios autentica ni autoriza peticiones. La comunicación entre ellos tampoco está protegida.
- **No hay integración continua.** Las suites se ejecutan a mano.
- **No hay observabilidad.** No hay comprobaciones de salud expuestas por los servicios, métricas ni trazas distribuidas. (Las comprobaciones de salud llegaron en el capítulo 26).
- **No hay documentación de la API generada** (OpenAPI).
- **No se ha verificado en esta guía que las suites pasen** (capítulo 22). (Verificado después: todas pasan).

24.3 Próximos pasos, por prioridad

↑ Índice

24.3.1 Prioridad 1: verificar y cerrar la fase 5 PENDIENTE

Tarea	Por qué	Capítulo
Ejecutar las dos suites y registrar el resultado	Nada de lo demás tiene sentido sobre una base sin verificar	18, 22
Sacar la llamada al catálogo de las transacciones	Evita agotar el *pool* de conexiones	20
Traducir los 4xx inesperados del catálogo a 502	Errores coherentes para el frontend	20
Alinear la configuración por defecto con el puerto del contenedor	Arranque sin sorpresas	3
Versionar la creación de `application_db`	Reproducibilidad del entorno	19
Retirar el volcado de datos de la carpeta del proyecto	Datos reales fuera del repositorio	19
Completar las pruebas: `CANCELLED`, tiempo de lectura, relectura con `clear()`	Cubrir los huecos conocidos	12, 19, 20
Unificar la clase de Testcontainers	Eliminar la API obsoleta	16
Coordinar con el frontend los campos de ordenación	Contrato coherente	21

24.3.2 Prioridad 2: robustez PENDIENTE

- **Manejador de último recurso** para cualquier excepción, con registro completo en el *log* y un 500 con el formato del proyecto.
- **Degradación del listado** cuando falte una aplicación, en lugar de fallar la página.
- **Comprobaciones de salud** con Spring Boot Actuator (`/actuator/health`), que permiten saber si un servicio está vivo y si puede trabajar.
- **Reintentos y cortacircuitos** para las llamadas al catálogo, por ejemplo con Resilience4j: un reintento con espera para las lecturas, que son idempotentes, y un cortacircuitos que deje de llamar durante un tiempo a un catálogo que está fallando, respondiendo 503 de inmediato.
- **Enumerados** con `@Enumerated(EnumType.STRING)` para estados, prioridades y criticidades.
- **Excepciones de dominio** en lugar de `ResponseStatusException` en los servicios.

24.3.3 Prioridad 3: operación PENDIENTE

- **Imágenes de contenedor** para ambos servicios. Spring Boot puede construirlas sin escribir un `Dockerfile` con `spring-boot:build-image`.
- **Despliegue conjunto con Docker Compose**: PostgreSQL con los *scripts* de inicialización de ambas bases de datos, los dos servicios con sus variables de entorno y dependencias de arranque basadas en las comprobaciones de salud. El esquema siguiente es una **propuesta**, no algo implementado:

Propuesta de compose para el sistema completo (no implementada) PENDIENTE

```yaml
services:
  postgres:
    image: postgres:18.6
    # … volumen, healthcheck y scripts en /docker-entrypoint-initdb.d

  application-catalog:
    image: application-catalog-service:0.0.1-SNAPSHOT
    environment:
      SPRING_DATASOURCE_URL: jdbc:postgresql://postgres:5432/application_db
      APPLICATION_DB_PASSWORD: ${APPLICATION_DB_PASSWORD}
    depends_on:
      postgres: { condition: service_healthy }

  demands:
    image: it-service-manager:0.0.1-SNAPSHOT
    environment:
      SPRING_DATASOURCE_URL: jdbc:postgresql://postgres:5432/it_service_manager
      DB_PASSWORD: ${DB_PASSWORD}
      APPLICATION_CATALOG_BASE_URL: http://application-catalog:8081
    depends_on:
      postgres: { condition: service_healthy }
```

Dentro de la red de Compose, los servicios se encuentran por su nombre (`postgres`, `application-catalog`) y usan los puertos internos, no los publicados. Por eso la URL de la base de datos usa el 5432 y no el 5433.

- **Observabilidad**: métricas con Micrometer y trazas distribuidas que sigan una petición desde el servicio de demandas hasta el catálogo. Spring Boot instrumenta automáticamente los `RestClient` construidos con el *builder* autoconfigurado, que es precisamente como lo construye el adaptador.
- **Documentación de la API** con OpenAPI.
- **Pruebas de contrato** entre el servicio de demandas (consumidor) y el catálogo (proveedor).

- **Integración continua** que ejecute ambas suites en cada cambio.

24.3.4 Prioridad 4: dominio PENDIENTE

- **Concurrencia optimista** con `@Version` en las entidades, para que dos personas que modifican la misma demanda a la vez no se pisen sin saberlo; un conflicto produciría un 409, que el frontend ya sabe tratar.
- **Nuevos módulos** (incidencias, equipos) que usen el catálogo a través de su API, siguiendo el patrón del puerto y el adaptador.
- **Seguridad**: autenticación de usuarios y, entre servicios, credenciales propias para que el catálogo sepa quién lo llama.

24.4 Lo que vino después

↑ Índice

Este capítulo describe el estado al terminar la fase 6, y así se ha dejado, porque la guía es cronológica. Pero la lista de prioridades no se quedó en papel: la **fase 7** (capítulos 25 a 27) ejecutó casi entera la prioridad 1, buena parte de la 2 y el despliegue con contenedores de la 3.

Pendiente de este capítulo	Dónde se resolvió
Ejecutar las suites	Capítulo 22 (resultado) y capítulo 25 (pruebas nuevas)
Llamada al catálogo fuera de la transacción	Capítulo 25, paso 7.2
Traducción de 4xx inesperados a 502 y manejador de último recurso	Capítulo 25, paso 7.3
Degradación del listado	Capítulo 25, paso 7.4
Configuración por defecto alineada con el contenedor	Capítulo 25, paso 7.5
Comprobaciones de salud	Capítulo 26, paso 7.6
Imágenes de contenedor	Capítulo 26, pasos 7.7 y 7.8
Versionar la creación de `application_db`	Capítulo 27, paso 7.10
Despliegue conjunto con Docker Compose	Capítulo 27

Siguen pendientes la seguridad, la integración continua, las métricas y trazas, OpenAPI, las pruebas de contrato, los reintentos y cortacircuitos, los enumerados, las excepciones de dominio, la concurrencia optimista, las pruebas que faltan (transiciones hacia `CANCELLED`, tiempo máximo de lectura, relectura con `clear()`), la unificación de la clase de Testcontainers y la coordinación de los campos de ordenación con el frontend.

24.5 Ejercicios

↑ Índice

Ejercicio 24.1 · Planificar la siguiente iteración Ver solución →

Elige cinco tareas de las prioridades 1 y 2 que harías en una primera iteración de dos semanas. Para cada una, indica cómo comprobarías que está terminada (qué prueba o qué comprobación manual).

Ejercicio 24.2 · Completar la propuesta de Compose Ver solución →

Completa el esquema de la propuesta de Docker Compose: añade el volumen y la comprobación de salud de PostgreSQL, los puertos publicados de los dos servicios y una comprobación de salud para cada servicio basada en Actuator. Indica qué tendrías que añadir a los proyectos para que esa comprobación funcione.

Autoevaluación del capítulo 24 Ver respuestas →

1. ¿Qué levanta exactamente el `compose.yaml` actual del proyecto?
2. ¿Por qué la verificación de las suites es la primera tarea de la lista?
3. ¿Qué aporta un cortacircuitos frente a un simple tiempo máximo?
4. Dentro de la red de Docker Compose, ¿por qué la URL de la base de datos usaría el puerto 5432 y no el 5433?
5. ¿Qué problema resolvería `@Version` y qué código HTTP produciría un conflicto?

FASE 7

Cierre de la separación y despliegue con Docker

25 Cerrar la separación pág. 165

26 Imágenes de contenedor pág. 175

27 La plataforma con Docker Compose pág. 182

↑ Volver al índice

CAPÍTULO 25

Cerrar la separación

↑ Volver al índice

EN ESTE CAPÍTULO

25.1 Por qué una fase más
25.2 Una red de seguridad: Git antes de tocar nada
25.3 La llamada remota, fuera de la transacción
25.4 Una traducción de errores completa
25.5 Degradar en lugar de fallar
25.6 La configuración, en variables de entorno
25.7 Errores típicos y depuración
25.8 Ejercicios

La fase 5 dejó dos servicios que funcionaban, y la fase 6 una lista de defectos y pendientes. La fase 7 empezó corrigiendo los que afectaban a la robustez antes de meter nada en contenedores. Este capítulo cuenta esos cambios: la red de seguridad con Git, la llamada remota fuera de la transacción, la traducción completa de errores, la degradación del listado y la configuración por variables de entorno.

25.1 Por qué una fase más

↑ Índice

La revisión del capítulo 23 encontró defectos que no se veían mientras todo funcionaba en local, pero que habrían aparecido en cuanto los servicios corrieran en contenedores, con la red de por medio. Meterlos en Docker tal cual habría trasladado los problemas a un entorno donde son más difíciles de diagnosticar.

El plan siguió el principio de la fase 5: pasos pequeños, cada uno comprobado, y lo más arriesgado al final.

Bloque	Pasos	Qué resuelve
Robustez (este capítulo)	7.1 a 7.5	Defectos 1, 2, 3, 4, 9 y 12 del capítulo 23
Imágenes (capítulo 26)	7.6 a 7.8	Comprobaciones de salud e imagen de cada servicio
Plataforma (capítulo 27)	7.9 a 7.13	Sistema completo con Docker Compose

Hubo también un cambio de método. Hasta la fase 6 la regla era no modificar los proyectos del backend; en esta fase se editaron directamente, pero con una condición: no se podía compilar desde el entorno de edición, así que **cada cambio se verificaba ejecutando las suites en Windows** antes de hacer su commit. Es la lección del capítulo 23 aplicada: ningún paso se da por terminado sin ver pasar las pruebas.

25.2 Una red de seguridad: Git antes de tocar nada

Paso 7.1 ↖ ↑ Índice

Los proyectos locales no tenían historial. Antes del primer cambio, cada servicio se convirtió en un repositorio con un commit inicial que recoge el estado validado:

Estado inicial de cada servicio IMPLEMENTADO

```
git init
git add .
```

```
git status          # revisar ANTES del commit
git commit -m "Estado inicial: microservicio de demandas validado"
```

El `git status` previo no es un trámite: en el primer intento aparecieron decenas de ficheros de `.metadata/`, la carpeta en la que Eclipse guarda su espacio de trabajo (índices, *logs*, historial del IDE). No son código, cambian cada vez que se abre el IDE y no deben versionarse. Se añadieron al `.gitignore` junto con los ficheros que nunca deben llegar a un repositorio:

it-service-manager/.gitignore (añadido) IMPLEMENTADO

```
### Secretos y volcados locales (no versionar) ###
.env
/applications_backup.sql
/it_service_manager_backup.dump
*.dump

### Espacio de trabajo de Eclipse ###
.metadata/
```

`git rm -r --cached .metadata` sacó la carpeta del índice sin borrarla del disco.

Cuidado: La identidad de los commits es pública

Git graba en cada commit el nombre y el correo del autor, y GitHub los muestra. Para no publicar el correo personal se usa la dirección anónima que proporciona GitHub (de la forma `12345678+usuario@users.noreply.github.com`), configurada una sola vez con `git config --global user.email`. Conviene activar también en GitHub la opción que rechaza los *push* que expongan el correo real.

Si vienes de Java EE...

En Java EE el control de versiones solía ser CVS, Subversion o ClearCase, con un servidor central. Git es distribuido: el repositorio completo vive en tu máquina, los commits son locales e instantáneos y publicarlos (`push`) es un paso aparte. Por eso es barato hacer un commit por cada paso pequeño.

25.3 La llamada remota, fuera de la transacción

Paso 7.2 ↖ ↑ Índice

El defecto más serio del capítulo 23 era este: `DemandService` era `@Transactional` y llamaba al catálogo por HTTP dentro de la transacción, de modo que cada conexión a la base de datos quedaba retenida mientras se esperaba a otro servicio (capítulo 20).

La corrección consiste en que **ninguna llamada al catálogo ocurra con una transacción abierta**:

demand/service/DemandService.java (extracto) ESTADO FINAL

```
@Service
public class DemandService {

    private final DemandRepository demandRepository;
    private final ApplicationCatalog applicationCatalog;
    private final TransactionOperations transactions;

    public DemandResponse createDemand(DemandCreateRequest request) {

        // 1. Llamada remota, sin transacción abierta.
```

```
        ApplicationSummary application = requireApplication(request.applicationId());

        Demand demand = new Demand(
                request.title(),
                request.description(),
                request.priority(),
                "OPEN",
                application.id()
        );

        // 2. Escritura: save() abre y cierra su propia transacción.
        Demand saved = demandRepository.save(demand);

        return toResponse(saved, application.name());
    }

    public DemandResponse updateDemand(
            Long id,
            DemandUpdateRequest request) {

        // 1. La demanda debe existir (lectura corta del repositorio).
        findDemand(id);

        // 2. Llamada remota, sin transacción abierta.
        ApplicationSummary application = requireApplication(request.applicationId());

        // 3. Transacción corta: recargar, modificar (dirty checking) y confirmar.
        return transactions.execute(transaction -> {

            Demand demand = findDemand(id);

            demand.update(
                    request.title(),
                    request.description(),
                    request.priority(),
                    application.id()
            );

            try {
                demand.changeStatus(request.status());
            } catch (IllegalStateException ex) {
                throw new ResponseStatusException(
                        HttpStatus.CONFLICT,
                        ex.getMessage()
                );
            }

            return toResponse(demand, application.name());
        });
    }
}
```

Cada método resuelve el problema de una forma:

Método	Antes	Ahora
`createDemand`	Transacción alrededor de catálogo + `save`	Catálogo sin transacción; `save` usa la transacción propia de `SimpleJpaRepository`
`getDemandById`	Transacción de solo lectura alrededor de todo	`findById` con su transacción; el catálogo después

Método	Antes	Ahora
`getDemands`	Ídem	`findAll` (datos y recuento en una transacción de solo lectura); la consulta por lotes al catálogo después
`updateDemand`	Transacción alrededor de todo	Comprobación corta, catálogo, y transacción corta solo para la escritura

Las lecturas no pierden nada al salir de la transacción del servicio: la entidad `Demand` solo tiene atributos simples (desde el paso 5.3 no hay asociaciones perezosas), así que construir el DTO fuera de la transacción es seguro.

25.3.1 `TransactionOperations`: una transacción delimitada en el código

`updateDemand` necesita el *dirty checking* (capítulo 5), y el *dirty checking* necesita una transacción. Pero no puede ser `@Transactional` en todo el método, porque incluiría la llamada remota. La solución es **delimitar la transacción en el código**:

```
return transactions.execute(transaction -> {
    // aquí dentro hay transacción: lo que se modifique se escribe al salir
});
```

`TransactionOperations` es la interfaz de `TransactionTemplate`, la forma programática de usar transacciones en Spring. Spring Boot crea automáticamente un `TransactionTemplate` cuando hay un único gestor de transacciones, así que basta con pedirlo en el constructor. El comportamiento es el mismo que el de `@Transactional`: si el bloque lanza una excepción no comprobada (el 409 de la transición inválida), la transacción se deshace; si termina bien, se confirma.

> **Si vienes de Java EE...**
>
> Es el equivalente a las transacciones gestionadas por el bean (BMT) de EJB con `UserTransaction.begin()` y `commit()`, pero sin tener que escribir el `try` / `finally`: la plantilla abre, confirma o deshace por ti. Como en EJB, lo habitual es la versión declarativa; la programática se reserva para cuando hay que delimitar solo una parte del método, como aquí.

25.3.2 Por qué se lee la demanda dos veces

`updateDemand` busca la demanda antes de llamar al catálogo y otra vez dentro de la transacción. No es un descuido:

- **La primera lectura conserva el orden de los errores** que fijaban las pruebas: si la demanda no existe, el cliente recibe «Demand not found» sin que se llegue a llamar al catálogo.
- **La segunda carga la entidad dentro de la transacción**, para que quede gestionada y el *dirty checking* funcione. Además, si alguien borrara la demanda entre las dos lecturas, la segunda lo detectaría.

Es una consulta por clave primaria más a cambio de no retener una conexión durante una llamada HTTP: un intercambio claramente favorable.

25.3.3 Las pruebas, sin Spring

Las pruebas unitarias construyen el servicio con una implementación de `TransactionOperations` que ejecuta el bloque sin transacción:

demand/service/DemandCatalogTest.java (extracto) ESTADO FINAL

```
@BeforeEach
void setUp() {
    // Sin Spring: las transacciones se ejecutan como simples llamadas.
    service = new DemandService(repository, catalog, TransactionOperations.withoutTransaction());
}
```

Antes estas clases usaban `@InjectMocks`. Al añadir un tercer argumento al constructor, Mockito habría inyectado `null` en silencio (el riesgo que señalaba el capítulo 15). Construir el servicio con `new` hace que un cambio de constructor no compile, que es exactamente lo que se quiere.

25.4 Una traducción de errores completa

Paso 7.3 ↖ ↑ Índice

El adaptador HTTP dejaba sin traducir los 4xx distintos del 404 (capítulo 20). La versión final separa la respuesta de negocio («no existe») de todo lo demás:

demand/adapter/RestApplicationCatalog.java (extracto) ESTADO FINAL

```
@Override
public Optional<ApplicationSummary> findById(Long id) {

    return executeRequest(() -> {
        try {
            ApplicationSummaryResponse response = restClient.get()
                    .uri("/api/applications/{id}", id)
                    .retrieve()
                    .body(ApplicationSummaryResponse.class);

            return Optional.ofNullable(response)
                    .map(ApplicationSummaryResponse::toSummary);

        } catch (HttpClientErrorException.NotFound ex) {
            // Respuesta de negocio: el catálogo confirma que no existe.
            return Optional.empty();
        }
    });
}

private <T> T executeRequest(Supplier<T> request) {

    try {
        return request.get();

    } catch (RestClientResponseException ex) {

        if (ex.getStatusCode().is5xxServerError()) {
            throw new ResponseStatusException(
                    HttpStatus.SERVICE_UNAVAILABLE, UNAVAILABLE, ex);
        }

        throw new ResponseStatusException(
                HttpStatus.BAD_GATEWAY, BAD_GATEWAY, ex);

    } catch (ResourceAccessException ex) {

        throw new ResponseStatusException(
                HttpStatus.SERVICE_UNAVAILABLE, UNAVAILABLE, ex);

    } catch (RestClientException ex) {
```

```
        // Por ejemplo, un cuerpo que no se puede convertir a JSON.
        throw new ResponseStatusException(
                HttpStatus.BAD_GATEWAY, BAD_GATEWAY, ex);
    }
}
```

El 404 se captura **dentro** de la petición, con la subclase específica `HttpClientErrorException.NotFound`, así que nunca llega a `executeRequest`. Todo lo que sale de la petición es, por tanto, un fallo:

Situación	Antes	Ahora
404 en `findById`	`Optional.empty()`	Igual
5xx, conexión rechazada, tiempo agotado	503	Igual
Otro 4xx (400, 401…)	Se relanzaba: 500 con otro formato	**502**
Respuesta que no se puede leer	500	**502**

El orden de los bloques `catch` importa: `RestClientResponseException` y `ResourceAccessException` son subclases de `RestClientException`, así que la general va al final. Java no compilaría al revés.

De paso se limpió el fichero: el `import` duplicado de `Duration`, el `import` sin usar y la sangría mezclada que recogía el defecto 12.

Dos pruebas nuevas en `RestApplicationCatalogTest` cubren los casos añadidos: un servidor que responde 400 y otro que responde 200 con un cuerpo que no es JSON. Ambos deben producir un 502.

25.4.1 El manejador de último recurso

El tratamiento de errores del proyecto tenía huecos (capítulo 10). Se añadieron dos manejadores en los dos servicios:

exception/GlobalExceptionHandler.java (añadido) ESTADO FINAL

```
@ExceptionHandler(MissingServletRequestParameterException.class)
public ResponseEntity<ApiErrorResponse> handleMissingParameter(
        MissingServletRequestParameterException ex) {

    ApiErrorResponse body = new ApiErrorResponse(
            400,
            "VALIDATION_ERROR",
            "Missing required parameter: " + ex.getParameterName(),
            Map.of()
    );

    return ResponseEntity.badRequest().body(body);
}

@ExceptionHandler(Exception.class)
public ResponseEntity<ApiErrorResponse> handleUnexpected(Exception ex) {

    if (ex instanceof ErrorResponse errorResponse) {
        HttpStatusCode status = errorResponse.getStatusCode();
        String detail = errorResponse.getBody().getDetail();

        ApiErrorResponse body = new ApiErrorResponse(
                status.value(),
                "RESOURCE_ERROR",
```

```
                detail != null ? detail : "Request could not be processed",
                Map.of()
        );

        return ResponseEntity.status(status).body(body);
    }

    log.error("Unexpected error", ex);

    ApiErrorResponse body = new ApiErrorResponse(
            500,
            "INTERNAL_ERROR",
            "Unexpected error",
            Map.of()
    );

    return ResponseEntity.internalServerError().body(body);
}
```

El detalle importante es la interfaz `ErrorResponse`. Las excepciones que Spring MVC lanza por sí mismo (ruta inexistente, verbo no permitido, tipo de contenido no soportado…) la implementan y llevan su propio código HTTP. Un manejador de `Exception` ingenuo las convertiría todas en 500: una URL mal escrita parecería un fallo del servidor. Comprobando `ErrorResponse`, esas excepciones conservan su código y solo lo realmente inesperado se convierte en 500, se registra completo en el *log* y llega al cliente con un mensaje genérico.

> **En el proyecto**
>
> El primer efecto visible de este manejador llegó por casualidad. Al probar `/actuator/health` (capítulo 26) en un servicio arrancado antes de añadir Actuator, la respuesta fue `{"status":404,"error":"RESOURCE_ERROR","message":"No static resource actuator/health."}`: un 404 correcto, **con el formato del proyecto**. Antes, esa misma petición habría devuelto el formato por defecto de Spring Boot.

25.5 Degradar en lugar de fallar

Paso 7.4 ↖ ↑ Índice

El capítulo 18 decidió que, si el catálogo no devolvía alguna aplicación de la página, el listado entero fallaba con un 404. Tras separar las bases de datos esa situación puede darse, y fallar toda la página por una referencia rota es desproporcionado. La versión final devuelve la fila con el nombre a `null`:

demand/service/DemandService.java · getDemands (extracto) ESTADO FINAL

```
// Una aplicación ausente del catálogo no tumba la página: la fila se
// devuelve con applicationName = null y el frontend la marca como no disponible.
Page<DemandResponse> responsePage = demands.map(demand -> {
    ApplicationSummary application = applications.get(demand.getApplicationId());
    return toResponse(demand, application == null ? null : application.name());
});
```

El detalle hace lo mismo con un método propio:

demand/service/DemandService.java (extracto) ESTADO FINAL

```
/** Nombre actual de la aplicación, o null si ya no existe en el catálogo. */
private String currentApplicationName(Long applicationId) {
    return applicationCatalog.findById(applicationId)
```

```
            .map(ApplicationSummary::name)
            .orElse(null);
}
```

La regla queda así: **leer** una demanda nunca falla porque falte su aplicación; **crear o modificar** una demanda sí exige que la aplicación exista (404). Es la distinción entre mostrar datos que ya existen y aceptar datos nuevos.

Situación	Respuesta
Listado con una aplicación ausente	200; esa fila con `applicationName: null`
Detalle con la aplicación ausente	200 con `applicationName: null`
Alta o modificación con una aplicación inexistente	404 `Application not found`
Catálogo caído	503 en todos los casos

El cambio de contrato se coordinó con el frontend: el tipo `applicationName` pasó a ser `string | null`, y las cuatro pantallas que lo muestran usan el texto «Aplicación no disponible». Tres pruebas nuevas de `DemandCatalogTest` fijan el comportamiento: la fila sin aplicación en el listado, el detalle sin aplicación y que una demanda inexistente se rechaza sin llamar al catálogo.

25.5.1 Mensajes de error para personas

El frontend mostraba tal cual el texto del servidor, de modo que con el catálogo caído el usuario leía «Application Catalog is unavailable». Los textos de los errores 5xx son técnicos, y además están en inglés. El cliente HTTP del frontend pasó a traducirlos **por código HTTP, no por texto**:

`it-service-manager-web · src/lib/api-client.ts (extracto)` ESTADO FINAL

```
function serverErrorMessage(status: number): string {
  switch (status) {
    case 502:
      return 'El catálogo de aplicaciones ha respondido de forma inesperada. Inténtalo de nuevo más
tarde.'
    case 503:
      return 'El catálogo de aplicaciones no está disponible en este momento. Inténtalo de nuevo en
unos segundos.'
    default:
      return 'Se ha producido un error inesperado en el servidor. Inténtalo de nuevo más tarde.'
  }
}
```

Hoy los únicos 502 y 503 con el formato del backend los produce el servicio de demandas cuando no puede consultar el catálogo, y por eso el mensaje puede nombrarlo. El aviso de «datos sin actualizar» de los listados incluye ahora la causa.

25.6 La configuración, en variables de entorno

Paso 7.5 ↖ ↑ Índice

Una imagen de contenedor debe ser **la misma en cualquier entorno**; lo que cambia de un entorno a otro tiene que llegar de fuera. La configuración de los dos servicios pasó a leerse entera de variables de entorno, con valores por defecto pensados para desarrollo:

it-service-manager/src/main/resources/application.properties ESTADO FINAL

```
spring.application.name=it-service-manager

# PostgreSQL
spring.datasource.url=${DB_URL:jdbc:postgresql://localhost:5433/it_service_manager}
spring.datasource.username=${DB_USERNAME:itmanager}
spring.datasource.password=${DB_PASSWORD}

# JPA / Hibernate
spring.jpa.hibernate.ddl-auto=validate
spring.jpa.open-in-view=false
spring.jpa.show-sql=${SHOW_SQL:false}

# Catálogo de aplicaciones (dentro de Docker: http://application-catalog-service:8081)
application-catalog.base-url=${APPLICATION_CATALOG_BASE_URL:http://localhost:8081}
```

La sintaxis `${VARIABLE:valor}` significa «el valor de la variable, o este si no existe». Cada decisión del fichero tiene su motivo:

- **El valor por defecto del puerto es el 5433**, el del contenedor. Esto resuelve el defecto 4: la configuración por defecto coincide con la forma habitual de trabajar, y el perfil `docker` deja de ser necesario. Se retiró `application-docker.properties`.
- **La contraseña no tiene valor por defecto.** Sin `DB_PASSWORD`, el servicio no arranca. Es preferible a arrancar con una contraseña vacía o, peor, con una escrita en el código.
- **`show-sql` pasa a ser `false` por defecto.** En un contenedor, el SQL en la consola solo añade ruido. Para verlo en desarrollo basta con definir `SHOW_SQL=true`.

En el catálogo, la contraseña admite dos nombres:

application-catalog-service · application.properties (extracto) ESTADO FINAL

```
spring.datasource.password=${DB_PASSWORD:${APPLICATION_DB_PASSWORD}}
```

Es un marcador anidado: primero `DB_PASSWORD` y, si no existe, el nombre antiguo, `APPLICATION_DB_PASSWORD`. Así los dos servicios usan el mismo nombre de variable (cada contenedor tiene su propio entorno, así que no chocan) sin romper la configuración de ejecución que ya existía en el IDE.

> **En el proyecto: Una confirmación inesperada**
>
> Al retirar el perfil `docker` de la configuración de ejecución de Spring Tools apareció un segundo argumento: `--spring.profiles.active=local-rest`. Ese perfil ya no hacía nada (el código no contiene ningún `@Profile`), pero es el rastro de la transición del capítulo 20, en la que convivieron el adaptador JPA y el REST. La guía la había descrito como probable sin poder afirmarla; la configuración del IDE la confirmó.

25.7 Errores típicos y depuración

↑ Índice

Síntoma	Causa probable	Cómo resolverlo
`NullPointerException` en `transactions.execute` en una prueba	El servicio se creó con `@InjectMocks` y falta `TransactionOperations`	Construirlo con `new` y `TransactionOperations.withoutTransaction()`

Síntoma	Causa probable	Cómo resolverlo
Un cambio en `updateDemand` no se guarda	La modificación se hizo fuera del bloque `execute`	Recargar y modificar la entidad dentro del bloque
Una URL inexistente responde 500	El manejador de `Exception` no distingue `ErrorResponse`	Respetar el código de las excepciones de Spring MVC
`Could not resolve placeholder 'DB_PASSWORD'`	Falta la variable en el entorno	Definirla en el IDE, en `.env` o en el contenedor
El IDE sigue conectando al 5432	Sigue activo un perfil o un argumento antiguo	Revisar los argumentos de la configuración de ejecución
El frontend muestra «null» como nombre de aplicación	El tipo no admitía `null`	Tipo `string | null` y texto alternativo

25.8 Ejercicios

↑ Índice

Ejercicio 25.1 · Comprobar que no hay transacción durante la llamada Ver solución →

Escribe una prueba de integración que simule el catálogo con `@MockitoBean` y, dentro de la respuesta simulada de `findById` (con `thenAnswer`), compruebe con `TransactionSynchronizationManager.isActualTransactionActive()` que no hay ninguna transacción activa. Ten en cuenta que la clase `DemandIntegrationTest` es `@Transactional`: ¿qué tendrías que cambiar para que la prueba tenga sentido?

Ejercicio 25.2 · La regla de la degradación Ver solución →

Explica por qué el listado y el detalle degradan (`applicationName: null`) mientras que el alta y la modificación responden 404. Pon un ejemplo de lo que pasaría si el alta también degradara.

Autoevaluación del capítulo 25 Ver respuestas →

1. ¿Por qué `createDemand` ya no necesita `@Transactional` y, aun así, el `INSERT` se hace dentro de una transacción?
2. ¿Qué aporta `TransactionOperations` frente a `@Transactional` en `updateDemand`?
3. ¿Por qué el 404 del catálogo se captura dentro de la petición y no en `executeRequest`?
4. ¿Qué problema evita comprobar `ErrorResponse` en el manejador de último recurso?
5. ¿Qué significa `${DB_PASSWORD:${APPLICATION_DB_PASSWORD}}` y por qué la contraseña no tiene un valor por defecto real?

CAPÍTULO 26

Imágenes de contenedor

↑ Volver al índice

EN ESTE CAPÍTULO

26.1 Comprobaciones de salud con Actuator
26.2 Qué son una imagen y un contenedor
26.3 El Dockerfile en dos etapas
26.4 Seguridad básica de la imagen
26.5 Probar una imagen sola
26.6 Errores típicos y depuración
26.7 Ejercicios

Con el código preparado, cada servicio necesitaba dos cosas para vivir dentro de Docker: una forma de decir si está sano y una imagen que lo empaquete. Este capítulo explica Spring Boot Actuator, qué son una imagen y un contenedor, el Dockerfile de dos etapas con el JAR por capas y las medidas básicas de seguridad de la imagen.

26.1 Comprobaciones de salud con Actuator

Paso 7.6 ↖ ↑ Índice

Un orquestador como Docker Compose necesita saber si un servicio funciona. Que el proceso esté arrancado no basta: puede estar vivo pero sin conexión a su base de datos. Spring Boot Actuator resuelve esto con **dos cambios de configuración y ninguna línea de Java**:

pom.xml (añadido en los dos servicios) ESTADO FINAL

```xml
<!-- Comprobaciones de salud (/actuator/health) para Docker -->
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>
```

application.properties (añadido en los dos servicios) ESTADO FINAL

```
# Actuator: solo se expone el estado de salud (usado por Docker Compose).
management.endpoints.web.exposure.include=health
management.endpoint.health.probes.enabled=true
management.endpoint.health.show-details=never
```

26.1.1 Qué hace la autoconfiguración

Al arrancar, Spring Boot detecta Actuator en el *classpath* y crea por sí mismo todo lo necesario (capítulo 2):

1. **Los *endpoints***, registrados en Spring MVC bajo `/actuator`, como si fueran un controlador más.
2. **Los indicadores de salud.** Como existe un `DataSource`, añade uno que valida una conexión con PostgreSQL; añade otros para el espacio en disco y el estado del proceso. `/actuator/health` combina todos: si alguno está `DOWN`, el conjunto está `DOWN` y la respuesta es un **503**.

3. **Los grupos `liveness` y `readiness`**, activados por `probes.enabled`. *Liveness* indica si el proceso debe reiniciarse; *readiness*, si puede recibir tráfico. Son los conceptos que usa Kubernetes, y Spring Boot solo los activa automáticamente cuando detecta que corre dentro de él.

```
GET /actuator/health
```

```
{"groups":["liveness","readiness"],"status":"UP"}
```

26.1.2 Por qué solo `health` y sin detalles

Actuator trae muchos más *endpoints*: variables de entorno, configuración completa, volcado de hilos, métricas, información de la aplicación... Varios revelan información sensible: rutas, versiones o propiedades de configuración. La regla es **exponer solo lo que se usa**, y aquí solo se usa la salud.

`show-details=never` hace que la respuesta diga solo `UP` o `DOWN`. Con `always`, cualquiera vería qué componente falla, la versión de PostgreSQL o el espacio libre en disco.

Decisión de diseño: Sin indicador propio para el catálogo

Habría sido fácil añadir un `HealthIndicator` que marcara el servicio de demandas como `DOWN` cuando el catálogo no responde. Se descartó a propósito: el servicio de demandas **sigue siendo útil** con el catálogo caído (responde 503 solo en lo que lo necesita, capítulo 20). Si su salud dependiera del catálogo, un orquestador podría reiniciarlo o retirarlo del tráfico por un fallo ajeno, y el problema de un servicio se convertiría en el de dos: el fallo en cascada del capítulo 20, provocado esta vez por la propia monitorización.

Cuidado: El IDE no ve la dependencia nueva

Las pruebas con `mvnw` incluyeron Actuator a la primera, porque Maven lee el `pom.xml` cada vez. Spring Tools, en cambio, mantiene su propia copia de las dependencias del proyecto: hasta ejecutar *Maven → Update Project* (Alt+F5) y reiniciar el servicio, `/actuator/health` devolvía un 404. Es un síntoma clásico: el código está bien y el IDE va por detrás.

26.2 Qué son una imagen y un contenedor

↑ Índice

Antes del Dockerfile conviene fijar tres conceptos:

Concepto	Qué es	Analogía Java
Imagen	Un sistema de ficheros de solo lectura con todo lo necesario para ejecutar algo: sistema base, JRE, aplicación y comando de arranque	La clase
Contenedor	Un proceso aislado que se ejecuta a partir de una imagen, con su propia capa escribible, red y variables de entorno	La instancia
Capa	Cada instrucción del Dockerfile que modifica ficheros produce una capa; las imágenes se componen apilando capas, y las capas se reutilizan entre imágenes y entre construcciones	—

Las capas son la clave del rendimiento. Docker guarda en caché cada capa y, al reconstruir, reutiliza todas hasta la primera instrucción cuyo resultado cambie. **El orden de las instrucciones determina cuánto se reconstruye.**

> **Si vienes de Java EE...**
>
> En WebSphere desplegabas un EAR sobre un servidor que alguien había instalado y configurado. Una imagen lleva **todo** dentro: el sistema operativo base, la JVM exacta y la aplicación. Por eso se comporta igual en tu equipo, en otro y en un servidor. La analogía falla en un punto: un contenedor es un proceso más del sistema anfitrión, no una máquina virtual; arranca en segundos y no tiene su propio núcleo.

26.3 El Dockerfile en dos etapas

Paso 7.7 ↖ ↑ Índice

it-service-manager/Dockerfile ESTADO FINAL

```
# syntax=docker/dockerfile:1

# ---------- Etapa 1: compilación ----------
FROM maven:3.9-eclipse-temurin-21 AS build
WORKDIR /workspace

# Primero solo el pom: las dependencias quedan en una capa en caché
# y no se vuelven a descargar mientras el pom no cambie.
COPY pom.xml .
RUN mvn -B -q dependency:go-offline

COPY src ./src
RUN mvn -B -q -DskipTests package \
 && cp target/*.jar app.jar \
 && java -Djarmode=tools -jar app.jar extract --layers --launcher --destination extracted

# ---------- Etapa 2: ejecución ----------
FROM eclipse-temurin:21-jre
WORKDIR /app

RUN groupadd --system spring && useradd --system --gid spring --no-create-home spring

# Capas de la menos a la más cambiante: al modificar el código
# solo se reconstruye la última.
COPY --from=build /workspace/extracted/dependencies/ ./
COPY --from=build /workspace/extracted/spring-boot-loader/ ./
COPY --from=build /workspace/extracted/snapshot-dependencies/ ./
COPY --from=build /workspace/extracted/application/ ./

USER spring

# La JVM respeta el límite de memoria del contenedor.
ENV JAVA_TOOL_OPTIONS="-XX:MaxRAMPercentage=75"

EXPOSE 8080

ENTRYPOINT ["java", "org.springframework.boot.loader.launch.JarLauncher"]
```

El del catálogo es idéntico salvo el puerto (`EXPOSE 8081`).

26.3.1 Dos etapas: compilar no es ejecutar

La primera etapa (`AS build`) parte de una imagen con Maven y el JDK completo, compila y prepara el resultado. La segunda parte de una imagen **solo con el JRE** y copia únicamente lo necesario con `COPY --from=build`. Todo lo de la primera etapa (Maven, el JDK, el código fuente, la caché de dependencias) se descarta: la imagen final no lo contiene.

Así la imagen es más pequeña y, sobre todo, tiene **menos superficie de ataque**: no lleva un compilador ni herramientas de construcción que alguien pudiera aprovechar.

26.3.2 El `pom.xml` antes que el código

```
COPY pom.xml .
RUN mvn -B -q dependency:go-offline
COPY src ./src
```

El orden no es casual. Las dependencias cambian poco y tardan en descargarse (unos 35 segundos la primera vez); el código cambia constantemente. Copiando primero solo el `pom.xml` y descargando las dependencias en su propia capa, Docker reutiliza esa capa mientras el `pom.xml` no cambie. Al tocar una clase, la reconstrucción empieza en `COPY src` y tarda unos segundos.

26.3.3 El JAR por capas

Un JAR de Spring Boot contiene dependencias que casi nunca cambian (decenas de megas) y el código propio, que cambia en cada versión (unos kilobytes). Si se copiara el JAR entero, cualquier cambio de una línea generaría una capa nueva con todo.

`java -Djarmode=tools -jar app.jar extract --layers --launcher` es una herramienta incluida en el propio JAR que lo descompone en cuatro carpetas, de la más estable a la más cambiante:

Carpeta	Contenido	Cambia…
`dependencies/`	Librerías publicadas (Spring, Hibernate, el driver…)	Al actualizar versiones
`spring-boot-loader/`	El lanzador de Spring Boot	Casi nunca
`snapshot-dependencies/`	Dependencias `-SNAPSHOT`	Rara vez (aquí, ninguna)
`application/`	Las clases y recursos propios	En cada cambio de código

Cada `COPY` crea una capa. Al cambiar el código solo cambia la última, de pocos kilobytes, y al publicar o desplegar la imagen solo viaja esa capa. El arranque se hace con `JarLauncher` sobre las clases ya extraídas, que además arranca algo más rápido que un JAR anidado.

> **Decisión de diseño: La imagen oficial de Maven en lugar de `mvnw`**
>
> El proyecto tiene Maven Wrapper (`mvnw`), y usarlo en el Dockerfile parecería lo natural. Pero el *wrapper* de estos proyectos es del tipo *only-script*: para descargar Maven necesita `curl` o `wget` en la imagen, y la imagen base no garantiza tenerlos. Partir de `maven:3.9-eclipse-temurin-21` , que ya trae Maven y Java 21, elimina esa dependencia. **Coste:** la versión de Maven de la imagen puede no coincidir exactamente con la del *wrapper*. No importa mientras sea Maven 3.9 o superior.

26.3.4 Las pruebas no se ejecutan dentro

`-DskipTests` omite las pruebas en el *build* de la imagen. No es pereza: las pruebas de integración usan Testcontainers, que necesita acceso a Docker, y Docker no está disponible dentro de un *build* de Docker sin configuraciones especiales. Las pruebas se ejecutan **antes**, con `mvnw test` , y la imagen se construye solo a partir de código ya verificado. En una integración continua (capítulo 24), ese orden lo garantiza el propio flujo.

26.4 Seguridad básica de la imagen

Paso 7.8 ↖ ↑ Índice

26.4.1 Un usuario sin privilegios

```
RUN groupadd --system spring && useradd --system --gid spring --no-create-home spring
USER spring
```

Por defecto, el proceso de un contenedor se ejecuta como `root`. Si alguien aprovechara una vulnerabilidad de la aplicación, tendría privilegios de administrador dentro del contenedor, y eso facilita escapar de él. Con `USER spring`, el proceso se ejecuta con un usuario sin privilegios. La primera línea del *log* del contenedor lo confirma: `started by spring in /app`.

26.4.2 Lo que no debe entrar en la imagen

`it-service-manager/.dockerignore` ESTADO FINAL

```
# Solo pom.xml y src/ son necesarios para construir la imagen.
target/
.git/
.metadata/
.settings/
.mvn/
.classpath
.project
.factorypath
.vscode/
.idea/
*.iml
.env
*.dump
*backup*.sql
docs/
HELP.md
```

Al construir, Docker envía la carpeta del proyecto (el **contexto**) al motor de construcción. El `.dockerignore` excluye lo que no hace falta, y eso tiene dos efectos: el *build* es más rápido, y **ningún secreto ni volcado de datos puede acabar dentro de una imagen**, aunque alguien escribiera por error `COPY . .`. Una imagen publicada con un `.env` dentro expondría las contraseñas a cualquiera que la descargara.

26.4.3 La memoria del contenedor

```
ENV JAVA_TOOL_OPTIONS="-XX:MaxRAMPercentage=75"
```

Java 21 detecta el límite de memoria del contenedor, pero por defecto solo usa una cuarta parte para el *heap*. Con `MaxRAMPercentage=75`, la JVM usa hasta el 75 % y deja el resto para la memoria fuera del *heap* (*metaspace*, hilos, *buffers*). La primera línea del *log* (`Picked up JAVA_TOOL_OPTIONS`) confirma que se aplica.

26.5 Probar una imagen sola

↑ Índice

Antes de montar el sistema completo, conviene comprobar que cada imagen funciona por sí misma:

```
docker build -t application-catalog-service:dev .

docker run --rm -p 8081:8081 `
  -e DB_URL=jdbc:postgresql://host.docker.internal:5433/application_db `
  -e DB_PASSWORD=<contraseña> `
  application-catalog-service:dev
```

Hay un detalle que confunde siempre la primera vez: **dentro de un contenedor, `localhost` es el propio contenedor**, no tu Windows. Para llegar al PostgreSQL publicado en el puerto 5433 de tu equipo, Docker Desktop ofrece el nombre especial `host.docker.internal`.

El *log* de esa prueba confirma, línea a línea, varias decisiones de la guía: la conexión llega por la variable de entorno (`jdbc:postgresql://host.docker.internal:5433/application_db`), Flyway valida la migración sin cambiar nada (`Schema "public" is up to date`) y Actuator expone un único *endpoint* (`Exposing 1 endpoint beneath base path '/actuator'`).

> **Cuidado: Contraseñas en la línea de comandos**
>
> Lo que se escribe en PowerShell queda en su historial. Para una prueba local puntual es aceptable; para el uso diario, la contraseña debe ir en un fichero `.env` que no se versiona, que es lo que hace el capítulo 27.

26.6 Errores típicos y depuración

↑ Índice

Síntoma	Causa probable	Cómo resolverlo
`/actuator/health` devuelve 404 en el IDE	Spring Tools no ha actualizado las dependencias	*Maven → Update Project* y reiniciar
`/actuator/health` devuelve `DOWN` y 503	El servicio no llega a su base de datos	Revisar `DB_URL` y que PostgreSQL esté arrancado
`Connection refused` a `localhost:5433` desde el contenedor	`localhost` es el propio contenedor	Usar `host.docker.internal` (o el nombre del servicio en Compose)
Cada *build* tarda lo mismo aunque solo cambie una clase	Se copia el código antes de descargar las dependencias	`COPY pom.xml` y `dependency:go-offline` antes de `COPY src`
`no main manifest attribute` o clase no encontrada al arrancar	El `ENTRYPOINT` no corresponde a la extracción por capas	Usar `org.springframework.boot.loader.launch.JarLauncher`
La imagen contiene el `.env`	Falta `.dockerignore` y se copió el contexto entero	Añadirlo y reconstruir; cambiar las contraseñas expuestas

26.7 Ejercicios

↑ Índice

Ejercicio 26.1 · Medir el efecto de las capas Ver solución →

Construye la imagen del catálogo, cambia un texto cualquiera en una clase y vuelve a construirla. Compara los tiempos y observa en la salida qué pasos aparecen como `CACHED`. Después cambia el `pom.xml` (por ejemplo, añade un comentario) y repite. Explica las diferencias.

Ejercicio 26.2 · Ver las capas de una imagen Ver solución →

Ejecuta `docker history it-service-manager:local` e identifica las capas que corresponden a las cuatro carpetas del JAR extraído. ¿Cuál ocupa más? ¿Cuál cambiaría al modificar una clase?

Autoevaluación del capítulo 26 Ver respuestas →

1. ¿Qué tres cosas crea la autoconfiguración de Actuator sin escribir Java?
2. ¿Por qué no se añadió un indicador de salud que dependa del catálogo?
3. ¿Qué gana la imagen final al separar la compilación y la ejecución en dos etapas?
4. ¿Por qué se copia el `pom.xml` y se descargan las dependencias antes de copiar `src/`?
5. ¿Qué significa `localhost` dentro de un contenedor y cómo se llega desde él al PostgreSQL de tu equipo?

CAPÍTULO 27

La plataforma con Docker Compose

↑ Volver al índice

EN ESTE CAPÍTULO

27.1 Un repositorio para la infraestructura
27.2 El `compose.yaml`, servicio a servicio
27.3 Reutilizar los datos: el volumen externo
27.4 La inicialización de las bases de datos
27.5 Comprobaciones de salud sin `curl`
27.6 El frontend detrás de nginx
27.7 Validar y retirar lo antiguo
27.8 Para terminar
27.9 Errores típicos y depuración
27.10 Ejercicios

El último tramo reunió todas las piezas: PostgreSQL, los dos servicios y el frontend, levantados con un solo comando. Este capítulo explica el repositorio de la plataforma, el `compose.yaml` servicio a servicio, cómo se reutilizaron los datos sin arriesgarlos, la inicialización de las bases de datos, las comprobaciones de salud sin `curl`, nginx delante del frontend y la validación final.

27.1 Un repositorio para la infraestructura

Paso 7.9 ↖ ↑ Índice

Hasta aquí, el único `compose.yaml` vivía dentro del servicio de demandas y solo levantaba PostgreSQL. Ampliarlo habría sido lo más rápido, pero habría convertido a un microservicio en **dueño de la infraestructura de todos**: para arrancar el catálogo habría que entrar en el proyecto de demandas.

La plataforma pasó a tener su propio repositorio, junto a los demás:

`C:\Workspace` ESTADO FINAL

```
├── it-service-platform\          ← compose.yaml, postgres/init, .env.example, README
├── it-service-manager\           ← servicio de demandas (Dockerfile)
├── application-catalog-service\  ← catálogo de aplicaciones (Dockerfile)
└── it-service-manager-web\       ← frontend (Dockerfile + nginx)
```

Cada servicio sabe construirse a sí mismo (su `Dockerfile`), pero no sabe nada de los demás. La plataforma es la que conoce el sistema completo: qué servicios hay, cómo se conectan y con qué configuración. Es la separación de responsabilidades de los capítulos 17 y 18, aplicada ahora al despliegue.

Decisión de diseño: Un repositorio de plataforma

Motivo: ningún servicio es dueño de la infraestructura común; la plataforma documenta el sistema completo y es el sitio natural para su README. **Alternativas:** el Compose dentro de uno de los servicios (más rápido, pero acopla) o un único repositorio con todo (*monorepo*), válido pero distinto de la estructura que ya existía. **Coste:** los repositorios deben estar uno junto a otro, porque el Compose los referencia con rutas relativas (`../it-service-manager`).

27.2 El `compose.yaml`, servicio a servicio

↑ Índice

27.2.1 PostgreSQL

it-service-platform/compose.yaml (extracto) ESTADO FINAL

```yaml
name: it-service-platform

services:

  postgres:
    image: postgres:18.6
    environment:
      POSTGRES_DB: it_service_manager
      POSTGRES_USER: itmanager
      POSTGRES_PASSWORD: ${DB_PASSWORD:?Define DB_PASSWORD en .env}
      # Lo usa el script de inicialización para crear el usuario del catálogo.
      APPLICATION_DB_PASSWORD: ${APPLICATION_DB_PASSWORD:?Define APPLICATION_DB_PASSWORD en .env}
    ports:
      # Publicado solo para herramientas de desarrollo (pgAdmin, Spring Tools).
      - "5433:5432"
    volumes:
      - postgres_data:/var/lib/postgresql
      # Solo se ejecuta si el volumen está vacío (instalación nueva).
      - ./postgres/init:/docker-entrypoint-initdb.d:ro
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U $${POSTGRES_USER} -d $${POSTGRES_DB}"]
      interval: 5s
      timeout: 5s
      retries: 10
      start_period: 10s
    restart: unless-stopped
```

`name: it-service-platform` fija el nombre del proyecto. Compose lo usa como prefijo de contenedores y redes (`it-service-platform-postgres-1`); sin él, se tomaría el nombre de la carpeta.

`${DB_PASSWORD:?mensaje}`. La sintaxis con `?` hace que Compose **se niegue a arrancar** si la variable no está definida, mostrando el mensaje. Las variables se leen del fichero `.env` de la carpeta, que no se versiona; el repositorio solo contiene `.env.example` con valores ficticios.

`restart: unless-stopped` reinicia el contenedor si se cae o si se reinicia Docker, salvo que se haya parado a mano.

27.2.2 Los dos servicios

it-service-platform/compose.yaml (extracto) ESTADO FINAL

```yaml
  application-catalog-service:
    build: ../application-catalog-service
    image: application-catalog-service:local
    environment:
      DB_URL: jdbc:postgresql://postgres:5432/application_db
      DB_USERNAME: application_user
      DB_PASSWORD: ${APPLICATION_DB_PASSWORD:?Define APPLICATION_DB_PASSWORD en .env}
    ports:
      - "8081:8081"
    depends_on:
      postgres:
        condition: service_healthy
```

```
  demand-service:
    build: ../it-service-manager
    image: it-service-manager:local
    environment:
      DB_URL: jdbc:postgresql://postgres:5432/it_service_manager
      DB_USERNAME: itmanager
      DB_PASSWORD: ${DB_PASSWORD:?Define DB_PASSWORD en .env}
      # Dentro de la red de Compose el catálogo se localiza por su nombre de servicio.
      APPLICATION_CATALOG_BASE_URL: http://application-catalog-service:8081
    ports:
      - "8080:8080"
    depends_on:
      # Solo necesita su base de datos para arrancar. Si el catálogo no está,
      # arranca igual y responde 503 en lo que dependa de él.
      postgres:
        condition: service_healthy
```

(Las comprobaciones de salud y `restart` de estos servicios se explican más abajo.)

`build` e `image`. `build` indica dónde está el `Dockerfile`; `image`, cómo se llamará la imagen construida. Con `docker compose up --build` se reconstruyen si el código ha cambiado.

La red de Compose y sus nombres. Compose crea una red privada para el proyecto y registra cada servicio con su nombre en un DNS interno. Por eso la URL de la base de datos es `postgres:5432` y la del catálogo `http://application-catalog-service:8081`. Fíjate en el puerto: dentro de la red se usa el **puerto interno** del contenedor (5432), no el publicado en tu equipo (5433). Es justo el cambio de configuración que el paso 7.5 hizo posible sin tocar el código.

`depends_on` con condición. `condition: service_healthy` hace que Compose no arranque el servicio hasta que la comprobación de salud de PostgreSQL esté en verde. Sin la condición, `depends_on` solo garantiza el orden de arranque, no que la base de datos esté lista, y el servicio podría fallar al conectar.

> **Decisión de diseño: El servicio de demandas no espera al catálogo**
>
> Habría sido natural añadir `application-catalog-service: condition: service_healthy` a las dependencias del servicio de demandas. Se descartó por la misma razón que en el capítulo 26: el servicio de demandas debe poder arrancar y trabajar con el catálogo caído. Si dependiera de él para arrancar, un catálogo roto dejaría sin servicio también a las demandas.

`ports`. Publicar un puerto lo hace accesible desde tu equipo. Los servicios se comunican entre sí por la red interna y no lo necesitan; los puertos 8080, 8081 y 5433 se publican para el frontend en desarrollo, para el IDE y para pgAdmin. En un despliegue real solo se publicaría el puerto de entrada (el de `web`).

27.3 Reutilizar los datos: el volumen externo

↑ Índice

La base de datos de desarrollo tenía datos reales del proyecto, guardados en el volumen `it-service-manager_postgres_data` que había creado el Compose antiguo (Compose nombra los volúmenes con el prefijo del proyecto). El Compose nuevo lo reutiliza:

`it-service-platform/compose.yaml (extracto)` **ESTADO FINAL**

```
volumes:
  postgres_data:
    # Volumen ya existente, creado por el compose.yaml de it-service-manager.
    # "external" significa que Compose lo usa pero nunca lo crea ni lo borra.
```

```
    external: true
    name: ${POSTGRES_VOLUME:-it-service-manager_postgres_data}
```

`external: true` es la decisión de seguridad más importante del fichero. Un volumen normal es **propiedad** del proyecto de Compose: `docker compose down -v` lo eliminaría. Un volumen externo no: Compose lo usa, pero no lo crea ni lo borra **nunca**, ni siquiera con `-v`. Si no existe, Compose se niega a arrancar en lugar de crear uno vacío.

> **Cuidado: Dos PostgreSQL sobre un mismo volumen**
>
> Antes del primer `up` hubo que parar el contenedor del Compose antiguo con `docker compose stop` (nunca con `down -v`). Dos instancias de PostgreSQL escribiendo sobre los mismos ficheros de datos **corrompen la base de datos**. En este caso, el conflicto del puerto 5433 habría impedido que el segundo arrancara, pero no conviene confiar en esa casualidad.

27.4 La inicialización de las bases de datos

Paso 7.10 ↖ ↑ Índice

El capítulo 19 dejó un pendiente: la base de datos `application_db` y su usuario se habían creado a mano, y nada lo reflejaba en el repositorio. La plataforma lo resuelve con un *script* de inicialización:

it-service-platform/postgres/init/10-application-db.sh ESTADO FINAL

```
#!/bin/bash
set -euo pipefail

psql -v ON_ERROR_STOP=1 \
     --username "$POSTGRES_USER" \
     --dbname "$POSTGRES_DB" \
     --set app_password="$APPLICATION_DB_PASSWORD" <<-'EOSQL'
    CREATE USER application_user WITH PASSWORD :'app_password';
    CREATE DATABASE application_db OWNER application_user;
    REVOKE CONNECT ON DATABASE application_db FROM PUBLIC;
    GRANT CONNECT ON DATABASE application_db TO application_user;
    REVOKE CONNECT ON DATABASE it_service_manager FROM PUBLIC;
EOSQL
```

Cuándo se ejecuta. La imagen oficial de PostgreSQL ejecuta los ficheros de `/docker-entrypoint-initdb.d` **solo cuando el directorio de datos está vacío**, es decir, la primera vez. Con el volumen actual, que ya tiene datos, no se ejecuta. Su propósito es que una instalación nueva (otra máquina, alguien que clone los repositorios) obtenga exactamente la misma estructura sin pasos manuales. La base de datos `it_service_manager` y su usuario los crea la propia imagen a partir de `POSTGRES_DB`, `POSTGRES_USER` y `POSTGRES_PASSWORD`.

La contraseña no está escrita. Llega como variable de entorno del contenedor y se pasa a `psql` como variable (`--set app_password=…`). `:'app_password'` la inserta en el SQL **correctamente entrecomillada**, sin riesgo de que un carácter especial rompa la sentencia. El *heredoc* `<<-'EOSQL'` va entre comillas para que el *shell* no interprete nada dentro del SQL.

Cada usuario, en su base de datos. Por defecto, en PostgreSQL cualquier usuario puede conectarse a cualquier base de datos (el permiso `CONNECT` lo tiene el rol especial `PUBLIC`). Los `REVOKE` lo retiran, y `application_user` solo puede conectarse a `application_db`. Es el patrón de base de datos por servicio del capítulo 19 garantizado por la propia base de datos.

> **Cuidado: `POSTGRES_USER` es superusuario**
>
> La imagen oficial crea el usuario de `POSTGRES_USER` (aquí, `itmanager`) como **superusuario**, y un superusuario se salta todos los permisos. Para desarrollo es aceptable; en un entorno real, el servicio de demandas debería usar un usuario sin privilegios especiales, igual que el catálogo, y el superusuario reservarse para administración.

Un último detalle del repositorio: un fichero `.gitattributes` con `*.sh text eol=lf`. El *script* se ejecuta dentro de un contenedor Linux, y si Git en Windows lo convirtiera a finales de línea CRLF, `bash` fallaría con un error confuso (`$'\r': command not found`).

27.5 Comprobaciones de salud sin `curl`

Paso 7.11 ↖ ↑ Índice

Actuator ofrece `/actuator/health`, pero alguien tiene que preguntar. Docker ejecuta periódicamente el comando de `healthcheck` **dentro del contenedor** y, según su resultado, lo marca como `starting`, `healthy` o `unhealthy`. Lo habitual es usar `curl`, pero la imagen `eclipse-temurin:21-jre` no lo incluye, y añadirlo solo para esto engordaría la imagen y le sumaría superficie de ataque.

La solución usa solo `bash`:

it-service-platform/compose.yaml (extracto) ESTADO FINAL

```
    healthcheck:
      # La imagen no incluye curl: petición HTTP mínima con bash.
      test: ["CMD-SHELL", "bash -c 'exec 3<>/dev/tcp/127.0.0.1/8081; printf \"GET /actuator/health H
TTP/1.0\\r\\nHost: localhost\\r\\n\\r\\n\" >&3; grep -q UP <&3'"]
      interval: 10s
      timeout: 5s
      retries: 10
      start_period: 40s
```

Desmontado, el comando hace tres cosas:

1. `exec 3<>/dev/tcp/127.0.0.1/8081` abre una conexión TCP con el propio servicio y la asigna al descriptor de fichero 3. `/dev/tcp` es una característica de `bash`, no un fichero real.
2. `printf "GET /actuator/health HTTP/1.0 …" >&3` escribe a mano una petición HTTP en esa conexión.
3. `grep -q UP <&3` lee la respuesta y termina con éxito solo si contiene `UP`.

Si Actuator responde `DOWN`, o el servicio no acepta conexiones, `grep` falla y Docker cuenta un intento fallido.

Parámetro	Valor	Significado
`interval`	10 s	Cada cuánto se comprueba
`timeout`	5 s	Cuánto se espera cada comprobación
`retries`	10	Fallos seguidos antes de marcar `unhealthy`
`start_period`	40 s	Margen inicial para que Spring Boot arranque, sin contar fallos

En la validación se vio la transición: el servicio de demandas en `health: starting` a los 10 segundos y `healthy` a los 30.

27.6 El frontend detrás de nginx

Paso 7.12 ↖ ↑ Índice

En desarrollo, el frontend lo sirve Vite, que además hace de proxy hacia los servicios. Para levantar el sistema completo sin `npm run dev`, el frontend tiene su propia imagen:

it-service-manager-web/Dockerfile ESTADO FINAL

```
# ---------- Etapa 1: compilación ----------
FROM node:22-alpine AS build
WORKDIR /app

# Dependencias primero: capa en caché mientras no cambie el lockfile.
COPY package.json package-lock.json ./
RUN npm ci

COPY . .
RUN npm run build

# ---------- Etapa 2: servidor ----------
FROM nginx:1.27-alpine

COPY docker/nginx.conf /etc/nginx/conf.d/default.conf
COPY --from=build /app/dist /usr/share/nginx/html

EXPOSE 80
```

Es el mismo patrón que los servicios: una etapa compila (con Node) y otra sirve el resultado. La imagen final no contiene Node ni el código fuente, solo `dist/` y nginx. `npm ci` instala exactamente las versiones del *lockfile*, como `mvn dependency:go-offline` en el backend.

nginx cumple dos funciones a la vez:

it-service-manager-web/docker/nginx.conf (extracto) ESTADO FINAL

```
server {
    listen 80;
    root  /usr/share/nginx/html;

    resolver 127.0.0.11 valid=10s ipv6=off;
    set $demand_service  http://demand-service:8080;
    set $catalog_service http://application-catalog-service:8081;

    location /api/demands {
        proxy_pass $demand_service;
    }

    location /api/applications {
        proxy_pass $catalog_service;
    }

    location / {
        try_files $uri $uri/ /index.html;
        add_header Cache-Control "no-cache";
    }
}
```

Proxy inverso. Las peticiones a `/api/demands` y `/api/applications` se reenvían a cada servicio por la red interna. El navegador habla con un único origen (`localhost:8088`), así que, igual que con Vite, no hace falta CORS.

Resolución en cada petición. Con un nombre escrito directamente en `proxy_pass`, nginx lo resuelve **al arrancar**, y si el servicio no existiera en ese momento, nginx no arrancaría. Con el nombre en una variable y `resolver 127.0.0.11` (el DNS interno de Docker), lo resuelve en cada petición. nginx arranca aunque un servicio esté parado, y en ese caso solo esa ruta responde 502.

Fallback de la SPA. Las rutas de React Router (`/demandas/3/editar`) no existen como ficheros. `try_files $uri $uri/ /index.html` sirve el fichero si existe y, si no, `index.html`, para que React Router resuelva la ruta. Sin esto, recargar la página en una ruta interna daría un 404 de nginx.

Caché. Los ficheros de `/assets/` llevan un *hash* en el nombre y se marcan como cacheables durante un año (esa regla está en el fichero completo); `index.html` se marca como `no-cache`, para que una versión nueva se vea en cuanto se despliega.

> **Si vienes de Java EE…**
>
> nginx hace aquí el papel que en tu entorno de WebSphere hacía el IBM HTTP Server con el *plugin* de WebSphere: un servidor web delante que sirve el contenido estático y reenvía las peticiones dinámicas al servidor de aplicaciones adecuado.

27.7 Validar y retirar lo antiguo

Paso 7.13 ↖ ↑ Índice

La validación siguió el mismo guion que el capítulo 22, ahora con todo en contenedores:

Comprobación	Resultado
`docker compose up -d --build`	Cuatro contenedores; `docker compose ps` los muestra `healthy`
Aplicaciones y demandas en la web	Datos de siempre, con los nombres resueltos por la red interna
`docker compose stop application-catalog-service`	Demandas muestra «El catálogo de aplicaciones no está disponible…»
`docker compose start application-catalog-service`	Recuperación automática, sin reiniciar nada más
`docker compose down` y `up -d` (sin `-v`)	Los datos siguen ahí, incluidas las demandas creadas ese día
Recargar `localhost:8088/demandas`	nginx devuelve la SPA, no un 404

> **Lección aprendida: El puerto ocupado**
>
> El primer `up` falló con `ports are not available: … 0.0.0.0:8080 … bind`: el servicio de demandas seguía arrancado en Spring Tools. PostgreSQL y el catálogo arrancaron bien y el servicio de demandas se quedó esperando el puerto, sin ningún riesgo para los datos. La regla que quedó: **un servicio se ejecuta en Docker o en el IDE, nunca en los dos**. Para depurar uno en el IDE se para solo su contenedor (`docker compose stop demand-service`) y se arranca en Spring Tools, que seguirá usando PostgreSQL y el otro servicio de Compose por los puertos publicados.

Con todo validado, se retiró el `compose.yaml` antiguo del servicio de demandas (`git rm compose.yaml`). Dejarlo habría mantenido **dos formas de levantar un PostgreSQL sobre el mismo volumen**, exactamente lo

que no debe ocurrir. Cada repositorio recibió además su README: qué hace, su API, sus variables de entorno, cómo ejecutarlo solo y cómo levantarlo con la plataforma.

27.7.1 El estado final

Repositorio	Contenido
`it-service-manager`	Servicio de demandas: código, 40 pruebas, Dockerfile y README
`application-catalog-service`	Catálogo de aplicaciones: código, 18 pruebas, Dockerfile y README
`it-service-platform`	Compose del sistema completo, inicialización de PostgreSQL, `.env.example` y README
`it-service-manager-web`	Frontend: código, 39 pruebas, Dockerfile, nginx y las dos guías

Todo el sistema se levanta con un comando y queda en **http://localhost:8088**:

```
cd C:\Workspace\it-service-platform
docker compose up -d --build
```

Lo que queda (seguridad, integración continua, métricas y trazas, OpenAPI, pruebas de contrato y los demás puntos del capítulo 24) ya no es deuda de esta etapa, sino la siguiente.

27.8 Para terminar

↑ Índice

El recorrido de esta guía va de un proyecto generado con Spring Initializr a dos servicios con bases de datos propias que se comunican por HTTP, empaquetados en imágenes y levantados con Docker Compose. Por el camino han aparecido casi todas las piezas de un backend Spring moderno: autoconfiguración, JPA e Hibernate, transacciones declarativas y programáticas, validación, manejo de errores, consultas dinámicas, el problema N+1, migraciones, pruebas con Mockito, MockMvc y Testcontainers, puertos y adaptadores, un cliente HTTP robusto, comprobaciones de salud y contenedores.

Pero la idea que más vale la pena llevarse no es ninguna tecnología concreta. Es el orden en que se hicieron las cosas: construir un monolito bien organizado, identificar con el código delante dónde se tocan los módulos, diseñar la frontera dentro del monolito, moverla a la red en pasos pequeños y reversibles, corregir lo que la red hizo visible **antes** de contenerizar, y decir con honestidad qué se ha verificado y qué no.

27.9 Errores típicos y depuración

↑ Índice

Síntoma	Causa probable	Cómo resolverlo
`ports are not available … bind`	El puerto lo usa otro proceso (a menudo, el IDE)	Parar ese proceso; `Get-NetTCPConnection -LocalPort 8080` lo identifica
`external volume … not found`	El volumen no existe o tiene otro nombre	Comprobar con `docker volume ls` y ajustar `POSTGRES_VOLUME`

Síntoma	Causa probable	Cómo resolverlo
`required variable DB_PASSWORD is missing`	Falta el `.env` o la variable	Copiar `.env.example` como `.env` y rellenarlo
Un servicio se queda en `health: starting` y pasa a `unhealthy`	No llega a su base de datos, o la contraseña no es correcta	`docker compose logs <servicio>`
El *script* de `postgres/init` no se ejecuta	El volumen ya tenía datos	Es lo esperado; solo actúa en instalaciones nuevas
`$'\r': command not found` en la inicialización	El `.sh` tiene finales de línea CRLF	`.gitattributes` con `*.sh text eol=lf`
404 de nginx al recargar una ruta interna	Falta el *fallback* a `index.html`	`try_files $uri $uri/ /index.html`
502 en `/api/demands` desde la web	El servicio de demandas está parado o arrancando	`docker compose ps` y esperar a `healthy`

27.10 Ejercicios

↑ Índice

Ejercicio 27.1 · Una instalación desde cero Ver solución →

Sin tocar tu volumen actual, simula una instalación nueva: en `.env` pon `POSTGRES_VOLUME=itsp_prueba`, crea ese volumen vacío con `docker volume create itsp_prueba`, para la plataforma y arráncala de nuevo. Comprueba con pgAdmin que existen las dos bases de datos y que `application_user` no puede conectarse a `it_service_manager`. Al terminar, vuelve a dejar `POSTGRES_VOLUME` como estaba.

Ejercicio 27.2 · Publicar solo la entrada Ver solución →

En un despliegue real solo debería ser accesible desde fuera el puerto del frontend. Indica qué líneas del `compose.yaml` quitarías y qué dejaría de funcionar en tu forma de trabajar actual (IDE, pgAdmin, Vite). Propón cómo mantener las dos formas de trabajar con un fichero `compose.override.yaml`.

Autoevaluación del capítulo 27 Ver respuestas →

1. ¿Por qué el `compose.yaml` vive en un repositorio propio y no en uno de los servicios?
2. ¿Qué diferencia hay entre un volumen normal y uno `external` cuando se ejecuta `docker compose down -v`?
3. ¿Por qué dentro de la red de Compose la URL de la base de datos usa el puerto 5432 y no el 5433?
4. ¿Cuándo se ejecuta el *script* de `postgres/init`, y por qué la contraseña se pasa con `--set` y `:'app_password'`?
5. ¿Qué problema resuelven el `resolver` de nginx y los nombres de los servicios en variables?

ANEXO A

Soluciones y respuestas

↑ Volver al índice

Soluciones de los ejercicios y respuestas de la autoevaluación de cada capítulo. Cada bloque enlaza de vuelta con su enunciado. Muchos ejercicios admiten más de una solución correcta: lo importante es el razonamiento.

A.1 Capítulo 2. Del servidor de aplicaciones a Spring Boot 4

↑ Índice

Solución 2.1 · Leer el informe de autoconfiguración ← Volver al ejercicio

En el informe aparecen en la sección *Positive matches*. La autoconfiguración del `DataSource` se activa porque existe la clase `DataSource` y un *driver* JDBC en el *classpath* y no hay ningún *bean* `DataSource` definido por el proyecto (`@ConditionalOnMissingBean`). La de Flyway se activa porque la clase `Flyway` está en el *classpath*, hay un `DataSource` disponible y la propiedad `spring.flyway.enabled` no es `false`. En Spring Boot 4 esta autoconfiguración solo existe si está el módulo que aporta `spring-boot-starter-flyway`.

Solución 2.2 · Romper el escaneo de componentes ← Volver al ejercicio

La aplicación arranca, pero `GET /api/applications` devuelve 404: el controlador está fuera del paquete `com.albertoderafael.itmanager` y sus subpaquetes, así que `@ComponentScan` no lo encuentra y no se registra ninguna ruta. No hay error al arrancar porque nadie depende del controlador. Si se moviera un servicio, en cambio, el arranque fallaría con `NoSuchBeanDefinitionException` en el controlador que lo necesita.

Respuestas · capítulo 2 ← Volver a las preguntas

1. Porque el *parent* `spring-boot-starter-parent` importa una lista de materiales (BOM) que fija versiones compatibles de todas las librerías del ecosistema, incluido Hibernate.
2. `@SpringBootConfiguration` (la clase es fuente de configuración), `@EnableAutoConfiguration` (activa la autoconfiguración según el *classpath*) y `@ComponentScan` (busca componentes en el paquete de la clase y sus subpaquetes).
3. Flyway está en el *classpath* pero no se ejecuta al arrancar, porque la autoconfiguración está en el módulo que aporta el *starter*. Las tablas no se crean y la validación de Hibernate falla.
4. Dependencias obligatorias e inmutables (`final`), clases instanciables con `new` en pruebas unitarias, dependencias visibles en la firma y ciclos detectados al arrancar.
5. Las dependencias entre módulos eran visibles como `import` entre paquetes (`itmanager.demand` importando de `itmanager.application`), lo que permitió localizar exactamente los puntos de contacto que había que cortar.

A.2 Capítulo 3. PostgreSQL en Docker y la configuración

↑ Índice

Solución 3.1 · Inspeccionar el contenedor

← Volver al ejercicio

`docker compose ps` muestra el estado `running (healthy)` cuando `pg_isready` responde. Con `\dt` aparecen las tablas del esquema `public`; en el servicio de demandas actual, `demands` y `flyway_schema_history` (la tabla `applications` la eliminó la V5).

Solución 3.2 · Provocar un fallo de validación del esquema

← Volver al ejercicio

La aplicación no arranca. El mensaje es del tipo `Schema-validation: missing column [owner] in table [applications]`, y lo produce Hibernate durante la creación del `EntityManagerFactory`, al validar el esquema por `ddl-auto=validate`. Flyway ya ha terminado y no ha encontrado ninguna migración nueva.

Respuestas · capítulo 3

← Volver a las preguntas

1. Para no chocar con una instalación local de PostgreSQL que use el puerto estándar 5432. Dentro del contenedor sigue siendo el 5432.
2. El contenedor se para y se elimina, pero el volumen con nombre se conserva, y con él los datos. No hay que añadir `-v`, que eliminaría también los volúmenes.
3. Docker Compose. Spring Boot no lee ficheros `.env`; necesita variables de entorno reales.
4. `validate` solo comprueba que el esquema coincide con las entidades y falla si no; `update` intenta modificar el esquema sin control ni historial. Con Flyway como única fuente de verdad, `validate` detecta desajustes sin tocar nada.
5. `LazyInitializationException`. Es una ventaja porque obliga a cargar los datos necesarios dentro del servicio y hace visibles consultas que con OSIV pasarían inadvertidas.

A.3 Capítulo 4. Migraciones de esquema con Flyway

↑ Índice

Solución 4.1 · Leer el historial

← Volver al ejercicio

`installed_rank` es el orden de aplicación; `version` y `description` salen del nombre del fichero; `checksum` es la huella del contenido, que Flyway compara en cada arranque; `success` indica si la migración terminó bien. En el servicio de demandas aparecen cinco filas, de la V1 a la V5.

Solución 4.2 · Diseñar una migración correctiva

← Volver al ejercicio

Fichero `V6__restrict_application_criticality.sql` (o el siguiente número libre) con:

```
ALTER TABLE applications
    ADD CONSTRAINT chk_applications_criticality
    CHECK (criticality IN ('LOW', 'MEDIUM', 'HIGH', 'CRITICAL'));
```

No se puede editar la V1 porque ya está aplicada en otras bases de datos: su *checksum* cambiaría, Flyway se negaría a arrancar en todas ellas y, además, la restricción nunca llegaría a las bases de datos existentes, que no vuelven a ejecutar la V1. Hoy la tabla vive en `application_db`, así que la migración iría en el proyecto del catálogo.

Respuestas · capítulo 4 ← Volver a las preguntas

1. Qué migraciones se han aplicado, en qué orden, cuándo y con qué resultado. El *checksum* sirve para detectar que el contenido de una migración aplicada ha cambiado.
2. Primero Flyway migra y después Hibernate valida. Importa porque Hibernate debe validar el esquema ya migrado; al revés fallaría en cuanto hubiera una migración pendiente.
3. `BY DEFAULT` permite insertar valores explícitos en la columna; `ALWAYS` lo prohíbe salvo con `OVERRIDING SYSTEM VALUE`.
4. Porque su V2 tiene otro contenido y, por tanto, otro *checksum*: Flyway detectaría la discrepancia y no arrancaría.
5. Porque reescribe el historial para que coincida con los ficheros, ocultando que la base de datos se creó con un contenido distinto del que hay ahora en el repositorio.

A.4 Capítulo 5. La entidad Application y el contexto de persistencia

↑ Índice

Solución 5.1 · Observar el *dirty checking* ← Volver al ejercicio

La primera petición produce un `SELECT` y un `UPDATE`. La segunda, con los mismos datos, solo el `SELECT`: al hacer *flush*, Hibernate compara con la instantánea, no encuentra diferencias y no genera `UPDATE`. (En las demandas no ocurre así, porque `update(...)` cambia siempre `updatedAt`.)

Solución 5.2 · La garantía de identidad ← Volver al ejercicio

Las dos instancias son la misma (`==` devuelve `true`) y solo hay un `SELECT`: la segunda búsqueda se resuelve en el contexto de persistencia. Con `entityManager.clear()` entre ambas, el contexto se vacía, la primera instancia queda separada, hay dos `SELECT` y `==` devuelve `false` (aunque representen la misma fila).

Respuestas · capítulo 5 ← Volver a las preguntas

1. JPA es la especificación (anotaciones e interfaces); Hibernate es la implementación que genera SQL y gestiona el contexto; Spring Data JPA es una capa superior que genera repositorios a partir de interfaces.
2. Porque una entidad nueva tiene `id` nulo, y Spring Data usa ese nulo para distinguir entidades nuevas de existentes. Un `long` primitivo valdría 0.
3. JPA lo necesita para instanciar entidades por reflexión, pero no debe usarse desde el código para crear entidades vacías e incompletas.
4. El valor actual de cada atributo de las entidades gestionadas con la instantánea guardada al cargarlas.
5. Porque el `id` lo asigna la base de datos al insertar, y Hibernate necesita conocerlo de inmediato para registrar la entidad, así que ejecuta cada `INSERT` en el momento en lugar de agruparlos en el *flush*.

A.5 Capítulo 6. Repositorios de Spring Data

↑ Índice

Solución 6.1 · Contar las consultas de una página ← Volver al ejercicio

Con `size=1` hay dos consultas: la página trae un elemento, igual al tamaño pedido, así que Spring Data no puede deducir el total y lanza el `count`. Con `size=50` hay una sola: la página trae menos elementos que el tamaño pedido y es la primera, así que el total es exactamente ese número.

Solución 6.2 · Un método derivado nuevo ← Volver al ejercicio

`Page<Application> findByNameContainingIgnoreCase(String text, Pageable pageable);` La consulta esperada es, en esencia, `select … from applications a where upper(a.name) like upper(?) escape '\' order by … offset … fetch first …`, con el parámetro envuelto en `%…%`, más la consulta de recuento con la misma condición.

Respuestas · capítulo 6 ← Volver a las preguntas

1. `SimpleJpaRepository`.
2. Si la entidad es nueva (`id` nulo), llama a `persist`; si no, a `merge`.
3. Al arrancar la aplicación, cuando se crean los repositorios.
4. Una consulta trae los datos de la página y otra cuenta el total. Se ahorra la segunda cuando puede deducir el total, por ejemplo si la primera página trae menos elementos que el tamaño pedido.
5. Se une a la transacción existente (propagación `REQUIRED`), en lugar de abrir una nueva.

A.6 Capítulo 7. El servicio y las transacciones

↑ Índice

Solución 7.1 · Ver el *rollback* en acción ← Volver al ejercicio

Con `IllegalStateException`, que es no comprobada, la transacción se deshace y el cambio no se guarda. Con una `IOException` declarada en el método, la transacción **se confirma**: Spring no hace *rollback* ante excepciones comprobadas por defecto, así que el `UPDATE` llega a la base de datos aunque el cliente reciba un error.

Solución 7.2 · Detectar la autoinvocación ← Volver al ejercicio

La llamada interna no pasa por el proxy, así que la anotación del método llamado se ignora y sigue activa la transacción de `getApplications`, de solo lectura. Con `readOnly`, Hibernate no hace *flush* automático, así que la modificación lo más probable es que no llegue a la base de datos, sin ningún error. Aunque no fuera así, el comportamiento dependería de un detalle invisible en el código, que es precisamente el problema.

Respuestas · capítulo 7 ← Volver a las preguntas

1. Porque la entidad está gestionada y Hibernate detecta el cambio al hacer *flush* antes del *commit*. La condición imprescindible es que el método sea transaccional.
2. `REQUIRED`: se une a una transacción existente o abre una si no hay ninguna.
3. Ante excepciones no comprobadas (`RuntimeException` y subclases) y `Error`.
4. Una llamada de un método a otro del mismo *bean* que no pasa por el proxy; como el proxy es quien aplica la transacción, la anotación del método llamado no tiene efecto.

5. Porque dentro de la transacción la entidad está gestionada y se pueden cargar sus asociaciones perezosas; fuera, con `open-in-view=false`, fallaría.

A.7 Capítulo 8. DTO y Bean Validation

↑ Índice

Solución 8.1 · Todos los errores a la vez ← Volver al ejercicio

Aparecen dos errores de campo: `name` (`@NotBlank` rechaza la cadena vacía) y `criticality` (`urgent` no encaja con `@Pattern`, que distingue mayúsculas). `description` no aparece porque es `null` y `@Size`, su única restricción, acepta `null`.

Solución 8.2 · Probar el validador sin Spring ← Volver al ejercicio

```
Validator validator = Validation.buildDefaultValidatorFactory().getValidator();
var request = new ApplicationCreateRequest("x".repeat(101), null, "HIGH");
var violations = validator.validate(request);
assertEquals(1, violations.size());
assertEquals("name",
        violations.iterator().next().getPropertyPath().toString());
```

Respuestas · capítulo 8 ← Volver a las preguntas

1. El contrato de la API queda independiente del modelo; cada operación acepta solo los campos que debe; no se filtran detalles de persistencia ni se disparan cargas perezosas; las reglas de entrada no contaminan la entidad.
2. Porque JPA necesita un constructor sin argumentos y modificar los atributos, y un *record* es inmutable y no tiene constructor vacío.
3. Porque `@Pattern` considera válido el `null`; sin `@NotBlank`, un valor ausente pasaría la validación.
4. `MethodArgumentNotValidException`. El método del controlador no llega a ejecutarse.
5. Los valores válidos se repiten como texto en varios sitios y el compilador no ayuda. Se mejoraría con un `enum` y `@Enumerated(EnumType.STRING)`, que mantiene el mismo texto en el JSON y en la columna.

A.8 Capítulo 9. El controlador REST y el contrato HTTP

↑ Índice

Solución 9.1 · Explorar el `Pageable` ← Volver al ejercicio

Con `sort=name,desc`, la página 0 contiene SIGMA y la página 1, ORION (orden alfabético descendente). En ambas, `totalElements` es 2 y `totalPages` es 2, porque el tamaño es 1.

Solución 9.2 · Seguir el flujo con el depurador ← Volver al ejercicio

Entre el controlador y el servicio aparecen clases de Spring AOP: un proxy (su nombre contiene `$$SpringCGLIB$$` o similar), `CglibAopProxy`, `ReflectiveMethodInvocation` y `TransactionInterceptor`, que es quien abre y cierra la transacción. Por encima del controlador se ven `DispatcherServlet` y los adaptadores de Spring MVC.

Respuestas · capítulo 9 ← Volver a las preguntas

1. `@RestController` añade `@ResponseBody`: el valor devuelto se serializa como cuerpo de la respuesta en lugar de interpretarse como nombre de una vista.
2. `page`, `size` y `sort`. Las páginas empiezan en 0.
3. El tamaño pasaría a 20 y no habría orden definido, así que los resultados podrían variar entre peticiones y páginas.
4. Porque la forma JSON de `PageImpl` es interna de Spring Data y puede cambiar; un DTO propio fija el contrato con el frontend.
5. El `INSERT` se ejecuta al llamar a `save` (por `IDENTITY`); la conexión se devuelve al *pool* al confirmar la transacción, antes de serializar la respuesta.

A.9 Capítulo 10. El tratamiento de errores

↑ Índice

Solución 10.1 · Recorrer todos los formatos ← Volver al ejercicio

Nombre vacío: 400 `VALIDATION_ERROR` con `fieldErrors.name`. `id` inexistente: 404 `RESOURCE_ERROR`. `id` no numérico: 400 `VALIDATION_ERROR` con «Invalid value for parameter: id». JSON mal formado: 400 `VALIDATION_ERROR` con «Malformed request body». `sort=pepito`: 400 `RESOURCE_ERROR` con «Invalid application sort field: pepito», porque procede de una `ResponseStatusException`.

Solución 10.2 · Diseñar el manejador de último recurso ← Volver al ejercicio

```
private static final Logger log =
        LoggerFactory.getLogger(GlobalExceptionHandler.class);

@ExceptionHandler(Exception.class)
public ResponseEntity<ApiErrorResponse> handleUnexpected(Exception ex) {
    log.error("Unexpected error", ex);
    return ResponseEntity.internalServerError().body(new ApiErrorResponse(
            500, "INTERNAL_ERROR", "Unexpected error", Map.of()));
}
```

No interfiere porque Spring elige el manejador con el tipo más específico: `MethodArgumentNotValidException` o `ResponseStatusException` siguen yendo a sus manejadores, y solo lo que no encaja con ninguno llega a `Exception`.

Respuestas · capítulo 10 ← Volver a las preguntas

1. `VALIDATION_ERROR`: la petición está mal construida (campos, tipos, JSON). `RESOURCE_ERROR`: errores que el servicio expresa con un código HTTP, como 404, 409 o los 400 de filtros.
2. `putIfAbsent` conserva un único mensaje por campo; `LinkedHashMap` mantiene el orden en que aparecieron los errores.
3. El que declara el tipo más específico de la jerarquía de la excepción.
4. La primera versión devolvía un 500 (la consulta fallaba con una propiedad inexistente). Después del paso 2.9 devuelve un 400 con el campo inválido en el mensaje.
5. Para no exponer al cliente detalles internos del analizador JSON ni nombres de clases.

A.10 Capítulo 11. Las pruebas del módulo Application

↑ Índice

Solución 11.1 · Una prueba nueva de servicio

← Volver al ejercicio

```
@Test
void shouldThrowNotFoundWhenUpdatingApplication42() {
    when(applicationRepository.findById(42L)).thenReturn(Optional.empty());
    var request = new ApplicationUpdateRequest("APP", null, "HIGH", "ACTIVE");

    var ex = assertThrows(ResponseStatusException.class,
            () -> applicationService.updateApplication(42L, request));

    assertEquals(HttpStatus.NOT_FOUND, ex.getStatusCode());
    assertEquals("Application not found: 42", ex.getReason());
    verify(applicationRepository, never()).save(any(Application.class));
}
```

Solución 11.2 · Probar la cabecera de un error

← Volver al ejercicio

```
@Test
void shouldRejectNonNumericId() throws Exception {
    mockMvc.perform(get("/api/applications/abc"))
            .andExpect(status().isBadRequest())
            .andExpect(jsonPath("$.error").value("VALIDATION_ERROR"))
            .andExpect(jsonPath("$.message")
                    .value("Invalid value for parameter: id"));
}
```

No hace falta preparar el *mock*: la conversión de `abc` a `Long` falla antes de invocar el controlador, así que el servicio nunca se llama.

Respuestas · capítulo 11

← Volver a las preguntas

1. Porque `@SpringBootTest` arranca el contexto completo y necesita una base de datos real configurada.
2. Devuelve como resultado de `save` el mismo objeto que recibió. Así se puede comprobar el estado con el que el servicio construyó la entidad.
3. Carga el controlador indicado, los `@ControllerAdvice`, Jackson, la validación y Spring MVC. No carga servicios, repositorios ni base de datos.
4. Sustituye a `@MockBean` (eliminado en Spring Boot 4). Es necesario porque el controlador necesita un `ApplicationService` y `@WebMvcTest` no carga servicios.
5. Que el cambio llegue a PostgreSQL: sin transacción ni contexto de persistencia reales, solo demuestra el cambio en memoria.

A.11 Capítulo 12. El modelo de la demanda

↑ Índice

Solución 12.1 · Completar las pruebas del ciclo de vida

← Volver al ejercicio

```
@Test
void shouldAllowOpenToCancelled() {
    Demand demand = createDemand("OPEN");
```

```
    assertDoesNotThrow(() -> demand.changeStatus("CANCELLED"));
    assertEquals("CANCELLED", demand.getStatus());
}

@Test
void shouldAllowInProgressToCancelled() {
    Demand demand = createDemand("IN_PROGRESS");
    assertDoesNotThrow(() -> demand.changeStatus("CANCELLED"));
    assertEquals("CANCELLED", demand.getStatus());
}

@Test
void shouldRejectCancelledToOpen() {
    Demand demand = createDemand("CANCELLED");
    assertThrows(IllegalStateException.class, () -> demand.changeStatus("OPEN"));
    assertEquals("CANCELLED", demand.getStatus());
}

@Test
void shouldRejectCompletedToCancelled() {
    Demand demand = createDemand("COMPLETED");
    assertThrows(IllegalStateException.class, () -> demand.changeStatus("CANCELLED"));
    assertEquals("COMPLETED", demand.getStatus());
}
```

Solución 12.2 · La fecha en la respuesta ← Volver al ejercicio

Con `@PreUpdate`, la fecha se asignaría durante el *flush*, que ocurre al confirmar la transacción (paso 11 de la figura 9.1). Pero el DTO se construye antes (paso 10). El cliente recibiría en la respuesta el `updatedAt` anterior, aunque en la base de datos quedaría el nuevo; la siguiente consulta mostraría otra fecha.

Respuestas · capítulo 12 ← Volver a las preguntas

1. Porque PostgreSQL crea índices automáticamente para claves primarias y restricciones únicas, pero no para claves foráneas.
2. Un instante en el tiempo, normalizado a UTC; no guarda la zona horaria original.
3. `EAGER`. Se cambió a `LAZY` para no cargar la aplicación en cada consulta de demandas y decidir explícitamente cuándo cargarla.
4. Porque el formulario de modificación envía siempre el estado; si solo cambia el título, el estado llega igual y no debe rechazarse.
5. Para que el dominio no dependa de HTTP. El servicio traduce la excepción al código adecuado (409).

A.12 Capítulo 13. Consultas dinámicas con `Specification`

↑ Índice

Solución 13.1 · Un cuarto filtro ← Volver al ejercicio

```
if (text != null && !text.isBlank()) {
    String pattern = "%" + text.toLowerCase() + "%";
    specification = specification.and((root, query, cb) ->
            cb.like(cb.lower(root.get("title")), pattern));
}
```

SQL esperado: `… where 1=1 and lower(d1_0.title) like ? …`. En un caso real conviene escapar `%` y `_` del texto del usuario.

Solución 13.2 · Ver el recuento ← Volver al ejercicio

La primera consulta trae la página con `where … d1_0.status = ?` y `fetch first 1 rows only`; la segunda es un `select count(d1_0.id) from demands d1_0 where … d1_0.status = ?`. Ambas aplican la misma especificación.

Respuestas · capítulo 13 ← Volver a las preguntas

1. Porque cada combinación de filtros presentes y ausentes necesita su propio método: con *n* filtros, 2ⁿ combinaciones.
2. Un predicado siempre verdadero. Sirve como elemento neutro sobre el que encadenar los `and` de los filtros informados.
3. Cuando Spring Data construye la consulta, al ejecutar `findAll`, no al declararlas.
4. `JpaSpecificationExecutor<T>`.
5. Una unión entre `demands` y `applications`. Funcionaba porque ambas tablas estaban en la misma base de datos.

A.13 Capítulo 14. Carga perezosa y el problema N+1

↑ Índice

Solución 14.1 · Contar consultas con estadísticas de Hibernate ← Volver al ejercicio

Con las estadísticas activadas, Hibernate registra al final de cada sesión cuántas sentencias JDBC ejecutó. En una prueba automática se puede obtener `Statistics` desde la `SessionFactory` (`entityManagerFactory.unwrap(SessionFactory.class).getStatistics()`), ponerla a cero, ejecutar el listado y comprobar que `getPrepareStatementCount()` no supera el número esperado (por ejemplo, dos: datos y recuento).

Solución 14.2 · Calcular el coste real ← Volver al ejercicio

Sin grafo: 1 (datos) + 1 (recuento, porque la página está llena y no se puede deducir el total) + 4 (una por aplicación distinta, gracias a la caché del contexto) = 6. Con grafo: 1 + 1 = 2. Con carga por lotes de tamaño 10: 1 + 1 + 1 (las cuatro aplicaciones en un solo `IN`) = 3.

Respuestas · capítulo 14 ← Volver a las preguntas

1. Porque una consulta trae *N* entidades y luego se ejecuta una más por cada una al acceder a la asociación. El número real de consultas extra es el de entidades asociadas distintas, gracias al contexto de persistencia.
2. `open-in-view=false` obliga a construir los DTO dentro del servicio, donde los accesos son visibles; `show-sql=true` muestra las consultas repetidas.
3. Indica a Spring Data que cargue la asociación `application` en la misma consulta, con una unión.
4. Porque una asociación a uno no multiplica filas; una colección sí, y Hibernate tendría que paginar en memoria.

5. Porque si la entidad asociada ya estuviera en el contexto, aparecería cargada aunque el grafo no funcionara.

A.14 Capítulo 15. Servicio, ciclo de vida y API de demandas

↑ Índice

Solución 15.1 · Comprobar el todo o nada ← Volver al ejercicio

El servicio responde 409. Al lanzar `changeStatus` la excepción, el servicio la traduce a `ResponseStatusException`, no comprobada, y el proxy transaccional deshace la transacción. El cambio de título, hecho por `update(...)` en memoria, nunca llega a la base de datos.

Solución 15.2 · Una prueba de servicio con transición inválida ← Volver al ejercicio

Hay que preparar dos *mocks*: el repositorio de demandas (`findById` devuelve una demanda creada con estado `COMPLETED`) y la dependencia que resuelve la aplicación (en el código final, `catalog.findById(…)` devuelve un `ApplicationSummary`; en el monolito, `applicationRepository.findById(…)`). Después:

```
var ex = assertThrows(ResponseStatusException.class,
        () -> service.updateDemand(9L,
                new DemandUpdateRequest("T", null, "HIGH", "IN_PROGRESS", 7L)));
assertEquals(HttpStatus.CONFLICT, ex.getStatusCode());
assertEquals("Invalid demand status transition: COMPLETED -> IN_PROGRESS",
        ex.getReason());
```

Respuestas · capítulo 15 ← Volver a las preguntas

1. Un 404 `Application not found: <id>`. Sobre una aplicación `INACTIVE` la demanda se crea: solo se comprueba la existencia.
2. Primero la existencia de la demanda, después la de la aplicación y por último la validez de la transición.
3. Porque la excepción no comprobada provoca el *rollback* y Hibernate no hace *flush* de la entidad modificada.
4. Un 400 indica una petición mal construida; un 409, una petición correcta que choca con el estado actual del recurso.
5. Que si el constructor cambia y falta un *mock*, Mockito puede inyectar `null` sin fallar, mientras que `new` no compilaría.

A.15 Capítulo 16. El monolito terminado

↑ Índice

Solución 16.1 · Observar el contenedor efímero ← Volver al ejercicio

`docker ps` muestra un contenedor `postgres:18` con un nombre aleatorio y un puerto publicado alto y aleatorio (por ejemplo, `0.0.0.0:55012->5432/tcp`), además del contenedor auxiliar de Testcontainers (Ryuk), que se encarga de limpiar. Al terminar la prueba ambos desaparecen; el contenedor de desarrollo sigue en el 5433 con sus datos.

Solución 16.2 · Clasificar el acoplamiento ← Volver al ejercicio

Orden razonable de más fácil a más difícil: 2 y 3 (cambiar dependencias de código detrás de una interfaz), 1 (cambiar el modelo a un identificador), 4 (sustituir la unión por una consulta por lotes), 7 (aceptar transacciones locales), 6 (migración para eliminar la clave foránea, irreversible) y 5 (imposible de mantener; hay que retirarlo o rediseñarlo). La fase 5 siguió prácticamente ese orden.

Respuestas · capítulo 16 ← Volver a las preguntas

1. Crea los detalles de conexión a partir del contenedor (URL, usuario, contraseña) y tienen prioridad sobre `spring.datasource.*` de `application.properties`.
2. Para que el contenedor se arranque una vez para toda la clase, en lugar de uno por método.
3. Cada prueba se ejecuta en una transacción que se deshace al terminar. Como nunca se confirma, Hibernate puede no enviar el SQL, y hay que forzarlo con `flush()` para comprobarlo.
4. Saber si existe y conocer su nombre.
5. Porque no basta con cambiar código Java: implican renunciar a consultas que hoy funcionan o cambiar el esquema de la base de datos.

A.16 Capítulo 17. Decidir la extracción

↑ Índice

Solución 17.1 · Elegir qué extraer ← Volver al ejercicio

El catálogo de aplicaciones seguiría siendo el primero: lo usan Demandas e Incidencias y no depende de nadie. Equipos sería el siguiente candidato si Incidencias lo referencia y no depende de otros módulos. Incidencias y Demandas, que dependen de los anteriores, serían los últimos. El criterio es extraer primero lo que otros usan y no depende de nadie.

Solución 17.2 · Descomponer en pasos ← Volver al ejercicio

1) Crear un módulo Maven `common-errors` con `ApiErrorResponse` y el manejador, sin usarlo todavía. 2) Hacer que un servicio dependa de él y elimine su copia, comprobando sus pruebas. 3) Repetir con el otro servicio. 4) Publicar la librería con versión. Después de cada paso, todo compila y las pruebas pasan. (También es razonable decidir que una librería compartida acopla los servicios y no merece la pena por dos clases.)

Respuestas · capítulo 17 ← Volver a las preguntas

1. Porque es un dato de referencia que usan otros módulos, cambia a otro ritmo y no depende de nadie; extraerlo no exige llamadas de vuelta.
2. Por ejemplo: la llamada remota puede fallar o tardar (tiempos máximos, 503); las transacciones dejan de ser compartidas (sin clave foránea); no se pueden unir datos de las dos bases de datos (consulta por lotes, ordenación retirada).
3. Introducir una abstracción entre el consumidor y la implementación, cambiar la implementación detrás de ella y retirar lo antiguo, manteniendo el sistema funcionando en cada paso.
4. Porque son los únicos irreversibles; se hacen cuando todo lo demás está probado.

5. Que una demanda apunte a una aplicación inexistente. Se mitiga comprobando la existencia en el servicio y no borrando aplicaciones.

A.17 Capítulo 18. Una frontera dentro del monolito

↑ Índice

Solución 18.1 · Comprobar la frontera ← Volver al ejercicio

Desde la raíz del proyecto: buscar `import com.albertoderafael.itmanager.application` en `src/main/java/com/albertoderafael/itmanager/demand`; no debe haber resultados. Con ArchUnit, una prueba con una regla del tipo `noClasses().that().resideInAPackage("..demand..").should().dependOnClassesThat().resideInAPackage("..application..")` fallaría en cuanto alguien añadiera esa dependencia.

Solución 18.2 · Diseñar la degradación ← Volver al ejercicio

```
Page<DemandResponse> responsePage = demands.map(demand -> {
    ApplicationSummary application = applications.get(demand.getApplicationId());
    String name = application == null ? null : application.name();
    return toResponse(demand, demand.getApplicationId(), name);
});
```

(con un `toResponse` que reciba el `id` y el nombre). El contrato cambia: `applicationName` puede ser `null` y el frontend debe mostrarlo como «no disponible». En `DemandCatalogTest` habría que añadir una prueba con una aplicación ausente que compruebe que la página se devuelve con `applicationName` nulo en esa fila.

Respuestas · capítulo 18 ← Volver a las preguntas

1. El consumidor. Así la dependencia apunta hacia el núcleo y cambiar de adaptador no obliga a cambiar quien usa el puerto.
2. Porque expresa solo lo necesario, no expone el modelo interno y puede construirse igual desde JPA que desde una respuesta HTTP.
3. No. La columna es la misma y la clave foránea se mantuvo; solo cambió la forma de verla en Java.
4. Una llamada para la página de diez demandas (con tres identificadores) y ninguna para la página vacía.
5. Porque la paginación ya se aplicó; ordenar los elementos de una página no los convierte en los que corresponden a esa página en el orden global.

A.18 Capítulo 19. Nace `application-catalog-service`

↑ Índice

Solución 19.1 · Probar el *endpoint* de lotes ← Volver al ejercicio

`ids=1,2,999` devuelve dos resúmenes (el 999 no existe y no aparece). `ids=1&ids=2` devuelve lo mismo que `ids=1,2`: Spring acepta ambas formas. `ids=abc` devuelve un 400 `VALIDATION_ERROR` con «Invalid value for parameter: ids».

Solución 19.2 · Escribir el *script* de inicialización ← Volver al ejercicio

Un fichero `/docker-entrypoint-initdb.d/10-application-db.sh` (los `.sql` no expanden variables) con:

```
#!/bin/bash
set -e
psql -v ON_ERROR_STOP=1 --username "$POSTGRES_USER" --dbname "$POSTGRES_DB" <<-EOSQL
  CREATE USER application_user WITH PASSWORD '${APPLICATION_DB_PASSWORD}';
  CREATE DATABASE application_db OWNER application_user;
EOSQL
```

y la variable `APPLICATION_DB_PASSWORD` pasada al contenedor en `compose.yaml` desde el entorno. No se ejecutaría sobre el volumen existente porque la imagen oficial solo ejecuta esos *scripts* cuando el directorio de datos está vacío, es decir, la primera vez.

Respuestas · capítulo 19 ← Volver a las preguntas

1. Que cada servicio es el único dueño de sus datos y los demás acceden a ellos solo por su API. No lo invalida porque la separación es lógica: bases de datos y usuarios distintos.
2. Porque los datos reales venían del monolito con sus identificadores; una V2 con datos de ejemplo habría ocupado o duplicado esos identificadores.
3. Elige la ruta más específica: un segmento literal (`/summaries`) gana a una variable (`/{id}`).
4. Porque las demandas existentes apuntan a las aplicaciones por su `id` . Después hay que ajustar la secuencia con `setval` para que las nuevas aplicaciones no reutilicen identificadores.
5. Demuestra que el `UPDATE` es SQL válido (el `flush` no falla), pero no que los valores leídos vengan de PostgreSQL, porque sin `clear()` la entidad se lee de la memoria.

A.19 Capítulo 20. El adaptador HTTP con `RestClient`

↑ Índice

Solución 20.1 · Provocar un 503 ← Volver al ejercicio

Con el catálogo parado, el listado de demandas responde 503 «Application Catalog is unavailable» casi al instante: el sistema operativo rechaza la conexión de inmediato porque no hay nadie escuchando en el 8081, sin esperar al tiempo máximo de conexión. Al arrancar el catálogo, la siguiente petición funciona sin reiniciar el servicio de demandas, porque cada petición abre su propia conexión.

Solución 20.2 · Probar el tiempo máximo de lectura ← Volver al ejercicio

Añadir en `setUp` un contexto `/api/applications/slow` cuyo manejador haga `Thread.sleep(4000)` antes de responder, y una prueba que llame a `catalog.findById(...)` de forma que la ruta termine en `/slow` (por ejemplo, ajustando el servidor para que esa ruta responda a un `id` concreto) y espere una `ResponseStatusException` con 503. Para no esperar cuatro segundos, se puede hacer configurable el tiempo de lectura (por constructor o propiedad) y usar, por ejemplo, 200 ms en la prueba con un retraso de 500 ms.

Respuestas · capítulo 20 ← Volver a las preguntas

1. La propagación del fallo de un servicio al que lo llama, por ejemplo agotando sus hilos en esperas indefinidas. Los tiempos máximos acotan la espera, liberan los recursos y convierten el problema en un error controlado.
2. Porque no depende del DTO de otro proyecto y solo lee los campos que necesita (lector tolerante); los demás se ignoran.
3. Porque un 404 es una respuesta de negocio («no existe»), mientras que un fallo de conexión significa «no puedo saberlo»; responder 404 en ese caso sería falso.
4. La transacción retiene una conexión a la base de datos mientras espera la respuesta HTTP, lo que puede agotar el *pool*.
5. Porque intercepta las peticiones dentro del propio `RestClient` sin abrir conexiones reales, así que no puede simular un servidor inaccesible.

A.20 Capítulo 21. Separar las bases de datos

↑ Índice

Solución 21.1 · Leer el historial completo ← Volver al ejercicio

Cinco filas: tras V1 existe `applications`; tras V2 tiene dos filas; tras V3 existe `demands` con la clave foránea hacia `applications`; tras V4 la clave foránea ha desaparecido; tras V5 ya no existe `applications` y solo queda `demands` (más el historial de Flyway).

Solución 21.2 · Diseñar la desnormalización ← Volver al ejercicio

Migración `V6__add_application_name_to_demands.sql` con `ALTER TABLE demands ADD COLUMN application_name VARCHAR(100);`, seguida de un relleno inicial con los nombres obtenidos del catálogo (un proceso puntual, no SQL, porque los datos están en otra base de datos) y, después, `NOT NULL` si se quiere. Habría que escribirla al crear y al modificar una demanda (el servicio ya tiene el nombre en ese momento). Si una aplicación se renombra, las demandas mostrarían el nombre antiguo hasta actualizarse; para enterarse, el catálogo podría publicar un evento «aplicación renombrada» que el servicio de demandas consumiera para actualizar sus filas.

Respuestas · capítulo 21 ← Volver a las preguntas

1. Porque las aplicaciones creadas en el catálogo no existirían en la tabla local, y la clave foránea impediría guardar demandas sobre ellas.
2. Porque PostgreSQL no permite borrar una tabla referenciada por una clave foránea.
3. Para no dejar una segunda fuente de verdad con datos congelados que alguien podría consultar por error.
4. Sustituye al adaptador `RestApplicationCatalog` (el puerto `ApplicationCatalog`). Siguen siendo reales Spring, Flyway, Hibernate, las transacciones y PostgreSQL.
5. Que se puede guardar una referencia a una aplicación inexistente, es decir, que la clave foránea ya no existe. `saveAndFlush` fuerza el `INSERT` y `clear` obliga a releer de la base de datos.

A.21 Capítulo 22. Validar el sistema completo

↑ Índice

Solución 22.1 · Ejecutar la batería de humo

← Volver al ejercicio

En condiciones normales, la comprobación 3 tarda unos milisegundos más que la consulta de un monolito por la llamada HTTP. Con el catálogo parado, responde casi al instante con 503 porque la conexión se rechaza de inmediato (no hay proceso escuchando en el puerto); el tiempo máximo de conexión solo se agotaría si el servidor no respondiera en absoluto, por ejemplo por un cortafuegos que descarta paquetes.

Solución 22.2 · Ejecutar y registrar las suites

← Volver al ejercicio

Maven muestra al final `Tests run: 18` en el catálogo y `Tests run: 35` en demandas si todas se ejecutan. Una prueba que falle se localiza por su clase: las de `*ServiceTest` y `DemandTest` son unitarias (capítulos 11, 12 y 18), las de `*ControllerTest` son de capa web (11, 15), `RestApplicationCatalogTest` es del adaptador (20) y las de `*IntegrationTest` y `*ApplicationTests` necesitan Docker (16, 19, 21).

Respuestas · capítulo 22

← Volver a las preguntas

1. Porque no llama al catálogo al arrancar, solo cuando una petición lo necesita. Implica que haber arrancado no garantiza que pueda trabajar.
2. La 5 frente a la 8: una aplicación inexistente da 404 y un catálogo parado da 503.
3. Porque las respuestas del catálogo en las pruebas de demandas están escritas a mano (*mocks* y servidor del JDK); nada contrasta esas respuestas con el catálogo real.
4. Se perdió la ordenación por aplicación, la integridad garantizada por la base de datos y la sencillez (latencia, nuevo modo de fallo). Se ganó un dueño y una API para el catálogo, bases de datos independientes, despliegues separados y preparación para nuevos módulos.
5. El final del paso 5.6: monolito con el puerto `ApplicationCatalog` y su adaptador JPA.

A.22 Capítulo 23. Errores y lecciones aprendidas

↑ Índice

Solución 23.1 · Priorizar los defectos

← Volver al ejercicio

Una priorización razonable pone primero el 1 (transacción abierta durante la llamada HTTP: con un catálogo lento puede dejar sin conexiones a todo el servicio; se detecta con una prueba que simule un catálogo lento y peticiones concurrentes), el 3 (una referencia rota bloquea todo el listado; prueba con una aplicación ausente) y el 8 (sin prueba del tiempo de lectura, el escenario más peligroso no está protegido; prueba con un servidor que responde tarde). El 12 va el último: no tiene efecto funcional.

Solución 23.2 · Arreglar la relectura

← Volver al ejercicio

Inyectar el `EntityManager` en la prueba (`@Autowired private EntityManager entityManager;`) y, justo después de `applicationRepository.flush();`, añadir `entityManager.clear();`. Así, el `findById` posterior ejecuta un `SELECT` real y los valores comprobados vienen de PostgreSQL.

Respuestas · capítulo 23 ← Volver a las preguntas

1. Que el código del servicio se escribió suponiendo una llamada local (rápida, siempre disponible, sin datos ausentes) y no se revisó al pasar a una llamada remota.
2. Porque no es reproducible: no hay procedimiento escrito para repetirlo en otro entorno, y dejó un volcado con datos reales en la carpeta del proyecto.
3. Una prueba de contrato entre frontend y backend, o una prueba del frontend que valide sus campos de ordenación contra la lista del servidor.
4. El análisis estático del IDE, un formateador automático, o herramientas en el *build* como Checkstyle o SpotBugs.
5. La sexta: «Ver pasar la suite es parte del trabajo».

A.23 Capítulo 24. Estado final y próximos pasos

↑ Índice

Solución 24.1 · Planificar la siguiente iteración ← Volver al ejercicio

Una selección razonable: ejecutar las dos suites (terminado cuando ambas pasan y el resultado queda registrado); sacar la llamada al catálogo de la transacción (prueba que verifique que no hay transacción activa durante la llamada, por ejemplo con `TransactionSynchronizationManager.isActualTransactionActive()` en un *mock* del catálogo); traducir 4xx a 502 (prueba en `RestApplicationCatalogTest`); *script* de inicialización de `application_db` (crear un volumen nuevo y arrancar el catálogo sin pasos manuales); y prueba del tiempo de lectura (la del ejercicio 20.2).

Solución 24.2 · Completar la propuesta de Compose ← Volver al ejercicio

Añadir a `postgres` el volumen `postgres_data:/var/lib/postgresql`, el `healthcheck` con `pg_isready` y los *scripts* de inicialización; publicar `8081:8081` y `8080:8080`; y para cada servicio un `healthcheck` como `curl -f http://localhost:8081/actuator/health` (si la imagen incluye `curl`). En los proyectos haría falta añadir `spring-boot-starter-actuator` y exponer el *endpoint* de salud. El servicio de demandas podría depender del catálogo con `condition: service_healthy`, aunque no es imprescindible, porque arranca sin él.

Respuestas · capítulo 24 ← Volver a las preguntas

1. Solo el contenedor de PostgreSQL 18.6, con su volumen y su comprobación de salud.
2. Porque todo lo demás se construiría sobre una base cuyo funcionamiento no se ha verificado.
3. Deja de llamar durante un tiempo a un servicio que está fallando y responde de inmediato, sin consumir recursos esperando; el tiempo máximo solo acota cada espera individual.
4. Porque dentro de la red de Compose los servicios se comunican por los puertos internos de los contenedores; el 5433 es solo el puerto publicado en el equipo.
5. Que dos modificaciones simultáneas se pisen sin saberlo. El conflicto produciría un 409.

A.24 Capítulo 25. Cerrar la separación

↑ Índice

Solución 25.1 · Comprobar que no hay transacción durante la llamada ← Volver al ejercicio

Como `DemandIntegrationTest` es `@Transactional`, cada prueba se ejecuta dentro de una transacción abierta por la propia prueba, y `isActualTransactionActive()` devolvería `true` aunque el servicio no abriera ninguna. La comprobación solo tiene sentido en una clase de prueba **sin** `@Transactional` (limpiando los datos a mano al terminar, o en una base de datos propia). Dentro de `thenAnswer` se comprueba `assertFalse(TransactionSynchronizationManager.isActualTransactionActive())` y se devuelve el `ApplicationSummary`; si alguien vuelve a poner `@Transactional` en el servicio, la prueba falla.

Solución 25.2 · La regla de la degradación ← Volver al ejercicio

Leer una demanda es **mostrar** algo que ya existe: si su aplicación desapareció, la información más útil es la demanda con un aviso, no un error que oculta todo el listado. Crear o modificar es **aceptar** datos nuevos: guardar una demanda que apunta a una aplicación inexistente crearía a sabiendas un dato incoherente. Si el alta degradara, un usuario podría crear demandas con un `applicationId` mal escrito, que aparecerían para siempre como «Aplicación no disponible» sin que nadie supiera que el error fue al darlas de alta.

Respuestas · capítulo 25 ← Volver a las preguntas

1. Porque `save()` de `SimpleJpaRepository` abre y cierra su propia transacción. El servicio ya no necesita una transacción alrededor de todo el método, y así la llamada al catálogo queda fuera.
2. Delimita la transacción solo alrededor de la parte que escribe en la base de datos, después de la llamada remota. `@Transactional` abarca el método entero.
3. Porque un 404 de `findById` es una respuesta de negocio («no existe»), no un fallo. Capturándolo dentro de la petición, todo lo que llega a `executeRequest` es un fallo y se traduce a 502 o 503.
4. Que las excepciones propias de Spring MVC (ruta inexistente, verbo no permitido…) se conviertan en un 500 y una URL mal escrita parezca un fallo del servidor.
5. Que se usa `DB_PASSWORD` si existe y, si no, `APPLICATION_DB_PASSWORD`. No tiene un valor por defecto real porque un servicio sin contraseña debe negarse a arrancar, y una contraseña escrita en el código acabaría en el repositorio.

A.25 Capítulo 26. Imágenes de contenedor

↑ Índice

Solución 26.1 · Medir el efecto de las capas ← Volver al ejercicio

Al cambiar una clase, todos los pasos hasta `COPY src ./src` aparecen como `CACHED` (incluida la descarga de dependencias) y solo se repiten la compilación y la última capa de la imagen: unos segundos. Al cambiar el `pom.xml`, la caché se invalida desde `COPY pom.xml`, Maven vuelve a ejecutar `dependency:go-offline` y el *build* tarda como la primera vez. Docker reutiliza las capas hasta la primera instrucción cuyo resultado cambia.

Solución 26.2 · Ver las capas de una imagen ← Volver al ejercicio

`docker history` lista las capas de la más reciente a la más antigua. Las cuatro creadas por `COPY --from=build` corresponden a `dependencies/` (la más grande, decenas de megas), `spring-boot-loader/` (pequeña), `snapshot-dependencies/` (vacía o casi) y `application/` (unos kilobytes). Al modificar una clase solo cambia la capa de `application/`.

Respuestas · capítulo 26 ← Volver a las preguntas

1. Los *endpoints* bajo `/actuator`, los indicadores de salud (base de datos, disco, estado del proceso) y, con `probes.enabled`, los grupos `liveness` y `readiness`.
2. Porque el servicio de demandas sigue siendo útil con el catálogo caído; si su salud dependiera de él, un orquestador podría reiniciarlo o retirarlo por un fallo ajeno y provocar un fallo en cascada.
3. Una imagen más pequeña y con menos superficie de ataque: no lleva Maven, ni el JDK completo, ni el código fuente.
4. Para que la descarga de dependencias quede en una capa que se reutiliza mientras el `pom.xml` no cambie; al modificar el código solo se repite la compilación.
5. El propio contenedor. Desde él se llega al PostgreSQL publicado en tu equipo con el nombre especial `host.docker.internal`.

A.26 Capítulo 27. La plataforma con Docker Compose

↑ Índice

Solución 27.1 · Una instalación desde cero ← Volver al ejercicio

Con `docker compose down` (sin `-v`) se eliminan los contenedores; al volver a arrancar con `POSTGRES_VOLUME=itsp_prueba`, PostgreSQL encuentra el volumen vacío, crea `it_service_manager` y `itmanager` a partir de las variables `POSTGRES_*` y ejecuta el *script* de `postgres/init`, que crea `application_user` y `application_db`. Al arrancar, Flyway migra cada base de datos desde cero (V1 a V5 en demandas, V1 en el catálogo). En pgAdmin, una conexión con `application_user` a `it_service_manager` debe fallar con un error de permiso de conexión. Al terminar: `down`, restaurar `POSTGRES_VOLUME` y `up -d`; el volumen de prueba se puede eliminar con `docker volume rm itsp_prueba`, porque no contiene nada que conservar.

Solución 27.2 · Publicar solo la entrada ← Volver al ejercicio

Se quitarían los bloques `ports` de `postgres`, `application-catalog-service` y `demand-service`, y quedaría solo `8088:80` en `web`. Dejarían de funcionar pgAdmin y Spring Tools contra la base de datos de Docker, y Vite, que llega a los servicios por los puertos 8080 y 8081. Para conservar las dos formas, el `compose.yaml` base publicaría solo `web`, y un `compose.override.yaml` (que Compose aplica automáticamente si existe) añadiría los demás puertos para desarrollo. En un entorno real se arrancaría solo con el fichero base (`docker compose -f compose.yaml up -d`).

Respuestas · capítulo 27 ← Volver a las preguntas

1. Para que ningún servicio sea dueño de la infraestructura común; la plataforma es la que conoce el sistema completo y cada servicio solo sabe construirse a sí mismo.
2. Un volumen normal es propiedad del proyecto y `down -v` lo elimina; uno externo nunca lo crea ni lo borra Compose, ni siquiera con `-v`.
3. Porque dentro de la red los contenedores se comunican por sus puertos internos; el 5433 es solo el puerto publicado en el equipo anfitrión.
4. Solo cuando el volumen de datos está vacío (instalación nueva). La contraseña llega como variable de entorno y se pasa a `psql` como variable para que `:'app_password'` la inserte correctamente entrecomillada, sin escribirla en el fichero.

5. Que nginx no pueda arrancar si un servicio todavía no existe: con el nombre en una variable y el DNS de Docker, el nombre se resuelve en cada petición y solo falla la ruta del servicio parado.

ANEXO B

Glosario

↑ Volver al índice

Términos técnicos usados en la guía, en orden alfabético. Entre paréntesis, el capítulo donde se explican con más detalle.

B.1 A – D

↑ Índice

Término	Definición
Actuator	Módulo de Spring Boot que expone información operativa; en el proyecto, solo `/actuator/health` (26)
Adaptador	Implementación de un puerto con una tecnología concreta: `JpaApplicationCatalog`, `RestApplicationCatalog` (18, 20)
`ApiErrorResponse`	*Record* con el formato común de error del proyecto: `status`, `error`, `message`, `fieldErrors` (10)
`ApplicationCatalog`	Puerto de solo lectura que el módulo de demandas usa para conocer las aplicaciones (18)
`ApplicationSummary`	*Record* con el `id` y el nombre de una aplicación; lo único que cruza la frontera (18)
Autoconfiguración	Mecanismo de Spring Boot que crea *beans* según lo que hay en el *classpath* y la configuración (2)
Autoinvocación	Llamada entre métodos del mismo *bean* que no pasa por el proxy y anula `@Transactional` (7)
Base de datos por servicio	Patrón en el que cada servicio es el único dueño de sus datos (19)
Bean Validation	Especificación de validación declarativa con anotaciones; la implementa Hibernate Validator (8)
Bind mount	Montaje de una carpeta del equipo dentro de un contenedor, como `postgres/init` (27)
BOM	Lista de materiales que fija versiones compatibles de un conjunto de librerías (2)
Branch by abstraction	Técnica de cambio gradual: abstracción, nueva implementación detrás y retirada de la antigua (17)
Capa (de imagen)	Resultado de una instrucción del Dockerfile; Docker las guarda en caché y las reutiliza (26)
Checksum	Huella del contenido de una migración que Flyway usa para detectar cambios (4)
Clave foránea	Restricción que obliga a que un valor exista en otra tabla (12, 21)
Consulta derivada	Método de repositorio cuya consulta se genera a partir de su nombre (6)
Consulta por lotes	Pedir de una vez todos los datos que necesita una página (18)

Término	Definición
Contenedor	Proceso aislado que se ejecuta a partir de una imagen (26)
Contexto de persistencia	Conjunto de entidades que gestiona un `EntityManager`; dura lo que la transacción (5)
Cortacircuitos (*circuit breaker*)	Mecanismo que deja de llamar a un servicio que falla durante un tiempo (24)
Criteria API	API de JPA para construir consultas con objetos Java (13)
`ddl-auto`	Propiedad que indica a Hibernate qué hacer con el esquema al arrancar (3)
Desnormalización	Copiar un dato de otro origen para poder consultarlo localmente (21)
Dirty checking	Comparación automática de entidades gestionadas con su instantánea para generar `UPDATE` (5)
Docker Compose	Herramienta que define y arranca varios contenedores relacionados con un fichero `compose.yaml` (27)
`.dockerignore`	Lista de ficheros que no se envían al construir una imagen (26)
DTO	Objeto de transferencia de datos entre la API y el exterior (8)

B.2 E – N

↑ Índice

Término	Definición
`EntityGraph`	Indicación de qué asociaciones cargar junto con la entidad en la misma consulta (14)
`EntityManager`	Interfaz de JPA para operar con entidades y su contexto de persistencia (5)
`ErrorResponse`	Interfaz de Spring que implementan las excepciones con código HTTP propio (25)
Estado gestionado (*managed*)	Entidad vigilada por el contexto de persistencia (5)
Estado separado (*detached*)	Entidad cuyo contexto se cerró; sus cambios ya no se sincronizan (5)
Fallo en cascada	Propagación del fallo de un servicio a los que lo llaman (20)
Flush	Envío a la base de datos de los cambios pendientes del contexto de persistencia (5)
Flyway	Herramienta de migraciones versionadas del esquema (4)
Healthcheck	Comando que Docker ejecuta periódicamente para marcar un contenedor como `healthy` o `unhealthy` (27)
Hibernate	Implementación de JPA usada por Spring Boot (5)
HikariCP	*Pool* de conexiones JDBC que usa Spring Boot por defecto (3)
`host.docker.internal`	Nombre con el que un contenedor llega al equipo anfitrión en Docker Desktop (26)

Término	Definición
`IDENTITY`	Estrategia de generación de identificadores por columna de identidad de la base de datos (5)
Imagen	Sistema de ficheros de solo lectura con todo lo necesario para ejecutar una aplicación (26)
Inversión de dependencias	Principio por el que el código de alto nivel define las interfaces que implementa el de bajo nivel (18)
JAR por capas	Extracción del JAR de Spring Boot en carpetas separadas para aprovechar la caché de capas (26)
JPA (Jakarta Persistence)	Especificación estándar de persistencia de objetos en Java (5)
`JpaRepository`	Interfaz de Spring Data que reúne las operaciones CRUD, paginación y extras de JPA (6)
`JpaSpecificationExecutor`	Interfaz que añade a un repositorio consultas con `Specification` (13)
`LAZY` / `EAGER`	Modos de carga de una asociación: bajo demanda o siempre (12)
`LazyInitializationException`	Excepción al acceder a una asociación perezosa sin contexto de persistencia (3, 14)
Lector tolerante	Consumidor que solo lee los campos que necesita e ignora el resto (20)
Línea base (*baseline*)	*Script* que crea directamente el estado actual del esquema para instalaciones nuevas (21)
`@MockitoBean`	Anotación que registra un *mock* como *bean* en un contexto de prueba (11)
MockMvc	Herramienta que ejecuta peticiones contra Spring MVC sin abrir puertos (11)
Monolito modular	Aplicación única organizada en módulos con fronteras claras (1, 16)
Multietapa (*multi-stage*)	Dockerfile con varias etapas: una compila y otra ejecuta solo con lo necesario (26)
N+1	Problema de rendimiento: una consulta más por cada elemento de una lista (14)

B.3 O – Z

↑ Índice

Término	Definición
Open Session in View (OSIV)	Patrón que mantiene el contexto de persistencia abierto durante toda la petición HTTP (3)
`Pageable` / `Page`	Petición de página (número, tamaño, orden) y resultado con metadatos (6, 9)
Perfil	Conjunto de configuración que se activa por nombre, como `docker` (3)
Probes (`liveness`, `readiness`)	Grupos de salud que indican si el proceso vive y si puede recibir tráfico (26)
Problem Details	Formato estándar de errores HTTP (RFC 9457) soportado por Spring con `ProblemDetail` (10)

Término	Definición
Propagación	Regla que decide qué hace un método transaccional si ya hay una transacción en curso (7)
Proxy	Objeto intermediario generado por Spring o Hibernate: transaccional, de repositorio o de carga perezosa (6, 7, 12)
Proxy inverso	Servidor que recibe las peticiones y las reenvía al servicio adecuado; en el proyecto, nginx (27)
Prueba de contrato	Prueba que verifica que proveedor y consumidor de una API están de acuerdo en su forma (22)
Puerto	Interfaz definida por el núcleo de una aplicación con lo que necesita del exterior (18)
Record	Clase inmutable de Java con constructor, accesos, `equals` y `hashCode` generados (8)
`ResponseStatusException`	Excepción de Spring que lleva un código HTTP y un motivo (7, 10)
`RestClient`	Cliente HTTP síncrono y fluido de Spring (20)
`@RestControllerAdvice`	Clase con manejadores de excepciones aplicables a todos los controladores (10)
Rollback	Deshacer una transacción (7, 15)
`@ServiceConnection`	Anotación que conecta Spring Boot a un contenedor de Testcontainers (16)
`SimpleJpaRepository`	Clase que implementa los métodos heredados de `JpaRepository` (6)
`Slice`	Resultado paginado sin recuento total (6)
SPA *fallback*	Regla que devuelve `index.html` para cualquier ruta de la aplicación de una sola página (27)
`Specification`	Condición reutilizable y combinable de Spring Data basada en la Criteria API (13)
Starter	Dependencia que agrupa librerías y activa su autoconfiguración (2)
Testcontainers	Librería que arranca contenedores Docker desde las pruebas (16)
Tiempo máximo (*timeout*)	Límite de espera de conexión o de lectura en una llamada remota (20)
`TIMESTAMP WITH TIME ZONE`	Tipo de PostgreSQL que guarda un instante en UTC (12)
`@Transactional`	Anotación que delimita una transacción alrededor de un método (7)
`TransactionOperations`	Interfaz de `TransactionTemplate` para delimitar una transacción en el código (25)
Volumen externo	Volumen que Compose usa pero nunca crea ni borra (27)
`@WebMvcTest`	Prueba que arranca solo la capa web de Spring MVC (11)

ANEXO C

Equivalencias Java EE ↔ Spring Boot

↑ Volver al índice

Tabla de traducción para quien viene de Java EE sobre WebSphere. La columna de notas señala dónde la equivalencia no es exacta.

C.1 Plataforma y despliegue

↑ Índice

Java EE / WebSphere	Spring Boot	Notas
Servidor de aplicaciones instalado aparte	Tomcat embebido en el JAR	La aplicación contiene su servidor (2)
EAR con WAR y EJB-JAR	Un JAR ejecutable	`java -jar` o `mvnw spring-boot:run`
Consola de administración	`application.properties`, variables de entorno, perfiles	La configuración viaja con el código, salvo los secretos (3)
Origen de datos JNDI	`spring.datasource.*` + HikariCP	El *pool* lo crea la aplicación (3)
Descriptores `web.xml`, `ejb-jar.xml`	Anotaciones + autoconfiguración	Sin XML en el proyecto
Librerías compartidas del servidor	Dependencias Maven gestionadas por el BOM	Cada servicio lleva las suyas (2)
Varias aplicaciones en un servidor	Un proceso por servicio	Puertos 8080 y 8081 (19)

C.2 Componentes y dependencias

↑ Índice

Java EE	Spring	Notas
`@Stateless` EJB	`@Service`	Los *beans* de Spring son *singleton* por defecto, no un *pool* de instancias
`@EJB`, `@Inject`	Inyección por constructor	Sin anotación si hay un único constructor (2)
Descubrimiento de componentes al desplegar	`@ComponentScan` desde la clase principal	Solo en el paquete raíz y subpaquetes (2)
`@Produces` (CDI)	Métodos `@Bean` en una clase `@Configuration`	No se usan en el proyecto
Interceptores (`@AroundInvoke`)	Spring AOP (proxies)	`@Transactional` funciona así (7)
Búsqueda JNDI de un EJB remoto	Cliente HTTP (`RestClient`) a otro servicio	Ver «llamadas remotas» (20)

C.3 Web y REST

↑ Índice

JAX-RS	Spring MVC	Notas
`@Path`	`@RequestMapping`	(9)
`@GET`, `@POST`, `@PUT`	`@GetMapping`, `@PostMapping`, `@PutMapping`	
`@QueryParam`	`@RequestParam`	
`@PathParam`	`@PathVariable`	
Parámetro de entidad del cuerpo	`@RequestBody`	Con `@Valid` para validar (8)
`Response.created(uri).entity(x)`	`ResponseEntity.created(uri).body(x)`	(9)
`ExceptionMapper<T>`	`@RestControllerAdvice` + `@ExceptionHandler`	Varios manejadores en una clase (10)
Servlet de la implementación JAX-RS	`DispatcherServlet`	
JSON-B / JSON-P	Jackson	Spring Boot 4 usa Jackson 3

C.4 Persistencia

↑ Índice

Java EE	Spring	Notas
`javax.persistence`	`jakarta.persistence`	Mismo API, otro paquete (5)
OpenJPA / EclipseLink	Hibernate	
`@PersistenceContext EntityManager` en un DAO	Repositorio de Spring Data (interfaz)	La implementación se genera (6)
JPQL con `TypedQuery`	Consultas derivadas, `@Query`, `Specification`	(6, 13)
`persistence.xml`	`spring.jpa.*`	Sin fichero de unidad de persistencia
Contexto de persistencia ligado a la transacción JTA	Contexto ligado a la transacción de Spring	Igual con `open-in-view=false` (3, 5)
Scripts SQL entregados al equipo de base de datos	Migraciones Flyway versionadas	Con historial en la propia base de datos (4)

C.5 Transacciones

↑ Índice

Java EE	Spring	Notas
Transacciones gestionadas por el contenedor (CMT)	`@Transactional`	Proxy en lugar de contenedor EJB (7)

Java EE	Spring	Notas
`@TransactionAttribute(REQUIRED)`	`@Transactional` (propagación `REQUIRED`)	Mismo valor por defecto
`@ApplicationException(rollback = true)`	`@Transactional(rollbackFor = …)`	Las comprobadas no hacen *rollback* por defecto en ninguno
JTA y transacciones XA entre recursos	Transacciones locales por servicio	El proyecto no usa transacciones distribuidas (17)
`EJBException` envolviendo la excepción	La excepción llega intacta al manejador	(15)

C.6 Validación

↑ Índice

Java EE	Spring	Notas
Bean Validation (`javax.validation`)	Bean Validation (`jakarta.validation`)	Misma especificación (8)
Validación automática en JAX-RS con `@Valid`	`@Valid` en el parámetro del controlador	Lanza `MethodArgumentNotValidException`
Transfer Object JavaBean	*Record* de Java	Inmutable (8)

C.7 Pruebas

↑ Índice

Java EE	Spring	Notas
Arquillian con servidor gestionado	`@SpringBootTest`	Arranca la aplicación completa en el proceso de la prueba (11, 16)
Base de datos de pruebas compartida	Testcontainers con PostgreSQL desechable	Una por ejecución (16)
Pruebas del recurso JAX-RS con cliente HTTP	`@WebMvcTest` + MockMvc	Sin abrir puertos (11)
Mockito	Mockito + `@MockitoBean`	(11)

C.8 Llamadas remotas

↑ Índice

Java EE	Spring	Notas
EJB remoto (RMI-IIOP)	`RestClient` sobre HTTP/JSON	Contrato explícito en lugar de interfaz Java compartida (20)
Tiempos de espera configurados en el servidor	Tiempos máximos en la fábrica de peticiones	En el código del adaptador (20)

Java EE	Spring	Notas
`RemoteException`	`ResourceAccessException`, `RestClientResponseException`	Traducidas a 503 en el proyecto (20)
Interfaz remota compartida en un JAR cliente	*Record* privado del consumidor	Lector tolerante (20)

C.9 Despliegue

↑ Índice

Java EE / WebSphere	Spring Boot y contenedores	Notas
Servidor instalado y configurado por administración	Imagen con el JRE y la aplicación	Se comporta igual en cualquier máquina (26)
EAR desplegado en un servidor	Contenedor ejecutado a partir de una imagen	La imagen es la clase; el contenedor, la instancia (26)
Estado del servidor en la consola de administración	`/actuator/health` y el *healthcheck* de Docker	(26, 27)
IBM HTTP Server con el *plugin* de WebSphere	nginx como proxy inverso	Sirve el estático y reenvía las peticiones (27)
Varios servidores y orígenes de datos configurados a mano	`compose.yaml` versionado	Todo el sistema en un fichero (27)
Transacciones gestionadas por el bean (`UserTransaction`)	`TransactionOperations` / `TransactionTemplate`	Sin `try`/`finally` manual (25)

ANEXO D

Chuleta de comandos

↑ Volver al índice

Los comandos de uso diario con los dos servicios, agrupados por herramienta. Ninguno contiene credenciales: las contraseñas se toman siempre de variables de entorno.

D.1 Puertos y nombres

↑ Índice

Pieza	Puerto	Base de datos	Usuario	Variable de la contraseña
PostgreSQL (Docker)	5433 (interno 5432)	—	—	`DB_PASSWORD` y `APPLICATION_DB_PASSWORD` en el `.env` de la plataforma
Servicio de demandas	8080	`it_service_manager`	`itmanager`	`DB_PASSWORD`
Catálogo de aplicaciones	8081	`application_db`	`application_user`	`DB_PASSWORD` (o `APPLICATION_DB_PASSWORD`)
Frontend con nginx (Docker)	8088	—	—	—
Frontend con Vite (desarrollo)	5173	—	—	—

D.2 Docker Compose (carpeta `it-service-platform`)

↑ Índice

```
docker compose up -d --build         # construir y arrancar todo el sistema
docker compose up -d                 # arrancar sin reconstruir
docker compose ps                    # estado y salud de cada servicio
docker compose logs -f demand-service
docker compose stop application-catalog-service    # parar un servicio
docker compose start application-catalog-service   # volver a arrancarlo
docker compose stop                  # parar todo sin eliminar contenedores
docker compose down                  # eliminar contenedores; el volumen se CONSERVA
docker volume ls                     # volúmenes (it-service-manager_postgres_data)
```

Docker (imágenes sueltas)	Qué hace
`docker build -t it-service-manager:dev .`	Construye la imagen de un servicio desde su carpeta
`docker history it-service-manager:local`	Muestra las capas de una imagen
`docker run --rm -p 8081:8081 -e DB_URL=… -e DB_PASSWORD=… application-catalog-service:dev`	Ejecuta una imagen suelta; desde el contenedor, el equipo es `host.docker.internal`

Docker (imágenes sueltas)	Qué hace
`Get-Process -Id (Get-NetTCPConnection -LocalPort 8080 -State Listen).OwningProcess`	Qué proceso ocupa un puerto

> **Cuidado**
>
> No uses `docker compose down -v` ni `docker volume rm`: eliminan el volumen y, con él, todos los datos.

```
# Consola SQL dentro del contenedor
docker compose exec postgres psql -U itmanager -d it_service_manager
```

Comando de `psql`	Qué hace
`\l`	Lista las bases de datos
`\c application_db`	Cambia de base de datos
`\dt`	Lista las tablas
`\d demands`	Describe una tabla: columnas, índices, restricciones
`\q`	Sale

```
-- Historial de Flyway
SELECT installed_rank, version, description, success
FROM flyway_schema_history ORDER BY installed_rank;
```

D.3 Maven Wrapper

↑ Índice

Comando (PowerShell)	Qué hace
`.\mvnw.cmd spring-boot:run`	Arranca el servicio
`.\mvnw.cmd test`	Ejecuta todas las pruebas (las de integración necesitan Docker)
`.\mvnw.cmd test -Dtest=DemandCatalogTest`	Una clase de pruebas
`.\mvnw.cmd test -Dtest="DemandTest#shouldReject*"`	Métodos por patrón
`.\mvnw.cmd -DskipTests package`	Compila y empaqueta sin pruebas
`.\mvnw.cmd dependency:tree`	Árbol de dependencias y sus versiones
`.\mvnw.cmd spring-boot:build-image`	Construye una imagen de contenedor (no se usa todavía)

D.4 Variables de entorno en PowerShell

↑ Índice

```
# Pedir una contraseña sin escribirla en el historial
$cred = Get-Credential -UserName itmanager -Message 'Base de datos de demandas'
$env:DB_PASSWORD = $cred.GetNetworkCredential().Password

# Cambiar la URL del catálogo sin tocar ficheros
$env:APPLICATION_CATALOG_BASE_URL = 'http://localhost:8081'

# Quitar una variable de la sesión
Remove-Item Env:DB_PASSWORD
```

D.5 Peticiones de comprobación

↑ Índice

```
# Catálogo
Invoke-RestMethod 'http://localhost:8081/api/applications?page=0&size=5'
Invoke-RestMethod 'http://localhost:8081/api/applications/1'
Invoke-RestMethod 'http://localhost:8081/api/applications/summaries?ids=1,2'

# Demandas
Invoke-RestMethod 'http://localhost:8080/api/demands?status=OPEN&sort=updatedAt,desc'

# Alta de una demanda
$body = @{ title = 'Prueba'; description = $null; priority = 'LOW'; applicationId = 1 } |
        ConvertTo-Json
Invoke-RestMethod -Method Post -Uri 'http://localhost:8080/api/demands' `
                  -ContentType 'application/json' -Body $body
```

Con `curl` (en PowerShell, `curl.exe` para no usar el alias):

```
curl.exe -i "http://localhost:8080/api/demands/999"
curl.exe -i -X POST "http://localhost:8080/api/demands" \
     -H "Content-Type: application/json" \
     -d "{\"title\":\"Prueba\",\"priority\":\"LOW\",\"applicationId\":1}"
```

La opción `-i` muestra el código de estado y las cabeceras, útil para ver `Location` o un 409.

D.6 Propiedades útiles para depurar

↑ Índice

Propiedad	Efecto
`debug=true`	Informe de autoconfiguración al arrancar
`spring.jpa.show-sql=true`	SQL de Hibernate en la consola
`logging.level.org.hibernate.SQL=DEBUG`	SQL de Hibernate por el sistema de *logs*
`logging.level.org.hibernate.orm.jdbc.bind=TRACE`	Valores de los parámetros del SQL
`spring.jpa.properties.hibernate.generate_statistics=true`	Estadísticas de sentencias por sesión
`logging.level.org.springframework.web=DEBUG`	Detalle de cómo Spring MVC resuelve cada petición

D.7 Códigos de estado del proyecto

↑ Índice

Código	Cuándo
200	Consulta o modificación correcta
201	Alta correcta, con cabecera `Location`
400	Campos inválidos, JSON mal formado, tipo incorrecto, filtro u orden desconocido
404	Recurso inexistente (demanda o aplicación)
409	Transición de estado inválida
503	El catálogo no responde o falla

D.8 Regenerar esta guía

↑ Índice

```
cd docs/guia-aprendizaje-backend/fuente
pip install weasyprint markdown pygments
python build.py
python check_links.py build/guia-backend-it-service-manager.pdf
```

ANEXO E

Mapa comentado de ficheros

↑ Volver al índice

Los ficheros de los tres proyectos del backend, con una línea que explica para qué sirven y el capítulo donde se estudian. Los ficheros de configuración local de los IDE y las carpetas generadas (`target`, `.mvn`) se omiten.

E.1 Repositorio publicado: `it-service-demand-manager` (fase 2)

↑ Índice

Fichero	Para qué sirve	Cap.
`README.md`	Estado, tecnologías y alcance de la copia pública	1
`docs/instalacion.md`	Puesta en marcha con variables de entorno y advertencia sobre Flyway	3, 4
`docs/arquitectura.md`	Resumen de capas y *endpoints*	9
`backend/pom.xml`	*Starters* de Spring Boot 4.1.1 y Java 21	2
`backend/…/application.properties`	Conexión por `DB_URL`, `DB_USERNAME`, `DB_PASSWORD`	3
`backend/…/V1__create_applications.sql`	Tabla `applications`	4
`backend/…/V2__insert_initial_applications.sql`	Datos ficticios (distintos del proyecto local)	4
`backend/…/application/**`	Módulo de aplicaciones completo	5-9
`backend/…/exception/**`	Formato de error y manejador con dos casos	10
`backend/…/ItServiceManagerApplicationTests.java`	`contextLoads` sin Testcontainers	11
`frontend/**`	Primera versión del frontend (fuera del alcance de esta guía)	—

E.2 `it-service-manager`: servicio de demandas (puerto 8080)

↑ Índice

Raíz

Fichero	Para qué sirve	Cap.
`pom.xml`	*Starters*, Testcontainers, `spring-boot-starter-restclient` y Actuator	2, 16, 20, 26
`Dockerfile`, `.dockerignore`	Imagen en dos etapas con el JAR por capas	26
`README.md`	API, variables, ejecución, pruebas y migraciones	27

Fichero	Para qué sirve	Cap.
`.gitignore`	Excluye `.env`, volcados y `.metadata/`	25
`compose.yaml`	PostgreSQL 18.6 con volumen, salud y puerto 5433. **Retirado en la fase 7**	3, 27
`.env`	Contraseña del Compose antiguo; no versionado ni reproducido en la guía	3
`applications_backup.sql`, `it_service_manager_backup.dump`	Volcados de datos; no reproducidos en la guía; conviene retirarlos	19, 23
`docs/frontera-catalogo.md`	Diseño del puerto, la consulta por lotes y la ordenación por nombre	18
`docs/verificacion-refactor.md`	Estado de entrega del refactor y verificación pendiente	18, 23
`HELP.md`	Enlaces generados por Spring Initializr	—

src/main

Fichero	Para qué sirve	Cap.
`ItServiceManagerApplication.java`	Clase principal	2
`resources/application.properties`	Conexión, JPA, URL del catálogo y Actuator por variables de entorno	3, 20, 25, 26
`resources/application-docker.properties`	Perfil `docker`: conexión al 5433. **Retirado en la fase 7**	3, 25
`db/migration/V1__create_applications.sql`	Tabla heredada `applications`	4
`db/migration/V2__insert_initial_applications.sql`	SIGMA y ORION	4
`db/migration/V3__create_demands.sql`	Tabla `demands`, clave foránea e índices	12
`db/migration/V4__detach_demands_from_applications.sql`	Elimina la clave foránea	21
`db/migration/V5__drop_legacy_applications.sql`	Elimina la tabla heredada	21
`demand/entity/Demand.java`	Entidad con `applicationId`, fechas y máquina de estados	12, 18
`demand/repository/DemandRepository.java`	`JpaRepository` + `JpaSpecificationExecutor`	13
`demand/service/DemandService.java`	Casos de uso, validación, lotes, 404, 409 y transacciones cortas	13, 15, 18, 25
`demand/controller/DemandController.java`	*Endpoints* `/api/demands`	15
`demand/dto/DemandCreateRequest.java`, `DemandUpdateRequest.java`	Entrada validada	15

Fichero	Para qué sirve	Cap.
`demand/dto/DemandResponse.java`, `DemandPageResponse.java`	Salida	15
`demand/port/ApplicationCatalog.java`	Puerto de solo lectura	18
`demand/port/ApplicationSummary.java`	*Record* que cruza la frontera	18
`demand/adapter/RestApplicationCatalog.java`	Adaptador HTTP con tiempos máximos, 503 y 502	20, 25
`exception/ApiErrorResponse.java`	Formato de error	10
`exception/GlobalExceptionHandler.java`	Seis manejadores, incluido el de último recurso	10, 25

`src/test`

Fichero	Para qué sirve	Cap.
`ItServiceManagerApplicationTests.java`	Arranque completo con Testcontainers (clase clásica)	16
`demand/entity/DemandTest.java`	Transiciones de estado (5)	12
`demand/service/DemandServiceTest.java`	Validación de filtros y orden (3)	15
`demand/service/DemandCatalogTest.java`	Uso del puerto: alta, ausencias, lotes, degradación (10)	18, 25
`demand/controller/DemandControllerTest.java`	Contrato HTTP con MockMvc (8)	15
`demand/adapter/RestApplicationCatalogTest.java`	Adaptador contra un servidor HTTP del JDK (7)	20, 25
`demand/DemandIntegrationTest.java`	PostgreSQL real, catálogo simulado, sin clave foránea (6)	21

E.3 `application-catalog-service`: catálogo (puerto 8081)

↑ Índice

Fichero	Para qué sirve	Cap.
`pom.xml`	*Starters* y módulos de prueba de Spring Boot 4, Testcontainers y Actuator	19, 26
`Dockerfile`, `.dockerignore`, `README.md`	Imagen del servicio y documentación	26, 27
`resources/application.properties`	Puerto 8081, `application_db` y Actuator, por variables de entorno	19, 25, 26
`db/migration/V1__create_applications.sql`	Tabla `applications`, sin datos	19
`ApplicationCatalogServiceApplication.java`	Clase principal	19
`application/entity/Application.java`	Entidad sin *setters* con `update`	5

Fichero	Para qué sirve	Cap.
`application/repository/ApplicationRepository.java`	Consultas derivadas paginadas	6
`application/service/ApplicationService.java`	Casos de uso, validación de filtros y resúmenes por lote	7, 10, 19
`application/controller/ApplicationController.java`	*Endpoints*, incluido `/summaries`	9, 19
`application/dto/*`	Entrada, salida, página y `ApplicationSummaryResponse`	8, 9, 19
`exception/*`	Formato de error y seis manejadores	10, 25
`test/…/ApplicationCatalogServiceApplicationTests.java`	Arranque con Testcontainers (clase de Testcontainers 2)	19
`test/…/service/ApplicationServiceTest.java`	Servicio con Mockito (8)	11
`test/…/controller/ApplicationControllerTest.java`	Contrato HTTP con MockMvc (7)	11, 19
`test/…/ApplicationIntegrationTest.java`	Catálogo vacío tras Flyway; alta y modificación reales (2)	19

E.4 `it-service-platform`: el sistema completo (fase 7)

↑ Índice

Fichero	Para qué sirve	Cap.
`compose.yaml`	PostgreSQL, los dos servicios y el frontend; volumen externo, salud y dependencias	27
`postgres/init/10-application-db.sh`	Crea `application_db` y su usuario en instalaciones nuevas	27
`.env.example`	Plantilla de las contraseñas (`.env` no se versiona)	27
`.gitignore`, `.gitattributes`	Excluyen `.env` y fuerzan LF en los *scripts*	27
`README.md`	Arranque, comandos y decisiones	27

E.5 `it-service-manager-web`: ficheros de despliegue

↑ Índice

Fichero	Para qué sirve	Cap.
`Dockerfile`, `.dockerignore`	Compila con Node y sirve con nginx	27
`docker/nginx.conf`	Proxy inverso hacia los dos servicios y *fallback* de la SPA	27
`docs/guia-aprendizaje-backend/`	Esta guía y su fuente	—

E.6 Dónde está cada concepto

↑ Índice

Concepto	Fichero de referencia
Dirty checking	`ApplicationService.updateApplication`, `DemandService.updateDemand`
Máquina de estados en la entidad	`Demand.changeStatus`
`Specification`	`DemandService.getDemands`
Consulta por lotes	`DemandService.getDemands` + `RestApplicationCatalog.findByIds`
Puertos y adaptadores	`demand/port/*` + `demand/adapter/*`
Tiempos máximos	`RestApplicationCatalog.createRequestFactory`
Transacción delimitada en el código	`DemandService.updateDemand` (`TransactionOperations`)
Comprobaciones de salud	`application.properties` (Actuator) + `healthcheck` en `compose.yaml`
Imagen por capas	`Dockerfile` de cada servicio
Proxy inverso	`it-service-manager-web/docker/nginx.conf`
404 frente a 503	`RestApplicationCatalog.findById` y `executeRequest`
Testcontainers	`*IntegrationTest`, `*ApplicationTests`
Formato de error	`GlobalExceptionHandler` en ambos servicios